

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN 1

Học có giám sát và ứng dụng

Môn học: Nhập môn học máy

GV lý thuyết: TS. Bùi Tiến Lên

GV thực hành: ThS. Lê Nhựt Nam

Lớp: 23_24

Nhóm: 13

TP. Hồ Chí Minh, tháng 4 năm 2026

Mục lục

Mục lục	2
Danh sách hình vẽ	4
Danh sách bảng	6
1 Bài toán hồi quy	7
1.1 Dữ liệu	7
1.1.1 Mô tả dữ liệu	7
1.1.2 Phân tích khám phá	7
1.1.3 Tiền xử lý	11
1.2 Các mô hình	12
1.2.1 Hồi quy tuyến tính	12
1.2.2 Kiểm tra giả thiết Gauss–Markov	16
1.2.3 Regularization	17
1.2.4 Mô hình với Hàm Cơ sở Phi Tuyến	25
1.2.5 Validation Curve	27
1.2.6 Ablation Study	30
1.3 Đánh giá và phân tích các mô hình	35
1.3.1 Đánh giá trên tập kiểm tra	35
1.3.2 Đánh giá qua cross-validation	36
1.3.3 Đánh giá kiểm định các cặp mô hình	37
1.3.4 Các biểu đồ so sánh	40
1.4 Phần nâng cao	43
1.4.1 Gaussian Process Regression	43
1.4.2 Kernel Ridge Regression	44
1.4.3 Phân tích Bias-Variance	46
1.4.4 Bayesian Linear Regression	48
1.4.5 Robust Regression với IRLS	52
2 Bài toán phân lớp	54
2.1 Dữ liệu	54
2.1.1 Mô tả dữ liệu	54
2.1.2 Phân tích khám phá	54
2.1.3 Tiền xử lý	62
2.2 Các mô hình	64
2.2.1 Hồi quy Logistic (Logistic Regression)	64
2.2.2 Linear Discriminant Analysis	68
2.2.3 Perceptron	73
2.2.4 Regularization trong Logistic Regression	75
2.2.5 Chọn tham số bằng Stratified K-Fold	79
2.2.6 Class-Weighted Loss	82
2.3 Đánh giá và phân tích các mô hình	86
2.3.1 Đánh giá trên tập kiểm tra	86
2.3.2 Đánh giá qua cross-validation	87
2.3.3 Đánh giá kiểm định các cặp mô hình	88
2.3.4 Các biểu đồ so sánh	89

2.4	Phần nâng cao	94
2.4.1	Mô hình Probit	94
2.4.2	Xấp xỉ Laplace	99
2.4.3	Kernel Logistic Regression	100
2.4.4	Gaussian Naive Bayes vs. LDA	102
2.4.5	Phân tích VC Dimension và Structural Risk Minimization	104
3	Kết nối hồi quy và phân lớp	108
3.1	Logistic Regression như một bài toán hồi quy	108
3.2	Khung Generalized Linear Models (GLM)	108
3.3	Vai trò của exponential family	109
3.4	Tổng kết	109
4	Phân tích mức độ nhạy cảm và tính bền vững	109
4.1	Độ nhạy cảm với tỉ lệ tập huấn luyện và tập kiểm tra	109
4.1.1	Hồi quy	109
4.1.2	Phân lớp	112
4.2	Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng	114
4.2.1	Hồi quy	114
4.2.2	Phân lớp	116
4.3	So sánh các chiến lược bổ sung dữ liệu thiếu (Imputation)	117
4.3.1	Hồi quy	117
4.3.2	Phân lớp	120
4.4	Khả năng hội tụ	122
4.4.1	Hồi quy	122
4.4.2	Phân lớp	122
5	Kiểm chuẩn	123
5.1	Hồi quy	123
5.2	Phân lớp	124
6	Bảng so sánh tất cả mô hình	125
A	Thông tin nhóm	127
A.1	Thông tin thành viên	127
A.2	Phân chia công việc và mức độ hoàn thành	127
A.3	Mức độ hoàn thành đề án	128
B	Mã nguồn bài làm đề án	131
	Tài liệu tham khảo	132

Danh sách hình vẽ

1	Phân phối biến mục tiêu của bộ dữ liệu California Housing	10
2	Biểu đồ phân tán giữa biến mục tiêu với các biến đặc trưng trong bộ dữ liệu California Housing	10
3	Ma trận tương quan giữa các biến trong bộ dữ liệu California Housing	11
4	Ngoại lai các biến đặc trưng trong bộ dữ liệu California Housing	12
5	Biểu đồ phần dư của mô hình hồi quy tuyến tính qua phương pháp bình phương tối thiểu	16
6	Biểu đồ phân vị-phân vị phần dư của mô hình hồi quy tuyến tính qua phương pháp bình phương tối thiểu khi so với phân phối chuẩn	17
7	Regularization path của Ridge Regression	23
8	Regularization path của Lasso Regression	24
9	Vùng tối ưu của Elastic Net	24
10	Validation curve: MSE theo bậc đa thức p	27
11	Validation curve: MSE theo số tâm RBF	28
12	Validation curve: MSE theo số hàm cơ sở Fourier	29
13	Ablation Study: Feature & Basis Importance (MSE trên validation, thang log)	31
14	Biểu đồ đường cong học của các mô hình dựa trên MSE từng kích thước của tập huấn luyện	41
15	Biểu đồ phần dư của các mô hình trên tập kiểm tra	42
16	Biểu đồ giá trị dự đoán-giá trị thực trên tập kiểm tra	42
17	Bias-Variance của Ridge trên regularization path	48
18	Bias-Variance của Lasso trên regularization path	49
19	Kết quả dự đoán của BLR: Đường trung bình (Mean) và dải tin cậy $\pm 2\sigma$	51
20	Biểu đồ đếm tần suất biến mục tiêu trong bộ dữ liệu Forest CoverType	58
21	Phân phối các biến đặc trưng định lượng của bộ dữ liệu Forest CoverType (phần 1)	59
22	Phân phối các biến đặc trưng định lượng của bộ dữ liệu Forest CoverType (phần 2)	59
23	Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 1)	60
24	Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 2)	60
25	Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 3)	61
26	Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 4)	61
27	Biểu đồ hộp từng đặc trưng định lượng theo từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType (phần 1)	62
28	Biểu đồ hộp từng đặc trưng định lượng theo từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType (phần 2)	63
29	Biểu đồ đếm tần suất biến mục tiêu theo từng khu vực trong bộ dữ liệu Forest CoverType	63
30	Biểu đồ độ chính xác từng lớp và ma trận nhầm lẫn của 3 chiến lược OvR, OvO và Softmax trực tiếp trên tập kiểm tra của bộ dữ liệu Forest CoverType	68
31	Đường biên quyết định của LDA trong không gian hai chiều	74

32	Kết quả Perceptron trên dữ liệu tuyến tính và phi tuyến	76
33	Ma trận nhầm lẫn của các mô hình trên tập kiểm tra	89
34	Biểu đồ đường cong ROC trên từng lớp của các mô hình	91
35	Biểu đồ đường cong theo từng lớp của các mô hình	92
36	Biểu đồ calibration các mô hình trên từng lớp	93
37	Đường cong mất mát huấn luyện trên từng epoch	94
38	Dữ liệu tạo sinh cho phần mô hình Probit và xấp xỉ Laplace	96
39	Đường biên quyết định của mô hình SGDClassifier (scikit-learn)	96
40	Đường biên quyết định của mô hình hồi quy Logistic	97
41	Đường biên quyết định của mô hình Probit	97
42	Xác suất đầu ra của mô hình hồi quy Logistic	98
43	Xác suất đầu ra của mô hình Probit	98
44	Vùng bất định của đường biên quyết định dựa trên xấp xỉ Laplace	100
45	Đường biên quyết định của Logistic Regression cho dữ liệu không phân chia tuyến tính	102
46	Đường biên quyết định của Kernel Logistic Regression cho dữ liệu không phân chia tuyến tính	103
47	Hiệu năng của Logistic Regression theo tham số C trên tập Covtype.	106
48	Biểu đồ hộp các chỉ số MSE của các mô hình hồi quy đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau	110
49	Biểu đồ hộp các chỉ số MAE của các mô hình hồi quy đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau	111
50	Biểu đồ hộp các chỉ số Accuracy của các mô hình phân lớp đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau	112
51	Biểu đồ hộp các chỉ số F1 của các mô hình phân lớp đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau	113
52	Biểu đồ đường cong lỗi của các mô hình hồi quy sử dụng thuật toán lặp	122
53	Biểu đồ đường cong lỗi của các mô hình phân lớp sử dụng thuật toán lặp	123

Danh sách bảng

1	Mô tả các biến trong bộ dữ liệu California Housing	8
2	Thống kê mô tả bộ dữ liệu California Housing (phần 1)	8
3	Thống kê mô tả bộ dữ liệu California Housing (phần 2)	8
4	Kết quả kiểm định Breusch–Pagan trên tập huấn luyện	17
5	Tổng hợp kết quả Ablation Study trên tập validation (MSE)	32
6	Kết quả các chỉ số của các mô hình trên tập kiểm tra	35
7	Kết quả các chỉ số của các mô hình hồi quy qua cross-validation (phần 1)	36
8	Kết quả các chỉ số của các mô hình hồi quy qua cross-validation (phần 2)	36
9	Kiểm định các cặp mô hình dựa trên các kết quả MSE thông qua cross-validation	37
10	Kiểm định các cặp mô hình dựa trên các kết quả MAE thông qua cross-validation	38
11	Kết quả cross-validation của các mô hình (phần 1)	45
12	Kết quả cross-validation của các mô hình (phần 2)	45
13	Kết quả trên tập kiểm tra	45
14	So sánh thống kê giữa các mô hình	46
15	Phân tích Bias-Variance cho Ridge và Lasso	47
16	So sánh độ nhạy với ngoại lai của các mô hình OLS, IRLS (Huber và Student-t) (phần 1)	53
17	So sánh độ nhạy với ngoại lai của các mô hình OLS, IRLS (Huber và Student-t) (phần 2)	53
18	Mô tả các biến trong bộ dữ liệu Forest CoverType	55
19	Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 1)	55
20	Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 2)	56
21	Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 3)	56
22	Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 4)	56
23	Thống kê số lượng từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType	57
24	Các chỉ số đánh giá của 3 chiến lược One-vs-Rest, One-vs-One và softmax trực tiếp trên tập kiểm tra của bộ dữ liệu Forest CoverType	67
25	So sánh giữa LDA và QDA	70
26	Top 5 đặc trưng theo Fisher ratio	73
27	So sánh Logistic Regression với L1 và L2 regularization	78
28	Kết quả các chỉ số đánh giá các mô hình trên tập kiểm tra	87
29	Kết quả trung bình và độ lệch chuẩn các chỉ số của các mô hình thông qua cross-validation (phần 1)	87
30	Kết quả trung bình và độ lệch chuẩn các chỉ số của các mô hình thông qua cross-validation (phần 2)	88
31	Kết quả kiểm định McNemar các cặp mô hình trên tập kiểm tra	88
32	Độ nhạy với nhãn nhiễu trên 2 mô hình Probit và hồi quy tuyến tính (phần 1)	99

33	Độ nhạy với nhãn nhiễu trên 2 mô hình Probit và hồi quy tuyến tính (phần 2)	99
34	Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng đối với các mô hình hồi quy	114
35	Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng đối với các mô hình phân lớp	116
36	Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình hồi quy	117
37	Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình phân lớp	120
38	Kết quả kiểm chuẩn các mô hình hồi quy (phần 1)	124
39	Kết quả kiểm chuẩn các mô hình hồi quy (phần 2)	124
40	Kết quả kiểm chuẩn các mô hình phân lớp (phần 1)	124
41	Kết quả kiểm chuẩn các mô hình phân lớp (phần 2)	125
42	So sánh toàn diện các mô hình hồi quy và phân lớp	126
43	Thông tin thành viên nhóm	127
44	Nhiệm vụ và mức độ hoàn thành yêu cầu được giao của mỗi thành viên .	127
45	Yêu cầu và mức độ hoàn thành từng yêu cầu	128

1 Bài toán hồi quy

1.1 Dữ liệu

Bộ dữ liệu California Housing được xây dựng từ cuộc điều tra dân số Hoa Kỳ năm 1990, trong đó mỗi quan sát tương ứng với một nhóm khu vực điều tra dân số (census block group), là đơn vị địa lý nhỏ nhất trong thống kê dân số, thường bao gồm từ vài trăm đến vài nghìn người (Pace & Barry, 1997).

Dữ liệu được thu thập trên toàn bang California với 20,640 quan sát và 9 đặc trưng. Mỗi nhóm khu vực được xem như một vùng dân cư và được đại diện bởi một điểm trung tâm (centroid) nhằm phục vụ cho phân tích không gian. Khoảng cách giữa các khu vực được tính dựa trên tọa độ địa lý (kinh độ và vĩ độ) của các điểm trung tâm này, từ đó phản ánh mức độ liên hệ không gian giữa các vùng (Pace & Barry, 1997).

Trung bình, mỗi nhóm khu vực có khoảng 1,425 cư dân sinh sống trong một khu vực địa lý nhỏ. Do đặc điểm phân bố dân cư, diện tích các khu vực thường giảm khi mật độ dân số tăng (Pace & Barry, 1997).

Trong quá trình xử lý dữ liệu, các quan sát có giá trị bằng 0 ở cả biến đầu vào và biến mục tiêu đã được loại bỏ để đảm bảo tính nhất quán (Pace & Barry, 1997).

Biến mục tiêu là giá trị trung vị nhà ở (median house value) trong từng khu vực, được đo theo đơn vị 100,000 USD. Một số đặc trưng như số phòng và số phòng ngủ được tính trung bình theo hộ gia đình (household), vì vậy có thể xuất hiện giá trị lớn ở những khu vực có ít hộ dân nhưng nhiều nhà trống, chẳng hạn khu nghỉ dưỡng.

Giá trị trung vị nhà bị ngắt ngưỡng ở 500,000 USD và tuổi nhà cũng bị ngắt ngưỡng ở 52 ((Géron, 2019, p. 51)).

1.1.1 Mô tả dữ liệu

Kích thước và kiểu dữ liệu

- Số lượng mẫu: 20,640
- Số lượng biến đầu vào: 8
- Số lượng biến mục tiêu: 1
- Kiểu dữ liệu: tất cả đều là số thực
- Giá trị thiếu: không có

Ý nghĩa các biến Bảng 1 mô tả ý nghĩa các biến trong bộ dữ liệu. Các biến chủ yếu gồm dữ về giá trị trung bình/trung vị các chỉ số của hộ gia đình. Dữ liệu về tọa độ khu vực được tính trên điểm trung tâm của khu vực đó.

1.1.2 Phân tích khám phá

Thống kê mô tả Bảng 2 và 3 trình bày các thống kê mô tả cơ bản của bộ dữ liệu California Housing. Các chỉ số bao gồm giá trị trung bình (mean), độ lệch chuẩn (standard deviation), giá trị nhỏ nhất (min), các phân vị (25%, 50%, 75%) và giá trị lớn nhất (max) của từng biến. Tất cả đều đã được làm tròn đến 2 chữ số thập phân.

Nhìn chung, các biến trong bộ dữ liệu có sự khác biệt đáng kể về thang đo và mức độ phân tán. Cụ thể, biến Population có giá trị trung bình là 1425.48 nhưng độ lệch chuẩn

Bảng 1: Mô tả các biến trong bộ dữ liệu California Housing

Tên biến	Kiểu dữ liệu	Ý nghĩa
MedInc	Số thực	Thu nhập trung vị của các hộ gia đình trong khu vực
HouseAge	Số thực	Tuổi trung vị của nhà trong khu vực
AveRooms	Số thực	Số phòng trung bình trên mỗi hộ gia đình
AveBedrms	Số thực	Số phòng ngủ trung bình trên mỗi hộ gia đình
Population	Số thực	Tổng dân số trong nhóm khu vực
AveOccup	Số thực	Số người trung bình trên mỗi hộ gia đình
Latitude	Số thực	Vĩ độ của khu vực
Longitude	Số thực	Kinh độ của khu vực
MedHouseVal	Số thực	Giá nhà trung vị (đơn vị: \$100,000 USD)

Bảng 2: Thống kê mô tả bộ dữ liệu California Housing (phần 1)

	HouseAge	AveRooms	AveBedrms	Population
count	20640.0	20640.0	20640.0	20640.0
mean	28.64	5.43	1.1	1425.48
std	12.59	2.47	0.47	1132.46
min	1.0	0.85	0.33	3.0
25%	18.0	4.44	1.01	787.0
50%	29.0	5.23	1.05	1166.0
75%	37.0	6.05	1.1	1725.0
max	52.0	141.91	34.07	35682.0

Bảng 3: Thống kê mô tả bộ dữ liệu California Housing (phần 2)

	MedInc	AveOccup	Latitude	Longitude	MedHouseVal
count	20640.0	20640.0	20640.0	20640.0	20640.0
mean	3.87	3.07	35.63	-119.57	2.07
std	1.9	10.39	2.14	2.0	1.15
min	0.5	0.69	32.54	-124.35	0.15
25%	2.56	2.43	33.93	-121.8	1.2
50%	3.53	2.82	34.26	-118.49	1.8
75%	4.74	3.28	37.71	-118.01	2.65
max	15.0	1243.33	41.95	-114.31	5.0

lên tới 1132.46, cho thấy sự phân bố rộng và tồn tại các khu vực có mật độ dân số rất cao. Tương tự, biến AveOccup có độ lệch chuẩn lớn (10.39) so với giá trị trung bình (3.07), phản ánh sự không đồng đều về số người trung bình trên mỗi hộ gia đình giữa các khu vực.

Đối với các đặc trưng liên quan đến nhà ở, biến AveRooms có giá trị trung bình khoảng 5.43 phòng mỗi hộ, trong khi AveBedrms xấp xỉ 1.1 phòng ngủ, cho thấy mối quan hệ hợp lý giữa số phòng và số phòng ngủ. Tuy nhiên, giá trị cực đại của AveRooms (141.91) và AveBedrms (34.07) cho thấy sự tồn tại của các giá trị ngoại lệ, xuất phát từ những khu vực như khu nghỉ dưỡng với ít hộ dân nhưng nhiều phòng trống.

Tương tự, các biến AveOccup, MedInc và MedHouseVal cũng xuất hiện các ngoại lai ở phía bên phải khi có sự chênh lệch lớn giữa giá trị phân vị thứ 3 so với giá trị cực đại ($15.0 - 4.74 = 10.26$) lớn hơn nhiều khi so với cực tiểu ($4.74 - 0.5 = 4.24$).

Các biến tọa độ địa lý Latitude và Longitude có phạm vi giá trị tương đối hẹp và phù hợp với vị trí địa lý của bang California, điều này đảm bảo tính nhất quán về không gian của dữ liệu.

Biến HouseAge có các chỉ số tương đối bình thường ngoại trừ giá trị cực đại là 52. Nó cho thấy dữ liệu có thể đã bị cắt ngưỡng.

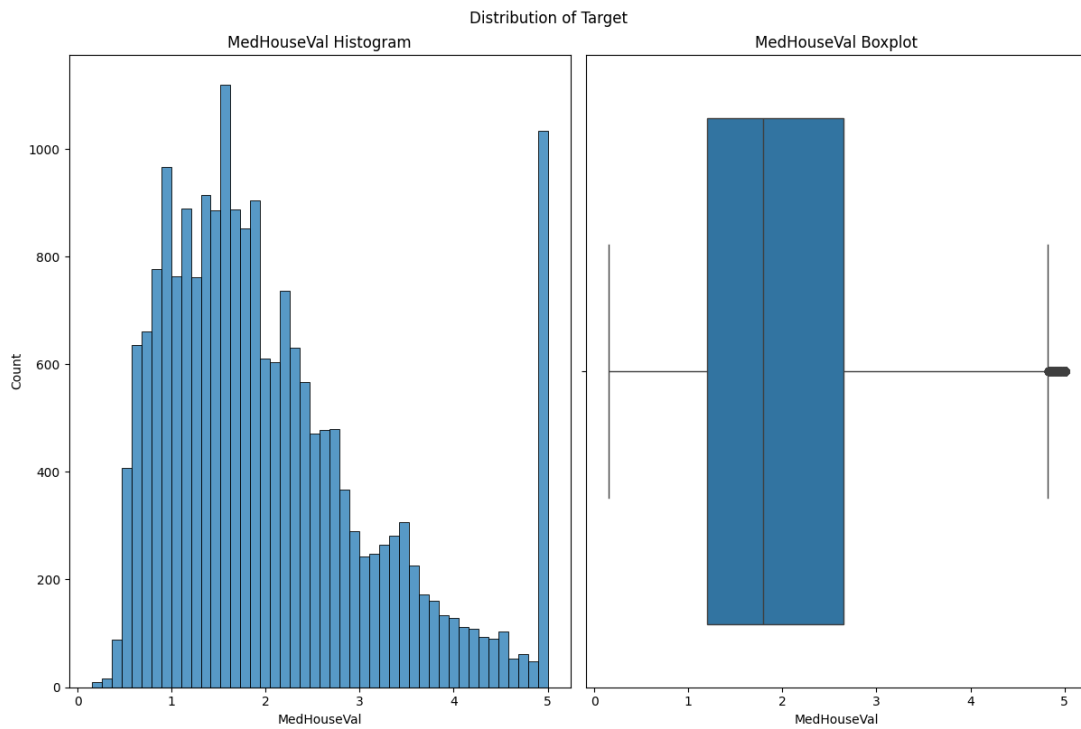
Biến mục tiêu MedHouseVal có giá trị trung bình là 2.07 (tương đương 207,000 USD), với trung vị là 1.8. Giá trị lớn nhất đạt 5.0 cho thấy dữ liệu có thể bị cắt ngưỡng ở mức giá cao.

Tổng thể, các thống kê mô tả cho thấy dữ liệu có sự phân bố không đồng đều và tồn tại ngoại lệ ở một số biến, do đó cần xem xét các bước tiền xử lý như chuẩn hóa hoặc xử lý ngoại lai trong các bước phân tích tiếp theo.

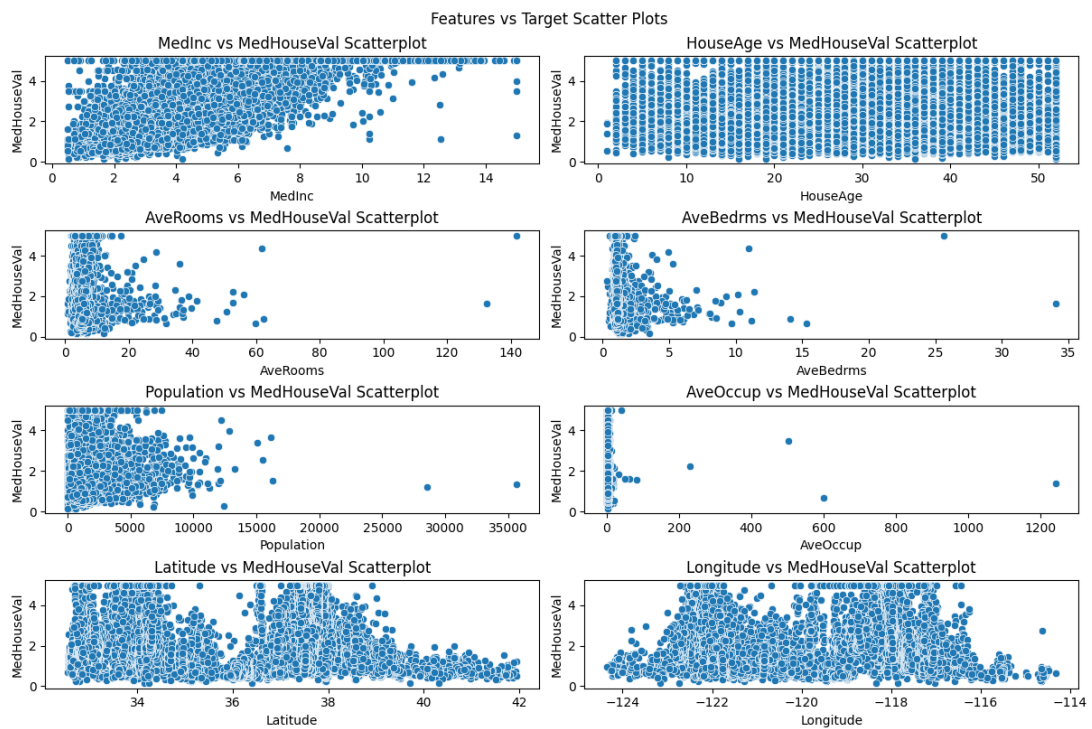
Phân phối biến mục tiêu Hình 1 gồm 2 biểu đồ là biểu đồ tần suất (histogram, bên trái) và biểu đồ hộp và râu (boxplot, bên phải), cho thấy phân phối của biến mục tiêu MedHouseVal. Ta thấy biểu đồ tần suất lệch trái, điều này đúng khi thông tin từ Bảng 3 cho ta biết trung vị lớn hơn trung bình đối với biến mục tiêu ($1.8 > 1.15$). Khoảng chia cuối cùng ở biểu đồ tần suất cho ta thấy sự bùng nổ về số lượng khu vực có chung khoảng giá trị. Điều này có thể được lý giải bằng việc cắt ngưỡng ở biến mục tiêu như đã được nói trước đó. Việc cắt ngưỡng này cũng có thể là nguyên nhân làm cho trung vị lớn hơn trung bình. Biểu đồ hộp cho thấy ngoại lai xuất hiện ở các giá trị ngắt ngưỡng này.

Mối quan hệ giữa biến mục tiêu và các biến đặc trưng Hình 2 thể hiện sự phân tán giữa từng biến đặc trưng so với mục tiêu. Ta thấy rằng biến MedInc có mối tương quan thuận mạnh so với MedHouseVal. Các biến AveRooms, AveBedrms, Population cũng có tương quan thuận với biến mục tiêu nhưng không rõ ràng. Các biến Latitude và Longitude cũng thể hiện mối quan hệ rõ rệt với mục tiêu nhưng không phải tuyến tính.

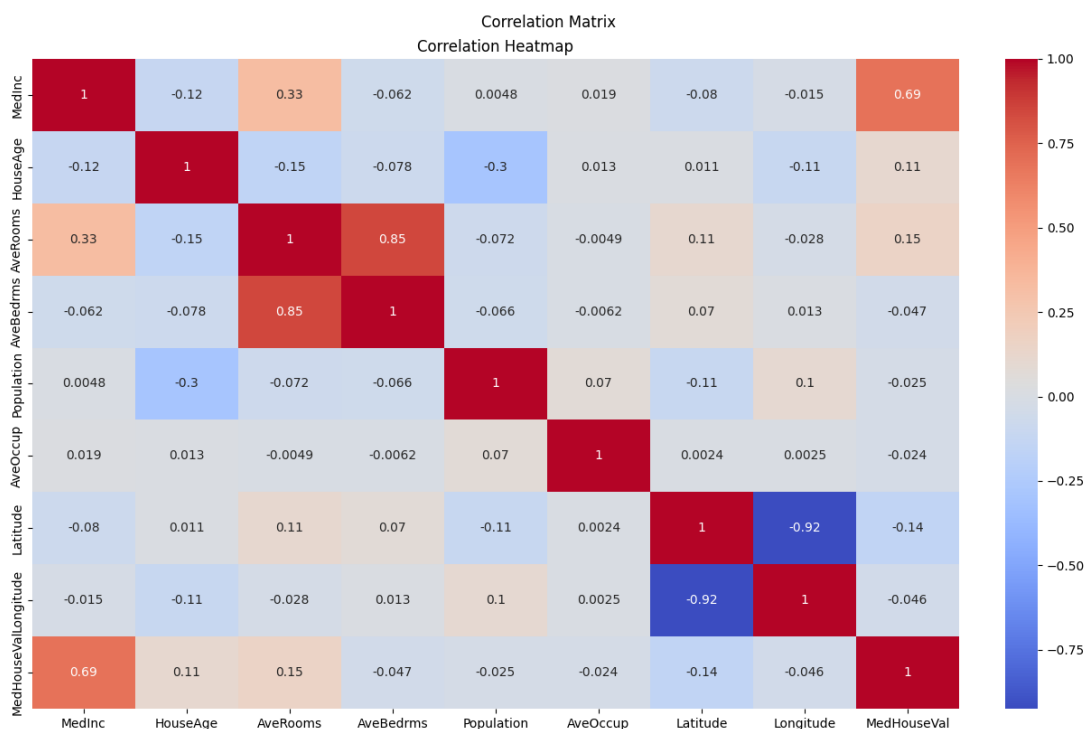
Hình 3 thể hiện các tương quan tuyến tính các biến. Ta thấy giữa Latitude và Longitude có mối tương quan nghịch rất mạnh (-0.92) điều này cũng thể hiện đúng với hình dáng chạy dọc theo trục Tây Bắc–Đông Nam của bang California. Giữa AveRooms và AveBedrms cũng có mối tương quan thuận mạnh (0.85) rất hợp lý khi số phòng ngủ thường tăng khi số phòng tăng. Đáng chú ý là giữa MedInc và biến mục tiêu có mối liên hệ tuyến tính mạnh (0.69). Do đó, MedInc có khả năng giải thích quan trọng trong các mô hình hồi quy tuyến tính.



Hình 1: Phân phối biến mục tiêu của bộ dữ liệu California Housing



Hình 2: Biểu đồ phân tán giữa biến mục tiêu với các biến đặc trưng trong bộ dữ liệu California Housing



Hình 3: Ma trận tương quan giữa các biến trong bộ dữ liệu California Housing

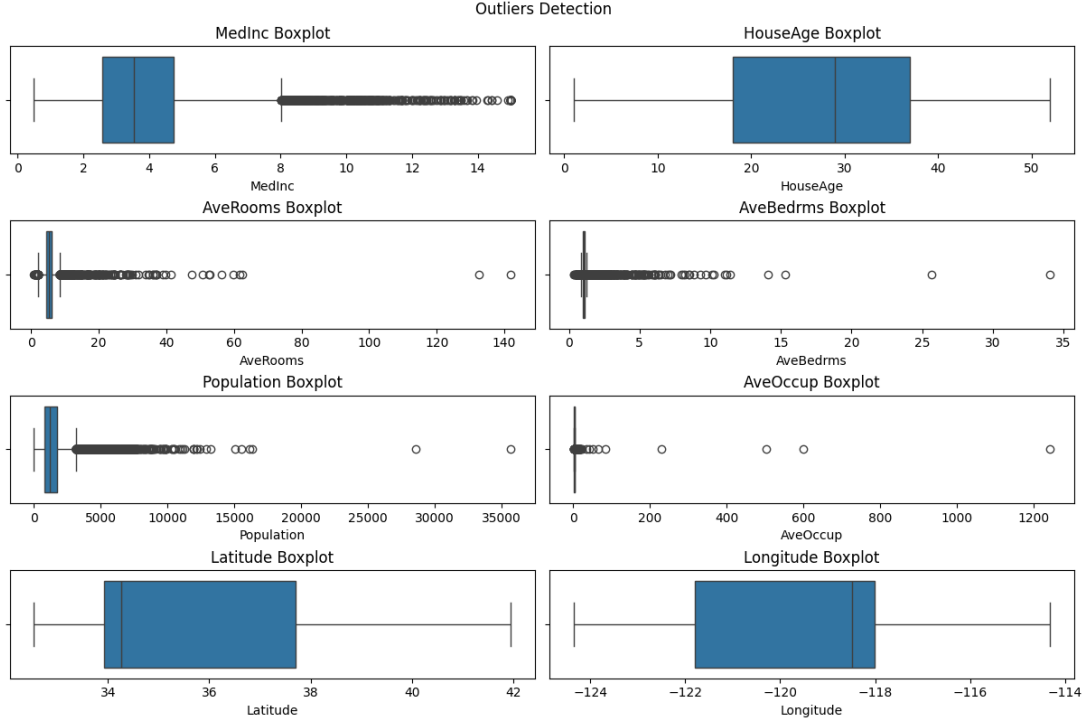
Ngoại lai Hình 4 gồm các biểu đồ hộp để thể hiện ngoại lai của các biến đặc trưng bằng phương pháp IQR. Ta thấy rằng các ngoại lai xuất hiện khá nhiều ở AveRooms, AveBedrms, Population, AveOccup và MedInc khi mà có nhiều khu vực rất giàu có, những khu vực nghỉ dưỡng có rất nhiều phòng, các khu đô thị tập trung đông đúc. Các biến HouseAge, Latitude và Longitude còn lại khá ổn định và hầu như không có ngoại lai.

1.1.3 Tiền xử lý

Xử lý các giá trị còn thiếu Bộ dữ liệu California Housing không có giá trị thiếu nên bước này được bỏ qua.

Chuẩn hóa dữ liệu Các biến đặc trưng được chuẩn hóa về trung bình $\mu = 0$ và phương sai $\sigma^2 = 1$. Điều này giúp các giá trị của các biến đặc trưng được đưa về cùng thang đo, giúp quá trình học bằng Gradient Descent của các mô hình nhanh hơn (các đường mức không còn bị kéo dẫn). Ngoài ra, việc chuẩn hóa như vậy giúp cho việc tính toán không bị tràn số (đã thử và bị với mô hình Mini-batch Gradient Descent). Lợi ích của cách chuẩn hóa này so với chuẩn hóa về $[0, 1]$ đó là sự xuất hiện các ngoại lai có giá trị rất lớn làm giá trị sau khi chuẩn hóa rất nhỏ. Điều này không làm các biến về cùng đúng thang đo so với mong muốn.

Chia dữ liệu Dữ liệu được chia thành 3 tập là huấn luyện (train), xác thực (validation) và kiểm tra (test) với tỉ lệ tương ứng là 0.6, 0.2 và 0.2.



Hình 4: Ngoại lai các biến đặc trưng trong bộ dữ liệu California Housing

1.2 Các mô hình

1.2.1 Hồi quy tuyến tính

Bình phương tối thiểu thông thường (Ordinary least squares, OLS) Phương pháp Bình phương tối thiểu thông thường (OLS) hướng đến việc tìm bộ trọng số $\mathbf{w}^*$ sao cho cực tiểu quá tổng bình phương sai số giữa giá trị thực tế và giá trị dự đoán hay cực tiểu hóa hàm lỗi:

$$E_D(\mathbf{w}) = 1/2 \cdot \|\mathbf{t} - \Phi\mathbf{w}\|^2. \quad (1)$$

Phương trình chuẩn (Normal Equation) Phương trình chuẩn cung cấp một lời giải đóng cho bài toán tối ưu trên bằng cách giải nghiệm đạo hàm bậc nhất bằng 0. Công thức nghiệm tối ưu là:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} E_D(\mathbf{w}) = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}. \quad (2)$$

Trong thực tế cài đặt, thay vì nghịch đảo trực tiếp $(\Phi^T \Phi)^{-1}$ —vốn dễ gây lỗi nếu ma trận bị suy biến (singular)—em sử dụng ma trận giả nghịch đảo Moore–Penrose $\Phi^\dagger$. Công thức triển khai trở thành:

$$\mathbf{w}^* = \Phi^\dagger \mathbf{t}. \quad (3)$$

Độ phức tạp tính toán nghiệm là $\Theta(NM^2 + NM + M^3 + M^2)$. Đối với bộ dữ liệu California Housing, khi mà $N \gg M$ ($20640 \gg 9$, trong đó 9 là số lượng đặc trưng cộng thêm 1 để học hệ số chặn), độ phức tạp tương đương $\Theta(NM^2)$.

Mini-batch Gradient Descent Đối với các bài toán có kích thước dữ liệu lớn, việc tính toán trực tiếp trên toàn bộ dữ liệu gây áp lực lớn cho bộ nhớ. Thuật toán Mini-batch Gradient Descent nhằm cân bằng giữa tốc độ hội tụ và hiệu năng tính toán.

Về hiệu suất, độ phức tạp tính toán của một lần xử lý một lô (batch) là $\Theta(BM)$. Khi đó, với một epoch, độ phức tạp là $\Theta(NM)$. Và với E epoch thì độ phức tạp là $\Theta(ENM)$.

Việc duy trì một tốc độ học η cố định trong suốt quá trình huấn luyện thường dẫn đến sự mất cân bằng giữa tốc độ hội tụ và độ ổn định của mô hình. Do đó, các chiến lược lập lịch tốc độ học được sử dụng nhằm điều chỉnh η theo tiến trình huấn luyện.

Một cách tiếp cận phổ biến là *Step Decay*, trong đó tốc độ học được giảm theo một hệ số suy giảm γ sau những khoảng chu kỳ (epochs) nhất định:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}, \quad (4)$$

trong đó η_0 là tốc độ học ban đầu, t là epoch hiện tại, s là số epoch giữa các lần giảm, và $\gamma \in (0, 1)$ là hệ số suy giảm.

Bên cạnh đó, chiến lược *Cosine Annealing* điều chỉnh tốc độ học theo một hàm cosine:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T}\right) \right), \quad (5)$$

trong đó $\eta_{\max}$ và $\eta_{\min}$ lần lượt là giá trị lớn nhất và nhỏ nhất của tốc độ học, t là epoch hiện tại, và T là tổng số epoch trong một chu kỳ.

Sự thay đổi mượt mà của Cosine Annealing giúp hạn chế dao động gradient lớn, trong khi Step Decay cung cấp cách điều chỉnh đơn giản và hiệu quả.

So sánh giữa phương pháp Normal Equation và Gradient Descent

Định lý Gauss–Markov Xét mô hình hồi quy tuyến tính

$$\mathbf{t} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}, \quad (6)$$

trong đó $\mathbf{t} \in \mathbb{R}^N$, $\mathbf{X} \in \mathbb{R}^{N \times M}$, $\mathbf{w} \in \mathbb{R}^M$, $\boldsymbol{\varepsilon} \in \mathbb{R}^N$.

Giả sử

$$\mathbb{E}[\boldsymbol{\varepsilon} \mid \mathbf{X}] = \mathbf{0}, \quad (7)$$

và

$$\text{Var}(\boldsymbol{\varepsilon} \mid \mathbf{X}) = \sigma^2 \mathbf{I}. \quad (8)$$

Ước lượng OLS được cho bởi

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}. \quad (9)$$

Thay $\mathbf{t} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}$ vào ta có

$$\hat{\mathbf{w}} = \mathbf{w} + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\varepsilon}. \quad (10)$$

Khi đó

$$\mathbb{E}[\hat{\mathbf{w}} \mid \mathbf{X}] = \mathbf{w}. \quad (11)$$

Phương sai của ước lượng là

$$\text{Var}(\hat{\mathbf{w}} \mid \mathbf{X}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}. \quad (12)$$

Với mọi ước lượng tuyến tính không chệch dạng $\tilde{\mathbf{w}} = \mathbf{A}\mathbf{t}$, điều kiện không chệch yêu cầu

$$\mathbf{A}\mathbf{X} = \mathbf{I}. \quad (13)$$

Ta có thể viết

$$\tilde{\mathbf{w}} = \mathbf{A}(\mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}) = \mathbf{w} + \mathbf{A}\boldsymbol{\varepsilon}. \quad (14)$$

Do đó

$$\text{Var}(\tilde{\mathbf{w}} \mid \mathbf{X}) = \mathbf{A} \text{Var}(\boldsymbol{\varepsilon} \mid \mathbf{X}) \mathbf{A}^\top = \sigma^2 \mathbf{A}\mathbf{A}^\top. \quad (15)$$

Với OLS, ta có

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}. \quad (16)$$

Đặt

$$\mathbf{A}_{OLS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top. \quad (17)$$

Khi đó với mọi $\mathbf{A}$ thỏa $\mathbf{A}\mathbf{X} = \mathbf{I}$, ta có thể viết

$$\mathbf{A} = \mathbf{A}_{OLS} + \mathbf{C} \quad \text{với } \mathbf{C}\mathbf{X} = \mathbf{0}. \quad (18)$$

Suy ra

$$\mathbf{A}\mathbf{A}^\top = \mathbf{A}_{OLS}\mathbf{A}_{OLS}^\top + \mathbf{C}\mathbf{C}^\top. \quad (19)$$

Do $\mathbf{C}\mathbf{C}^\top$ là ma trận nửa xác định dương nên

$$\text{Var}(\tilde{\mathbf{w}} \mid \mathbf{X}) \succeq \text{Var}(\hat{\mathbf{w}} \mid \mathbf{X}). \quad (20)$$

Do đó $\hat{\mathbf{w}}$ là ước lượng tuyến tính không chệch tốt nhất (Best Linear Unbiased Estimator, BLUE).

Bình phương tối thiểu có trọng số (Weighted Least Squares, WLS) Phương pháp Bình phương tối thiểu có trọng số (WLS) mở rộng từ OLS nhằm xử lý trường hợp phương sai của sai số không đồng nhất (heteroskedasticity), tức là:

$$\text{Var}(\varepsilon_i) = \sigma_i^2. \quad (21)$$

Ý tưởng chính là gán trọng số khác nhau cho từng quan sát, trong đó các quan sát có phương sai lớn sẽ có trọng số nhỏ hơn. Hàm lỗi của WLS được viết lại như sau:

$$E_D(\mathbf{w}) = \frac{1}{2}(\mathbf{t} - \Phi\mathbf{w})^\top \mathbf{W}(\mathbf{t} - \Phi\mathbf{w}), \quad (22)$$

trong đó $\mathbf{W} = \text{diag}(w_1, \dots, w_N)$ là ma trận trọng số, với $w_i = 1/\sigma_i^2$.

Phương trình chuẩn cho WLS Tương tự OLS, nghiệm tối ưu của bài toán trên được xác định bằng cách giải phương trình:

$$\mathbf{w}^* = (\Phi^\top \mathbf{W} \Phi)^{-1} \Phi^\top \mathbf{W} \mathbf{t}. \quad (23)$$

Trong thực tế, để tránh vấn đề ma trận suy biến, ta có thể sử dụng giả nghịch đảo:

$$\mathbf{w}^* = (\Phi^\top \mathbf{W} \Phi)^\dagger \Phi^\top \mathbf{W} \mathbf{t}. \quad (24)$$

Ước lượng khả thi bằng FGLS (Feasible Generalized Least Squares) Trong thực tế, các phương sai σ_i^2 thường không biết trước, do đó ma trận trọng số $\mathbf{W}$ cũng chưa xác định. Phương pháp FGLS được sử dụng để ước lượng các trọng số này từ dữ liệu.

Bước 1: Ước lượng ban đầu bằng OLS Ước lượng nghiệm ban đầu:

$$\hat{\mathbf{w}}_{OLS} = (\mathbf{\Phi}^\top \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \mathbf{t}. \quad (25)$$

Sau đó, tính phần dư:

$$\hat{\varepsilon}_i = t_i - \phi_i^\top \hat{\mathbf{w}}_{OLS}. \quad (26)$$

Bước 2: Mô hình hóa phương sai Giả sử phương sai phụ thuộc vào đặc trưng theo công thức:

$$\sigma_i^2 = \exp(\phi_i^\top \boldsymbol{\gamma}). \quad (27)$$

Khi đó, ta thực hiện hồi quy phụ:

$$\log(\hat{\varepsilon}_i^2) = \phi_i^\top \boldsymbol{\gamma} + u_i, \quad (28)$$

để ước lượng $\boldsymbol{\gamma}$.

Bước 3: Ước lượng trọng số Từ đó, ta có:

$$\hat{\sigma}_i^2 = \exp(\phi_i^\top \hat{\boldsymbol{\gamma}}), \quad \hat{w}_i = \frac{1}{\hat{\sigma}_i^2}. \quad (29)$$

Xây dựng ma trận trọng số:

$$\hat{\mathbf{W}} = \text{diag}(\hat{w}_1, \dots, \hat{w}_N). \quad (30)$$

Bước 4: Thực hiện WLS Cuối cùng, nghiệm FGLS được xác định:

$$\hat{\mathbf{w}}_{FGLS} = (\mathbf{\Phi}^\top \hat{\mathbf{W}} \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \hat{\mathbf{W}} \mathbf{t}. \quad (31)$$

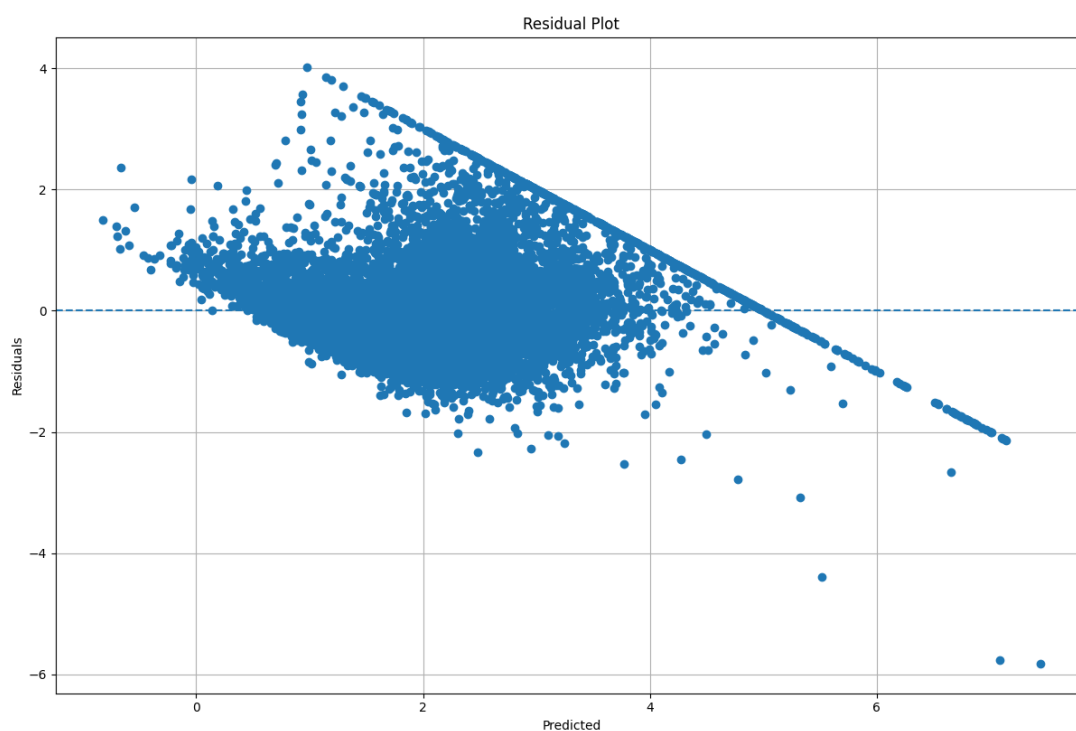
Độ phức tạp tính toán Chi phí tính toán của FGLS bao gồm ba bước chính:

- Ước lượng OLS ban đầu: $\Theta(NM^2 + M^3)$, trong đó $\Theta(NM^2)$ để tính $\mathbf{\Phi}^\top \mathbf{\Phi}$ và $\Theta(M^3)$ để nghịch đảo ma trận.
- Hồi quy phụ để ước lượng phương sai: $\Theta(NM^2 + M^3)$, do đây cũng là một bài toán OLS với cùng số chiều đặc trưng.
- Ước lượng WLS: $\Theta(NM^2 + M^3)$, bao gồm việc tính $\mathbf{\Phi}^\top \hat{\mathbf{W}} \mathbf{\Phi}$ và nghịch đảo ma trận tương ứng.

Do đó, tổng chi phí tính toán là $\Theta(NM^2 + M^3)$. Tương tự với $N \gg M$ thì độ phức tạp có thể xấp xỉ $\Theta(NM^2)$, tức là cùng bậc với OLS.

1.2.2 Kiểm tra giả thiết Gauss–Markov

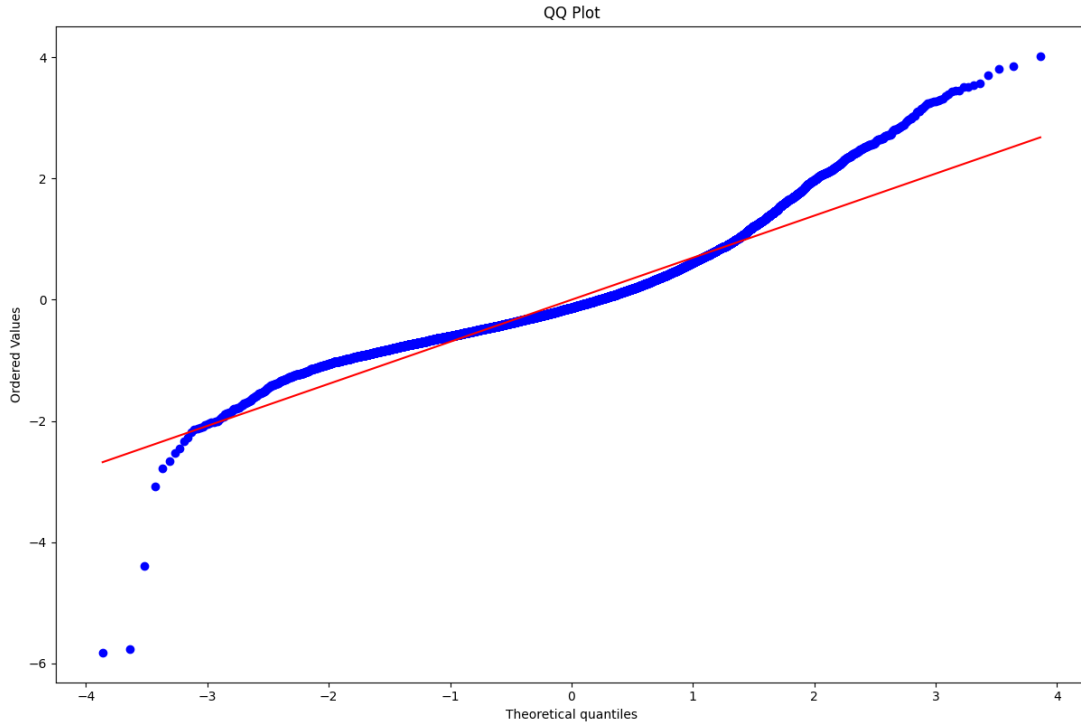
Biểu đồ phần dư (residual plot) Hình 5 thể hiện phần dư trên tập huấn luyện. Normal Equation được dùng để tìm nghiệm bình phương tối thiểu. Nhìn vào hình ta thấy có xuất hiện vệt của đường thẳng. Đây là do việc ngắt ngưỡng của biến mục tiêu. Độ phân tán không đồng đều (trong đoạn $[0, 1]$ và $[4, 6]$) thấp và không ổn định như trong đoạn $[2, 4]$. Điều này cho thấy giả thiết về phương sai bằng nhau giữa các mẫu là không đúng. Phân tán phần dư trong đoạn $[0, 1]$ và $[4, 6]$ nằm trên đường thẳng $e = 0$ cho thấy trung bình phần dư trên đoạn này cũng không bằng 0. Trên đoạn $[6, 8]$ cũng thấy trung bình phần dư nhỏ hơn 0 nhưng nguyên nhân chính là do việc ngắt ngưỡng. Trên đoạn $[2, 4]$, giả thiết Gauss–Markov vẫn giữ tính đúng.



Hình 5: Biểu đồ phần dư của mô hình hồi quy tuyến tính qua phương pháp bình phương tối thiểu

Biểu đồ phân vị–phân vị (Quantile-Quantile plot) Hình 6 cho thấy sự tuân theo phân phối chuẩn của phần dư trên tập huấn luyện. Ở vùng trung tâm ($[-1, 2]$) các điểm bám sát với đường tham chiếu. Điều này thể hiện mô hình ổn định và phần dư bám sát phân phối chuẩn. Vùng đuôi bên trái ($[-3, -11]$), đường biểu diễn cong nhẹ lên đường tham chiếu. Điều này cho thấy hiện tượng đuôi mỏng, nghĩa là mô hình hiếm khi xảy ra sai số âm cực lớn. Vùng đuôi bên phải ($[2, 4]$), xuất hiện hiện tượng vênh cao rõ rệt. Đây là dấu hiệu hiện tượng đuôi dày, phản ánh sự tồn tại của các sai số dương lớn. Hiện tượng này chủ yếu do dữ liệu bị chặn ngưỡng tại mức. Khi giá trị thực tế cao vượt ngưỡng nhưng dữ liệu ghi nhận thấp hơn, mô hình tuyến tính OLS khó có thể khớp chính xác, dẫn đến bùng nổ sai số dương ở phân khúc này.

Kiểm định Breusch–Pagan Bảng 4 cho kết quả đánh giá giả thuyết về phương sai sai số không đổi (Homoscedasticity)—một trong điều kiện tiên quyết của định lý Gauss–Markov để ước lượng bình phương tối thiểu là không chệch tuyến tính tốt nhất (BLUE).



Hình 6: Biểu đồ phân vị-phân vị phần dư của mô hình hồi quy tuyến tính qua phương pháp bình phương tối thiểu khi so với phân phối chuẩn

Cả hai giá trị thống kê (LM Stat khoảng 1248.62 và F Stat khoảng 173.45) đều rất lớn. Điều này cho thấy sự sai lệch đáng kể giữa phân phối phần dư thực tế và giả thiết phương sai sai số đồng nhất. Và với mức ý nghĩa $\alpha = 0.05$, ta có p -value nhỏ hơn α . Do đó ta bác bỏ giả thuyết H_0 (phương sai sai số không đổi) và chấp nhận giả thuyết H_1 (phương sai sai số thay đổi). Kết quả này nhất quán với nội dung biểu đồ phần dư khi độ rộng phân tán không đều.

Bảng 4: Kết quả kiểm định Breusch–Pagan trên tập huấn luyện

Thông số	Giá trị
LM Stat	1248.6156
LM p-value	2.9971×10^{-264}
F Stat	173.4518
F p-value	1.0491×10^{-278}

1.2.3 Regularization

Ý tưởng Để điều khiển hiện tượng quá khớp (*overfitting*), ta thêm vào hạng tử điều chỉnh vào hàm lỗi sao cho hàm lỗi tổng cần được tối thiểu hóa có dạng:

$$E_D(\mathbf{w}) + \lambda E_W(\mathbf{w}), \quad (32)$$

trong đó λ là hệ số điều chỉnh (regularization coefficient) dùng để kiểm soát mức độ quan trọng tương đối giữa sai số phụ thuộc dữ liệu $E_D(\mathbf{w})$ và hạng tử điều chỉnh $E_W(\mathbf{w})$. Sau đây là các hàm $E_W(\mathbf{w})$ hay được dùng.

Ridge Regression

Hạng tử điều chỉnh Công thức hạng tử điều chỉnh trong Ridge Regression có dạng:

$$\frac{1}{2}\|\mathbf{w}\|_2 \quad (33)$$

Hạng tử này chính là tổng bình phương các trọng số, còn được gọi là chuẩn L_2 của vector trọng số. Việc đưa hạng tử này vào hàm lỗi có tác dụng làm "phạt" các giá trị trọng số lớn.

Hàm mục tiêu Khi đó, hàm mục tiêu của Ridge Regression là:

$$E(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2. \quad (34)$$

Nghiệm giải Ta giải được nghiệm của Ridge Regression là:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{t}. \quad (35)$$

So với nghiệm của hồi quy tuyến tính thông thường:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}. \quad (36)$$

Ridge Regression bổ sung thêm $\lambda \mathbf{I}$ vào ma trận $\mathbf{X}^\top \mathbf{X}$. Điều này mang lại hai lợi ích quan trọng:

- **Tránh ma trận suy biến:** Khi $\mathbf{X}^\top \mathbf{X}$ không khả nghịch, việc cộng $\lambda \mathbf{I}$ giúp ma trận luôn khả nghịch.
- **Ổn định nghiệm:** Giảm độ nhạy của nghiệm đối với nhiễu trong dữ liệu.

Diễn giải trực quan Có thể hiểu Ridge Regression là bài toán tối ưu với ràng buộc:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 \quad \text{với} \quad \|\mathbf{w}\|^2 \leq C. \quad (37)$$

Tức là ta tìm nghiệm vừa khớp dữ liệu tốt, vừa nằm trong một "quả cầu" giới hạn độ lớn của trọng số. Điều này giúp mô hình tránh học các hệ số quá lớn gây overfitting.

Lasso Regression

Hạng tử điều chỉnh Khác với Ridge Regression sử dụng chuẩn L_2 , Lasso Regression sử dụng chuẩn L_1 cho vector trọng số. Hạng tử điều chỉnh có dạng:

$$E_W(\mathbf{w}) = \|\mathbf{w}\|_1 \quad (38)$$

Hạng tử này là tổng giá trị tuyệt đối của các trọng số. Việc sử dụng chuẩn L_1 cũng nhằm "phạt" các trọng số lớn, tuy nhiên theo cách khác với Ridge.

Hàm mục tiêu Hàm mục tiêu của Lasso Regression được viết như sau:

$$E(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 + \lambda\|\mathbf{w}\|_1. \quad (39)$$

Nghiệm giải Không giống như Ridge Regression, Lasso Regression **không có nghiệm đóng dạng tường minh**. Nguyên nhân là do hàm $|w|$ không khả vi tại $w = 0$. Do đó, nghiệm của Lasso thường được tìm bằng các phương pháp tối ưu số, chẳng hạn như:

- Gradient Descent (với subgradient)
- Coordinate Descent
- Least Angle Regression (LARS)

Đặc điểm Một tính chất quan trọng của Lasso Regression là khả năng tạo ra nghiệm thưa (“sparse solution”), tức là nhiều trọng số có giá trị đúng bằng 0.

- Ridge Regression: làm nhỏ các trọng số nhưng hiếm khi bằng 0
- Lasso Regression: có thể đưa một số trọng số về đúng 0

Nhờ đó, Lasso có thể được sử dụng như một phương pháp **lựa chọn đặc trưng** (feature selection).

Diễn giải trực quan Có thể hiểu Lasso Regression là bài toán tối ưu với ràng buộc:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 \quad \text{với} \quad \|\mathbf{w}\|_1 \leq C. \quad (40)$$

Miền ràng buộc $\|\mathbf{w}\|_1 \leq C$ có dạng hình thoi (diamond). Khi kết hợp với các đường đồng mức của hàm lỗi, nghiệm tối ưu thường rơi vào các đỉnh của hình thoi, dẫn đến nhiều hệ số bằng 0.

Elastic Net

Ý tưởng Elastic Net là phương pháp kết hợp cả hai dạng điều chuẩn L_1 (Lasso) và L_2 (Ridge). Mục tiêu là tận dụng ưu điểm của cả hai:

- L_1 : tạo nghiệm thưa (feature selection)
- L_2 : ổn định nghiệm và xử lý tốt khi các biến có tương quan cao

Hạng tử điều chỉnh Hạng tử điều chỉnh trong Elastic Net có dạng:

$$E_W(\mathbf{w}) = \alpha \|\mathbf{w}\|_1 + \frac{1 - \alpha}{2} \|\mathbf{w}\|_2^2 \quad (41)$$

Hàm mục tiêu Hàm mục tiêu của Elastic Net là:

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 + \lambda \left(\alpha \|\mathbf{w}\|_1 + \frac{1 - \alpha}{2} \|\mathbf{w}\|^2 \right) \quad (42)$$

Các trường hợp đặc biệt Elastic Net bao gồm hai phương pháp trước như các trường hợp riêng:

- $\alpha = 1$: trở thành Lasso Regression
- $\alpha = 0$: trở thành Ridge Regression

Nghiệm giải Tương tự như Lasso, Elastic Net **không có nghiệm đóng tường minh** do sự xuất hiện của chuẩn L_1 . Nghiệm thường được tìm bằng các phương pháp tối ưu số:

- Coordinate Descent (phổ biến nhất)
- Gradient Descent với kỹ thuật proximal

Ưu điểm Elastic Net khắc phục được một số hạn chế của Lasso:

- Khi các biến đầu vào có tương quan cao:
 - Lasso có xu hướng chọn một biến và bỏ các biến còn lại
 - Elastic Net có thể giữ lại nhóm biến liên quan
- Vẫn giữ được tính chất nghiệm thưa (sparse)
- Ổn định hơn so với Lasso khi dữ liệu có nhiễu

Diễn giải trực quan Elastic Net có thể được hiểu là bài toán với ràng buộc kết hợp:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 \quad \text{với} \quad \alpha\|\mathbf{w}\|_1 + (1 - \alpha)\|\mathbf{w}\|^2 \leq C \quad (43)$$

Elastic Net có thể được hiểu là bài toán tối ưu với ràng buộc:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 \quad \text{với} \quad \alpha\|\mathbf{w}\|_1 + (1 - \alpha)\|\mathbf{w}\|^2 \leq C \quad (44)$$

Miền ràng buộc trong trường hợp này là sự kết hợp giữa hình thoi (L_1) và hình cầu (L_2).

Do đó:

- Giống Lasso: nghiệm có xu hướng thưa (nhiều hệ số bằng 0)
- Giống Ridge: nghiệm ổn định hơn và ít nhạy với nhiễu

Elastic Net tạo ra sự cân bằng giữa việc lựa chọn đặc trưng và duy trì tính ổn định của mô hình, đặc biệt hiệu quả khi các biến đầu vào có tương quan cao.

Chọn hệ số điều chỉnh λ tốt nhất Việc lựa chọn hệ số điều chỉnh λ là một bước quan trọng trong các mô hình regularization, vì tham số này quyết định sự cân bằng giữa mức độ khớp dữ liệu và khả năng tổng quát hóa của mô hình. Nếu λ quá nhỏ, mô hình dễ xảy ra hiện tượng *overfitting*; ngược lại, nếu λ quá lớn, mô hình có thể bị *underfitting*. Một phương pháp phổ biến để lựa chọn λ là sử dụng ***k*-fold Cross Validation**.

Phương pháp k -fold Cross Validation Tập dữ liệu huấn luyện được chia thành k phần có kích thước xấp xỉ bằng nhau:

$$D = D_1 \cup D_2 \cup \dots \cup D_k. \quad (45)$$

Với mỗi giá trị λ trong tập giá trị ứng viên, ta thực hiện:

1. Chọn lần lượt từng tập D_i làm tập kiểm định.
2. Sử dụng $k - 1$ tập còn lại để huấn luyện mô hình.
3. Tính sai số trên tập kiểm định.
4. Lặp lại cho $i = 1, 2, \dots, k$.

Sai số cross-validation ứng với mỗi λ được tính bằng trung bình sai số trên k lần:

$$CV(\lambda) = \frac{1}{k} \sum_{i=1}^k E_i(\lambda), \quad (46)$$

trong đó $E_i(\lambda)$ là sai số trên fold thứ i . Giá trị tối ưu của λ được chọn theo tiêu chuẩn:

$$\lambda^* = \arg \min_{\lambda} CV(\lambda). \quad (47)$$

Grid Search Để tìm giá trị λ tối ưu, ta xây dựng một tập giá trị ứng viên theo lưới tìm kiếm như sau:

$$\lambda \in \{\lambda_1, \lambda_2, \dots, \lambda_{n-1}, \lambda_n\}. \quad (48)$$

Mỗi giá trị trong lưới sẽ được đánh giá bằng quy trình k -fold cross-validation ở trên. Phương pháp này được gọi là **grid search**.

Warm Start Để tăng tốc quá trình huấn luyện khi thử nhiều giá trị λ , ta sử dụng kỹ thuật **warm start**. Cụ thể, khi huấn luyện với giá trị λ_j , thay vì khởi tạo ngẫu nhiên tham số $\mathbf{w}$, ta sử dụng nghiệm tối ưu thu được từ giá trị trước đó λ_{j-1} :

$$\mathbf{w}_{\lambda_j}^{(0)} = \mathbf{w}_{\lambda_{j-1}}^*. \quad (49)$$

Kỹ thuật này giúp giảm số vòng lặp hội tụ, đặc biệt khi các giá trị λ được sắp xếp liên tiếp theo thang logarit.

Quy trình tổng quát Toàn bộ quy trình chọn λ được thực hiện như sau:

1. Xây dựng tập giá trị ứng viên của λ .
2. Thực hiện grid search kết hợp k -fold cross-validation.
3. Sử dụng warm start để tăng tốc quá trình huấn luyện.
4. Chọn giá trị λ^* cho sai số validation nhỏ nhất.

Lựa chọn đặc trưng Lựa chọn đặc trưng nhằm mục tiêu lựa chọn một tập con các đặc trưng quan trọng nhất, giúp giảm độ phức tạp của mô hình, cải thiện khả năng tổng quát hóa và giảm hiện tượng overfitting. Dưới đây là một số phương pháp phổ biến.

Forward Stepwise Selection Phương pháp này bắt đầu với một mô hình rỗng (không có đặc trưng nào). Tại mỗi bước, ta thêm vào đặc trưng giúp cải thiện hiệu năng mô hình nhiều nhất (ví dụ: giảm sai số hoặc tăng độ chính xác).

- Khởi đầu với tập đặc trưng rỗng.
- Tại mỗi bước, thử thêm từng đặc trưng chưa được chọn vào mô hình.
- Chọn đặc trưng làm cải thiện tốt nhất theo tiêu chí đánh giá.
- Lặp lại cho đến khi không còn cải thiện đáng kể hoặc đạt số lượng đặc trưng mong muốn.

Backward Elimination Ngược lại với forward selection, phương pháp này bắt đầu với toàn bộ tập đặc trưng và loại bỏ dần các đặc trưng kém quan trọng.

- Khởi đầu với tất cả các đặc trưng.
- Tại mỗi bước, loại bỏ đặc trưng ít đóng góp nhất (ví dụ: có trọng số nhỏ hoặc không có ý nghĩa thống kê).
- Đánh giá lại mô hình sau khi loại bỏ.
- Lặp lại cho đến khi việc loại bỏ làm giảm hiệu năng đáng kể.

Feature Selection bằng Lasso Phương pháp này dựa trên regularization với chuẩn L_1 (Lasso). Khi huấn luyện mô hình với Lasso, một số hệ số w_j sẽ bị đưa về đúng 0.

- Huấn luyện mô hình với regularization L_1 .
- Các đặc trưng có hệ số $w_j = 0$ được xem là không quan trọng và bị loại bỏ.
- Các đặc trưng có hệ số khác 0 được giữ lại.

Ưu điểm của phương pháp này là thực hiện feature selection một cách tự động trong quá trình huấn luyện, đặc biệt hiệu quả khi số chiều lớn.

Thí nghiệm và kết quả

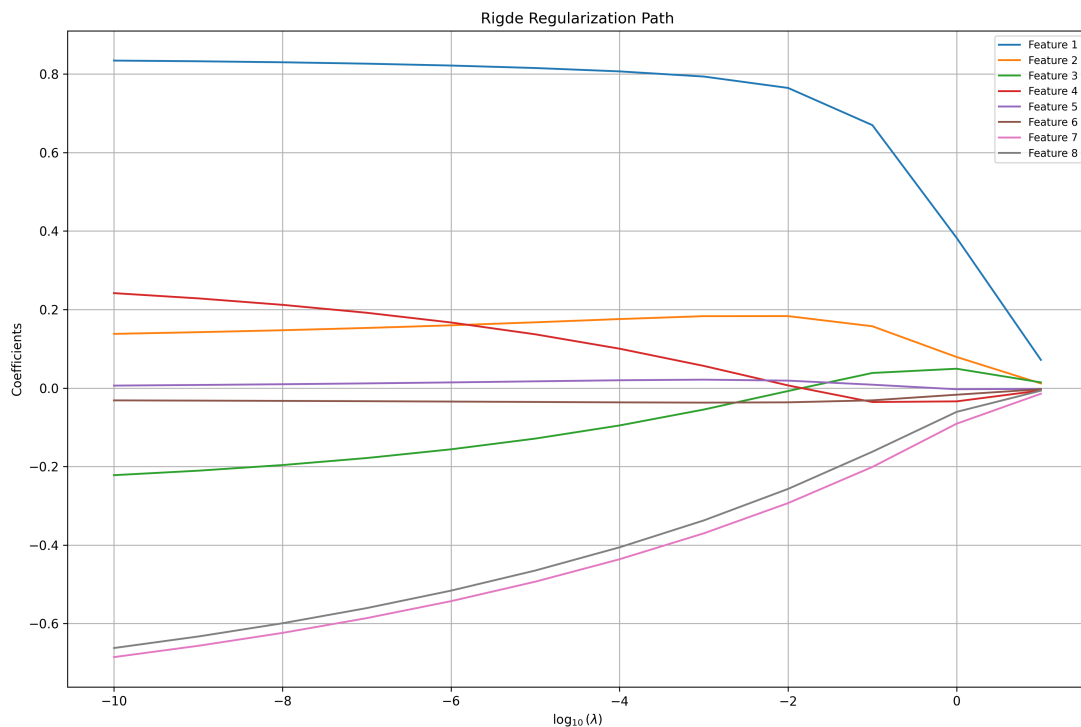
Chọn λ tốt nhất bằng k -fold Chọn tập λ ứng viên là:

$$\lambda \in \{10^{-10}, 10^{-12}, \dots, 10^{-1}, 1, 10\}. \quad (50)$$

Kết quả Kết quả thực nghiệm trên cả Ridge Regression và Lasso Regression cho thấy giá trị λ tối ưu nằm ở biên nhỏ nhất của tập ứng viên, cụ thể là $\lambda = 10^{-10}$. Điều này cho thấy mô hình gần như không cần đến regularization và nghiệm thu được xấp xỉ nghiệm của bài toán hồi quy tuyến tính thông thường (Ordinary Least Squares).

Giải thích Hiện tượng lựa chọn λ rất nhỏ có thể xuất phát từ nguyên nhân mô hình không cần regularization. Dữ liệu có số lượng đặc trưng nhỏ, mức nhiễu thấp hoặc không có hiện tượng đa cộng tuyến mạnh, dẫn đến việc mô hình tuyến tính đã tổng quát hóa tốt. Khi đó, việc thêm regularization chỉ làm tăng bias mà không cải thiện variance, nên giá trị λ tối ưu tiến về 0.

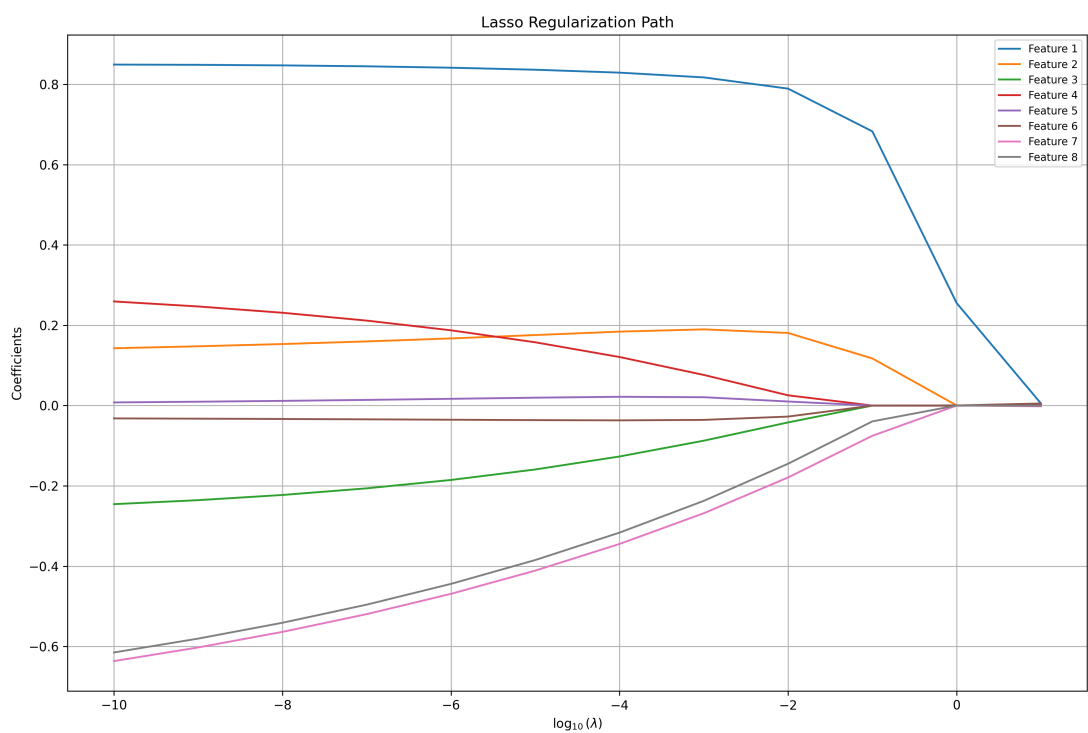
Regularization path Hình 7 và Hình 8 thể hiện sự thay đổi của các hệ số hồi quy của Ridge và Lasso khi hệ số điều chỉnh (regularization coefficient) thay đổi trên thang logarit. Khi λ lớn (bên phải đồ thị), các hệ số bị co lại đáng kể về gần 0 do tác động mạnh của regularization. Ngược lại, khi λ giảm dần (bên trái đồ thị), các hệ số tiến gần tới nghiệm của bài toán hồi quy tuyến tính không regularization (Ordinary Least Squares). Điều này thể hiện rõ qua việc các đường biểu diễn trở nên ổn định và đạt giá trị lớn hơn khi λ dần đến 0. Do nghiệm của Lasso Regression thừa nên khi λ tăng dần (bên phải Hình 8) nhiều hệ số hồi quy bị ép bằng 0.



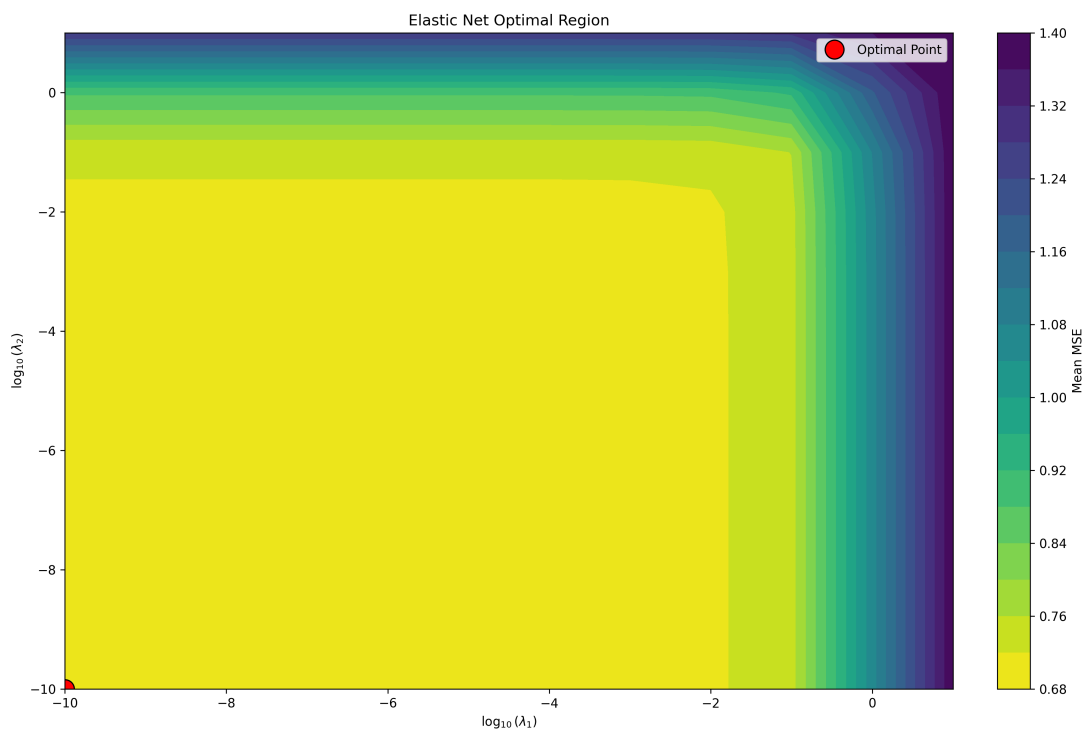
Hình 7: Regularization path của Ridge Regression

Vùng tối ưu Elastic Net Hình 9 thể hiện sự thay đổi của MSE trung bình theo sự thay đổi của λ_1 và λ_2 trên thang logarit. Có thể quan sát rằng vùng tối ưu nằm ở vùng mà λ_1 và λ_2 rất nhỏ (góc dưới trái của đồ thị). Điều này cho thấy mô hình đạt hiệu năng tốt khi hai hệ số điều chỉnh (regularization coefficient) tiến dần về 0. Điều này chỉ ra rằng trong bài toán hiện tại, việc áp dụng regularization không mang lại cải thiện rõ rệt về khả năng tổng quát hóa của mô hình.

Lựa chọn đặc trưng Kết quả thực nghiệm cho thấy khi chọn đặc trưng cả Forward Stepwise Selection và Backward Elimination bằng mô hình OLS đều cho ra cùng một tập đặc trưng (*MedInc*, *HouseAge*, *Latitude*, *Longitude*) và chọn đặc trưng dựa trên lasso cho kết quả là (*MedInc*, *HouseAge*, *AveRooms*, *AveBedrms*, *AveOccup*, *Latitude*, *Longitude*)



Hình 8: Regularization path của Lasso Regression



Hình 9: Vùng tối ưu của Elastic Net

Giải thích Sự khác biệt giữa kết quả lựa chọn đặc trưng của OLS và Lasso xuất phát từ cơ chế regularization của hai phương pháp.

Đối với OLS, mô hình không áp dụng regularization nên việc lựa chọn đặc trưng hoàn toàn dựa trên khả năng giải thích phương sai của từng biến. Trong trường hợp này, cả Forward Stepwise Selection và Backward Elimination đều hội tụ về cùng một tập đặc trưng (*MedInc*, *HouseAge*, *Latitude*, *Longitude*), cho thấy đây là các biến mang thông tin quan trọng và ít dư thừa nhất trong dữ liệu.

Ngược lại, Lasso sử dụng chuẩn ℓ_1 để regularize các hệ số, từ đó có khả năng đưa một số hệ số về 0 và thực hiện lựa chọn đặc trưng một cách tự động. Tuy nhiên, trong thực nghiệm này, tập đặc trưng được chọn bởi Lasso vẫn bao gồm nhiều biến, cụ thể là (*MedInc*, *HouseAge*, *AveRooms*, *AveBedrms*, *AveOccup*, *Latitude*, *Longitude*). Điều này cho thấy giá trị λ tối ưu được lựa chọn tương đối nhỏ, khiến tác động của regularization chưa đủ mạnh để loại bỏ hoàn toàn các đặc trưng có đóng góp nhỏ.

1.2.4 Mô hình với Hàm Cơ sở Phi Tuyến

Các mô hình hàm cơ sở phi tuyến Tất cả các mô hình đều tuân theo dạng tổng quát:

$$f(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T \Phi(\mathbf{x}) \quad (51)$$

trong đó $\Phi(\mathbf{x})$ được xây dựng từ các hệ cơ sở khác nhau.

Mô hình đa thức (Polynomial Basis): Mô hình đa thức mở rộng vector đầu vào thành các lũy thừa phi tuyến:

$$\Phi(\mathbf{x}) = [1, x, x^2, \dots, x^p] \quad (52)$$

Đây là dạng đơn giản nhất, giúp mô hình xấp xỉ các quan hệ phi tuyến mượt.

Ưu điểm:

- Dễ cài đặt
- Diễn giải tốt

Nhược điểm:

- Dễ overfitting khi bậc lớn
- Không ổn định với dữ liệu lớn

Mô hình RBF (Radial Basis Function): Mô hình RBF sử dụng các hàm Gaussian làm cơ sở:

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma^2}\right) \quad (53)$$

Trong đó $\mathbf{c}_j$ là các tâm và σ điều khiển độ lan tỏa.

Mô hình tập trung vào khoảng cách trong không gian đầu vào, giúp biểu diễn tốt cấu trúc phi tuyến cục bộ.

Ưu điểm:

- Xấp xỉ tốt dữ liệu phi tuyến
- Linh hoạt theo phân bố dữ liệu

Mô hình Fourier Basis: Mô hình Fourier biểu diễn dữ liệu bằng các hàm sin/cos:

$$\Phi(\mathbf{x}) = [\sin(\omega_1 x), \cos(\omega_1 x), \dots, \sin(\omega_k x), \cos(\omega_k x)] \quad (54)$$

Biến đổi dữ liệu sang miền tần số, phù hợp với dữ liệu có tính chu kỳ.

Ưu điểm:

- Trục giao tốt
- Ổn định số học cao

So sánh tổng quát: Ba nhóm mô hình có đặc điểm khác nhau:

- Polynomial: đơn giản nhưng dễ overfit
- RBF: mạnh với dữ liệu cục bộ
- Fourier: tốt cho dữ liệu tuần hoàn

Phân tích ảnh hưởng của hiệu ứng tương tác Nhằm khảo sát vai trò của các mối quan hệ phi tuyến giữa các biến đầu vào, chúng tôi mở rộng tập đặc trưng bằng cách bổ sung các thành phần tương tác dạng $x_i x_j$. Các đặc trưng này giúp mô hình có khả năng biểu diễn tốt hơn các quan hệ phụ thuộc phức tạp giữa các biến.

Thiết lập mô hình Xuất phát từ tập đặc trưng gốc $\{x_1, x_2, \dots, x_d\}$, chúng tôi xây dựng thêm các biến tương tác theo dạng:

$$x_i x_j, \quad \text{với } i \neq j \quad (55)$$

Sau đó, mô hình hồi quy tuyến tính được huấn luyện trên tập dữ liệu đã được mở rộng bởi các đặc trưng này.

Kết quả thực nghiệm Hiệu năng mô hình được đánh giá trên tập validation như sau:

- MSE (không sử dụng tương tác): 2.0967
- MSE (có sử dụng tương tác): 4.0124
- Chênh lệch: -1.9157

Phân tích kết quả Kết quả thực nghiệm cho thấy việc bổ sung các đặc trưng tương tác $x_i x_j$ không mang lại cải thiện về hiệu năng, mà còn làm gia tăng sai số dự đoán. Điều này có thể được giải thích bởi một số nguyên nhân:

- Việc mở rộng đặc trưng làm tăng đáng kể độ phức tạp của mô hình, dẫn đến nguy cơ overfitting cao hơn.
- Sự xuất hiện của các đặc trưng tương tác có thể gây ra hiện tượng đa cộng tuyến, làm giảm tính ổn định của mô hình.
- Các phương pháp biểu diễn phi tuyến như RBF hoặc Fourier đã đủ khả năng mô hình hóa quan hệ giữa các biến, khiến các thành phần tương tác trở nên dư thừa.

Kết luận Từ kết quả thu được, có thể thấy rằng việc bổ sung các đặc trưng tương tác $x_i x_j$ trong mô hình hồi quy tuyến tính không cải thiện hiệu năng, thậm chí còn làm giảm độ chính xác dự đoán. Do đó, trong bài toán này, việc sử dụng các đặc trưng tương tác không mang lại lợi ích đáng kể.

1.2.5 Validation Curve

Để đánh giá một cách hệ thống ảnh hưởng của độ phức tạp mô hình đến khả năng tổng quát hóa, chúng tôi xây dựng các validation curve cho ba nhóm mô hình phi tuyến: Polynomial Basis, Radial Basis Function (RBF) và Fourier Basis.

Hiệu năng mô hình được đo bằng sai số bình phương trung bình (Mean Squared Error - MSE) trên cả tập huấn luyện và tập validation, nhằm quan sát sự đánh đổi giữa bias và variance khi độ phức tạp tăng dần.

Về mặt lý thuyết, bài toán có thể được xem như:

$$\hat{f} = \arg \min_f \mathbb{E} [(y - f(x))^2] \quad (56)$$

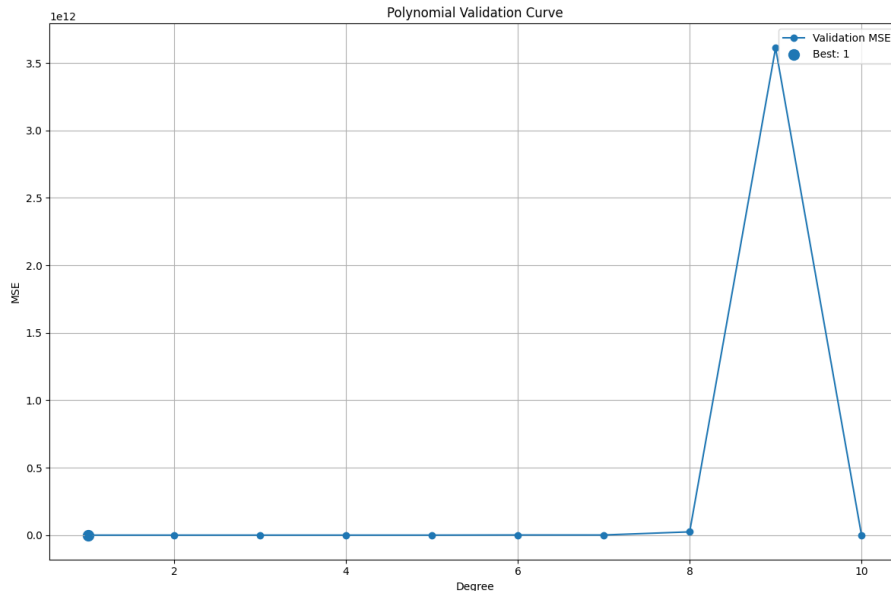
trong đó sai số có thể phân rã thành:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise} \quad (57)$$

1. Polynomial Basis (theo bậc đa thức p) Trong thí nghiệm này, chúng tôi mở rộng không gian đặc trưng theo dạng đa thức:

$$\phi(x) = [1, x, x^2, \dots, x^p]$$

và khảo sát ảnh hưởng của bậc p lên hiệu năng mô hình.



Hình 10: Validation curve: MSE theo bậc đa thức p

Phân tích chi tiết: Khi $p = 1$, mô hình tương đương hồi quy tuyến tính đơn giản, đạt MSE thấp khoảng 0.5349, cho thấy giả thiết tuyến tính ban đầu phù hợp tương đối với dữ liệu.

Tuy nhiên, khi tăng p , ma trận thiết kế $X^T X$ trở nên ngày càng kém điều kiện (ill-conditioned), do sự xuất hiện của các cột x^p có độ lớn tăng theo cấp số nhân:

$$|x^p| \gg |x^{p-1}|$$

Điều này dẫn đến hiện tượng khuếch đại sai số trong nghiệm:

$$\hat{w} = (X^T X)^{-1} X^T y$$

Kết quả thực nghiệm cho thấy:

- $p = 2$: MSE tăng nhẹ nhưng vẫn ổn định (2.0967)
- $p = 3$: MSE tăng đột biến (65.1358)
- $p \geq 4$: MSE bùng nổ theo cấp số mũ (lên tới 10^{12})

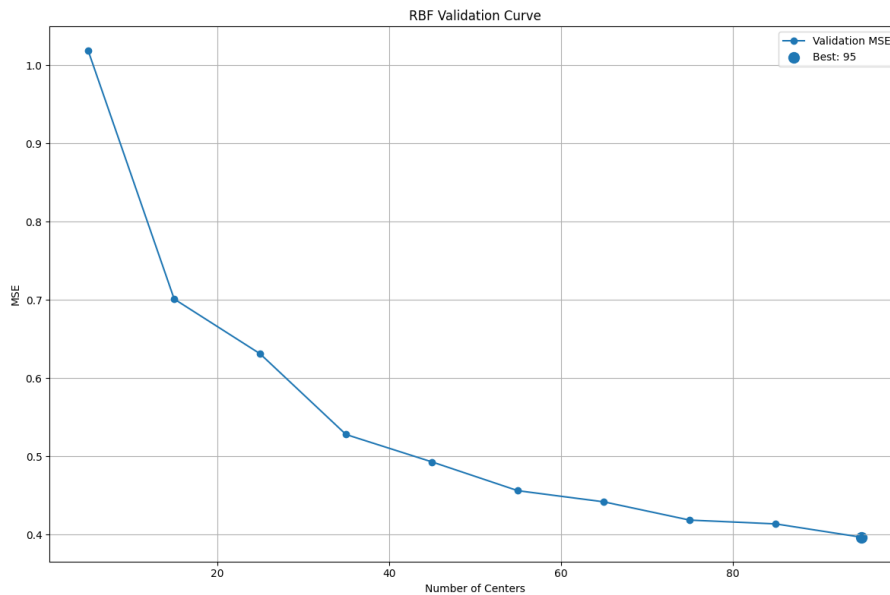
Hiện tượng này phản ánh rõ ràng sự mất ổn định số học (numerical instability) và overfitting nghiêm trọng do mô hình cố gắng nội suy chính xác từng điểm dữ liệu.

Ngoài ra, Polynomial basis còn chịu ảnh hưởng mạnh bởi hiện tượng Runge phenomenon, gây dao động lớn ở biên miền dữ liệu.

2. RBF Basis (theo số tâm M) Trong mô hình RBF, hàm cơ sở được định nghĩa:

$$\phi_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma^2}\right)$$

Trong đó c_i là các tâm (centers) và σ điều khiển độ lan tỏa.



Hình 11: Validation curve: MSE theo số tâm RBF

Phân tích chi tiết: Khi số lượng tâm nhỏ ($M = 5$), mô hình có bias cao do không gian biểu diễn còn hạn chế, dẫn đến MSE tương đối lớn (1.0185).

Khi tăng M , không gian hàm cơ sở trở nên giàu biểu diễn hơn, giúp giảm bias:

$$\text{Bias} \downarrow \quad \text{when } M \uparrow$$

Đồng thời, do tính chất local support của RBF, các hàm cơ sở có độ tương quan thấp:

$$\text{Cov}(\phi_i, \phi_j) \approx 0 \quad (i \neq j)$$

Điều này giúp kiểm soát variance hiệu quả.

Kết quả thực nghiệm:

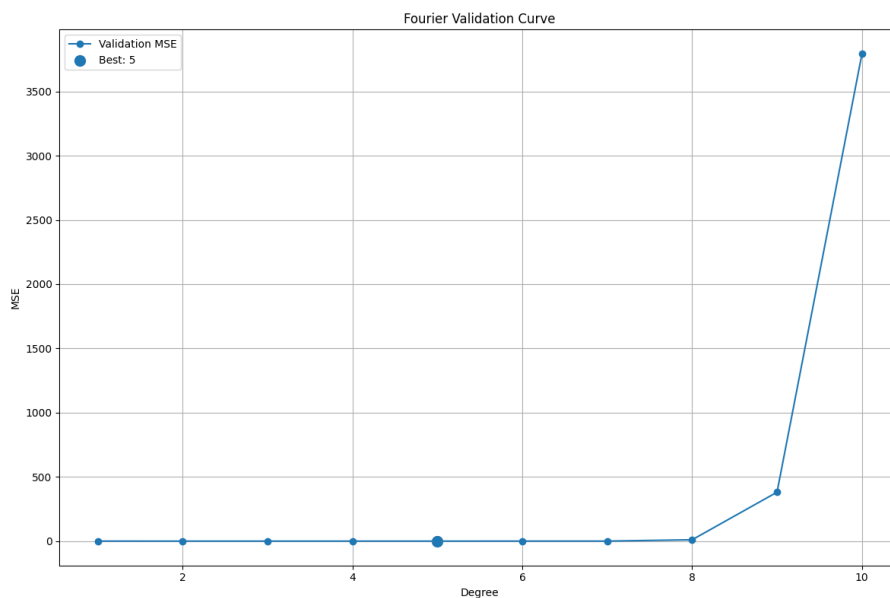
- MSE giảm đều từ 1.0185 $\rightarrow$ 0.3966
- Không xuất hiện hiện tượng bùng nổ sai số
- Mô hình đạt tối ưu quanh $M = 85\text{--}95$

Như vậy, RBF thể hiện tính ổn định cao nhờ cấu trúc kernel cục bộ, giúp cân bằng tốt giữa bias và variance.

3. Fourier Basis (theo số hàm cơ sở M) Fourier basis mở rộng hàm theo miền tần số:

$$\phi_k(x) = \sin(kx), \quad \cos(kx)$$

với k đóng vai trò tần số (frequency component).



Hình 12: Validation curve: MSE theo số hàm cơ sở Fourier

Phân tích chi tiết: Khi M nhỏ, mô hình chỉ biểu diễn được các thành phần tần số thấp, tương ứng với hàm trơn (low-frequency signal), dẫn đến underfitting:

$$\text{MSE} \approx 0.55$$

Khi M tăng lên khoảng 3–5, mô hình đạt trạng thái tối ưu:

$$\text{MSE}_{\min} \approx 0.42$$

Tại đây, mô hình cân bằng tốt giữa:

- thành phần tín hiệu (signal)
- và độ phức tạp biểu diễn

Tuy nhiên, khi $M > 6$, các thành phần tần số cao (high-frequency components) bắt đầu chi phối mô hình. Điều này dẫn đến việc mô hình học cả nhiễu (noise fitting), gây ra hiện tượng spectral overfitting.

Thực nghiệm cho thấy:

- $M = 8$: MSE tăng mạnh (10.8597)
- $M = 9$: MSE bùng nổ (381.1882)
- $M = 10$: MSE cực lớn (3791.8363)

Hiện tượng này tương tự Gibbs phenomenon trong phân tích Fourier, khi khai triển chuỗi Fourier với số hạng hữu hạn trên dữ liệu có nhiễu.

Nhận xét tổng quát Từ ba nhóm mô hình, có thể rút ra các kết luận quan trọng:

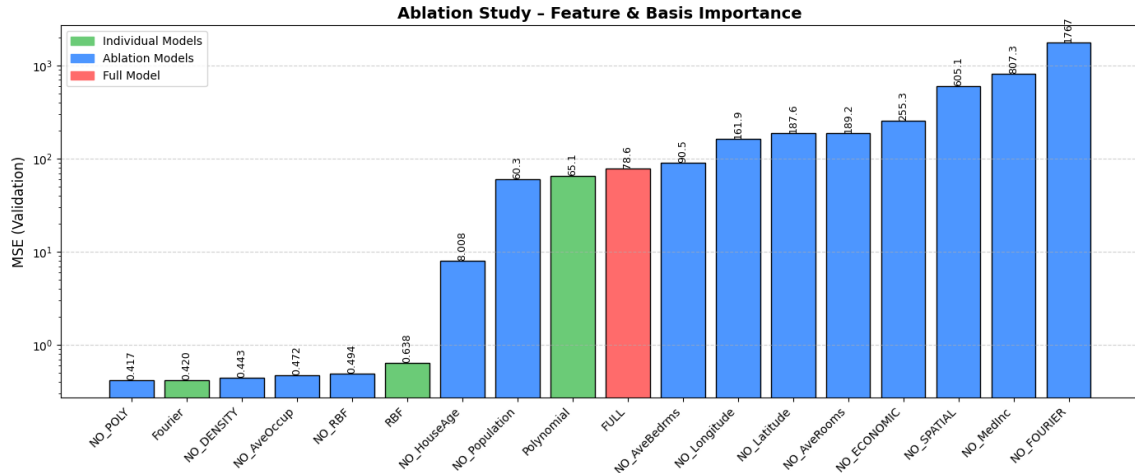
- **Polynomial Basis:** có độ nhạy cao với bậc p , dễ gây mất ổn định số học và overfitting nghiêm trọng do tính toàn cục của các hàm cơ sở.
- **RBF Basis:** thể hiện tính ổn định cao nhờ tính cục bộ hóa (localization), giúp kiểm soát tốt variance và giảm thiểu hiện tượng overfitting.
- **Fourier Basis:** hiệu quả tốt trong miền tần số thấp nhưng dễ bị overfitting khi mở rộng sang miền tần số cao do học cả nhiễu.

Tổng thể, các kết quả thực nghiệm khẳng định rằng việc tăng độ phức tạp mô hình không đảm bảo cải thiện hiệu năng, mà cần tuân theo nguyên lý *bias-variance tradeoff* và đặc tính cấu trúc của không gian hàm cơ sở (function space geometry).

1.2.6 Ablation Study

Để đánh giá một cách toàn diện mức độ đóng góp của từng nhóm đặc trưng và từng họ hàm cơ sở, chúng tôi thực hiện ablation study bằng cách lần lượt loại bỏ từng thành phần trong mô hình và quan sát sự thay đổi của sai số bình phương trung bình (MSE) trên tập validation.

Mục tiêu chính của thí nghiệm này gồm: (i) xác định các đặc trưng quan trọng nhất, (ii) đánh giá vai trò của từng loại basis function, (iii) phân tích mức độ nhạy của mô hình dưới các perturbation khác nhau.



Hình 13: Ablation Study: Feature & Basis Importance (MSE trên validation, thang log)

1. Biểu diễn trực quan kết quả Ablation Hình 13 thể hiện kết quả ablation study dưới dạng biểu đồ cột, trong đó:

- Màu xanh lá: các mô hình đơn lẻ (individual models)
- Màu xanh dương: các mô hình loại bỏ từng thành phần (ablation models)
- Màu đỏ: mô hình đầy đủ (full model)

Do độ chênh lệch giữa các giá trị MSE rất lớn (từ 10^{-1} đến 10^4), trục tung được biểu diễn theo thang logarit ($\log_{10}$) nhằm đảm bảo khả năng so sánh trực quan giữa các cấu hình.

2. Bảng tổng hợp kết quả Ablation Study Bảng 5 tổng hợp toàn bộ giá trị MSE tương ứng với từng cấu hình mô hình.

2. Phân tích tổng quan hành vi mô hình Quan sát từ biểu đồ, có thể rút ra một số đặc điểm quan trọng:

- Nhóm mô hình Fourier và NO_POLY đạt MSE thấp nhất ($\approx 0.41-0.42$), cho thấy đây là vùng tối ưu về mặt biểu diễn.
- Polynomial basis thể hiện hành vi cực kỳ không ổn định với MSE tăng mạnh, phản ánh sự bùng nổ sai số do tính chất ill-conditioned của không gian đa thức.
- Các cấu hình loại bỏ Fourier (NO_Fourier) dẫn đến MSE tăng đột biến ($> 10^3$), cho thấy Fourier basis đóng vai trò cốt lõi trong việc tái tạo cấu trúc dữ liệu.

3. Phân tích theo nhóm đặc trưng (Feature Importance) Từ các cấu hình loại bỏ từng feature, ta có thể xếp hạng mức độ quan trọng của các đặc trưng như sau:

MedInc $\gg$ Spatial Features (Latitude, Longitude) $\gg$
HouseAge $>$ Population $>$ Rooms Features $>$ AveOccup

Cụ thể:

Model / Configuration	Validation MSE
RBF	0.6379
Fourier	0.4199
Polynomial	65.1358
NO_RBF	0.4939
NO_Fourier	1767.3249
NO_Polynomial	0.4175
NO_Economic	255.3254
NO_Density	0.4427
NO_Spatial	605.1461
NO_MedInc	807.2610
NO_HouseAge	8.0081
NO_AveRooms	189.1508
NO_AveBedrms	90.5460
NO_Population	60.2631
NO_AveOccup	0.4718
NO_Latitude	187.5998
NO_Longitude	161.8832
FULL	78.5531

Bảng 5: Tổng hợp kết quả Ablation Study trên tập validation (MSE)

- **MedInc (Median Income)**: khi loại bỏ làm MSE tăng lên 807.26, chứng tỏ đây là biến có sức dự báo mạnh nhất, phản ánh mối quan hệ kinh tế trực tiếp đến giá nhà.
- **Spatial features (Latitude, Longitude)**: khi loại bỏ gây tăng MSE lớn ($\approx 160-187$), cho thấy giá nhà phụ thuộc mạnh vào vị trí địa lý (spatial dependency).
- **Population, AveRooms, AveBedrms**: ảnh hưởng trung bình, phản ánh cấu trúc khu dân cư.
- **AveOccup**: gần như không ảnh hưởng (0.4718), cho thấy biến này ít thông tin dự báo.

4. Phân tích theo basis function (Model Sensitivity) Xét theo nhóm hàm cơ sở:

- **Fourier Basis**: ổn định nhất, đạt MSE thấp nhất (0.4199), chứng tỏ dữ liệu có cấu trúc mang tính chu kỳ/tần số thấp.
- **RBF Basis**: hoạt động tốt (0.6379), nhưng phụ thuộc vào phân bố center, do tính chất local approximation.
- **Polynomial Basis**: mất ổn định nghiêm trọng (65.13), phản ánh hiện tượng numerical explosion khi tăng bậc đa thức.

Đặc biệt, cấu hình NO_Fourier làm MSE tăng lên hơn 10^3 , chứng minh Fourier basis là thành phần không thể thiếu trong việc mô hình hóa cấu trúc dữ liệu.

5. Hiện tượng đáng chú ý: FULL model kém hiệu năng Một kết quả đáng chú ý là mô hình FULL có MSE cao bất thường (78.55), trong khi nhiều cấu hình rút gọn lại đạt MSE thấp hơn rất nhiều.

Điều này có thể được giải thích bởi:

- **Over-parameterization**: số lượng feature quá lớn gây nhiễu gradient
- **Feature interaction không kiểm soát**: các basis kết hợp gây collinearity
- **Scaling mismatch**: một số nhóm feature có scale khác nhau làm lệch tối ưu

Do đó, mô hình đầy đủ không đồng nghĩa với mô hình tốt nhất, mà cần có cơ chế chọn lọc đặc trưng và regularization phù hợp.

6. Nhận xét tổng quát Từ toàn bộ kết quả ablation study, có thể kết luận:

- Fourier basis là thành phần quan trọng nhất trong việc giảm sai số tổng thể.
- RBF basis giúp cải thiện độ ổn định nhưng không mạnh bằng Fourier.
- Polynomial basis gây mất ổn định số học khi tăng độ phức tạp.
- MedInc và Spatial features là hai nhóm đặc trưng quyết định chính.
- Mô hình tốt không phải là mô hình đầy đủ, mà là mô hình cân bằng giữa biểu diễn và khả năng tổng quát hóa.

Tổng thể, kết quả nhấn mạnh vai trò của *feature importance under structural perturbation*, trong đó việc loại bỏ hoặc thay đổi từng thành phần giúp làm rõ cấu trúc thông tin tiềm ẩn trong dữ liệu.

Kết luận Tổng hợp từ cả validation curve và ablation study, có thể thấy hiệu năng của mô hình không chỉ phụ thuộc vào độ phức tạp của từng thành phần riêng lẻ, mà còn phụ thuộc mạnh vào sự tương tác giữa các nhóm đặc trưng và không gian hàm cơ sở.

Xét từ góc nhìn bias-variance tradeoff, các mô hình thể hiện hành vi nhất quán:

- Khi độ phức tạp tăng (Polynomial degree hoặc số basis tăng), bias giảm nhưng variance tăng mạnh, dẫn đến hiện tượng overfitting rõ rệt.
- Các mô hình có cấu trúc cục bộ (RBF) hoặc theo miền tần số (Fourier) cho khả năng kiểm soát variance tốt hơn so với Polynomial basis toàn cục.
- Hiệu năng tối ưu đạt được khi mô hình nằm ở vùng cân bằng giữa biểu diễn (representation power) và độ ổn định số học (numerical stability).

Đặc biệt, từ ablation study, có thể thấy rằng:

- Các đặc trưng kinh tế (MedInc) và vị trí địa lý (Latitude, Longitude) đóng vai trò quyết định trong bài toán dự đoán.
- Một số đặc trưng có ảnh hưởng rất nhỏ (AveOccup, Density), cho thấy tồn tại redundancy trong không gian dữ liệu.
- Việc loại bỏ Fourier basis gây suy giảm nghiêm trọng hiệu năng, chứng tỏ dữ liệu có thành phần cấu trúc theo miền tần số thấp.

Một điểm quan trọng là mô hình full không phải lúc nào cũng tối ưu. Hiện tượng FULL model có MSE cao hơn nhiều cấu hình rút gọn cho thấy:

- Sự xuất hiện của noise amplification khi số lượng feature quá lớn
- Collinearity giữa các basis function khác nhau
- Thiếu regularization phù hợp trong không gian đặc trưng mở rộng

Kết luận Trong nghiên cứu này, chúng tôi đã xây dựng và đánh giá các mô hình hồi quy phi tuyến dựa trên ba họ hàm cơ sở: Polynomial, RBF và Fourier, đồng thời thực hiện ablation study để phân tích mức độ quan trọng của từng thành phần.

Kết quả thực nghiệm cho thấy:

- Fourier basis cho hiệu năng tốt nhất và ổn định nhất trong hầu hết các cấu hình, đặc biệt trong việc mô hình hóa các thành phần dao động của dữ liệu.
- RBF basis cung cấp sự cân bằng tốt giữa độ linh hoạt và tính ổn định, phù hợp với các bài toán có cấu trúc phi tuyến cục bộ.
- Polynomial basis dễ dẫn đến mất ổn định số học và overfitting khi tăng bậc.
- Các đặc trưng kinh tế và vị trí địa lý là yếu tố quan trọng nhất trong bài toán dự đoán giá nhà.
- Mô hình tốt không nhất thiết là mô hình phức tạp nhất, mà là mô hình có cấu trúc phù hợp với phân bố dữ liệu.

Tổng thể, nghiên cứu khẳng định vai trò quan trọng của việc lựa chọn không gian biểu diễn (feature space design) và kiểm soát độ phức tạp mô hình, đồng thời nhấn mạnh nguyên lý cốt lõi trong machine learning: *generalization is achieved through controlled complexity, not maximal complexity.*

1.3 Đánh giá và phân tích các mô hình

1.3.1 Đánh giá trên tập kiểm tra

Bảng 6 cho kết quả đánh giá của các mô hình trên tập kiểm tra. Các chỉ số được sử dụng gồm MSE, MAE, RMSE và R^2 .

Bảng 6: Kết quả các chỉ số của các mô hình trên tập kiểm tra

Model	MSE	RMSE	MAE	R^2
OLS	0.5253	0.7248	0.5299	0.6097
MBGD	0.5254	0.7249	0.5299	0.6097
WLS	0.5276	0.7264	0.5205	0.608
Ridge	0.7297	0.8542	0.5953	0.4579
Lasso	0.7477	0.8647	0.6051	0.4445
Elastic Net	0.766	0.8752	0.6153	0.4309
Polynomial	0.494	0.7029	0.5145	0.633
RBF	0.6971	0.8349	0.6278	0.4821
Fourier	0.3873	0.6223	0.4466	0.7123

Nhìn chung, các mô hình hồi quy tuyến tính cơ bản (OLS, MBGD và WLS) cho kết quả rất tương đồng. Cụ thể, OLS và MBGD gần như trùng khớp trên tất cả các chỉ số (MSE xấp xỉ 0.5253–0.5254, R^2 xấp xỉ 0.6097), cho thấy MBGD đã hội tụ tốt và tái hiện lại nghiệm của OLS dù sử dụng phương pháp tối ưu lặp. Kết quả này cũng cho thấy chiến lược lập lịch tốc độ học trong MBGD hoạt động hiệu quả, giúp quá trình tối ưu vừa ổn định vừa đạt được nghiệm gần tối ưu toàn cục.

WLS có MAE thấp hơn một chút (0.5205 so với 0.5299), tuy nhiên các chỉ số còn lại không cải thiện đáng kể, thậm chí MSE và RMSE còn cao hơn nhẹ. Điều này cho thấy việc gán trọng số chưa mang lại lợi ích rõ ràng, có thể do giả định về phương sai của sai số không phù hợp với dữ liệu.

Đối với các mô hình chính quy hóa (Ridge, Lasso và Elastic Net), kết quả kém hơn đáng kể so với nhóm mô hình tuyến tính cơ bản. Giá trị R^2 giảm xuống khoảng 0.43–0.46, đồng thời các sai số (MSE, RMSE, MAE) đều tăng lên rõ rệt. Điều này cho thấy mô hình tuyến tính ban đầu không bị quá khớp (overfitting) nghiêm trọng; ngược lại, chính quy hóa đã làm giảm độ linh hoạt của mô hình và dẫn đến hiện tượng chưa khớp (underfitting).

Mô hình Polynomial cho thấy sự cải thiện so với OLS, với MSE giảm từ 0.5253 xuống 0.494 và R^2 tăng từ 0.6097 lên 0.633. Kết quả này cho thấy dữ liệu tồn tại yếu tố phi tuyến và việc mở rộng không gian đặc trưng giúp mô hình biểu diễn tốt hơn mối quan hệ giữa các biến.

Ngược lại, mô hình RBF không đạt được kết quả như kỳ vọng khi có MSE cao hơn (0.6971) và R^2 thấp hơn (0.4821) so với OLS. Nguyên nhân có thể đến từ việc lựa chọn siêu tham số (như số lượng tâm hoặc độ rộng kernel) chưa phù hợp, hoặc bản chất dữ liệu không phù hợp với dạng biểu diễn của RBF.

Cuối cùng, mô hình Fourier thể hiện hiệu năng vượt trội so với tất cả các mô hình còn lại. Cụ thể, mô hình này đạt MSE thấp nhất (0.3873), RMSE thấp nhất (0.6223), MAE thấp nhất (0.4466) và R^2 cao nhất (0.7123). Điều này cho thấy dữ liệu có thể chứa

các thành phần mang tính chu kỳ hoặc lặp lại, và Fourier là phương pháp phù hợp để khai thác đặc điểm này, thậm chí hiệu quả hơn cả Polynomial trong trường hợp này.

1.3.2 Đánh giá qua cross-validation

Bảng 7 và 8 cho kết quả đánh giá trung bình và độ lệch chuẩn của từng chỉ số MSE, MAE, RMSE và R2 thông qua cross-validation.

Bảng 7: Kết quả các chỉ số của các mô hình hồi quy qua cross-validation (phần 1)

Model	MSE	RMSE
OLS	0.5345 ± 0.0384	0.7306 ± 0.0256
MBGD	0.5344 ± 0.0382	0.7306 ± 0.0255
WLS	0.6220 ± 0.1587	0.7829 ± 0.0953
Ridge	0.7263 ± 0.0328	0.8520 ± 0.0191
Lasso	0.7431 ± 0.0310	0.8618 ± 0.0179
Elastic Net	0.7608 ± 0.0313	0.8721 ± 0.0178
Polynomial	0.6877 ± 0.5260	0.7943 ± 0.2383
RBF	0.7295 ± 0.0219	0.8540 ± 0.0128
Fourier	1.0607 ± 2.0013	0.8303 ± 0.6093

Bảng 8: Kết quả các chỉ số của các mô hình hồi quy qua cross-validation (phần 2)

Model	MAE	R2
OLS	0.5315 ± 0.0068	0.5961 ± 0.0237
MBGD	0.5316 ± 0.0068	0.5961 ± 0.0235
WLS	0.5263 ± 0.0088	0.5287 ± 0.1230
Ridge	0.5972 ± 0.0149	0.4504 ± 0.0295
Lasso	0.6056 ± 0.0147	0.4378 ± 0.0271
Elastic Net	0.6158 ± 0.0151	0.4244 ± 0.0265
Polynomial	0.5201 ± 0.0120	0.4853 ± 0.3738
RBF	0.6524 ± 0.0133	0.4479 ± 0.0249
Fourier	0.4597 ± 0.0270	0.2262 ± 1.4222

Bảng kết quả cho thấy sự khác biệt rõ rệt giữa các nhóm mô hình tuyến tính, chính quy hóa, và các phương pháp mở rộng đặc trưng phi tuyến.

Nhìn chung, các mô hình tuyến tính cơ bản như OLS và MBGD đạt hiệu năng tốt và ổn định nhất. Cụ thể, cả hai mô hình có MSE trung bình khoảng 0.534 và R^2 xấp xỉ 0.596 với độ lệch chuẩn nhỏ, cho thấy khả năng tổng quát hóa tốt và ít phụ thuộc vào từng tập con dữ liệu. MBGD tiếp tục cho kết quả gần như trùng khớp với OLS, khẳng định quá trình tối ưu bằng mini-batch gradient descent đã hội tụ hiệu quả và chiến lược lập lịch tốc độ học giúp đảm bảo tính ổn định trong huấn luyện.

WLS cho kết quả kém ổn định hơn đáng kể. Mặc dù MAE thấp hơn nhẹ (0.5263 so với 0.5315), nhưng MSE và RMSE cao hơn và đặc biệt độ lệch chuẩn lớn (ví dụ

$\sigma_{MSE} = 0.1587$, cao hơn nhiều so với OLS). Điều này cho thấy mô hình nhạy cảm với từng fold dữ liệu và giả định về phương sai sai số có thể không phù hợp.

Các mô hình chính quy hóa (Ridge, Lasso và Elastic Net) có hiệu năng kém hơn so với OLS cả về giá trị trung bình lẫn khả năng giải thích (R^2 chỉ khoảng 0.42–0.45). Tuy nhiên, độ lệch chuẩn của các mô hình này tương đối nhỏ, cho thấy chúng ổn định nhưng bị hạn chế về độ linh hoạt, dẫn đến chưa khớp.

Mô hình Polynomial không mang lại cải thiện rõ rệt về hiệu năng trung bình, trong khi độ lệch chuẩn rất lớn (đặc biệt $\sigma_{MSE} = 0.5260$, $\sigma_{R^2} = 0.3738$). Điều này cho thấy mô hình thiếu ổn định và có khả năng bị quá khớp trên một số fold.

Mô hình RBF có hiệu năng trung bình thấp hơn OLS (MSE xấp xỉ 0.7295, R^2 xấp xỉ 0.4479) nhưng lại có độ lệch chuẩn nhỏ, cho thấy mô hình ổn định nhưng bị giới hạn về khả năng biểu diễn.

Đáng chú ý, mô hình Fourier cho kết quả rất không ổn định với MSE trung bình cao (1.0607) và độ lệch chuẩn cực lớn ($\sigma_{MSE} \approx 2.0013$, $\sigma_{R^2} \approx 1.4222$). Mặc dù đạt kết quả rất tốt trên tập kiểm tra, hiệu năng không nhất quán qua các fold cho thấy mô hình có dấu hiệu quá khớp mạnh và phụ thuộc nhiều vào từng phân chia dữ liệu.

Tổng thể, OLS và MBGD là hai mô hình cân bằng tốt nhất giữa hiệu năng và độ ổn định. Các phương pháp mở rộng phi tuyến như Polynomial và Fourier có thể cải thiện kết quả trong một số trường hợp, nhưng đi kèm với rủi ro giảm khả năng tổng quát hóa nếu không được kiểm soát phù hợp.

1.3.3 Đánh giá kiểm định các cặp mô hình

Bảng 9 và 10 trình bày kết quả kiểm định các cặp mô hình dựa trên chỉ số MSE và MAE thông qua cross-validation, sử dụng t-test và Wilcoxon signed-rank test.

Bảng 9: Kiểm định các cặp mô hình dựa trên các kết quả MSE thông qua cross-validation

Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
OLS	MBGD	0.8751	0.9219
OLS	WLS	0.0781	0.0645
OLS	Ridge	0.0	0.002
OLS	Lasso	0.0	0.002
OLS	Elastic Net	0.0	0.002
OLS	Polynomial	0.3746	0.625
OLS	RBF	0.0	0.002
OLS	Fourier	0.4487	0.084
MBGD	WLS	0.0786	0.084
MBGD	Ridge	0.0	0.002
MBGD	Lasso	0.0	0.002
MBGD	Elastic Net	0.0	0.002
MBGD	Polynomial	0.3748	0.625
MBGD	RBF	0.0	0.002

Còn tiếp trang sau

Bảng 9: Kiểm định các cặp mô hình dựa trên các kết quả MSE thông qua cross-validation (Tiếp theo)

Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
MBGD	Fourier	0.4487	0.084
WLS	Ridge	0.0931	0.1602
WLS	Lasso	0.0589	0.1309
WLS	Elastic Net	0.0347	0.0488
WLS	Polynomial	0.654	0.1309
WLS	RBF	0.095	0.1055
WLS	Fourier	0.5336	0.084
Ridge	Lasso	0.0	0.002
Ridge	Elastic Net	0.0	0.002
Ridge	Polynomial	0.8355	0.084
Ridge	RBF	0.746	0.5566
Ridge	Fourier	0.6248	0.084
Lasso	Elastic Net	0.0	0.002
Lasso	Polynomial	0.7656	0.084
Lasso	RBF	0.1587	0.3223
Lasso	Fourier	0.6421	0.084
Elastic Net	Polynomial	0.6936	0.084
Elastic Net	RBF	0.0068	0.0039
Elastic Net	Fourier	0.6605	0.084
Polynomial	RBF	0.82	0.084
Polynomial	Fourier	0.6112	0.084
RBF	Fourier	0.63	0.084

Bảng 10: Kiểm định các cặp mô hình dựa trên các kết quả MAE thông qua cross-validation

Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
OLS	MBGD	0.235	0.2324
OLS	WLS	0.0251	0.0371
OLS	Ridge	0.0	0.002
OLS	Lasso	0.0	0.002
OLS	Elastic Net	0.0	0.002
OLS	Polynomial	0.0131	0.0371

Còn tiếp trang sau

Bảng 10: Kiểm định các cặp mô hình dựa trên các kết quả MAE thông qua cross-validation (Tiếp theo)

Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
OLS	RBF	0.0	0.002
OLS	Fourier	0.0	0.002
MBGD	WLS	0.0253	0.0371
MBGD	Ridge	0.0	0.002
MBGD	Lasso	0.0	0.002
MBGD	Elastic Net	0.0	0.002
MBGD	Polynomial	0.0133	0.0371
MBGD	RBF	0.0	0.002
MBGD	Fourier	0.0	0.002
WLS	Ridge	0.0	0.002
WLS	Lasso	0.0	0.002
WLS	Elastic Net	0.0	0.002
WLS	Polynomial	0.0324	0.0645
WLS	RBF	0.0	0.002
WLS	Fourier	0.0	0.0039
Ridge	Lasso	0.0	0.002
Ridge	Elastic Net	0.0	0.002
Ridge	Polynomial	0.0	0.002
Ridge	RBF	0.0	0.002
Ridge	Fourier	0.0	0.002
Lasso	Elastic Net	0.0	0.002
Lasso	Polynomial	0.0	0.002
Lasso	RBF	0.0	0.002
Lasso	Fourier	0.0	0.002
Elastic Net	Polynomial	0.0	0.002
Elastic Net	RBF	0.0001	0.002
Elastic Net	Fourier	0.0	0.002
Polynomial	RBF	0.0	0.002
Polynomial	Fourier	0.0001	0.0039
RBF	Fourier	0.0	0.002

Đối với chỉ số MSE, không phải tất cả các cặp mô hình đều có sự khác biệt có ý nghĩa thống kê. Cụ thể, giữa OLS và MBGD không tồn tại sự khác biệt có ý nghĩa (p-value lớn hơn 0.05 ở cả hai kiểm định), phù hợp với kết quả trước đó khi hai mô hình này cho hiệu năng gần như trùng khớp. Tương tự, các cặp như OLS–Polynomial hoặc OLS–Fourier

cũng không cho thấy sự khác biệt có ý nghĩa thống kê, mặc dù giá trị trung bình có thể khác nhau. Điều này cho thấy độ biến thiên lớn trong các mô hình phi tuyến đã làm giảm khả năng kết luận sự khác biệt rõ ràng.

Ngược lại, OLS và MBGD đều có sự khác biệt có ý nghĩa thống kê so với các mô hình chính quy hóa (Ridge, Lasso, Elastic Net) và RBF (p-value rất nhỏ), phản ánh sự suy giảm hiệu năng của các mô hình này so với nhóm tuyến tính cơ bản.

Đối với chỉ số MAE, nhiều cặp mô hình thể hiện sự khác biệt có ý nghĩa thống kê, bao gồm cả OLS so với WLS, Polynomial và Fourier (p-value < 0.05). Tuy nhiên, giữa OLS và MBGD vẫn không có sự khác biệt có ý nghĩa, cho thấy hai phương pháp này tương đương không chỉ về mặt giá trị trung bình mà còn về phân phối sai số.

Kết quả thực nghiệm cho thấy các mô hình WLS, Polynomial và Fourier đạt MAE thấp hơn so với OLS trong một số thiết lập cross-validation. Điều này cho thấy việc mở rộng không gian biểu diễn (Polynomial, Fourier) hoặc điều chỉnh trọng số quan sát (WLS) có thể giúp mô hình khai thác tốt hơn cấu trúc dữ liệu, từ đó cải thiện độ chính xác dự đoán.

Tuy nhiên, sự cải thiện này không hoàn toàn đồng nhất trên tất cả các phép kiểm định. Một số cặp so sánh vẫn cho thấy khác biệt có ý nghĩa thống kê, trong khi một số khác có mức chênh lệch nhỏ, cho thấy hiệu quả cải thiện phụ thuộc vào phân phối dữ liệu và cấu trúc nhiễu.

Nhìn chung, khi so sánh với các mô hình còn lại, trên các chỉ số còn lại, OLS thể hiện kết quả ổn định và có tính nhất quán cao trên các phép đo đánh giá. Mặc dù một số mô hình mở rộng như WLS, Polynomial hay Fourier có thể đạt MAE tốt hơn trong một số trường hợp cụ thể, sự cải thiện này không mang tính đồng đều và thường đi kèm với độ biến thiên cao hơn giữa các fold.

Tổng thể, xét trên cả độ chính xác trung bình, tính ổn định và mức độ đơn giản của mô hình, OLS và MBGD vẫn cho thấy hiệu quả tốt nhất trong bài toán này, đồng thời là lựa chọn cân bằng giữa hiệu năng và tính tổng quát hóa.

1.3.4 Các biểu đồ so sánh

Biểu đồ đường cong học Hình 14 thể hiện các đường cong học của từng mô hình.

Ta thấy rằng, hầu hết các mô hình cần khoảng 7000 mẫu huấn luyện để đạt MSE cực tiểu trên tập huấn luyện và tập xác thực. Riêng mô hình Fourier thì cần khoảng 400 mẫu đã có thể hội tụ.

Biểu đồ phần dư Hình 15 thể hiện biểu đồ phần dư của các mô hình trên tập kiểm tra

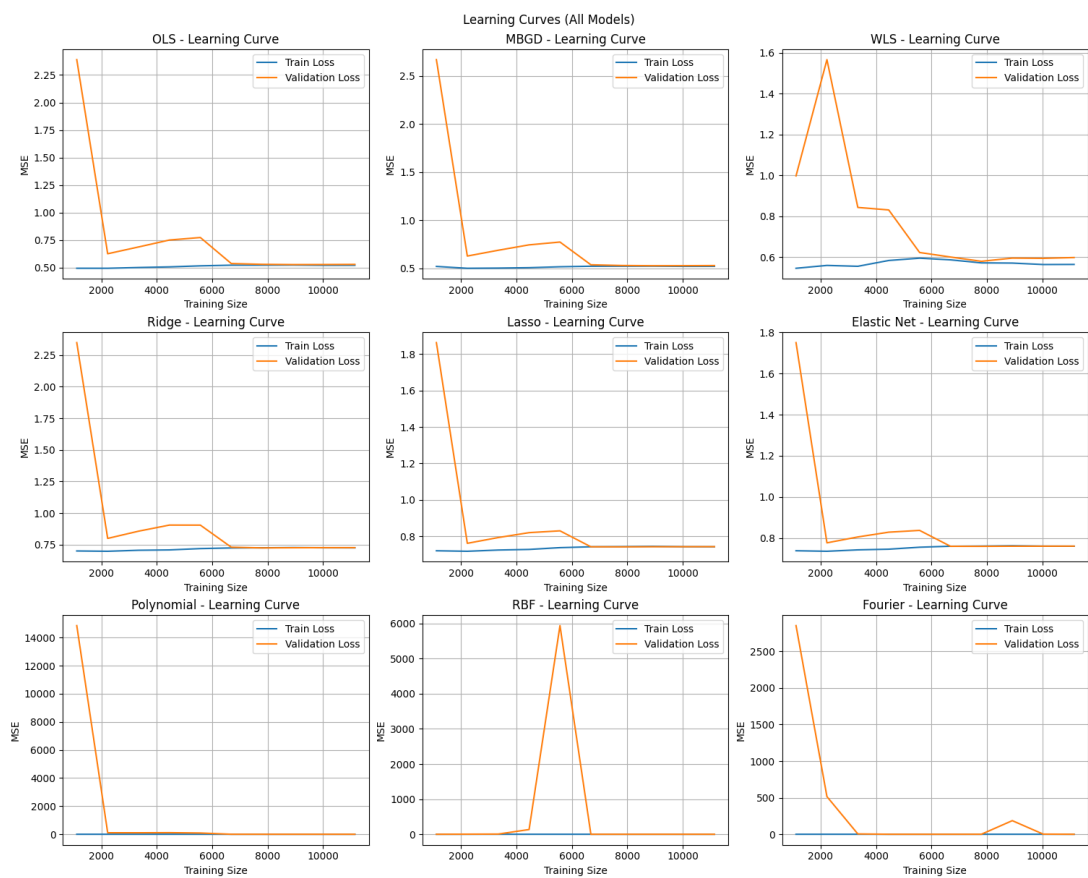
Ta thấy rằng các mô hình hồi quy tuyến tính cơ bản (OLS, MBGD, WLS) có độ khớp vừa phải, không quá khớp của sự ngắt ngưỡng.

Ngược lại, các mô hình cơ sở phi tuyến (Polynomial, RBF và Fourier) có việc học sự ngắt ngưỡng khi mô hình rất ít khi dự đoán trên 5.

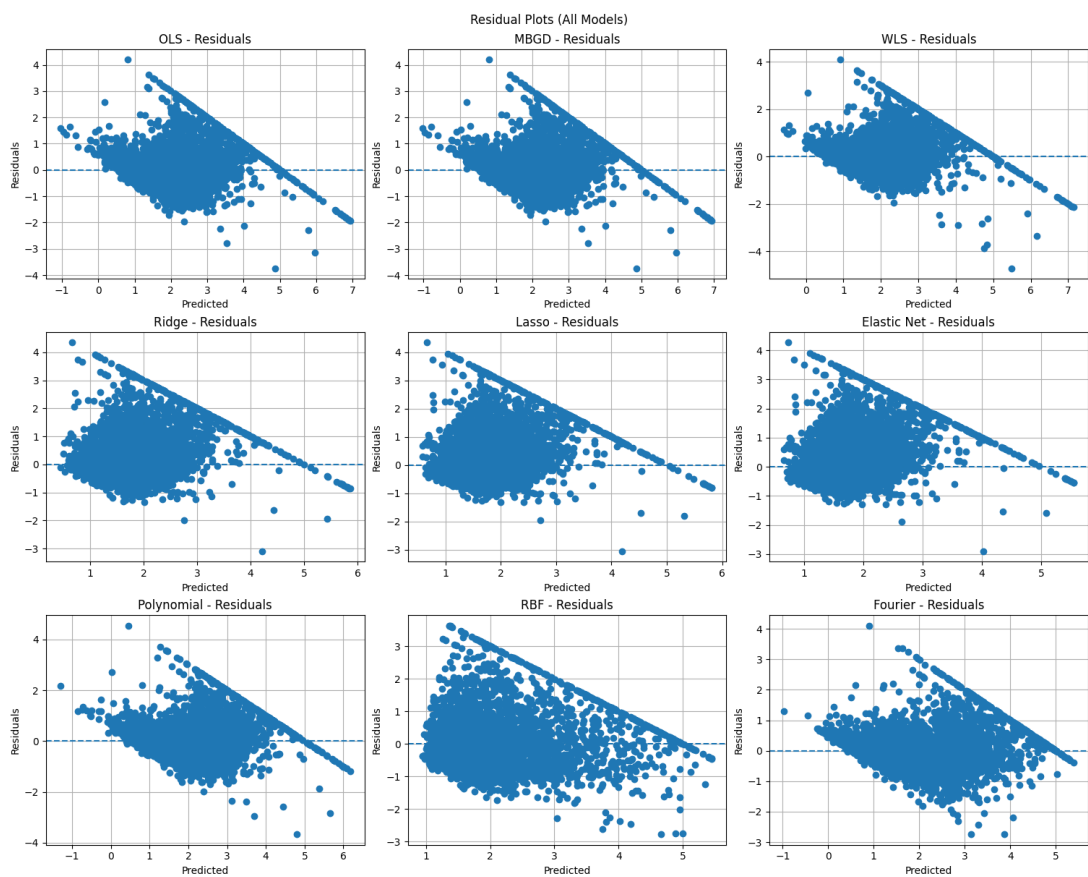
Trong khi đó, các mô hình chính quy hoá (Lasso, Ridge, Elastic Net) có đường tham chiếu $e = 0$ thấp hơn trung bình so với phân tán của phần dư.

Biểu đồ giá trị dự đoán-giá trị thực Hình 16 thể hiện tương quan giữa giá trị dự đoán và giá trị thực của các mô hình trên tập kiểm tra.

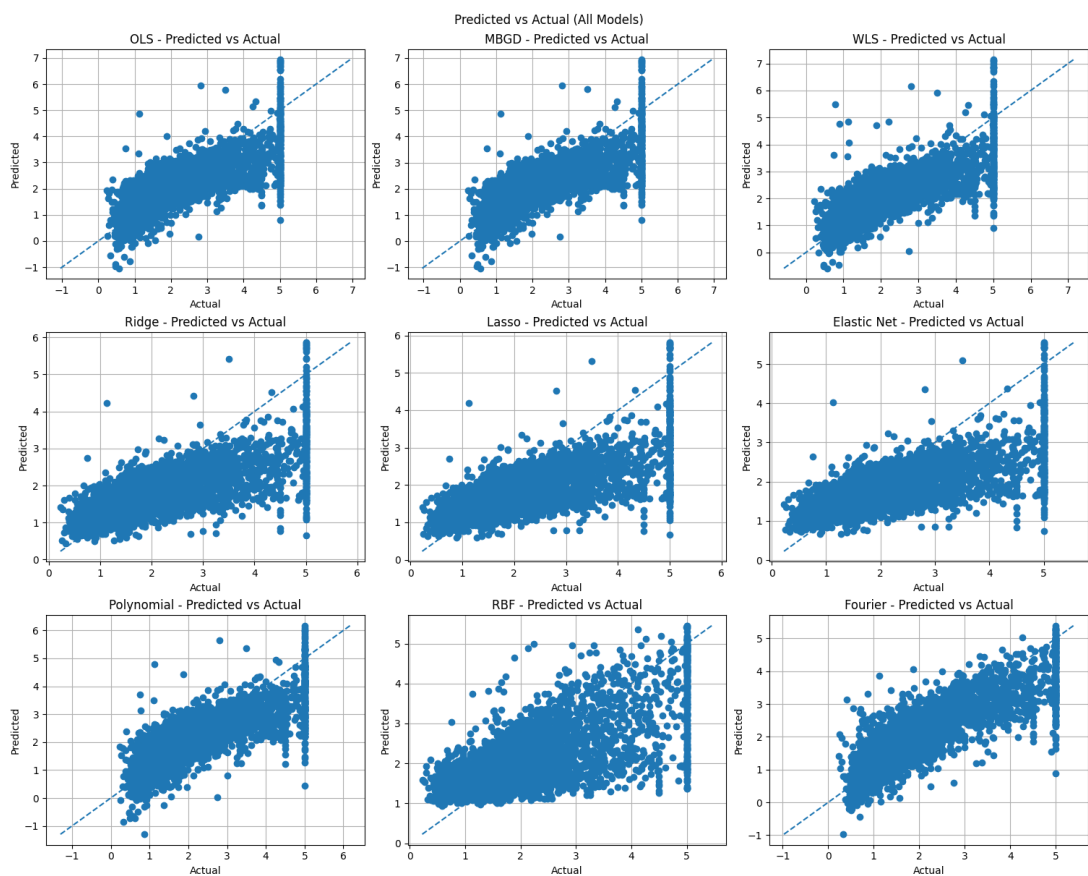
Ta thấy rằng các mô hình chính quy hoá cho thấy các điểm phân tán lệch cao hơn so với đường tham chiếu. Điều này cho thấy các mô hình này học chưa khớp.



Hình 14: Biểu đồ đường cong học của các mô hình dựa trên MSE từng kích thước của tập huấn luyện



Hình 15: Biểu đồ phần dư của các mô hình trên tập kiểm tra



Hình 16: Biểu đồ giá trị dự đoán-giá trị thực trên tập kiểm tra

Ngược lại, các mô hình cơ sở phí tuyến có sự khớp cao với cả dữ liệu và việc ngắt ngưỡng. Điều này có thể ảnh hưởng đến dự đoán thực tế khi không có sự ngắt ngưỡng này.

Đối với các mô hình hồi quy tuyến tính cơ bản, các điểm phân tán hợp lý hơn. Mô hình không quá bị ảnh hưởng bởi việc ngắt ngưỡng của dữ liệu. Điều này cho thấy mô hình khớp tốt với dữ liệu và có khả năng dự đoán thực tế ổn định hơn.

1.4 Phần nâng cao

1.4.1 Gaussian Process Regression

Gaussian Process Regression (GPR) là một phương pháp hồi quy phi tham số (non-parametric Bayesian approach), trong đó ta không trực tiếp học một hàm ánh xạ cố định, mà thay vào đó định nghĩa một phân phối trên không gian các hàm.

Khái niệm Gaussian Process Một Gaussian Process được định nghĩa như sau:

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')), \quad (58)$$

trong đó:

- $m(\mathbf{x})$ là hàm mean (thường giả sử bằng 0)
- $k(\mathbf{x}, \mathbf{x}')$ là hàm kernel mô tả độ tương quan giữa hai điểm dữ liệu

Với bài toán hồi quy, ta giả sử dữ liệu quan sát:

$$t_i = f(\mathbf{x}_i) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_n^2). \quad (59)$$

Kernel RBF Trong thực nghiệm, ta sử dụng Radial Basis Function (RBF) kernel:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \quad (60)$$

trong đó:

- ℓ là tham số length-scale, điều khiển độ mượt của hàm
- σ_f^2 là variance, điều khiển biên độ dao động

RBF kernel giả định rằng các điểm gần nhau trong không gian input sẽ có giá trị đầu ra tương tự nhau.

Posterior dự đoán Với tập huấn luyện $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^n$ và $\mathbf{t} = \{t_i\}_{i=1}^n$, ta xây dựng ma trận kernel:

$$\mathbf{K} = K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}. \quad (61)$$

Với một điểm kiểm tra $\mathbf{x}_*$, ta có:

$$\mathbf{k}_* = K(\mathbf{X}, \mathbf{x}_*), \quad k_{**} = K(\mathbf{x}_*, \mathbf{x}_*). \quad (62)$$

Khi đó phân phối hậu nghiệm (posterior predictive distribution) là Gaussian:

$$p(f_* | \mathbf{x}_*, \mathbf{X}, \mathbf{t}) = \mathcal{N}(\mu_*, \sigma_*^2), \quad (63)$$

với:

$$\mu_* = \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{t}, \quad (64)$$

$$\sigma_*^2 = k_{**} - \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{k}_*. \quad (65)$$

Log-Marginal Likelihood Các tham số kernel $\boldsymbol{\theta} = \{\ell, \sigma_f, \sigma_n\}$ được học bằng cách tối ưu log-marginal likelihood:

$$\log p(\mathbf{t} \mid \mathbf{X}) = -\frac{1}{2} \mathbf{t}^\top \mathbf{K}^{-1} \mathbf{t} - \frac{1}{2} \log |\mathbf{K}| - \frac{n}{2} \log(2\pi). \quad (66)$$

Hàm này gồm ba thành phần:

- **Data fit term:** $\mathbf{t}^\top \mathbf{K}^{-1} \mathbf{t}$
- **Complexity penalty:** $\log |\mathbf{K}|$
- **Normalization constant**

Tối ưu tham số bằng Gradient Ascent Ta tối ưu tham số kernel bằng gradient ascent:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla_{\boldsymbol{\theta}} \log p(\mathbf{t} \mid \mathbf{X}) \quad (67)$$

Quy trình huấn luyện:

1. Khởi tạo tham số ℓ, σ_f, σ_n
2. Tính ma trận kernel $\mathbf{K}$
3. Tính log-marginal likelihood
4. Tính gradient theo từng tham số
5. Cập nhật tham số

Quá trình này giúp mô hình tự động học độ mượt và độ nhiễu phù hợp với dữ liệu.

Dự đoán và bất định Sau khi huấn luyện, mô hình không chỉ đưa ra giá trị dự đoán mà còn cung cấp độ bất định:

$$f_* \sim \mathcal{N}(\mu_*, \sigma_*^2). \quad (68)$$

Trong đó:

- μ_* là giá trị dự đoán trung bình
- σ_*^2 biểu diễn độ không chắc chắn

1.4.2 Kernel Ridge Regression

Trong phần này, ta triển khai Kernel Ridge Regression với hai loại kernel là RBF và Polynomial, đồng thời so sánh với Ridge Regression tuyến tính.

Đối với RBF kernel, tham số γ được lựa chọn thông qua grid search với tập giá trị $\{0.01, 0.1, 1\}$. Kết quả tốt nhất đạt được tại $\gamma = 0.1$.

Đối với Polynomial kernel, các tham số được tìm kiếm trên tập giá trị degree là $\{2, 3\}$ và coef0 là $\{0, 1\}$. Kết quả tối ưu là degree = 2 và coef0 = 1.

Bảng 11 và Bảng 12 trình bày kết quả cross-validation của các mô hình. RBF Ridge cho kết quả tốt nhất, trong khi Ridge tuyến tính kém nhất và Polynomial Ridge có độ biến động lớn.

Bảng 11: Kết quả cross-validation của các mô hình (phần 1)

Model	MSE	RMSE
Ridge	0.8875 ± 0.0352	0.9419 ± 0.0186
RBF Ridge	0.3583 ± 0.0275	0.5982 ± 0.0224
Polynomial Ridge	1.4810 ± 2.6136	0.9806 ± 0.7208

Bảng 12: Kết quả cross-validation của các mô hình (phần 2)

Model	MAE	R^2
Ridge	0.6829 ± 0.0163	0.3288 ± 0.0244
RBF Ridge	0.4185 ± 0.0098	0.7289 ± 0.0210
Polynomial Ridge	0.4788 ± 0.0331	-0.1673 ± 2.1098

Kết quả cross-validation

Kết quả trên tập kiểm tra Bảng 13 trình bày kết quả trên tập kiểm tra của các mô hình. Có thể thấy rằng mô hình RBF Ridge tiếp tục đạt hiệu năng tốt nhất với các giá trị MSE, RMSE và MAE thấp nhất, đồng thời hệ số R^2 cao nhất, cho thấy khả năng tổng quát hóa tốt. Polynomial Ridge cũng cải thiện so với Ridge tuyến tính, tuy nhiên vẫn kém hơn RBF Ridge. Trong khi đó, Ridge Regression tuyến tính cho kết quả thấp nhất do không thể mô hình hóa các quan hệ phi tuyến trong dữ liệu.

Bảng 13: Kết quả trên tập kiểm tra

Model	MSE	RMSE	MAE	R^2
Ridge	0.8951	0.9461	0.6807	0.3350
RBF Ridge	0.3571	0.5976	0.4175	0.7347
Polynomial Ridge	0.4340	0.6588	0.4666	0.6776

Kiểm định thống kê Bảng 14 cho thấy kết quả kiểm định thống kê giữa các mô hình. Cụ thể, sự khác biệt giữa Ridge và RBF Ridge là có ý nghĩa thống kê với p-value rất nhỏ trong cả t-test (1.99×10^{-13}) và Wilcoxon test (0.001953), nhỏ hơn mức ý nghĩa $\alpha = 0.05$. Điều này xác nhận rằng RBF Ridge cải thiện hiệu năng một cách đáng kể so với Ridge tuyến tính. Đối với cặp Ridge và Polynomial Ridge, cả hai kiểm định đều cho p-value lớn hơn 0.05 (0.5153 và 0.4316), cho thấy không có sự khác biệt có ý nghĩa thống kê giữa hai mô hình này. Điều này phù hợp với các kết quả thực nghiệm trước đó, khi Polynomial Ridge không cho thấy sự cải thiện ổn định so với Ridge. Đối với cặp RBF Ridge và Polynomial Ridge, kết quả không hoàn toàn nhất quán giữa hai kiểm định: t-test cho p-value = 0.2299 (không có ý nghĩa thống kê), trong khi Wilcoxon test cho p-value = 0.0020 (có ý nghĩa thống kê). Sự khác biệt này có thể xuất phát từ việc t-test giả định phân phối chuẩn, trong khi Wilcoxon test là kiểm định phi tham số và nhạy hơn với sự khác biệt về phân phối. Tổng hợp lại, có thể kết luận rằng RBF Ridge là mô hình vượt trội nhất, với sự cải thiện có ý nghĩa thống kê so với Ridge, trong khi Polynomial Ridge không cho thấy sự khác biệt rõ ràng và ổn định so với các mô hình còn lại.

Bảng 14: So sánh thống kê giữa các mô hình

Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
Ridge	RBF Ridge	1.99×10^{-13}	0.001953
Ridge	Polynomial Ridge	0.5153	0.4316
RBF Ridge	Polynomial Ridge	0.2299	0.0020

Nhận xét Kết quả cho thấy mô hình RBF Ridge đạt hiệu năng tốt nhất trên cả cross-validation và tập kiểm tra, với sai số thấp nhất và hệ số R^2 cao nhất. Điều này cho thấy dữ liệu có mối quan hệ phi tuyến mà kernel RBF có thể nắm bắt hiệu quả.

Polynomial Ridge cải thiện kết quả so với Ridge tuyến tính trên tập kiểm tra, tuy nhiên kết quả cross-validation có độ biến động lớn, cho thấy mô hình không ổn định và có khả năng overfitting.

Kiểm định thống kê cho thấy sự khác biệt giữa Ridge và RBF Ridge là có ý nghĩa thống kê (p-value rất nhỏ), trong khi sự khác biệt giữa Ridge và Polynomial Ridge không có ý nghĩa thống kê (p-value lớn hơn 0.05).

Tóm lại, RBF kernel là lựa chọn phù hợp nhất trong bài toán này, trong khi Ridge tuyến tính đóng vai trò baseline và Polynomial kernel cần được điều chỉnh cẩn thận để đạt hiệu quả tốt.

1.4.3 Phân tích Bias-Variance

Thiết lập bài toán Giả sử dữ liệu được sinh theo mô hình:

$$t = f(\mathbf{x}) + \varepsilon, \quad (69)$$

$$\mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2. \quad (70)$$

Xét mô hình dự đoán: $\hat{f}(\mathbf{x})$.

Phân rã sai số kỳ vọng

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(t - \hat{f}(\mathbf{x}) \right)^2 \right] = \mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(f(\mathbf{x}) + \varepsilon - \hat{f}(\mathbf{x}) \right)^2 \right] \quad (71)$$

$$= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}) - f(\mathbf{x}) \right)^2 \right] + \sigma^2. \quad (72)$$

Phân rã tiếp:

$$\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}) - f(\mathbf{x}) \right)^2 \right] = \left(\mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x})] - f(\mathbf{x}) \right)^2 + \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x})] \right)^2 \right]. \quad (73)$$

Định nghĩa

$$\text{Bias}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x})] - f(\mathbf{x}), \quad (74)$$

$$\text{Var}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x})] \right)^2 \right]. \quad (75)$$

Kết quả cuối cùng

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(t - \hat{f}(\mathbf{x}) \right)^2 \right] = \text{Bias}^2(\mathbf{x}) + \text{Var}(\mathbf{x}) + \sigma^2. \quad (76)$$

Kết quả ước lượng Trong phần này, ta thực hiện phân tích Bias-Variance cho hai mô hình Ridge và Lasso nhằm hiểu rõ hơn về khả năng tổng quát hóa của từng mô hình.

Phân tích được thực hiện bằng phương pháp bootstrap với 200 lần lặp. Cụ thể, với mỗi lần lặp, chúng tôi lấy mẫu lại từ tập huấn luyện, huấn luyện mô hình và ghi nhận dự đoán. Từ đó, ước lượng được bias^2 và variance theo công thức thực nghiệm. Bảng 15 trình bày kết quả ước lượng bias^2 và variance cho hai mô hình.

Bảng 15: Phân tích Bias-Variance cho Ridge và Lasso

Model	Bias ²	Variance
Ridge	0.5252	0.0378
Lasso	0.6702	0.0235

Kết quả cho thấy Ridge có bias^2 thấp hơn so với Lasso, nhưng lại có variance cao hơn. Điều này cho thấy sự đánh đổi giữa bias^2 và variance của hai phương pháp regularization:

- Ridge (l_2 regularization) giữ lại toàn bộ các đặc trưng và phân phối trọng số một cách mượt mà, do đó giảm bias nhưng có xu hướng tăng variance.
- Lasso (l_1 regularization) thực hiện lựa chọn đặc trưng bằng cách đưa một số hệ số về 0, dẫn đến mô hình đơn giản hơn, variance thấp hơn nhưng bias cao hơn.

Sự đánh đổi giữa bias và variance thể hiện rõ ràng trong kết quả thực nghiệm: Ridge đạt được sự cân bằng tốt hơn giữa hai thành phần, trong khi Lasso thiên về giảm variance với chi phí là tăng bias.

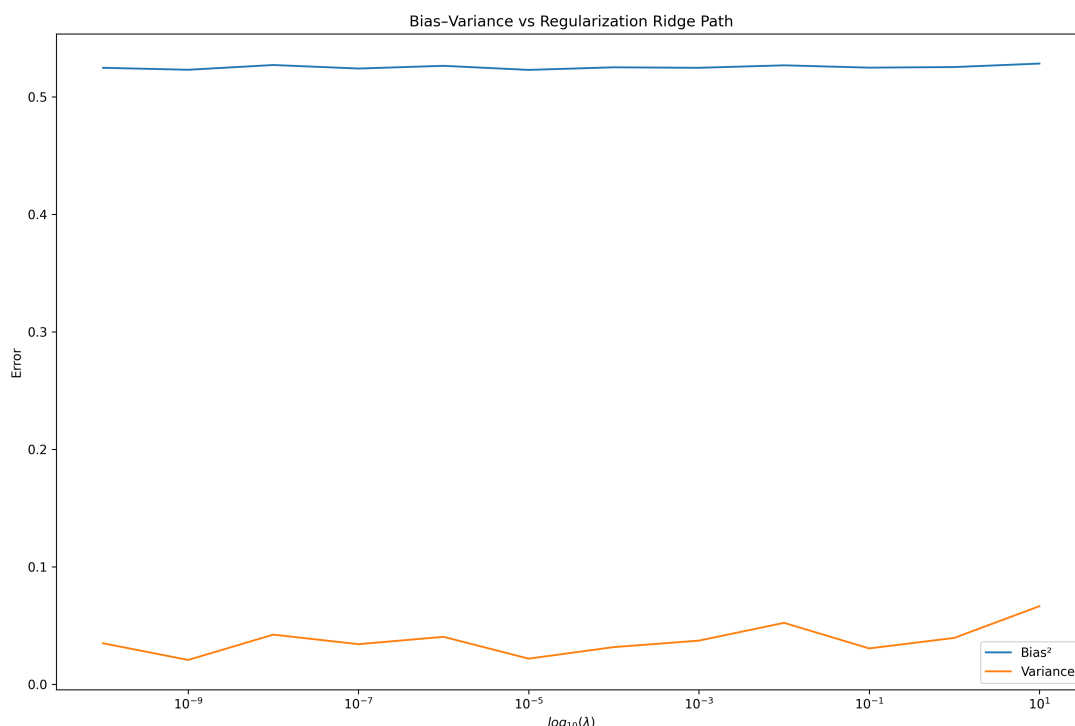
Ngoài ra, khi quan sát regularization path (biểu diễn sự thay đổi của hệ số theo tham số điều chuẩn), có thể thấy rằng:

- Với Ridge, các hệ số giảm dần một cách liên tục khi tăng mức regularization.
- Với Lasso, nhiều hệ số bị triệt tiêu hoàn toàn về 0, làm giảm độ phức tạp của mô hình.

Tóm lại, kết quả phân tích Bias-Variance cho thấy Ridge phù hợp hơn trong bài toán này nhờ đạt được sự cân bằng tốt giữa bias và variance, trong khi Lasso có xu hướng underfitting do bias cao.

Thay đổi của bias-variance theo hệ số điều chỉnh

Ridge Regression Hình 17 minh họa sự thay đổi của bias^2 và variance theo hệ số điều chuẩn λ đối với Ridge Regression. Có thể quan sát rằng bias^2 của Ridge gần như ổn định khi λ thay đổi, trong khi variance có xu hướng tăng nhẹ khi λ lớn. Điều này phản ánh đặc điểm của Ridge Regression: các hệ số bị co lại một cách liên tục nhưng không bị triệt tiêu hoàn toàn, giúp mô hình duy trì mức bias thấp và kiểm soát variance tương đối tốt. Do đó, Ridge đạt được sự cân bằng ổn định giữa bias và variance trên toàn bộ miền giá trị của λ .



Hình 17: Bias-Variance của Ridge trên regularization path

Lasso Regression Hình 18 trình bày sự thay đổi của bias^2 và variance theo λ đối với Lasso Regression. Có thể thấy, khi λ tăng, bias^2 tăng mạnh, đặc biệt ở vùng λ lớn. Nguyên nhân là do Lasso thực hiện regularization dạng l_1 , khiến nhiều hệ số bị đưa về 0, làm giảm độ phức tạp của mô hình nhưng đồng thời làm mất thông tin quan trọng, dẫn đến underfitting. Ngược lại, variance giảm dần và tiến gần về 0 khi λ lớn, cho thấy mô hình trở nên rất ổn định nhưng kém linh hoạt. Hiện tượng này thể hiện rõ ràng sự đánh đổi giữa bias và variance: Lasso ưu tiên giảm variance bằng cách tăng bias.

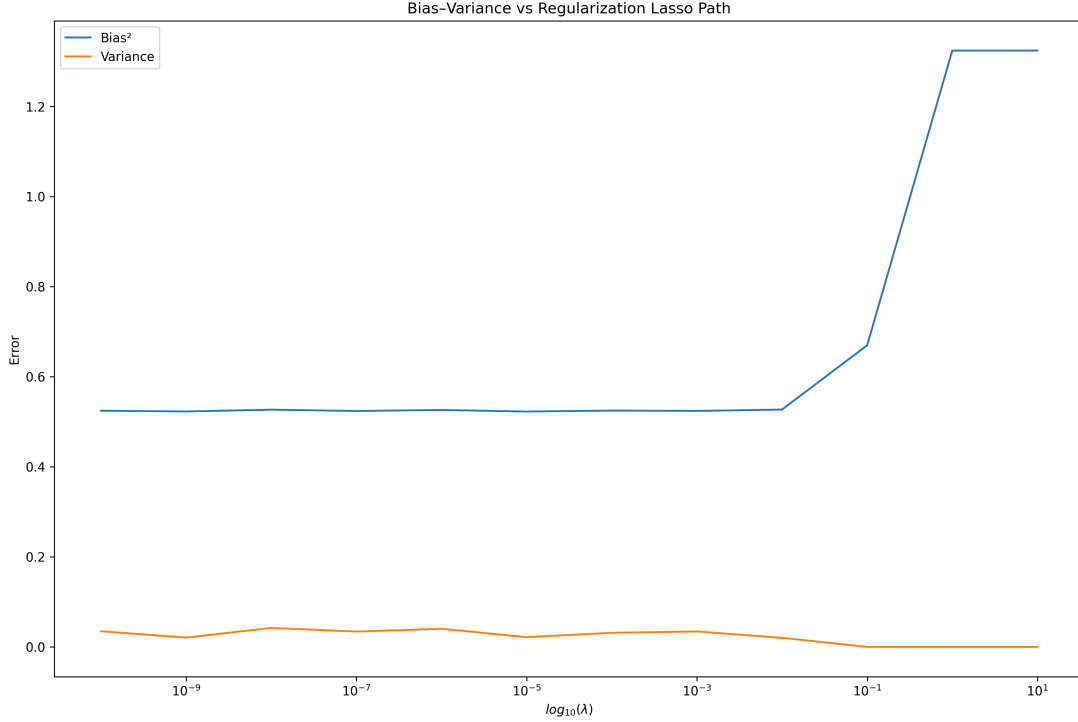
So sánh và nhận xét Từ hai hình trên, có thể thấy:

- Ridge duy trì bias thấp và variance ở mức trung bình, cho thấy khả năng tổng quát hóa ổn định.
- Lasso tạo ra mô hình đơn giản hơn với variance thấp, nhưng phải trả giá bằng bias cao khi λ lớn.

Tóm lại, Ridge phù hợp hơn trong trường hợp dữ liệu có nhiều đặc trưng đóng góp nhỏ, trong khi Lasso hữu ích khi cần lựa chọn đặc trưng, nhưng dễ dẫn đến underfitting nếu regularization quá mạnh.

1.4.4 Bayesian Linear Regression

Trong khi hồi quy tuyến tính truyền thống (OLS) tìm kiếm một bộ tham số tối ưu duy nhất thông qua việc cực tiểu hóa hàm mất mát, Bayesian Regression tiếp cận bài toán bằng cách tích hợp tri thức tiên nghiệm (prior knowledge) và dữ liệu thực nghiệm để thu được một phân phối xác suất trên không gian tham số. Phương pháp này cho phép mô hình hóa không chỉ giá trị dự báo mà còn cả độ tin cậy của dự báo đó.



Hình 18: Bias-Variance của Lasso trên regularization path

Mô hình xác suất và Likelihood Xét tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^d$. Giả định mỗi liên hệ tuyến tính bị nhiễu bởi một thành phần nhiễu trắng Gaussian:

$$y = \mathbf{w}^\top \mathbf{x} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \beta^{-1})$$

Trong đó β là độ chụm (precision) của nhiễu. Hàm hợp lý (likelihood) của toàn bộ tập dữ liệu được mô tả bởi phân phối chuẩn đa biến:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}, \beta) = \prod_{i=1}^n \mathcal{N}(y_i \mid \mathbf{w}^\top \mathbf{x}_i, \beta^{-1}) = \mathcal{N}(\mathbf{y} \mid \mathbf{X}\mathbf{w}, \beta^{-1}\mathbf{I})$$

Phân phối Tiên nghiệm (Prior) và Ý nghĩa của Sự điều chuẩn Chúng ta đặt một phân phối tiên nghiệm Gaussian lên các tham số $\mathbf{w}$ để thể hiện niềm tin ban đầu về cấu trúc mô hình:

$$p(\mathbf{w} \mid \alpha) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \alpha^{-1}\mathbf{I})$$

Việc sử dụng kỳ vọng bằng $\mathbf{0}$ đóng vai trò như một cơ chế điều chuẩn (regularization) tự nhiên. Trong ngôn ngữ tối ưu hóa, việc tìm cực đại hậu nghiệm (MAP) với prior này tương đương chính xác với hồi quy Ridge (L_2 regularization), nơi α/β đóng vai trò là hệ số phạt.

Suy diễn Hậu nghiệm (Posterior Inference) Nhờ tính liên hợp của phân phối Gaussian, phân phối hậu nghiệm — sự kết hợp giữa niềm tin tiên nghiệm và bằng chứng từ dữ liệu — cũng là một phân phối Gaussian:

$$p(\mathbf{w} \mid \mathbf{y}, \mathbf{X}, \alpha, \beta) \propto p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}, \beta)p(\mathbf{w} \mid \alpha) = \mathcal{N}(\mathbf{w} \mid \mathbf{m}_n, \mathbf{S}_n)$$

Các tham số của phân phối hậu nghiệm được xác định qua hệ phương trình:

$$\mathbf{S}_n^{-1} = \alpha\mathbf{I} + \beta\mathbf{X}^\top\mathbf{X}, \quad \mathbf{m}_n = \beta\mathbf{S}_n\mathbf{X}^\top\mathbf{y}$$

Ở đây, ma trận độ chụm hậu nghiệm $\mathbf{S}_n^{-1}$ là tổng của độ chụm tiên nghiệm và thông tin thu hồi từ dữ liệu.

Phân phối Dự báo (Predictive Distribution) Để dự báo nhãn y^* cho một quan sát mới $\mathbf{x}^*$, thay vì chỉ sử dụng vector trung bình $\mathbf{m}_n$, ta tích phân trên toàn bộ không gian tham số (marginalization):

$$p(y^* | \mathbf{x}^*, \mathcal{D}) = \int p(y^* | \mathbf{x}^*, \mathbf{w}) p(\mathbf{w} | \mathcal{D}) d\mathbf{w} = \mathcal{N}(y^* | \mathbf{m}_n^\top \mathbf{x}^*, \sigma_N^2(\mathbf{x}^*))$$

Trong đó, phương sai dự báo được phân tách thành hai thành phần cốt lõi:

$$\sigma_N^2(\mathbf{x}^*) = \underbrace{\frac{1}{\beta}}_{\text{Aleatoric}} + \underbrace{\mathbf{x}^{*\top} \mathbf{S}_n \mathbf{x}^*}_{\text{Epistemic}}$$

- **Aleatoric Uncertainty:** Nhiều nội tại của dữ liệu, không thể giảm bớt bằng cách thêm dữ liệu huấn luyện.
- **Epistemic Uncertainty:** Sự bất định về tham số mô hình. Thành phần này sẽ lớn tại những vùng không gian $\mathbf{x}^*$ thừa thớt dữ liệu huấn luyện và thu hẹp dần khi lượng dữ liệu tăng lên.

Nhận xét bản chất Bayesian Regression không chỉ đơn thuần là một công cụ dự báo, mà là một khung tư duy thống kê chặt chẽ:

1. **Tính toán xác suất:** Nó thay thế các câu trả lời "đơn trị" bằng "khoảng tin cậy", cho phép quản trị rủi ro trong các quyết định quan trọng.
2. **Khả năng học trực tuyến (Online Learning):** Phân phối hậu nghiệm tại bước này có thể trở thành phân phối tiên nghiệm cho bước tiếp theo khi có dữ liệu mới, giúp mô hình cập nhật liên tục mà không cần huấn luyện lại từ đầu.
3. **Tự điều chuẩn tối ưu:** Thông qua việc tối ưu hóa Marginal Likelihood (Evidence), mô hình có thể tự tìm ra các siêu tham số α, β tối ưu mà không cần tập validation tách biệt.

Nhận xét lý thuyết Bayesian Regression cung cấp một cách tiếp cận đầy đủ hơn so với hồi quy tuyến tính truyền thống. Không chỉ đưa ra dự đoán điểm, mô hình còn ước lượng được độ bất định thông qua phân phối dự đoán.

Phương pháp này đặc biệt phù hợp trong các bài toán có dữ liệu nhiễu hoặc kích thước mẫu nhỏ, nơi việc đánh giá độ tin cậy của dự đoán là quan trọng.

Tuy nhiên, chi phí tính toán cao hơn do cần xử lý ma trận nghịch đảo trong không gian tham số.

Kết quả thực nghiệm của Bayesian Linear Regression Trong thực nghiệm này, mô hình Bayesian Linear Regression (BLR) được cấu hình với các tham số tiên định (hyperparameters) nhằm cân bằng giữa tri thức ưu tiên và dữ liệu thực tế:

$$\alpha = 1.0 \quad (\text{Prior precision}), \quad \beta = 10.0 \quad (\text{Noise precision})$$

Việc lựa chọn $\beta = 10.0$ tương ứng với giả định nhiễu quan sát có phương sai $\sigma^2 = 0.1$, cho thấy mô hình tin cậy khá cao vào chất lượng dữ liệu huấn luyện.

Phân tích phân phối hậu nghiệm (Posterior Analysis) Khác với Hồi quy Tuyến tính thông thường chỉ tìm điểm tối ưu (Point Estimation), BLR xác định toàn bộ phân phối xác suất của trọng số $\mathbf{w}$ dưới dạng Gaussian đa biến:

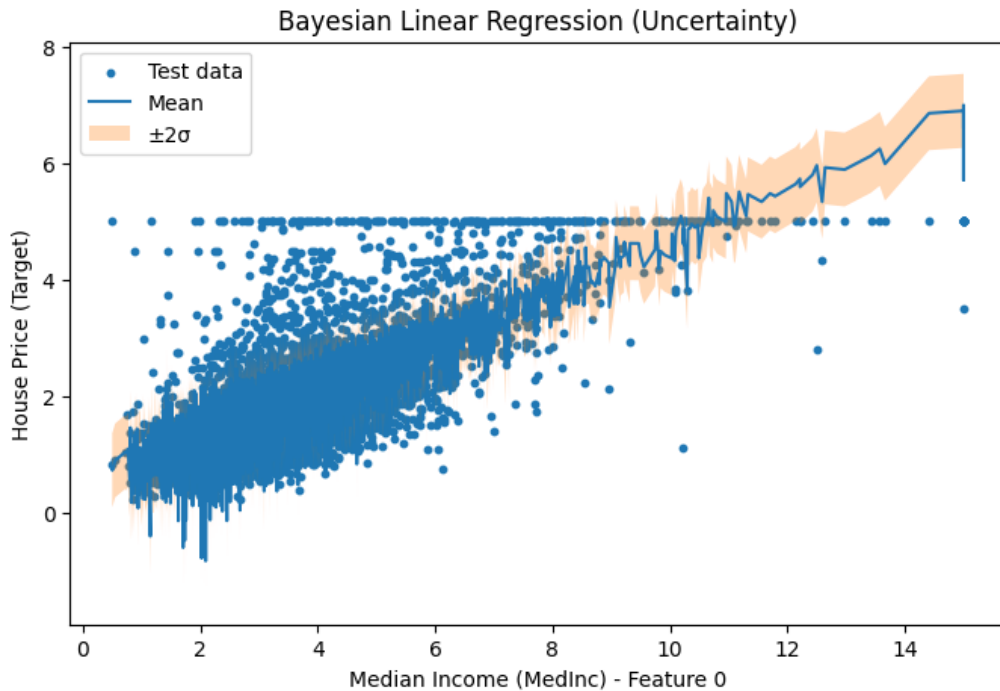
$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \mathcal{N}(\mu_N, \Sigma_N)$$

Trong đó:

- $\mu_N \in \mathbb{R}^{9 \times 1}$: Vector kỳ vọng hậu nghiệm, đại diện cho bộ trọng số hội tụ sau khi quan sát dữ liệu.
- $\Sigma_N \in \mathbb{R}^{9 \times 9}$: Ma trận hiệp phương sai, thể hiện độ bất định và mối tương quan giữa 9 đặc trưng đầu vào.

Phân phối dự đoán và Độ bất định (Predictive Distribution) Khả năng vượt trội của BLR nằm ở việc cung cấp phân phối dự đoán cho các điểm dữ liệu mới x^* :

$$p(y^* \mid \mathbf{x}^*, \mathbf{X}, \mathbf{y}) = \mathcal{N}(\mu_N^T \mathbf{x}^*, \sigma_N^2(\mathbf{x}^*))$$



Hình 19: Kết quả dự đoán của BLR: Đường trung bình (Mean) và dải tin cậy $\pm 2\sigma$

Phân tích trực quan từ Hình 19:

- **Đường màu xanh (Predictive Mean):** Phản ánh xu hướng trung tâm của giá nhà. Có thể thấy tại các vùng dữ liệu tập trung (mật độ điểm cao), đường này đi qua trung tâm của đám mây điểm một cách ổn định.
- **Vùng màu cam (Epistemic Uncertainty):** Thể hiện khoảng tin cậy $\pm 2\sigma$ (xấp xỉ 95% xác suất). Tại các vùng biên hoặc nơi dữ liệu thưa thớt (trục hoành > 10), dải màu cam mở rộng rõ rệt. Đây là minh chứng cho việc mô hình tự lượng hóa sự "thiếu tự tin" khi dự báo ở những không gian chưa có nhiều thông tin huấn luyện.

Đánh giá hiệu năng Sai số bình phương trung bình (MSE) thu được trên tập kiểm tra:

$$\text{MSE} = 0.524602$$

Mức sai số này cho thấy mô hình có khả năng khớp tốt với dữ liệu thực tế trong khi vẫn duy trì được tính tổng quát hóa.

Biện luận thực nghiệm Từ kết quả thu được, chúng ta rút ra các nhận định quan trọng:

- **Khả năng tự điều chuẩn (Self-regularization):** Tham số α đóng vai trò như một bộ điều chuẩn L_2 . Với $\alpha = 1.0$, mô hình ngăn chặn hiệu quả hiện tượng các trọng số quá lớn, giúp MSE đạt mức thấp mà không bị Overfitting.
- **Xử lý nhiễu và dữ liệu chặn (Capped Data):** Biểu đồ cho thấy có một dải dữ liệu bị chặn tại ngưỡng giá bằng 5.0. Dù dữ liệu nhiễu này gây áp lực lên mô hình, BLR nhờ có thành phần β (noise precision) vẫn duy trì được đường dự đoán xu hướng mà không bị kéo lệch cực đoan bởi các điểm outlier.
- **Tin cậy trong ra quyết định:** Việc cung cấp σ_N^2 cho phép người dùng không chỉ biết giá nhà dự kiến mà còn biết "độ rủi ro" của dự báo đó, một yếu tố mà các mô hình hồi quy truyền thống thường bỏ qua.

Tổng kết Thực nghiệm khẳng định Bayesian Linear Regression là công cụ mạnh mẽ cho bài toán dự báo giá nhà. Nó không chỉ đảm bảo độ chính xác (MSE thấp) mà còn cung cấp một khung lý thuyết vững chắc để quản trị độ bất định trong dữ liệu, giúp tăng tính minh bạch và tin cậy cho các hệ thống hỗ trợ ra quyết định.

1.4.5 Robust Regression với IRLS

Trong hồi quy tuyến tính thông thường (Ordinary Least Squares – OLS), mô hình giả định rằng nhiễu phương sai bằng nhau và tối ưu hóa tổng bình phương sai số. Tuy nhiên, giả định này khiến OLS trở nên rất nhạy cảm với các điểm ngoại lai, do sai số lớn bị bình phương và khuếch đại ảnh hưởng.

Để khắc phục hạn chế này, phương pháp Robust Regression được sử dụng nhằm giảm ảnh hưởng của ngoại lai thông qua việc gán trọng số thích nghi cho từng quan sát. Một trong những kỹ thuật phổ biến là Iteratively Reweighted Least Squares (IRLS), trong đó bài toán được giải lặp bằng cách cập nhật trọng số dựa trên phần dư.

Tại mỗi bước lặp, phần dư được tính theo:

$$\mathbf{r} = \mathbf{t} - \Phi \mathbf{w}. \quad (77)$$

Dựa trên phần dư này, trọng số được cập nhật theo một hàm loss robust. Với loss Huber, các quan sát có sai số nhỏ được giữ nguyên trọng số bằng 1, trong khi các quan sát có sai số lớn bị giảm trọng số theo tỉ lệ nghịch:

$$w_i = \begin{cases} 1 & \text{nếu } |r_i| \leq \delta, \\ \frac{\delta}{|r_i|} & \text{nếu } |r_i| > \delta. \end{cases} \quad (78)$$

Trong trường hợp sử dụng phân phối Student-t, trọng số được suy ra từ likelihood của phân phối này:

$$w_i = \frac{\nu + 1}{\nu + r_i^2}, \quad (79)$$

trong đó ν là bậc tự do, điều khiển độ “heavy-tailed” của phân phối. Giá trị ν nhỏ giúp mô hình trở nên robust hơn với ngoại lai.

Sau khi có trọng số, bài toán trở thành một dạng Weighted Least Squares:

$$\mathbf{w} = (\Phi^T \mathbf{W} \Phi)^{-1} \Phi^T \mathbf{W} \mathbf{t}, \quad (80)$$

trong đó $\mathbf{W}$ là ma trận đường chéo chứa các trọng số.

So với OLS, IRLS làm giảm đáng kể ảnh hưởng của các điểm ngoại lai bằng cách giảm trọng số của các quan sát có sai số lớn. Điều này giúp mô hình ổn định hơn trong các tập dữ liệu có nhiều không thỏa giả thiết Gauss–Markov hoặc chứa ngoại lai, đồng thời cải thiện khả năng tổng quát hóa trong thực tế.

Kiểm tra độ nhạy với ngoại lai Bảng 16 và 17 cho thấy sự khác biệt rõ rệt giữa OLS và các phương pháp Robust (IRLS với Huber và Student-t) khi dữ liệu bị nhiễu bởi ngoại lai. Dữ liệu được tạo nhiễu bằng cách lấy ngẫu nhiên 10% dữ liệu và cộng vào 5 lần độ lệch chuẩn của dữ liệu ban đầu.

Bảng 16: So sánh độ nhạy với ngoại lai của các mô hình OLS, IRLS (Huber và Student-t) (phần 1)

Model	MSE_clean	MSE_noisy	MSE_sensitivity
OLS	0.5253	0.8595	0.6362
IRLS (Huber)	0.518	0.5271	0.0176
IRLS (Student-t)	0.4726	0.4776	0.0106

Bảng 17: So sánh độ nhạy với ngoại lai của các mô hình OLS, IRLS (Huber và Student-t) (phần 2)

Model	MAE_clean	MAE_noisy	MAE_sensitivity
OLS	0.5299	0.7942	0.4988
IRLS (Huber)	0.5129	0.5379	0.0487
IRLS (Student-t)	0.4901	0.5048	0.03

Với mô hình OLS, khi thêm nhiễu vào dữ liệu huấn luyện, các chỉ số tăng mạnh. Cụ thể, MSE tăng từ 0.5253 đến 0.8595 (lên đến 63.62%). Tương tự, MAE gần 50%. Điều này cho thấy OLS rất nhạy với ngoại lai.

Trong khi đó, các mô hình Robust có sự thay đổi các chỉ số rất nhỏ (dưới 5%). Việc dùng phân phối Student-t cho độ ổn định nhỉnh hơn một chút so với Huber loss (độ nhạy 0.0106 so với 0.176 đối với MSE và 0.03 so với 0.0487 với MAE).

2 Bài toán phân lớp

2.1 Dữ liệu

Bộ dữ liệu Forest CoverType được xây dựng nhằm dự đoán loại thảm thực vật rừng dựa trên các biến bản đồ học, không sử dụng dữ liệu viễn thám. Mỗi quan sát tương ứng với một ô rừng kích thước 30×30 mét vuông, trong đó nhãn được xác định từ dữ liệu của Cơ quan Lâm nghiệp Hoa Kỳ (USFS), còn các đặc trưng được tổng hợp từ dữ liệu của USGS và USFS (Blackard, 1998).

Khu vực nghiên cứu bao gồm bốn vùng hoang dã trong rừng quốc gia Roosevelt ở phía bắc bang Colorado, gồm Rawah (khu vực 1), Neota (khu vực 2), Comanche Peak (khu vực 3) và Cache la Poudre (khu vực 4). Đây là các khu vực ít chịu tác động của con người, do đó đặc trưng thảm thực vật chủ yếu phản ánh các quá trình sinh thái tự nhiên (Blackard, 1998). Về đặc điểm địa hình, Neota có độ cao trung bình lớn nhất, tiếp theo là Rawah và Comanche Peak, trong khi Cache la Poudre có độ cao trung bình thấp nhất (Blackard, 1998).

Xét về thành phần loài cây, Neota chủ yếu gồm spruce/fir (lớp 1); Rawah và Comanche Peak có lodgepole pine (lớp 2) là loài chính, kèm theo spruce/fir và aspen (lớp 5); trong khi Cache la Poudre đặc trưng bởi Ponderosa pine (lớp 3), Douglas-fir (lớp 6) và cottonwood/willow (lớp 4) (Blackard, 1998). Nhìn chung, Rawah và Comanche Peak mang tính đại diện cao hơn cho toàn bộ tập dữ liệu, còn Cache la Poudre có xu hướng khác biệt hơn do phạm vi độ cao thấp và thành phần loài đặc trưng (Blackard, 1998).

2.1.1 Mô tả dữ liệu

Kích thước và kiểu dữ liệu

- Số lượng mẫu: 581,012
- Số lượng biến đầu vào: 54
- Số lượng biến mục tiêu: 1
- Kiểu dữ liệu: số nguyên, số thực (đã được làm tròn thành số nguyên) và biến nhị phân
- Giá trị thiếu: không có

Ý nghĩa các biến Bảng 18 mô tả ý nghĩa các biến trong bộ dữ liệu. Các biến đầu vào bao gồm các đặc trưng địa hình như độ cao, hướng dốc, độ dốc; khoảng cách đến các đối tượng như nguồn nước, đường giao thông và điểm cháy rừng; các chỉ số chiếu sáng theo thời điểm trong ngày; cùng với các biến nhị phân biểu diễn khu vực hoang dã và loại đất.

2.1.2 Phân tích khám phá

Thống kê mô tả Bảng 19, 20, 21, và 22 cho ta thấy thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType.

Kết quả thống kê mô tả cho các biến định lượng trong bộ dữ liệu cho thấy sự phân bố đa dạng về thang đo và độ biến thiên giữa các đặc trưng địa hình.

Biến Elevation có giá trị trung bình khoảng 2959.37 với độ lệch chuẩn 279.98, cho thấy sự chênh lệch đáng kể về độ cao giữa các khu vực quan sát.

Bảng 18: Mô tả các biến trong bộ dữ liệu Forest CoverType

Tên biến	Kiểu dữ liệu	Ý nghĩa
Elevation	Số thực	Độ cao so với mực nước biển (mét)
Aspect	Số thực	Hướng dốc (đơn vị góc azimuth)
Slope	Số thực	Độ dốc của địa hình (độ)
Horizontal_Distance_ To_Hydrology	Số thực	Khoảng cách ngang đến nguồn nước gần nhất (mét)
Vertical_Distance_ To_Hydrology	Số thực	Khoảng cách dọc đến nguồn nước gần nhất (mét)
Horizontal_Distance_ To_Roadways	Số thực	Khoảng cách ngang đến đường giao thông gần nhất (mét)
Hillshade_9am	Số thực	Chỉ số che bóng lúc 9 giờ sáng (0–255)
Hillshade_Noon	Số thực	Chỉ số che bóng lúc 12 giờ trưa (0–255)
Hillshade_3pm	Số thực	Chỉ số che bóng lúc 3 giờ chiều (0–255)
Horizontal_Distance_ To_Fire_Points	Số thực	Khoảng cách ngang đến điểm cháy rừng gần nhất (mét)
Wilderness_Area	Nhị phân	4 biến nhị phân biểu diễn khu vực hoang dã (0: không thuộc, 1: thuộc)
Soil_Type	Nhị phân	40 biến nhị phân biểu diễn loại đất (0: không thuộc, 1: thuộc)
Cover_Type	Số nguyên	Loại thảm thực vật rừng (giá trị từ 1 đến 7)

Bảng 19: Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 1)

	Elevation	Aspect	Slope
count	581012.0	581012.0	581012.0
mean	2959.37	155.66	14.1
std	279.98	111.91	7.49
min	1859.0	0.0	0.0
25%	2809.0	58.0	9.0
50%	2996.0	127.0	13.0
75%	3163.0	260.0	18.0
max	3858.0	360.0	66.0

Bảng 20: Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 2)

	Hillshade_9am	Hillshade_Noon	Hillshade_3pm
count	581012.0	581012.0	581012.0
mean	212.15	223.32	142.53
std	26.77	19.77	38.27
min	0.0	0.0	0.0
25%	198.0	213.0	119.0
50%	218.0	226.0	143.0
75%	231.0	237.0	168.0
max	254.0	254.0	254.0

Bảng 21: Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 3)

	Horizontal_Distance__ To_Hydrology	Vertical_Distance_To__ Hydrology
count	581012.0	581012.0
mean	269.43	46.42
std	212.55	58.3
min	0.0	-173.0
25%	108.0	7.0
50%	218.0	30.0
75%	384.0	69.0
max	1397.0	601.0

Bảng 22: Thống kê mô tả các biến định lượng của bộ dữ liệu Forest CoverType (phần 4)

	Horizontal_Distance__ To_Roadways	Horizontal_Distance__ To_Fire_Points
count	581012.0	581012.0
mean	2350.15	1980.29
std	1559.25	1324.2
min	0.0	0.0
25%	1106.0	1024.0
50%	1997.0	1710.0
75%	3328.0	2550.0
max	7117.0	7173.0

Biến Aspect dao động trong khoảng từ 0 đến 360, phản ánh hướng địa hình phân bố trên toàn bộ không gian góc phương vị.

Biến Slope có giá trị trung bình 14.10 và độ lệch chuẩn 7.49, cho thấy phần lớn khu vực có độ dốc thấp đến trung bình, tuy nhiên vẫn tồn tại các khu vực có độ dốc lớn.

Khoảng cách đến nguồn nước gần nhất (Horizontal_Distance_To_Hydrology) có giá trị trung bình 269.43, trong khi khoảng cách theo phương thẳng đứng có trung bình 46.42 và dao động từ -173 đến 601, phản ánh sự biến thiên địa hình theo độ cao tương đối.

Khoảng cách đến đường giao thông gần nhất (Horizontal_Distance_To_Roadways) có giá trị trung bình 2350.15 với độ lệch chuẩn lớn (1559.25), cho thấy mức độ phân tán không gian đáng kể giữa các khu vực rừng và hệ thống giao thông.

Các biến Hillshade_9am, Hillshade_Noon và Hillshade_3pm nằm trong khoảng từ 0 đến 254, thể hiện mức độ chiếu sáng tại các thời điểm khác nhau trong ngày. Giá trị trung bình lần lượt là 212.15, 223.32 và 142.53, cho thấy cường độ ánh sáng cao nhất vào buổi trưa.

Biến Horizontal_Distance_To_Fire_Points có giá trị trung bình 1980.29 với độ lệch chuẩn 1324.20, phản ánh sự khác biệt lớn về khoảng cách đến các điểm cháy rừng giữa các khu vực.

Tổng thể, các biến định lượng trong bộ dữ liệu thể hiện sự đa dạng về phạm vi giá trị và mức độ biến thiên, phản ánh đặc trưng địa hình và sinh thái phức tạp của khu vực nghiên cứu.

Phân phối biến mục tiêu Bảng 23 và Hình 20 cho thấy được phân phối của biến mục tiêu trong bộ dữ liệu Forest CoverType.

Bảng 23: Thống kê số lượng từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType

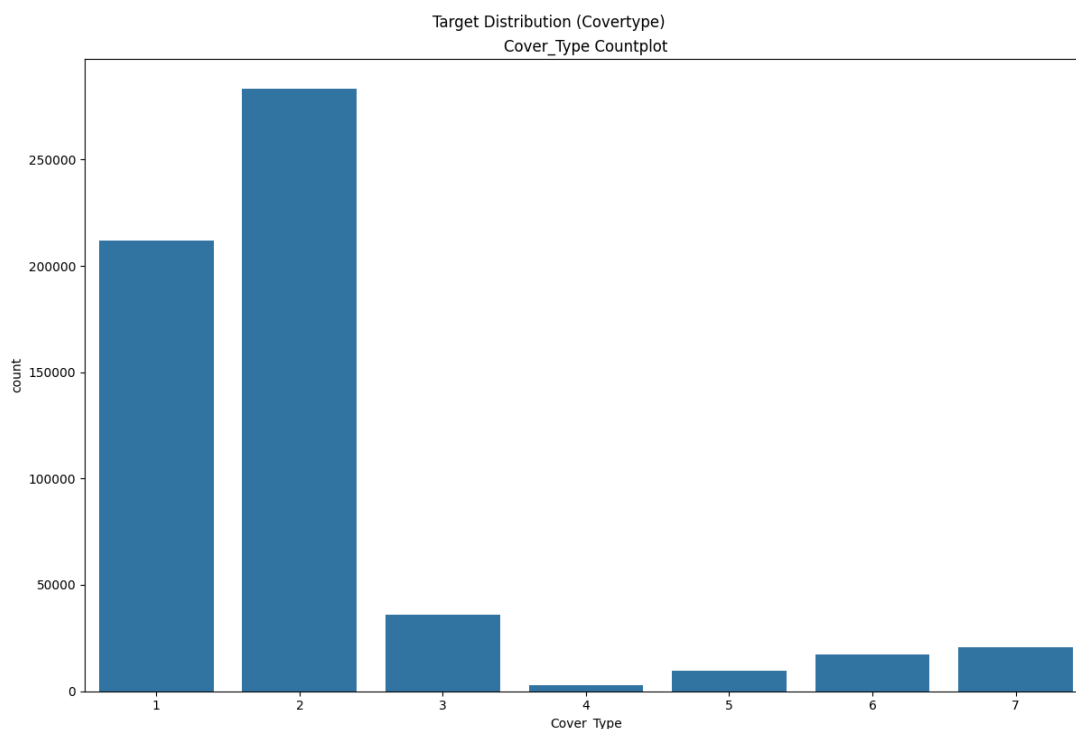
Cover_Type	count	percent
1	211840	36.46
2	283301	48.76
3	35754	6.15
4	2747	0.47
5	9493	1.63
6	17367	2.99
7	20510	3.53

Biến mục tiêu Cover_Type trong bộ dữ liệu biểu diễn loại thảm thực vật rừng và bao gồm 7 lớp khác nhau, tương ứng với các kiểu rừng chính trong khu vực nghiên cứu.

Kết quả thống kê cho thấy phân bố các lớp là không cân bằng. Lớp 2 chiếm tỷ lệ lớn nhất với 48.76%, tiếp theo là lớp 1 với 36.46%. Hai lớp này chiếm phần lớn tổng số quan sát, cho thấy dữ liệu bị lệch mạnh về hai loại rừng chủ đạo.

Các lớp còn lại có tỷ lệ thấp hơn đáng kể. Cụ thể, lớp 3 chiếm 6.15%, lớp 7 chiếm 3.53%, lớp 6 chiếm 2.99%, lớp 5 chiếm 1.63%, và lớp 4 chỉ chiếm 0.47%, là lớp hiếm nhất trong bộ dữ liệu.

Sự mất cân bằng này cho thấy bài toán phân loại rừng là một bài toán đa lớp không cân bằng, trong đó một số lớp chiếm ưu thế rõ rệt so với các lớp còn lại. Điều này có thể ảnh hưởng đến hiệu năng của các mô hình học máy, đặc biệt là xu hướng thiên lệch về các lớp phổ biến nếu không có biện pháp xử lý phù hợp.



Hình 20: Biểu đồ đếm tần suất biến mục tiêu trong bộ dữ liệu Forest CoverType

Các biến đặc trưng định lượng Hình 21 và 22 cho ta thấy phân phối của các biến đặc trưng định lượng của bộ dữ liệu Forest CoverType. Hình gồm hai cột, cột trái là biểu đồ hộp của biến đặc trưng, cột phải là biểu đồ tần suất của biến tương ứng.

Các biến Slope, Aspect, Horizontal_Distance_To_Hydrology, Vertical_Distance_To_Hydrology, Horizontal_Distance_To_Roadways và Horizontal_Distance_To_Fire_Points có phân phối lệch phải. Điều này cho thấy phần lớn các quan sát nằm trong các khu vực có địa hình tương đối “thuận lợi”, tức là độ dốc thấp đến trung bình và ở khoảng cách không quá xa các yếu tố địa lý như nguồn nước, đường giao thông và điểm cháy rừng. Ngược lại, chỉ một số ít quan sát thuộc các khu vực xa biệt lập hoặc có điều kiện địa hình cực đoan, tạo thành đuôi kéo dài về phía các giá trị lớn.

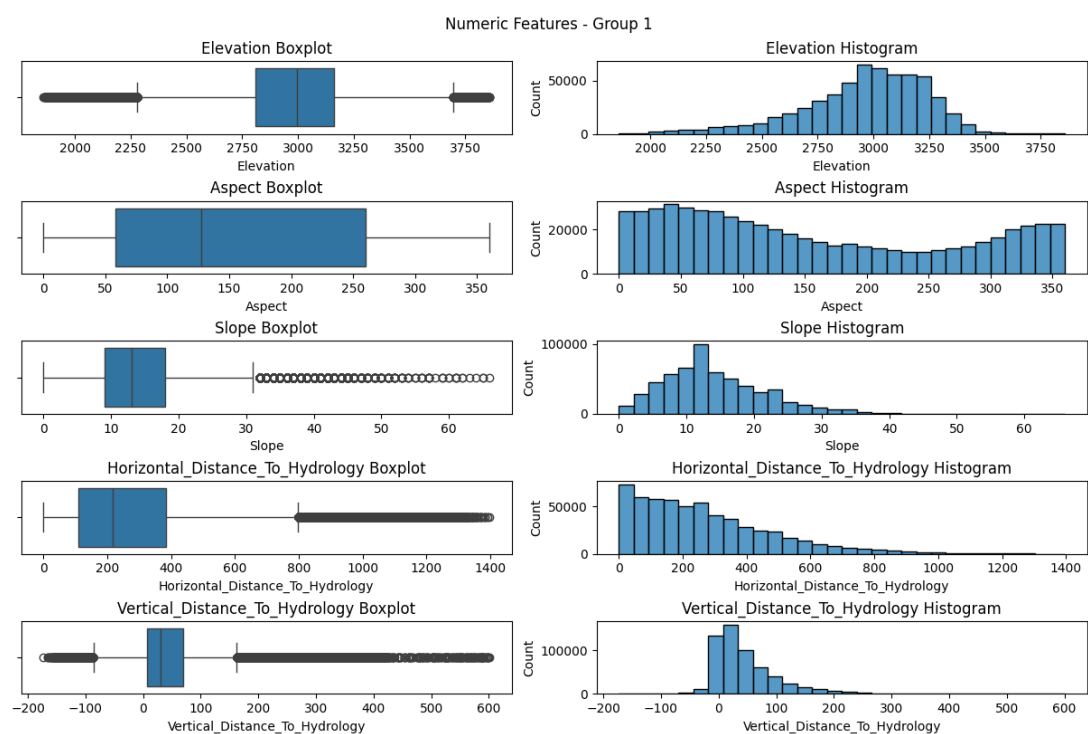
Các biến Hillshade_9am và Hillshade_Noon có phân phối lệch trái, cho thấy phần lớn các khu vực có mức độ chiếu sáng cao vào buổi sáng và giữa trưa, tương ứng với các bề mặt ít bị che khuất bởi địa hình hoặc thảm thực vật. Chỉ một số ít khu vực có mức chiếu sáng thấp, thường tương ứng với các vị trí bị che bóng hoặc địa hình phức tạp.

Các biến Elevation và Hillshade_3pm có phân phối tương đối cân bằng. Đối với Elevation, sự lệch trái nhẹ cho thấy khu vực nghiên cứu chủ yếu nằm ở độ cao trung bình đến cao, với một số ít vùng có độ cao thấp hơn. Trong khi đó, Hillshade_3pm phân bố khá đối xứng, phản ánh sự đa dạng tương đối đồng đều của điều kiện chiếu sáng vào buổi chiều trong khu vực khảo sát.

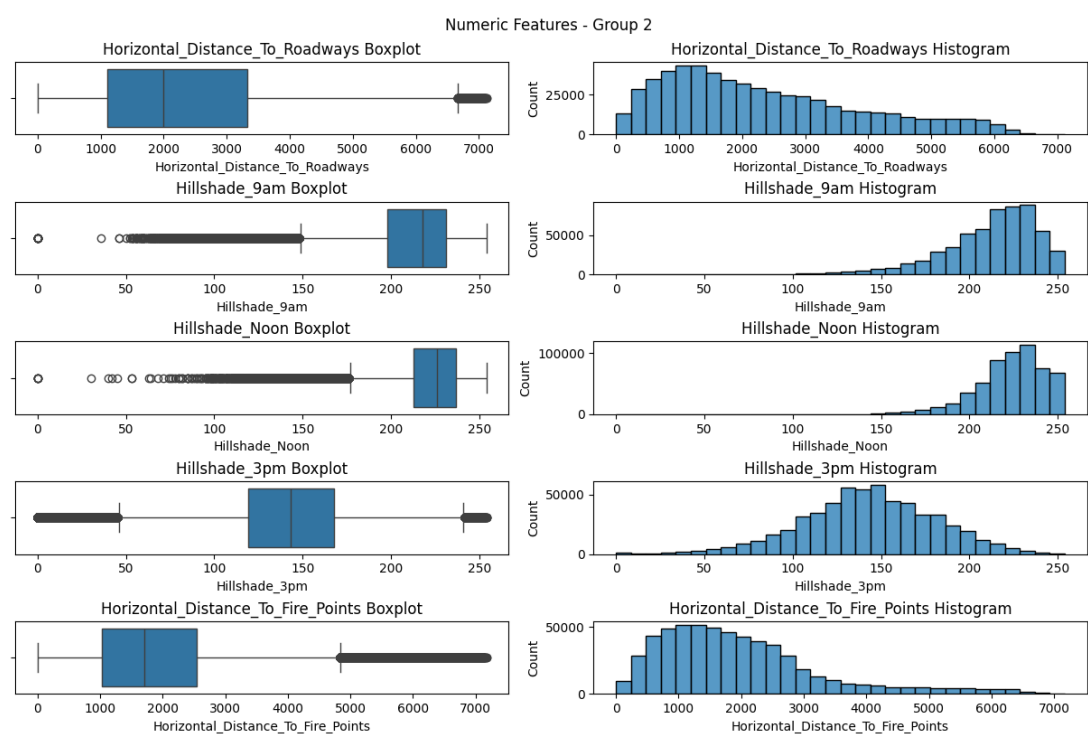
Các biến đặc trưng định tính Hình 23, 24, 25 và 26 cho ta thấy phân phối của các đặc trưng trong bộ dữ liệu Forest CoverType.

Ta thấy Wilderness_Area_0 và Wilderness_Area_2 chiếm đa số trong bộ dữ liệu. Điều này có thể do độ cao trung bình của hai khu vực này thấp (được nói ở phần giới thiệu) có diện tích phủ lớn hơn.

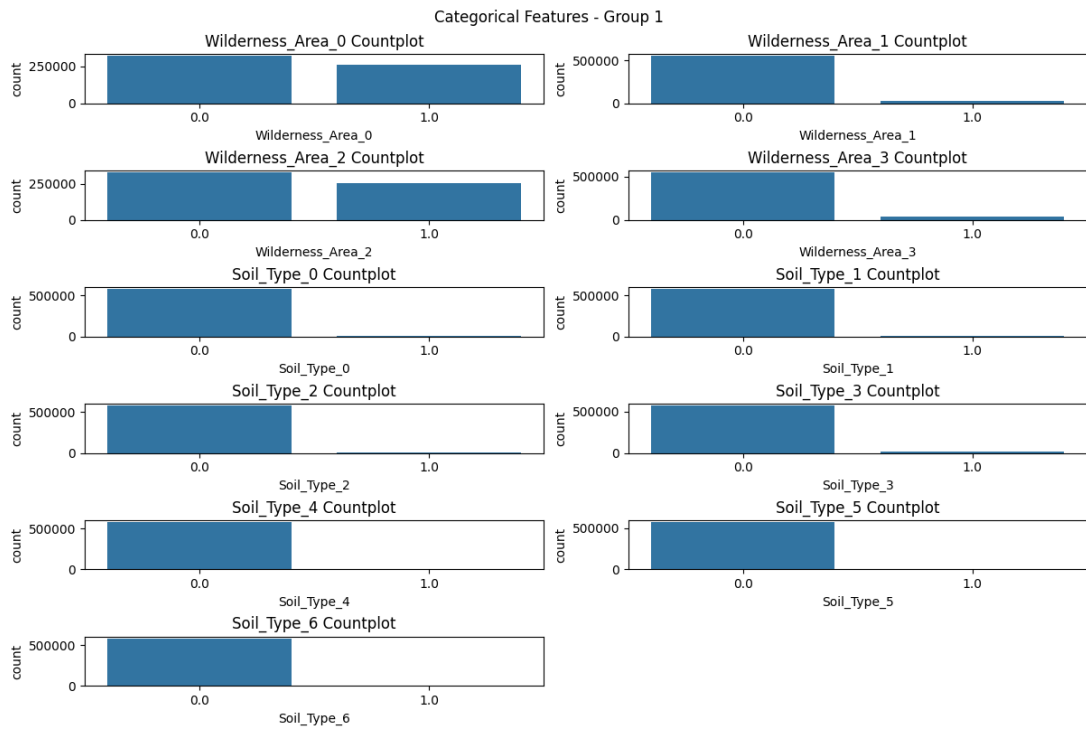
Các loại đất Soil_Type_28 và Soil_Type_22 có số lượng lớn hơn hẳn các loại đất



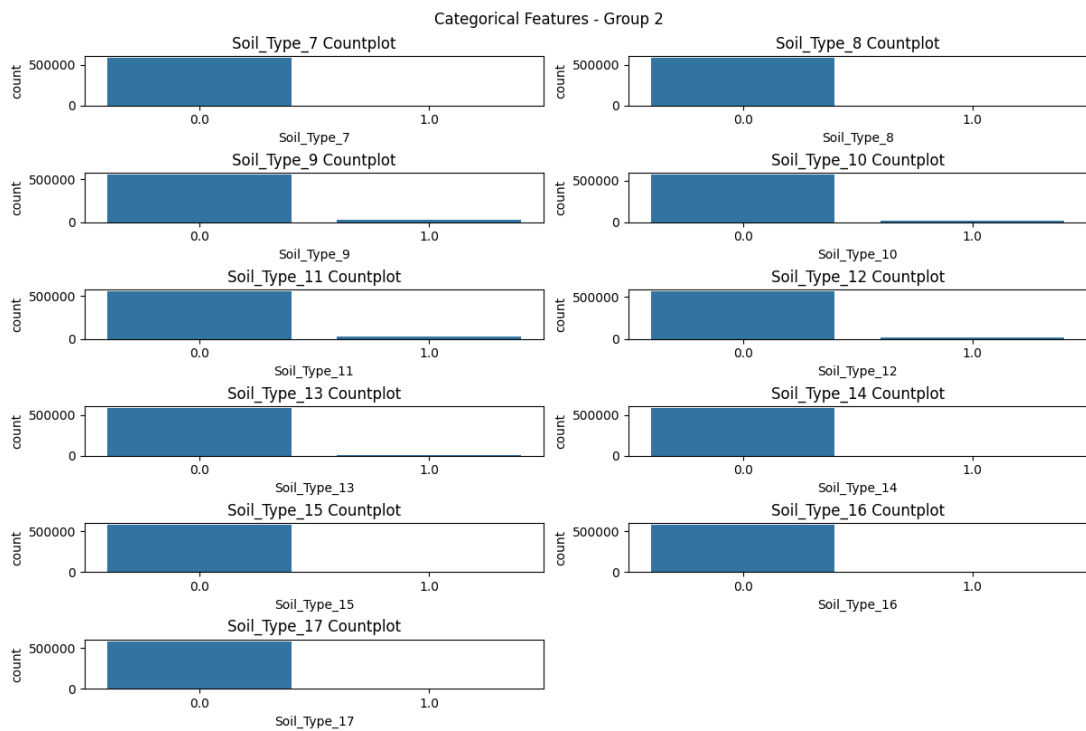
Hình 21: Phân phối các biến đặc trưng định lượng của bộ dữ liệu Forest CoverType (phần 1)



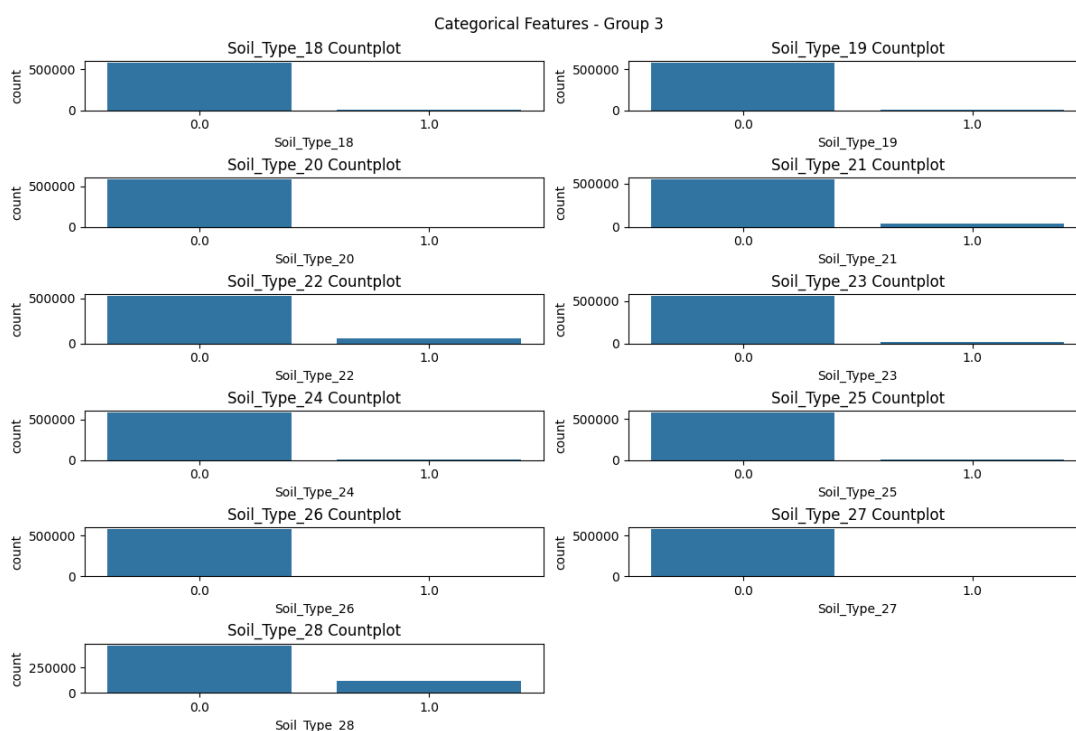
Hình 22: Phân phối các biến đặc trưng định lượng của bộ dữ liệu Forest CoverType (phần 2)



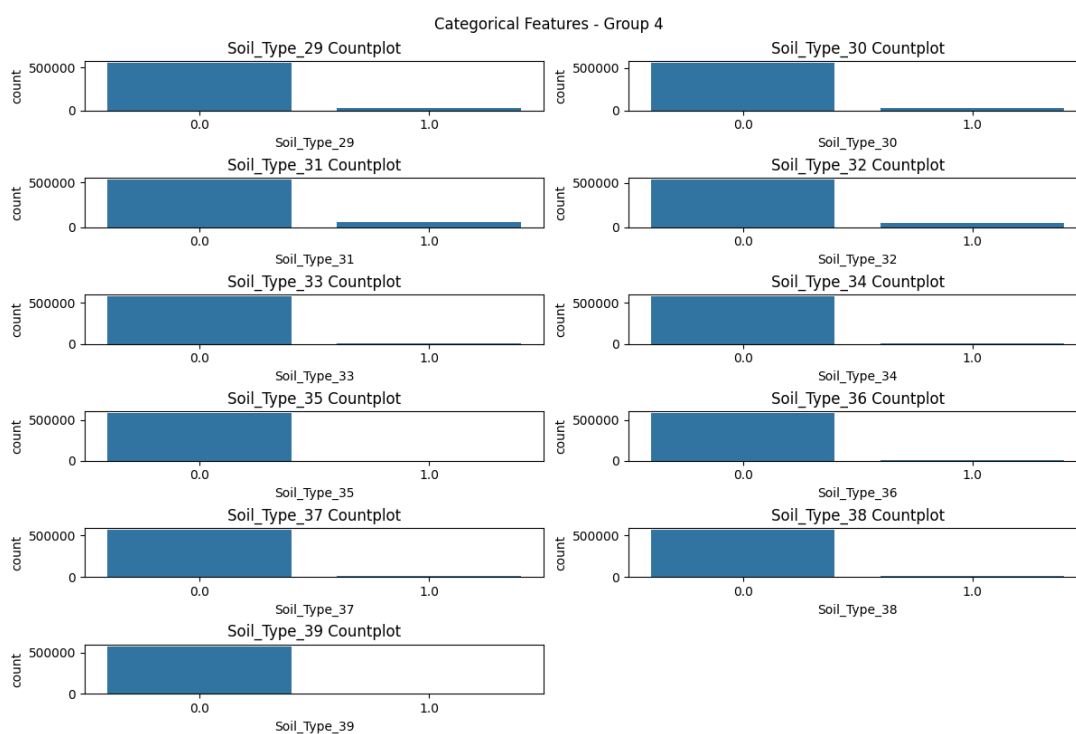
Hình 23: Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 1)



Hình 24: Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 2)



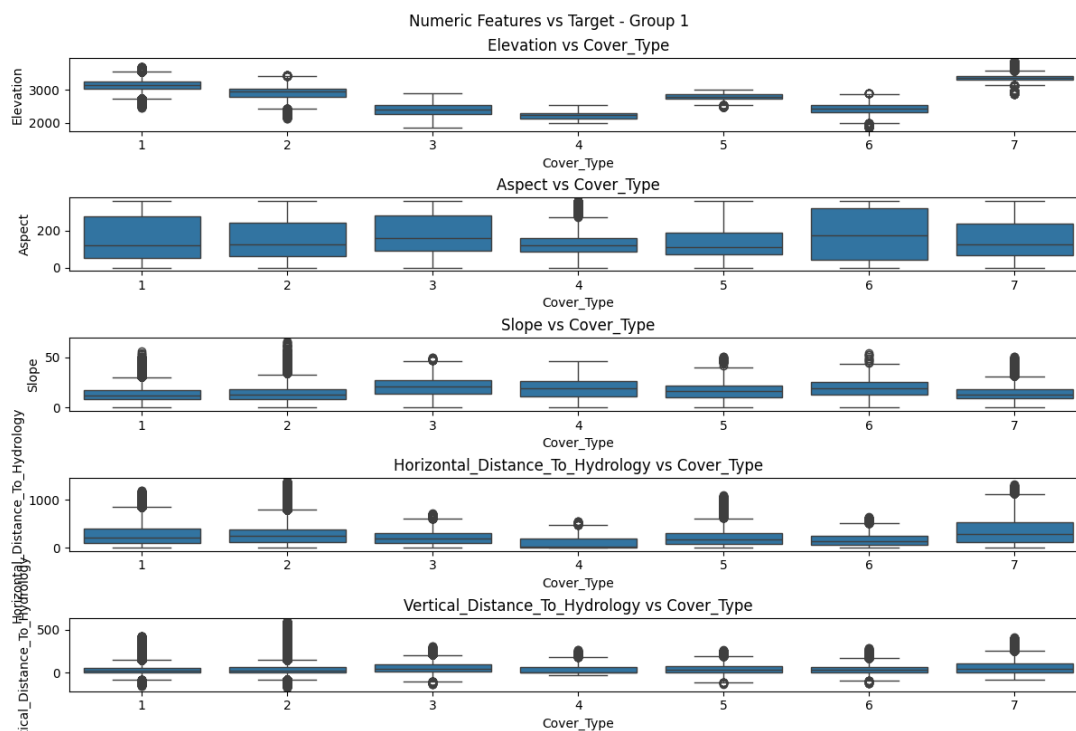
Hình 25: Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 3)



Hình 26: Biểu đồ đếm tần suất các biến đặc trưng định tính của bộ dữ liệu Forest CoverType (phần 4)

khác.

Phân bố từng từng đặc trưng định lượng theo từng lớp của biến mục tiêu
Hình 27 và 28 cho ta thấy phân bố từng đặc trưng theo từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType.



Hình 27: Biểu đồ hộp từng đặc trưng định lượng theo từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType (phần 1)

Qua 2 hình, ta thấy biến Elevation có sự phân bố khác nhau tương đối giữa các lớp. Điều này gợi ý rằng độ cao có thể là một đặc trưng quan trọng trong việc phân biệt các loại rừng phủ.

Các đặc trưng khác thì chưa thấy sự phân biệt nào rõ ràng.

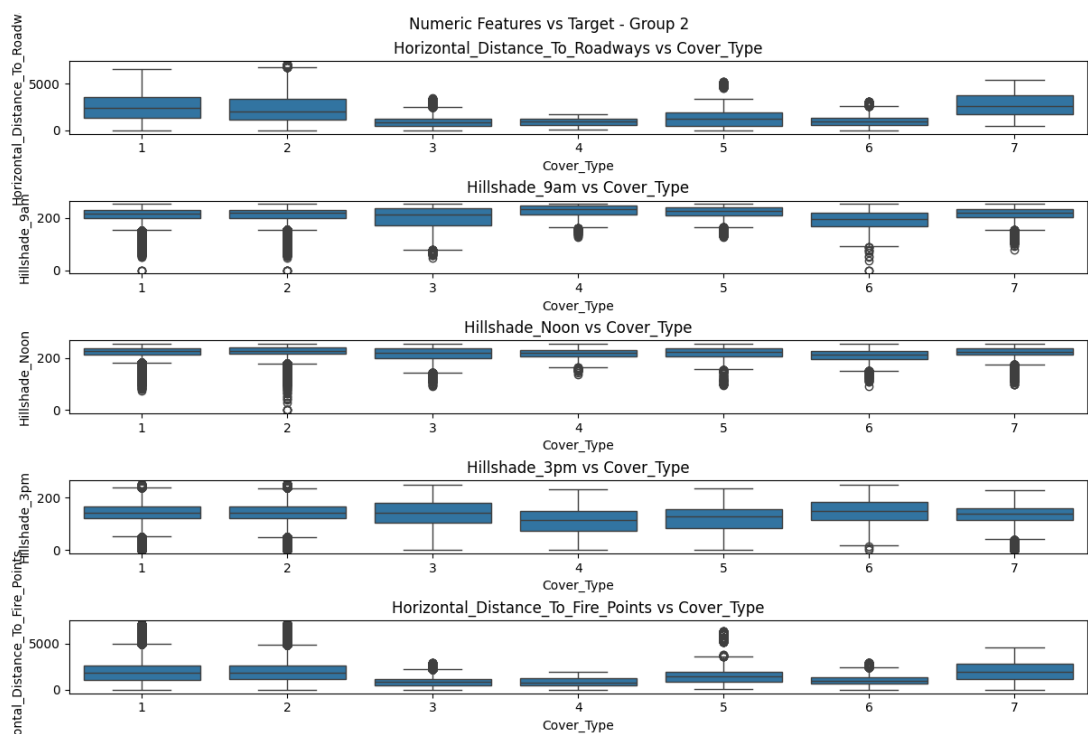
Phân bố của biến mục tiêu theo từng khu vực Hình 29 cho ta thấy phân bố của từng loại rừng phủ theo từng khu vực.

Như đã được nói ở phần giới thiệu, các loại rừng phủ chủ yếu giúp ta có thể loại trừ khả năng các loại phủ thiếu số trong khu vực đã biết. Điều này cũng cho thấy đặc trưng Wilderness_Area quan trọng trong việc phân biệt các lớp của biến mục tiêu.

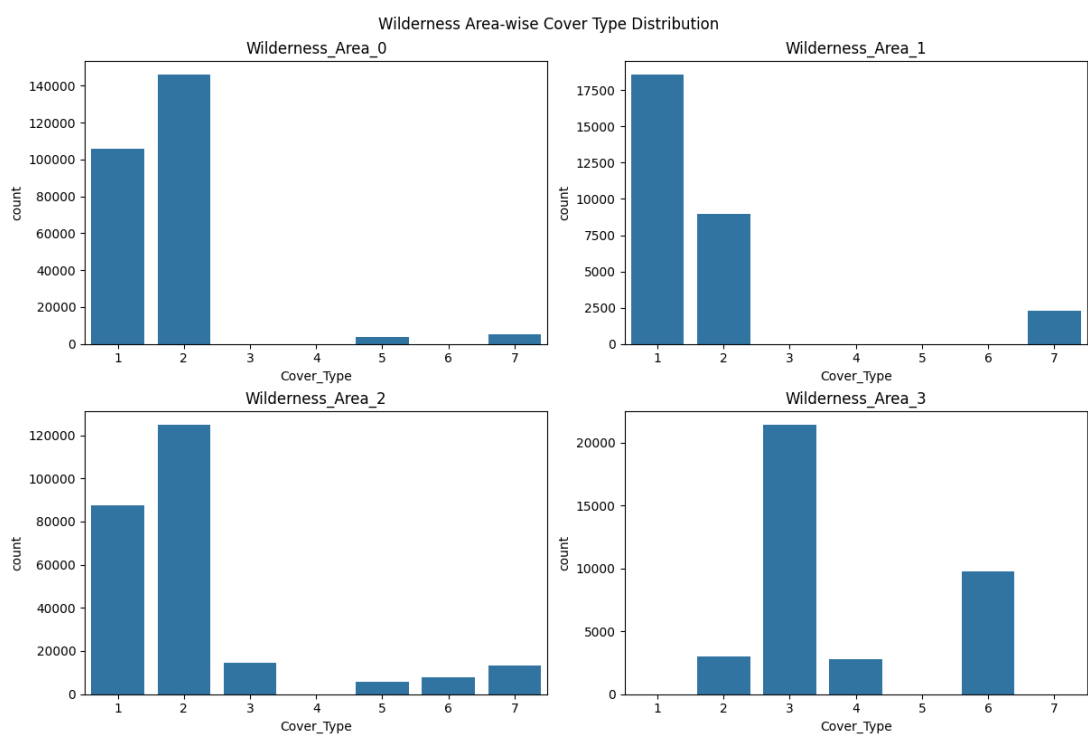
2.1.3 Tiền xử lý

Xử lý các giá trị còn thiếu Bộ dữ liệu Forest CoverType không có giá trị thiếu nên bước này được bỏ qua.

Chuẩn hóa dữ liệu Các biến đặc trưng được chuẩn hoá về trung bình $\mu = 0$ và phương sai $\sigma^2 = 1$. Điều này giúp các giá trị được đưa về cùng thang đo, giúp quá trình học bằng Gradient ổn định và nhanh hơn.



Hình 28: Biểu đồ hộp từng đặc trưng định lượng theo từng lớp của biến mục tiêu trong bộ dữ liệu Forest CoverType (phần 2)



Hình 29: Biểu đồ đếm tần suất biến mục tiêu theo từng khu vực trong bộ dữ liệu Forest CoverType

Chia dữ liệu Dữ liệu được chia thành 3 tập là huấn luyện, xác thực và tập kiểm tra với tỉ lệ tương ứng là 0.6, 0.2 và 0.2. Do dữ liệu bị mất cân bằng nên quá trình chia đã được chỉnh để giữ tỉ lệ giá trị các lớp của biến mục tiêu trên cả 3 tập.

2.2 Các mô hình

2.2.1 Hồi quy Logistic (Logistic Regression)

Hồi quy Logistic là một mô hình phân lớp tuyến tính, trong đó ranh giới quyết định được xây dựng từ một tổ hợp tuyến tính của biến đầu vào và được ánh xạ qua một hàm phi tuyến nhằm tạo đầu ra trong khoảng $(0, 1)$ đối với bài toán nhị phân, hoặc một phân phối chuẩn hóa trên nhiều lớp đối với bài toán đa lớp.

Đối với bài toán phân lớp nhị phân, mô hình xây dựng một hàm tuyến tính:

$$a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0, \quad (81)$$

sau đó ánh xạ giá trị này thông qua hàm sigmoid:

$$\sigma(a) = \frac{1}{1 + e^{-a}}. \quad (82)$$

Khi đó đầu ra của mô hình được viết là:

$$y = \sigma(\mathbf{w}^\top \mathbf{x} + w_0). \quad (83)$$

Đối với bài toán phân lớp đa lớp, mô hình sử dụng một vector điểm số (logits) cho từng lớp:

$$a_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + w_{k0}, \quad k = 1, \dots, K. \quad (84)$$

Các giá trị này được chuẩn hóa thông qua hàm softmax:

$$y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}. \quad (85)$$

Hàm mất mát được sử dụng trong huấn luyện là hàm cross-entropy:

$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_{nk}, \quad (86)$$

trong đó t_{nk} là nhãn one-hot và y_{nk} là đầu ra của mô hình.

Gradient của hàm mất mát theo tham số $\mathbf{w}_k$ được tính như sau:

$$\nabla_{\mathbf{w}_k} E = \sum_{n=1}^N (y_{nk} - t_{nk}) \mathbf{x}_n, \quad (87)$$

trong đó y_{nk} là đầu ra tại lớp k của mẫu thứ n .

Mô hình Logistic Regression có thể được xem như một mô hình tuyến tính kết hợp với một hàm kích hoạt phi tuyến, cho phép xây dựng ranh giới quyết định phức tạp hơn trong không gian đặc trưng thông qua tối ưu hóa hàm mất mát lỗi.

Thuật toán Newton–Raphson/IRLS Hồi quy Logistic không tồn tại nghiệm dạng đóng cho bài toán tối ưu tham số. Tuy nhiên, hàm mất mát cross-entropy là hàm lồi và khả vi hai lần, do đó có thể sử dụng phương pháp Newton–Raphson dựa trên khai triển Taylor bậc hai.

Trong mỗi bước lặp, tham số được cập nhật theo công thức:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{H}^{-1} \nabla E, \quad (88)$$

trong đó gradient có dạng:

$$\nabla E = \Phi^T(\mathbf{y} - \mathbf{t}), \quad (89)$$

và Hessian được viết dưới dạng:

$$\mathbf{H} = \Phi^T \mathbf{R} \Phi + \lambda \mathbf{I}, \quad (90)$$

với:

$$\mathbf{R} = \text{diag}(y_1(1 - y_1), \dots, y_N(1 - y_N)). \quad (91)$$

Thuật toán này còn được gọi là IRLS (Iterative Reweighted Least Squares), do có thể viết lại dưới dạng bài toán bình phương tối thiểu có trọng số:

$$\mathbf{w}^{(t+1)} = (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T \mathbf{R} \mathbf{z}, \quad (92)$$

với:

$$\mathbf{z} = \Phi \mathbf{w} + \mathbf{R}^{-1}(\mathbf{y} - \mathbf{p}). \quad (93)$$

IRLS thường hội tụ trong $O(\log(1/\varepsilon))$ bước, nhanh hơn Gradient Descent cần $O(1/\varepsilon)$ bước. Tuy nhiên, chi phí mỗi bước cao do cần tính nghịch đảo Hessian, với độ phức tạp xấp xỉ:

$$O(NM^2 + M^3), \quad (94)$$

trong đó N là số mẫu và M là số đặc trưng.

Jacobian của softmax Xét vector logits $\mathbf{a} = (a_1, a_2, \dots, a_K)^T$. Hàm softmax được định nghĩa bởi

$$y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}, \quad k = 1, \dots, K. \quad (95)$$

Đạo hàm riêng của phần tử đầu ra theo logits được cho bởi

$$\frac{\partial y_i}{\partial a_j} = \frac{\partial}{\partial a_j} \left(\frac{\exp(a_i)}{\sum_{k=1}^K \exp(a_k)} \right). \quad (96)$$

Đặt $Z = \sum_{k=1}^K \exp(a_k)$, khi đó $y_i = \frac{\exp(a_i)}{Z}$, ta có

$$\frac{\partial y_i}{\partial a_j} = \frac{Z \exp(a_i) \delta_{ij} - \exp(a_i) \exp(a_j)}{Z^2}. \quad (97)$$

Suy ra

$$\frac{\partial y_i}{\partial a_j} = y_i(\delta_{ij} - y_j), \quad (98)$$

trong đó δ_{ij} là delta Kronecker. Từ đó Jacobian của softmax được viết dưới dạng ma trận

$$\mathbf{J} = \text{diag}(\mathbf{y}) - \mathbf{y} \mathbf{y}^T. \quad (99)$$

Hessian của hàm mất mát theo trọng số trong softmax Ta xét quan hệ giữa logits và trọng số:

$$\mathbf{a}_n = \mathbf{W}^\top \boldsymbol{\phi}_n. \quad (100)$$

Gradient theo logits Với hàm mất mát cross-entropy:

$$E_n = - \sum_{k=1}^K t_{nk} \log y_{nk}, \quad (101)$$

ta có đạo hàm theo logits:

$$\frac{\partial E_n}{\partial a_{nk}} = y_{nk} - t_{nk}. \quad (102)$$

Hessian theo logits Lấy đạo hàm thêm lần nữa theo a_{nl} :

$$\frac{\partial^2 E_n}{\partial a_{nk} \partial a_{nl}} = \frac{\partial y_{nk}}{\partial a_{nl}}. \quad (103)$$

Với hàm softmax:

$$y_{nk} = \frac{\exp(a_{nk})}{\sum_j \exp(a_{nj})}, \quad (104)$$

ta có:

$$\frac{\partial y_{nk}}{\partial a_{nl}} = y_{nk}(\delta_{kl} - y_{nl}). \quad (105)$$

Do đó, Hessian theo logits có dạng:

$$\mathbf{H}_n^{(a)} = \frac{\partial^2 E_n}{\partial \mathbf{a}_n \partial \mathbf{a}_n^\top} = \text{diag}(\mathbf{y}_n) - \mathbf{y}_n \mathbf{y}_n^\top. \quad (106)$$

Chuyển sang Hessian theo trọng số Áp dụng quy tắc móc xích bậc hai:

$$\mathbf{H} = \sum_{n=1}^N \left(\frac{\partial \mathbf{a}_n}{\partial \mathbf{W}} \right)^\top \mathbf{H}_n^{(a)} \left(\frac{\partial \mathbf{a}_n}{\partial \mathbf{W}} \right). \quad (107)$$

Do $\mathbf{a}_n$ là hàm tuyến tính theo $\mathbf{W}$, đạo hàm bậc hai của $\mathbf{a}_n$ bằng 0.

Với:

$$\frac{\partial \mathbf{a}_n}{\partial \mathbf{W}} = \boldsymbol{\phi}_n, \quad (108)$$

suy ra:

$$\mathbf{H} = \sum_{n=1}^N \boldsymbol{\phi}_n \boldsymbol{\phi}_n^\top \otimes \mathbf{H}_n^{(a)}, \quad (109)$$

trong đó $\otimes$ kí hiệu tích Kronecker.

Dạng block của Hessian Hessian có thể được biểu diễn dưới dạng ma trận block, với mỗi block (k, l) :

$$\mathbf{H}_{kl} = \sum_{n=1}^N y_{nk}(\delta_{kl} - y_{nl}) \boldsymbol{\phi}_n \boldsymbol{\phi}_n^\top, \quad (110)$$

trong đó δ_{kl} là delta Kronecker.

Dạng ma trận gọn Gọi $\Phi \in \mathbb{R}^{N \times D}$ là ma trận thiết kế. Khi đó:

$$\mathbf{H} = (\mathbf{I}_K \otimes \Phi)^\top \mathbf{R} (\mathbf{I}_K \otimes \Phi), \quad (111)$$

trong đó $\mathbf{R}$ là ma trận block-diagonal:

$$\mathbf{R} = \text{blockdiag}(\mathbf{H}_1^{(a)}, \dots, \mathbf{H}_N^{(a)}). \quad (112)$$

So sánh các chiến lược One-vs-Rest, One-vs-One và Softmax trực tiếp Bảng kết quả 24 cho thấy sự khác biệt rõ ràng giữa ba chiến lược phân loại đa lớp gồm One-vs-Rest (OvR), One-vs-One (OvO) và Softmax trực tiếp.

Bảng 24: Các chỉ số đánh giá của 3 chiến lược One-vs-Rest, One-vs-One và softmax trực tiếp trên tập kiểm tra của bộ dữ liệu Forest CoverType

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
OvR Sigmoid	0.7073	0.5743	0.3959	0.3864
OvO Sigmoid	0.7162	0.564	0.4249	0.4185
Softmax	0.7148	0.5901	0.4306	0.4319

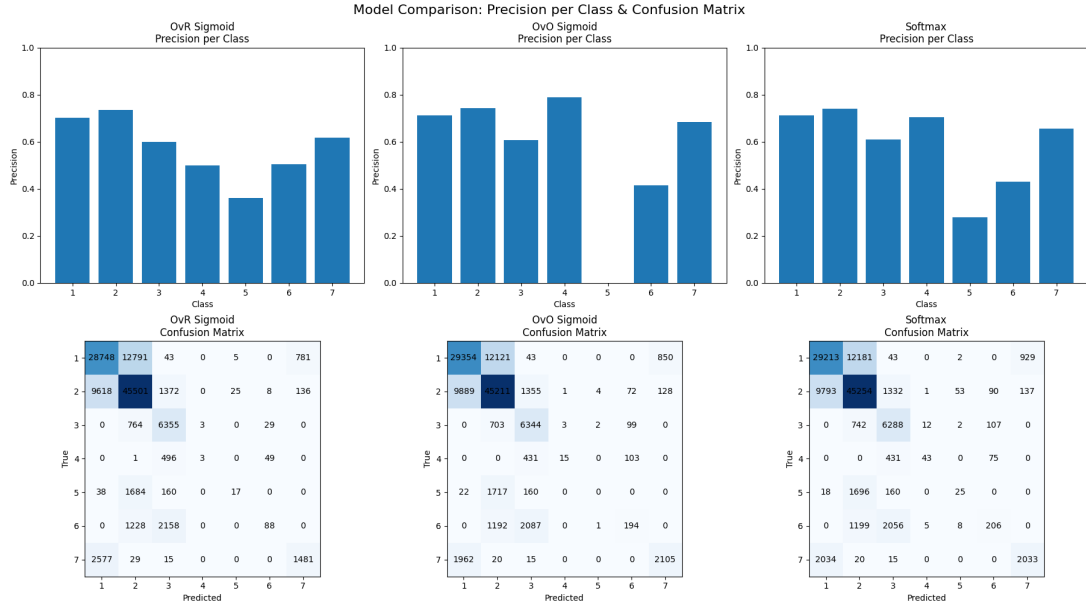
Về độ chính xác tổng thể (Accuracy), OvO Sigmoid đạt 0.7162, cao nhất nhưng chỉ nhỉnh hơn Softmax (0.7148) và OvR (0.7073) không đáng kể. Tuy nhiên, khi xét các chỉ số cân bằng hơn như Precision, Recall và F1-score macro, Softmax thể hiện hiệu quả ổn định và tốt nhất với Precision bằng 0.5901, Recall bằng 0.4306 và F1-score bằng 0.4319.

OvO Sigmoid cho thấy cải thiện nhẹ về Recall so với OvR (0.4249 so với 0.3959), đồng thời F1-score cũng cao hơn (0.4185 so với 0.3864), phản ánh khả năng cân bằng tốt hơn giữa các lớp so với OvR. OvR lại có hiệu suất thấp nhất ở cả Recall và F1-score, cho thấy hạn chế trong việc xử lý mất cân bằng giữa các lớp khi huấn luyện theo từng lớp độc lập.

Nhìn chung, Softmax trực tiếp là lựa chọn ổn định hơn khi xét toàn bộ các chỉ số macro, nhờ khả năng tối ưu hóa đồng thời xác suất trên tất cả lớp, giúp giảm xung đột giữa các bộ phân loại nhị phân như trong OvR và OvO. OvO có ưu thế nhẹ về Accuracy nhưng không vượt trội đáng kể, trong khi OvR tỏ ra kém hiệu quả nhất trong bối cảnh bài toán này.

Hình 30 độ chính xác trên từng lớp và ma trận nhầm lẫn trên tập kiểm tra của 3 chiến lược. Ta thấy rằng độ chính xác của Softmax khá ổn định. Trong khi đó, OvO gặp khó ở lớp 5 với độ chính xác bằng 0. Điều này cũng đã được thể hiện khi độ chính xác theo trung bình macro của OvO là thấp nhất (0.564). Còn OvR bị gặp khó ở lớp 4 khi độ chính xác tụt thấp hơn hẳn so với OvO và Softmax. Như nhìn chung, OvR có độ chính xác khá ổn định giữa các lớp. Nhìn chung, ma trận nhầm lẫn giữa 3 chiến lược khá tương đồng. OvR thì hay nhầm lẫn giữa lớp 7 thành 1 hơn. OvR và Softmax thì ít nhầm lẫn đặc biệt hơn.

Kết quả thực nghiệm cho thấy Softmax trực tiếp là phương pháp ổn định nhất khi xét theo các chỉ số macro, trong khi OvO Sigmoid đạt Accuracy cao nhất nhưng không vượt trội đáng kể và thiếu tính ổn định trên từng lớp. Đáng chú ý, OvO không cải thiện hiệu năng trên các lớp khó như kỳ vọng, thậm chí còn thất bại hoàn toàn ở một số lớp, cho thấy hạn chế của cách tiếp cận phân rã theo từng cặp trong bối cảnh dữ liệu có sự chồng lấn đa lớp. Trong khi đó, OvR Sigmoid cho kết quả tương đối ổn định nhưng mang tính bảo thủ, dẫn đến Recall và F1-score thấp hơn.



Hình 30: Biểu đồ độ chính xác từng lớp và ma trận nhầm lẫn của 3 chiến lược OvR, OvO và Softmax trực tiếp trên tập kiểm tra của bộ dữ liệu Forest CoverType

Xét về mặt lý thuyết, OvR và OvO lần lượt yêu cầu K và $K(K-1)/2$ mô hình, trong khi Softmax chỉ cần một mô hình duy nhất, mang lại lợi thế rõ rệt về chi phí tính toán. Tuy OvO có khả năng học ranh giới phân lớp chi tiết theo từng cặp, kết quả thực nghiệm cho thấy lợi thế này không được phát huy trong trường hợp cụ thể, có thể do cấu trúc dữ liệu không phù hợp với giả định phân rã cặp.

Nhìn chung, Softmax là lựa chọn phù hợp nhất cho bài toán này nhờ sự cân bằng giữa hiệu năng và độ phức tạp.

2.2.2 Linear Discriminant Analysis

Linear Discriminant Analysis Linear Discriminant Analysis (LDA) có thể được hiểu dưới hai góc nhìn tương đương: góc nhìn xác suất (probabilistic) và góc nhìn hình học (projection).

Góc nhìn xác suất LDA giả định rằng dữ liệu của mỗi lớp tuân theo phân phối Gaussian đa biến với cùng ma trận hiệp phương sai:

$$\mathbf{x} \mid t = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}). \quad (113)$$

Dựa trên định lý Bayes, ta thu được hàm phân biệt tuyến tính:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k. \quad (114)$$

Mẫu $\mathbf{x}$ được gán vào lớp có giá trị $\delta_k(\mathbf{x})$ lớn nhất. Do các lớp chia sẻ cùng $\boldsymbol{\Sigma}$, biên quyết định là tuyến tính.

Góc nhìn hình học Từ góc nhìn hình học, LDA tìm một phép chiếu tuyến tính $\mathbf{w}$ sao cho khi chiếu dữ liệu:

$$z = \mathbf{w}^\top \mathbf{x}, \quad (115)$$

các lớp được phân tách tốt nhất.

Tiêu chí tối ưu là cực đại hóa tỷ lệ giữa phương sai giữa các lớp và phương sai trong lớp:

$$\mathbf{w}^* = \arg \max_{\mathbf{w}} \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}. \quad (116)$$

Trong đó:

- $\mathbf{S}_B$ (between-class variance) đo mức độ phân tách giữa các lớp, được định nghĩa:

$$\mathbf{S}_B = \sum_{k=1}^K N_k (\boldsymbol{\mu}_k - \boldsymbol{\mu})(\boldsymbol{\mu}_k - \boldsymbol{\mu})^\top, \quad (117)$$

với:

- K là số lớp
 - N_k là số mẫu của lớp k
 - $\boldsymbol{\mu}_k$ là trung bình của lớp k
 - $\boldsymbol{\mu}$ là trung bình toàn bộ dữ liệu
- $\mathbf{S}_W$ (within-class variance) đo mức độ phân tán của các điểm trong từng lớp:

$$\mathbf{S}_W = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top, \quad (118)$$

trong đó C_k là tập các mẫu thuộc lớp k .

Thực quan:

- $\mathbf{S}_B$ càng lớn $\Rightarrow$ các lớp càng tách xa nhau
- $\mathbf{S}_W$ càng nhỏ $\Rightarrow$ các điểm trong lớp càng tập trung

Do đó, LDA tìm $\mathbf{w}$ sao cho:

- khoảng cách giữa các lớp là lớn nhất
- độ phân tán trong mỗi lớp là nhỏ nhất

Nhận xét Hai cách tiếp cận trên là tương đương và dẫn đến cùng một nghiệm. Góc nhìn xác suất giúp diễn giải LDA như một mô hình sinh (generative model), trong khi góc nhìn hình học cho thấy LDA là một phương pháp giảm chiều có giám sát, tối đa hóa khả năng phân tách giữa các lớp.

Trong thực tế, LDA hoạt động tốt khi dữ liệu gần với giả định Gaussian và các lớp có phương sai tương tự nhau, nhưng có thể kém hiệu quả khi các giả định này bị vi phạm hoặc khi quan hệ giữa các lớp là phi tuyến.

Quadratic Discriminant Analysis Quadratic Discriminant Analysis (QDA) là một phương pháp phân loại dựa trên mô hình xác suất, tương tự như LDA, nhưng cho phép mỗi lớp có ma trận hiệp phương sai riêng. Điều này giúp QDA linh hoạt hơn trong việc mô hình hóa dữ liệu.

Góc nhìn xác suất QDA giả định rằng dữ liệu của mỗi lớp tuân theo phân phối Gaussian đa biến:

$$\mathbf{x} \mid t = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \quad (119)$$

trong đó mỗi lớp k có ma trận hiệp phương sai riêng $\boldsymbol{\Sigma}_k$.

Hàm phân biệt khi đó có dạng:

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \log |\boldsymbol{\Sigma}_k| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k. \quad (120)$$

Do sự khác biệt giữa các $\boldsymbol{\Sigma}_k$, biên quyết định của QDA là phi tuyến (cụ thể là bậc hai).

Góc nhìn hình học Từ góc nhìn hình học, QDA không tìm một phép chiếu tuyến tính chung như LDA, mà thay vào đó học trực tiếp các biên quyết định phi tuyến giữa các lớp. Các biên này thường có dạng các mặt bậc hai (quadratic surfaces), cho phép mô hình thích ứng tốt hơn với dữ liệu có cấu trúc phức tạp và phân phối khác nhau giữa các lớp.

Nhận xét QDA phù hợp khi dữ liệu có cấu trúc phức tạp và phương sai giữa các lớp khác nhau đáng kể. Tuy nhiên, do số lượng tham số cần ước lượng lớn hơn (mỗi lớp một ma trận hiệp phương sai), QDA yêu cầu nhiều dữ liệu hơn để đạt hiệu quả tốt và dễ bị overfitting trong trường hợp dữ liệu hạn chế.

Tóm lại, QDA là một mở rộng linh hoạt của LDA, thể hiện rõ sự đánh đổi giữa độ phức tạp của mô hình và khả năng tổng quát hóa.

So sánh LDA và QDA Linear Discriminant Analysis (LDA) và Quadratic Discriminant Analysis (QDA) đều là các phương pháp phân loại dựa trên mô hình xác suất, với giả định rằng dữ liệu của mỗi lớp tuân theo phân phối Gaussian đa biến. Điểm khác biệt cốt lõi giữa hai phương pháp nằm ở giả định về ma trận hiệp phương sai, dẫn đến sự khác biệt về dạng biên quyết định, số lượng tham số và khả năng tổng quát hóa.

So sánh tổng quan Bảng 25 trình bày các đặc điểm chính của hai phương pháp.

Bảng 25: So sánh giữa LDA và QDA

Tiêu chí	LDA	QDA
Giả định hiệp phương sai	Chung ($\boldsymbol{\Sigma}$)	Riêng ($\boldsymbol{\Sigma}_k$)
Biên quyết định	Tuyến tính	Phi tuyến (bậc hai)
Số tham số	Ít	Nhiều
Nguy cơ overfitting	Thấp	Cao
Yêu cầu dữ liệu	Ít	Nhiều
Độ linh hoạt	Thấp	Cao

Phân tích theo số mẫu và số chiều Hiệu quả của LDA và QDA phụ thuộc mạnh vào mối quan hệ giữa số mẫu n và số chiều d của dữ liệu:

- **Trường hợp $n \gg d$:**

- Khi số mẫu lớn hơn nhiều so với số chiều, việc ước lượng các ma trận hiệp phương sai riêng Σ_k trong QDA trở nên chính xác.
- Khi đó, QDA có thể tận dụng sự linh hoạt của mình để mô hình hóa tốt các biên quyết định phi tuyến, thường cho hiệu năng cao hơn LDA.

- **Trường hợp n nhỏ hoặc $n \approx d$:**

- Việc ước lượng Σ_k trong QDA trở nên không ổn định và dễ dẫn đến ma trận suy biến.
- LDA có lợi thế vì chỉ cần ước lượng một ma trận hiệp phương sai chung Σ , do đó ít tham số hơn và ổn định hơn.
- Trong trường hợp này, LDA thường tổng quát hóa tốt hơn.

- **Trường hợp $d \gg n$:**

- Cả LDA và QDA đều gặp khó khăn trong việc ước lượng ma trận hiệp phương sai.
- Tuy nhiên, QDA bị ảnh hưởng nghiêm trọng hơn do cần ước lượng nhiều ma trận Σ_k .
- Vì vậy, LDA thường là lựa chọn phù hợp hơn.

Kết luận Tóm lại, việc lựa chọn giữa LDA và QDA phụ thuộc vào đặc điểm của dữ liệu:

- LDA phù hợp khi dữ liệu nhỏ, số chiều cao hoặc các lớp có phương sai tương tự nhau.
- QDA phù hợp khi dữ liệu đủ lớn và tồn tại sự khác biệt rõ rệt giữa phân phối của các lớp.

Sự lựa chọn này phản ánh sự đánh đổi giữa độ phức tạp của mô hình và khả năng tổng quát hóa.

Fisher Ratio Fisher Ratio là một tiêu chí đánh giá mức độ phân biệt của từng đặc trưng trong bài toán phân loại. Ý tưởng chính là đo lường mức độ tách biệt giữa các lớp so với mức độ phân tán bên trong mỗi lớp.

Định nghĩa Với mỗi đặc trưng j , Fisher Ratio được định nghĩa là:

$$J_j = \frac{\sum_k n_k (\mu_{k,j} - \mu_j)^2}{\sum_k n_k \sigma_{k,j}^2}, \quad (121)$$

trong đó:

- n_k là số lượng mẫu của lớp k
- $\mu_{k,j}$ là giá trị trung bình của đặc trưng j trong lớp k
- μ_j là giá trị trung bình của đặc trưng j trên toàn bộ dữ liệu
- $\sigma_{k,j}^2$ là phương sai của đặc trưng j trong lớp k

Ý nghĩa Fisher Ratio thể hiện sự cân bằng giữa:

- **Between-class variance** (tử số): đo mức độ khác biệt giữa các lớp
- **Within-class variance** (mẫu số): đo mức độ phân tán trong từng lớp
- Nếu J_j lớn: đặc trưng j giúp phân tách các lớp tốt (mean khác nhau rõ rệt, variance trong lớp nhỏ)
- Nếu J_j nhỏ: đặc trưng ít hữu ích cho phân loại

Liên hệ với LDA Fisher Ratio chính là dạng đơn giản (theo từng đặc trưng độc lập) của tiêu chí Fisher được sử dụng trong Linear Discriminant Analysis (LDA):

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}. \quad (122)$$

Trong khi LDA tìm một phép chiếu tối ưu $\mathbf{w}$ trong không gian nhiều chiều, Fisher Ratio đánh giá trực tiếp từng đặc trưng riêng lẻ.

Ứng dụng Fisher Ratio thường được sử dụng trong:

- **Lựa chọn đặc trưng**: chọn các đặc trưng có J_j lớn
- **Phân tích dữ liệu**: đánh giá mức độ phân biệt của từng biến đầu vào

Nhận xét Fisher Ratio là một tiêu chí đơn giản và hiệu quả, đặc biệt phù hợp khi cần đánh giá nhanh tầm quan trọng của các đặc trưng. Tuy nhiên, do xét từng đặc trưng độc lập, phương pháp này không nắm bắt được mối quan hệ giữa các đặc trưng, do đó có thể bỏ sót các tương tác quan trọng trong dữ liệu.

Thí nghiệm và kết quả

Fisher ratio Để đánh giá mức độ phân biệt của từng đặc trưng, ta sử dụng tiêu chí Fisher ratio. Giá trị Fisher ratio càng lớn cho thấy đặc trưng đó có khả năng phân tách các lớp càng tốt.

Bảng 26 trình bày 5 đặc trưng có giá trị Fisher ratio cao nhất.

Từ kết quả trên, có thể thấy rằng đặc trưng *Elevation* có giá trị Fisher ratio cao nhất, cho thấy đây là biến quan trọng nhất trong việc phân biệt các lớp. Điều này gợi ý rằng độ cao có ảnh hưởng đáng kể đến sự khác biệt giữa các nhóm dữ liệu.

Ngoài ra, các đặc trưng như *Wilderness_Area_3* cũng có giá trị tương đối cao, cho thấy vai trò đáng kể trong việc phân tách lớp. Ngược lại, các đặc trưng thuộc nhóm *Soil Type* có giá trị Fisher ratio thấp hơn, cho thấy khả năng phân biệt yếu hơn khi xét riêng lẻ.

Bảng 26: Top 5 đặc trưng theo Fisher ratio

Feature	Fisher Ratio
Elevation	1.6043
Wilderness_Area_3	1.1836
Soil_Type_9	0.2889
Soil_Type_3	0.1328
Soil_Type_37	0.1295

Nhận xét Kết quả này cho thấy rằng chỉ một số ít đặc trưng đóng vai trò quan trọng trong việc phân loại, trong khi nhiều đặc trưng khác có đóng góp hạn chế. Điều này gợi ý rằng có thể áp dụng các phương pháp lựa chọn đặc trưng (feature selection) để giảm số chiều của dữ liệu mà vẫn giữ được hiệu năng của mô hình.

Tuy nhiên, cần lưu ý rằng Fisher ratio đánh giá từng đặc trưng một cách độc lập, do đó không phản ánh được mối quan hệ hoặc tương tác giữa các đặc trưng. Vì vậy, việc kết hợp với các phương pháp khác (như Lasso hoặc các mô hình học máy) có thể mang lại cái nhìn toàn diện hơn.

Đường biên quyết định của LDA Hình 31 minh họa đường biên quyết định của mô hình LDA sau khi chiếu dữ liệu xuống không gian hai chiều. Có thể thấy rằng các biên quyết định của LDA là các đường thẳng, phù hợp với giả định rằng các lớp chia sẻ cùng ma trận hiệp phương sai. Điều này dẫn đến việc không gian đặc trưng được chia thành các vùng tuyến tính, mỗi vùng tương ứng với một lớp. Trong hình, các lớp dữ liệu có xu hướng phân bố dọc theo một cấu trúc gần tuyến tính, do đó LDA vẫn có khả năng phân tách tương đối tốt ở một số vùng. Tuy nhiên, ở các khu vực mà các lớp chồng lấn mạnh hoặc có cấu trúc phi tuyến, các biên tuyến tính của LDA không đủ linh hoạt để phân tách hoàn toàn, dẫn đến sự xen lẫn giữa các lớp. Điều này phản ánh hạn chế của LDA là mô hình có độ linh hoạt thấp (high bias), nên mặc dù ổn định và tổng quát hóa tốt, nhưng có thể không đạt hiệu năng tối ưu khi dữ liệu có ranh giới phân lớp phức tạp.

2.2.3 Perceptron

Giới thiệu Perceptron là một trong những mô hình phân loại tuyến tính nền tảng trong học máy, được đề xuất bởi Frank Rosenblatt vào năm 1957. Đây là một thuật toán học có giám sát đơn giản nhưng đóng vai trò quan trọng trong sự phát triển của các mô hình học sâu hiện đại. Mục tiêu của Perceptron là tìm một siêu phẳng trong không gian đặc trưng để phân tách hai lớp dữ liệu.

Mô hình toán học Cho tập dữ liệu huấn luyện:

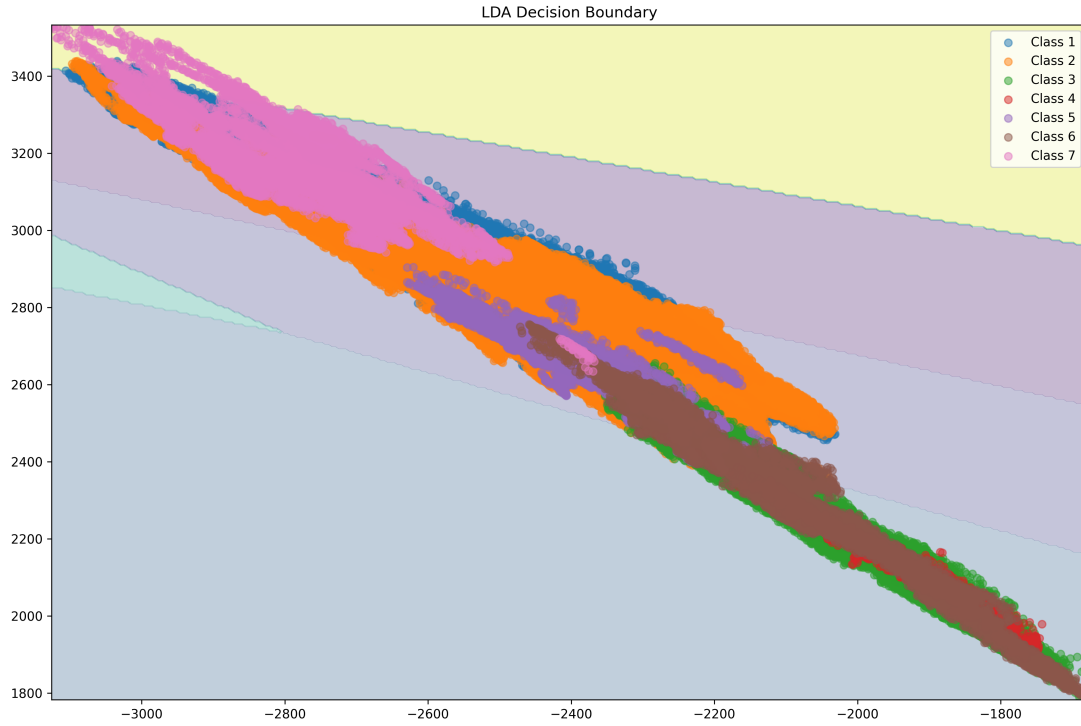
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad y_i \in \{-1, +1\}, \quad \mathbf{x}_i \in \mathbb{R}^d$$

Perceptron xây dựng một hàm quyết định tuyến tính dưới dạng:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

và dự đoán nhãn thông qua:

$$\hat{y} = \text{sign}(f(\mathbf{x}))$$



Hình 31: Đường biên quyết định của LDA trong không gian hai chiều

Siêu phẳng quyết định $\mathbf{w}^T \mathbf{x} + b = 0$ chia không gian đặc trưng thành hai nửa không gian, tương ứng với hai lớp. Vector $\mathbf{w}$ xác định hướng pháp tuyến của siêu phẳng, trong khi b điều khiển vị trí dịch chuyển của nó.

Nguyên lý học Perceptron có thể được diễn giải như một thuật toán tối ưu hóa nhằm tìm kiếm một siêu phẳng phân tách sao cho:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) > 0, \quad \forall i$$

Khi điều kiện này bị vi phạm (tức là một điểm bị phân loại sai hoặc nằm trên ranh giới), tham số được cập nhật theo quy tắc:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \quad b \leftarrow b + \eta y_i$$

Quy tắc này có thể được hiểu dưới góc nhìn hình học: mỗi lần cập nhật sẽ “đẩy” siêu phẳng theo hướng làm tăng giá trị $y_i(\mathbf{w}^T \mathbf{x}_i + b)$, tức là đưa điểm sai về phía đúng của ranh giới phân lớp.

Từ góc nhìn tối ưu hóa, Perceptron tương đương với việc thực hiện gradient descent trên một hàm mất mát không trơn:

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \max(0, -y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

được gọi là *Perceptron loss*.

Tính chất lý thuyết

- **Định lý hội tụ (Perceptron Convergence Theorem):** Nếu tồn tại một siêu phẳng phân tách dữ liệu với biên $\gamma > 0$, thì thuật toán Perceptron sẽ hội tụ sau hữu hạn số lần cập nhật. Số bước lặp bị chặn trên bởi:

$$O\left(\frac{R^2}{\gamma^2}\right)$$

trong đó $R = \max_i \|\mathbf{x}_i\|$.

- **Không tối ưu biên (non-margin maximizing):** Không giống như SVM, Perceptron không tìm siêu phẳng có biên lớn nhất, mà chỉ tìm một nghiệm bất kỳ thỏa mãn điều kiện phân tách.
- **Giới hạn biểu diễn:** Do giả định tuyến tính, Perceptron không thể học các cấu trúc dữ liệu phi tuyến. Điều này giới hạn khả năng áp dụng trong các bài toán thực tế có ranh giới phức tạp.
- **Nhạy cảm với thứ tự dữ liệu:** Quá trình hội tụ và nghiệm thu được có thể phụ thuộc vào thứ tự duyệt dữ liệu, do bản chất cập nhật online của thuật toán.

Thực nghiệm Trong thực nghiệm, chúng tôi đánh giá Perceptron trên hai loại dữ liệu:

- Dữ liệu tuyến tính (linearly separable)
- Dữ liệu phi tuyến (non-linearly separable)

Kết quả được minh họa trong Hình 32.

Phân tích Kết quả thực nghiệm cho thấy:

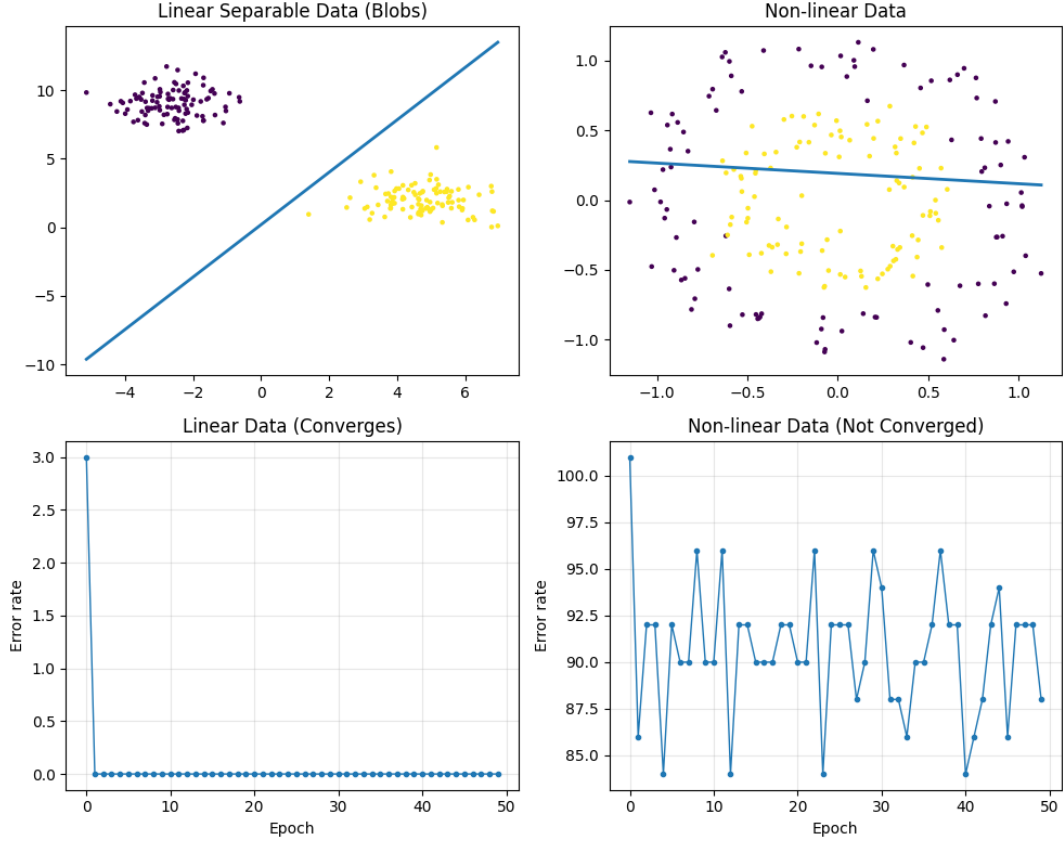
- Với dữ liệu tuyến tính, Perceptron nhanh chóng hội tụ và tìm được một ranh giới phân lớp hiệu quả.
- Với dữ liệu phi tuyến, mô hình không thể tìm được siêu phẳng phân tách phù hợp, dẫn đến việc sai số dao động và không hội tụ.

Điều này phản ánh rõ hạn chế cốt lõi của Perceptron: chỉ phù hợp với các bài toán có cấu trúc tuyến tính.

Kết luận Perceptron là một mô hình cơ bản nhưng mang tính nền tảng trong học máy. Mặc dù có những hạn chế về khả năng biểu diễn, nó đặt nền móng cho các kiến trúc phức tạp hơn như mạng nơ-ron nhiều lớp (Multi-layer Perceptron) và các phương pháp kernel, giúp mở rộng khả năng học các ranh giới phi tuyến trong không gian đặc trưng.

2.2.4 Regularization trong Logistic Regression

Giới thiệu Trong Logistic Regression, regularization đóng vai trò quan trọng trong việc kiểm soát độ phức tạp của mô hình và cải thiện khả năng tổng quát hóa. Bằng cách bổ sung một hạng phạt vào hàm mất mát, mô hình bị ràng buộc không học các tham số quá lớn, từ đó giảm hiện tượng overfitting và tăng tính ổn định khi làm việc với dữ liệu thực tế.



Hình 32: Kết quả Perceptron trên dữ liệu tuyến tính và phi tuyến

Hàm mất mát có regularization Hàm mất mát tổng quát của Logistic Regression với regularization được viết dưới dạng:

$$\mathcal{L}(\mathbf{w}, b) = - \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \Omega(\mathbf{w})$$

Trong đó $\Omega(\mathbf{w})$ là hạng phạt và $\lambda > 0$ điều khiển mức độ regularization. Giá trị λ càng lớn thì ràng buộc càng mạnh, dẫn đến mô hình đơn giản hơn.

L2 Regularization Với L2 regularization, hạng phạt được xác định bởi chuẩn ℓ_2 :

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2$$

Khi đó:

$$\mathcal{L}_{L2} = \mathcal{L} + \lambda \|\mathbf{w}\|_2^2$$

L2 regularization làm giảm giá trị của các trọng số một cách trơn tru và liên tục, giúp mô hình ổn định hơn trước nhiễu và hiện tượng đa cộng tuyến. Tuy nhiên, do không triệt tiêu hoàn toàn các trọng số, mô hình vẫn giữ lại toàn bộ đặc trưng.

L1 Regularization Với L1 regularization, hạng phạt sử dụng chuẩn ℓ_1 :

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=1}^d |w_j|$$

Khi đó:

$$\mathcal{L}_{L1} = \mathcal{L} + \lambda \|\mathbf{w}\|_1$$

Đặc điểm nổi bật của L1 regularization là khả năng đưa nhiều trọng số về đúng bằng 0, từ đó tạo ra mô hình thưa (sparse). Điều này cho phép mô hình tự động thực hiện lựa chọn đặc trưng và giảm đáng kể độ phức tạp.

Trực giác hình học Sự khác biệt giữa L1 và L2 có thể được lý giải thông qua miền ràng buộc trong không gian tham số:

- L2 regularization tương ứng với miền ràng buộc dạng hình cầu, dẫn đến nghiệm tối ưu phân bố đều trên các trọng số.
- L1 regularization tương ứng với miền ràng buộc dạng hình thoi, khiến nghiệm tối ưu thường rơi vào các đỉnh, nơi một số trọng số bằng 0.

Thiết lập thí nghiệm Để đánh giá tác động của các kỹ thuật regularization, chúng tôi tiến hành so sánh hai mô hình Logistic Regression sử dụng L1 và L2 regularization trên cùng một tập dữ liệu.

Tập dữ liệu được chia thành hai phần: tập huấn luyện dùng để học tham số và tập kiểm tra dùng để đánh giá khả năng tổng quát hóa của mô hình.

Cấu hình mô hình Hai mô hình được thiết lập với các cấu hình như sau:

- **L1 Regularization:**
 - Hệ số điều chuẩn: $C = 0.01$, tương ứng với mức regularization mạnh nhằm khuyến khích sparsity
 - Thuật toán tối ưu: *SAGA*, phù hợp với các bài toán có thành phần không trơn như chuẩn ℓ_1
 - Số vòng lặp tối đa: 2000 để đảm bảo hội tụ
- **L2 Regularization:**
 - Hệ số điều chuẩn: sử dụng giá trị mặc định, tương ứng với mức regularization trung bình
 - Thuật toán tối ưu: *LBFGS*, một phương pháp quasi-Newton hiệu quả cho các hàm mất mát trơn
 - Số vòng lặp tối đa: 1000

Tiêu chí đánh giá Hiệu năng của mô hình được đánh giá thông qua hai tiêu chí:

- **Accuracy:** đo lường tỷ lệ dự đoán đúng trên tập kiểm tra, phản ánh khả năng phân loại của mô hình.
- **Sparsity:** được định nghĩa là tỷ lệ các trọng số có giá trị gần bằng 0:

$$\text{Sparsity} = \frac{1}{d} \sum_{j=1}^d \mathbb{I}(|w_j| < \epsilon), \quad \epsilon = 10^{-6}$$

Chỉ số này phản ánh mức độ đơn giản của mô hình và khả năng loại bỏ các đặc trưng không quan trọng.

Mục tiêu thí nghiệm Thí nghiệm được thiết kế nhằm làm rõ:

- Ảnh hưởng của L1 và L2 regularization đến hiệu năng phân loại
- Khả năng tạo sparsity của L1 so với L2
- Sự đánh đổi giữa độ chính xác và độ phức tạp của mô hình

Kết quả Kết quả thực nghiệm được trình bày trong Bảng 27.

Mô hình	Accuracy	Sparsity
L1 Regularization	0.7988	0.30
L2 Regularization	0.7989	0.00

Bảng 27: So sánh Logistic Regression với L1 và L2 regularization

Phân tích Kết quả cho thấy hai mô hình đạt độ chính xác gần như tương đương, cho thấy việc lựa chọn loại regularization không ảnh hưởng đáng kể đến hiệu năng dự đoán trong trường hợp này. Tuy nhiên, sự khác biệt rõ rệt xuất hiện ở cấu trúc tham số của mô hình.

Cụ thể, L1 regularization tạo ra mô hình thưa với khoảng 30% trọng số bị triệt tiêu, trong khi L2 regularization giữ lại toàn bộ các trọng số. Điều này cho thấy L1 không chỉ đóng vai trò regularization mà còn thực hiện lựa chọn đặc trưng một cách hiệu quả.

Từ góc nhìn bias-variance trade-off, L1 regularization có xu hướng tăng bias nhưng giảm đáng kể độ phức tạp của mô hình, trong khi L2 regularization giữ được sự cân bằng tốt hơn giữa bias và variance bằng cách phân phối trọng số đều hơn.

Kết luận Từ các kết quả thực nghiệm, có thể rút ra một số nhận định quan trọng về vai trò của regularization trong Logistic Regression. Trước hết, cả hai mô hình sử dụng L1 và L2 regularization đều đạt độ chính xác gần như tương đương trên tập kiểm tra, cho thấy trong bối cảnh dữ liệu hiện tại, việc lựa chọn loại regularization không ảnh hưởng đáng kể đến hiệu năng dự đoán tổng thể.

Tuy nhiên, sự khác biệt cốt lõi giữa hai phương pháp thể hiện rõ ở cấu trúc của vector trọng số. Cụ thể, L1 regularization tạo ra mô hình thưa với tỷ lệ đáng kể các trọng số bị triệt tiêu hoàn toàn, trong khi L2 regularization duy trì tất cả các trọng số ở mức giá trị nhỏ nhưng khác 0. Điều này phản ánh đúng bản chất lý thuyết của hai phương pháp: L1 thực hiện lựa chọn đặc trưng một cách ngẫu nhiên, còn L2 chủ yếu đóng vai trò làm mượt và phân bố lại tầm quan trọng giữa các đặc trưng.

Từ góc nhìn bias-variance trade-off, L1 regularization có xu hướng làm tăng bias do loại bỏ một phần thông tin từ dữ liệu, nhưng đồng thời giúp giảm phương sai và đơn giản hóa mô hình. Ngược lại, L2 regularization duy trì đầy đủ các đặc trưng nên có khả năng giữ bias thấp hơn, đồng thời giảm variance thông qua việc co nhỏ các trọng số. Sự khác biệt này dẫn đến việc mỗi phương pháp phù hợp với những bối cảnh khác nhau.

Về mặt ứng dụng, L1 regularization đặc biệt hữu ích trong các bài toán có số chiều lớn hoặc khi cần mô hình dễ diễn giải, chẳng hạn trong các hệ thống yêu cầu tính minh bạch cao. Trong khi đó, L2 regularization phù hợp hơn khi các đặc trưng đều mang thông tin hữu ích và mục tiêu là tối đa hóa hiệu năng một cách ổn định.

Tổng thể, thí nghiệm cho thấy rằng việc lựa chọn phương pháp regularization không chỉ phụ thuộc vào độ chính xác mà còn cần cân nhắc đến cấu trúc mô hình, khả năng

diễn giải và đặc điểm của dữ liệu. Điều này nhấn mạnh vai trò của regularization như một công cụ quan trọng không chỉ để cải thiện hiệu năng mà còn để kiểm soát và hiểu rõ hơn hành vi của mô hình học máy.

2.2.5 Chọn tham số bằng Stratified K-Fold

Động cơ và bài toán Trong các mô hình học máy có regularization như Logistic Regression hay Ridge Regression, siêu tham số λ đóng vai trò kiểm soát độ phức tạp của mô hình thông qua việc phạt các tham số lớn. Dưới góc nhìn tối ưu hóa, bài toán học có thể được biểu diễn như sau:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \ell(y_i, f_{\mathbf{w}}(x_i)) + \lambda \Omega(\mathbf{w})$$

Trong đó:

- $\ell(\cdot)$ là hàm mất mát
- $\Omega(\mathbf{w})$ là hàm regularization (ví dụ: $\|\mathbf{w}\|^2$)
- λ điều khiển mức độ đánh đổi giữa *data fitting* và *generalization*

Cụ thể, khi λ nhỏ, mô hình ưu tiên khớp sát dữ liệu huấn luyện, dễ dẫn đến overfitting (variance cao). Ngược lại, khi λ lớn, mô hình bị ép đơn giản hóa, dẫn đến underfitting (bias cao). Do đó, việc lựa chọn λ tối ưu thực chất là tìm điểm cân bằng trong trade-off giữa bias và variance.

Ước lượng rủi ro tổng quát hóa Mục tiêu cuối cùng của mô hình là tối thiểu hóa rủi ro kỳ vọng trên phân phối dữ liệu thực:

$$R(\lambda) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(y, f_{\lambda}(x))]$$

Tuy nhiên, do phân phối $\mathcal{D}$ không biết trước, ta cần một phương pháp ước lượng thực nghiệm. K-Fold Cross Validation cung cấp một cơ chế hiệu quả để xấp xỉ đại lượng này.

Cụ thể, tập dữ liệu được chia thành K phần rời nhau:

$$\mathcal{D} = \bigcup_{k=1}^K \mathcal{D}_k$$

Tại mỗi lần lặp, mô hình được huấn luyện trên $\mathcal{D} \setminus \mathcal{D}_k$ và đánh giá trên $\mathcal{D}_k$. Khi đó, ước lượng rủi ro được tính:

$$\hat{R}_{CV}(\lambda) = \frac{1}{K} \sum_{k=1}^K \mathcal{L}^{(k)}(\lambda)$$

Giá trị λ tối ưu được xác định theo:

$$\lambda^* = \arg \min_{\lambda} \hat{R}_{CV}(\lambda)$$

Stratified K-Fold và tính ổn định Trong bài toán phân loại, đặc biệt khi dữ liệu mất cân bằng, việc chia fold ngẫu nhiên có thể làm sai lệch phân phối nhãn giữa các fold. Điều này dẫn đến ước lượng rủi ro có phương sai cao và không ổn định.

Để khắc phục, ta sử dụng Stratified K-Fold, đảm bảo rằng:

$$P(y \mid \mathcal{D}_k) \approx P(y), \quad \forall k$$

Nhờ đó, mỗi fold là một đại diện tốt của toàn bộ dataset, giúp giảm phương sai của $\hat{R}_{CV}$ và cải thiện độ tin cậy của quá trình lựa chọn siêu tham số.

Hàm đánh giá Trong thực nghiệm này, chúng tôi sử dụng log loss (cross-entropy loss):

$$\mathcal{L}(y, \hat{p}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Log loss là một hàm mất mát lỗi và có các đặc tính quan trọng:

- Là *strictly proper scoring rule*, đảm bảo tối ưu khi xác suất dự đoán đúng với phân phối thật
- Nhạy với độ tự tin của mô hình, phạt mạnh các dự đoán sai nhưng có xác suất cao
- Tương đương với việc tối đa hóa likelihood trong mô hình xác suất

Do đó, log loss không chỉ đo độ chính xác mà còn phản ánh chất lượng dự đoán xác suất.

Quy trình lựa chọn λ Dựa trên các nguyên lý trên, quy trình lựa chọn λ được thực hiện như sau:

1. Xác định một tập hữu hạn các giá trị ứng viên $\{\lambda_1, \dots, \lambda_M\}$
2. Với mỗi λ_m :
 - Thực hiện Stratified K-Fold ($k = 5$)
 - Tại mỗi fold:
 - Huấn luyện mô hình trên tập train
 - Đánh giá log loss trên tập validation

3. Tính trung bình log loss trên 5 fold để thu được $\hat{R}_{CV}(\lambda_m)$

4. Chọn giá trị tối ưu:

$$\lambda^* = \arg \min_{\lambda_m} \hat{R}_{CV}(\lambda_m)$$

Thiết lập thực nghiệm Thực nghiệm được tiến hành trên tập dữ liệu Covtype với kích thước:

$$N = 581,012, \quad D = 54$$

Phân phối lớp cho thấy dữ liệu bị mất cân bằng đáng kể, trong đó một số lớp chiếm ưu thế rõ rệt. Để đơn giản hóa bài toán, chúng tôi chuyển đổi về bài toán phân loại nhị phân bằng cách ánh xạ:

$$y = \begin{cases} 1 & \text{nếu lớp gốc} = 1 \\ 0 & \text{ngược lại} \end{cases}$$

Tập giá trị λ được xét:

$$\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$$

Việc lựa chọn λ được thực hiện thông qua Stratified 5-Fold Cross Validation với thước đo log loss.

Kết quả thực nghiệm Kết quả log loss trung bình trên 5 fold được trình bày như sau:

$$\begin{aligned} \lambda = 0.001 & \rightarrow \mathcal{L} = 0.6290 \\ \lambda = 0.01 & \rightarrow \mathcal{L} = 0.6290 \\ \lambda = 0.1 & \rightarrow \mathcal{L} = 0.6290 \\ \lambda = 1 & \rightarrow \mathcal{L} = 0.6290 \\ \lambda = 10 & \rightarrow \mathcal{L} = 0.6292 \end{aligned}$$

Giá trị tối ưu được chọn:

$$\lambda^* = 0.01, \quad \mathcal{L}_{min} \approx 0.6290$$

Phân tích Một quan sát quan trọng là log loss gần như không thay đổi đáng kể trong khoảng:

$$\lambda \in [0.001, 1]$$

Điều này cho thấy:

- Mô hình tương đối **ổn định** với regularization trong khoảng này
- Dữ liệu có thể có **tín hiệu tuyến tính mạnh**, khiến nghiệm tối ưu không quá nhạy với λ

Khi λ tăng lên 10 (regularization mạnh), log loss tăng nhẹ, cho thấy:

- Mô hình bắt đầu bị *underfitting*
- Các tham số bị co lại quá mức, làm giảm khả năng biểu diễn

Diễn giải dưới góc nhìn bias–variance Kết quả này phản ánh rõ đặc điểm của trade-off bias–variance:

- λ nhỏ $\rightarrow$ variance cao, bias thấp
- λ lớn $\rightarrow$ variance thấp, bias cao

Tuy nhiên, trong bài toán này:

- Sai số gần như không thay đổi $\Rightarrow$ mô hình đang ở vùng *bias–variance balance*
- Điều này thường xảy ra khi kích thước dữ liệu lớn ($N \gg D$)

Nhận xét

- Việc chọn $\lambda^* = 0.01$ là hợp lý vì đạt log loss nhỏ nhất.
- Tuy nhiên, do sự khác biệt giữa các giá trị là rất nhỏ, có thể kết luận rằng:
 - Regularization không phải là yếu tố chi phối chính trong bài toán này
 - Chất lượng mô hình chủ yếu phụ thuộc vào dữ liệu và đặc trưng
- Trong các trường hợp như vậy, việc mở rộng không gian đặc trưng (feature engineering) hoặc sử dụng mô hình phi tuyến có thể mang lại cải thiện đáng kể hơn so với việc tinh chỉnh λ .

Kết luận Kết quả thực nghiệm cho thấy Logistic Regression với regularization L_2 hoạt động ổn định trên tập dữ liệu Covtype. Việc lựa chọn λ thông qua cross-validation giúp đảm bảo khả năng tổng quát hóa, tuy nhiên ảnh hưởng của λ trong bài toán này là tương đối hạn chế do kích thước dữ liệu lớn và cấu trúc tuyến tính rõ ràng của bài toán.

2.2.6 Class-Weighted Loss

Lý thuyết Trong các bài toán phân loại thực tế, dữ liệu thường không phân bố đồng đều giữa các lớp, dẫn đến hiện tượng *class imbalance*. Khi đó, các lớp chiếm đa số (majority classes) có số lượng mẫu lớn sẽ chi phối quá trình tối ưu, khiến mô hình học được thiên lệch về phía các lớp này.

Dưới góc nhìn tối ưu hóa, hàm mất mát thực nghiệm:

$$\frac{1}{N} \sum_{n=1}^N \ell(y_n, f(x_n))$$

bị chi phối bởi các lớp có N_k lớn. Do đó, mô hình có xu hướng tối thiểu hóa sai số trên lớp đa số, trong khi bỏ qua lớp thiểu số (minority classes), dẫn đến:

- Hiệu năng thấp trên lớp hiếm
- Recall kém trong các bài toán quan trọng (fraud detection, medical diagnosis)
- Decision boundary bị lệch về phía lớp thiểu số

Để khắc phục, ta điều chỉnh hàm mất mát nhằm tái cân bằng đóng góp của từng lớp.

Hàm mất mát có trọng số Xét bài toán phân loại K lớp với biểu diễn one-hot:

$$t_{nk} = \begin{cases} 1 & \text{nếu mẫu } n \text{ thuộc lớp } k \\ 0 & \text{ngược lại} \end{cases}$$

Gọi $y_{nk} = P(y = k \mid x_n)$ là xác suất dự đoán. Khi đó, hàm mất mát cross-entropy chuẩn có dạng:

$$E = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_{nk}$$

Để xử lý mất cân bằng, ta đưa vào trọng số lớp c_k :

$$E = - \sum_{n=1}^N \sum_{k=1}^K c_k t_{nk} \log y_{nk}$$

Trong đó $c_k > 0$ điều chỉnh mức độ ảnh hưởng của lớp k trong quá trình tối ưu.

Lựa chọn trọng số Một lựa chọn phổ biến là đặt:

$$c_k \propto \frac{1}{N_k} \quad \text{hoặc} \quad c_k = \frac{N}{K \cdot N_k}$$

với N_k là số mẫu thuộc lớp k . Khi đó:

- Lớp thiếu số (N_k nhỏ) $\Rightarrow c_k$ lớn $\Rightarrow$ tăng ảnh hưởng
- Lớp đa số (N_k lớn) $\Rightarrow c_k$ nhỏ $\Rightarrow$ giảm ảnh hưởng

Cách thiết kế này đảm bảo rằng tổng đóng góp của mỗi lớp trong hàm mất mát trở nên cân bằng hơn.

Diễn giải dưới góc nhìn xác suất Class-weighted loss có thể được hiểu như việc thay đổi phân phối dữ liệu hiệu dụng. Cụ thể, thay vì tối thiểu hóa kỳ vọng theo phân phối gốc $P(x, y)$, ta tối thiểu hóa theo một phân phối được tái trọng số:

$$P'(x, y = k) \propto c_k \cdot P(x, y = k)$$

Do đó, quá trình học tương đương với việc “phóng đại” tầm quan trọng của các lớp thiếu số trong không gian xác suất.

Diễn giải dưới góc nhìn tối ưu hóa Hàm mục tiêu có thể được viết lại:

$$\min_{\mathbf{w}} \sum_{k=1}^K c_k \sum_{n \in \mathcal{D}_k} \ell(y_n, f_{\mathbf{w}}(x_n))$$

Điều này tương đương với việc mỗi mẫu thuộc lớp k được nhân với hệ số c_k , hoặc có thể hiểu như việc lặp lại mẫu đó c_k lần trong quá trình huấn luyện.

Ảnh hưởng đến gradient Gradient của hàm mất mát theo tham số $\mathbf{w}$ có dạng:

$$\nabla_{\mathbf{w}} E = - \sum_{n=1}^N \sum_{k=1}^K c_k t_{nk} \nabla_{\mathbf{w}} \log y_{nk}$$

Trong trường hợp Logistic Regression, điều này dẫn đến:

$$\nabla_{\mathbf{w}} E \propto \sum_{n=1}^N c_{y_n} (y_n - \hat{y}_n) \mathbf{x}_n$$

Cho thấy rằng:

- Sai số từ lớp thiểu số được khuếch đại (gradient lớn hơn)
- Sai số từ lớp đa số bị giảm ảnh hưởng

Do đó, decision boundary sẽ được điều chỉnh theo hướng cân bằng hơn giữa các lớp.

Ưu điểm:

- Không cần thay đổi dữ liệu (không cần over/under-sampling)
- Dễ tích hợp vào hầu hết các mô hình học máy
- Hiệu quả với dữ liệu mất cân bằng vừa phải

Hạn chế:

- Nhạy với cách chọn trọng số c_k
- Có thể gây mất ổn định nếu c_k quá lớn (gradient explosion)
- Không xử lý tốt khi dữ liệu cực kỳ mất cân bằng

Thực nghiệm xử lý mất cân bằng lớp (Class Imbalance) Tập dữ liệu Covtype có phân phối lớp không đồng đều, thể hiện rõ qua số lượng mẫu giữa các lớp:

Class distribution = [211840, 283301, 35754, 2747, 9493, 17367, 20510]

Sự mất cân bằng này dẫn đến việc mô hình có xu hướng ưu tiên các lớp chiếm đa số, làm suy giảm hiệu năng trên các lớp thiểu số.

Các phương pháp so sánh Ba mô hình được sử dụng để đánh giá ảnh hưởng của class imbalance:

- **Logistic Regression (No Weight):** mô hình chuẩn, không xử lý mất cân bằng
- **Logistic Regression (Balanced):** sử dụng `class_weight='balanced'` trong thư viện sklearn
- **Weighted Softmax (Custom):** cài đặt thủ công hàm mất mát có trọng số lớp

Kết quả thực nghiệm

Model	Accuracy	Macro F1
Logistic (No Weight)	0.7263	0.5317
Logistic (Balanced)	0.5991	0.5063
Weighted Softmax	0.5969	≈ 0.50

Phân tích chi tiết

- **Logistic Regression (No Weight):**
 - Đạt accuracy cao nhất ($\approx 72.6\%$)
 - Tuy nhiên, hiệu năng trên các lớp nhỏ rất kém:
 - * Class 5: recall gần 0
 - * Class 4, 6: recall thấp
 - Điều này cho thấy mô hình bị **bias mạnh về lớp lớn**
- **Logistic Regression (Balanced):**
 - Accuracy giảm đáng kể ($\approx 59.9\%$)
 - Nhưng recall của các lớp nhỏ tăng mạnh:
 - * Class 5: từ $\approx 0.006 \rightarrow 0.78$
 - * Class 4: từ $\approx 0.41 \rightarrow 0.86$
 - Đánh đổi rõ ràng giữa:
 - * Accuracy tổng thể
 - * Khả năng nhận diện lớp thiểu số
- **Weighted Softmax (Custom):**
 - Kết quả gần tương đương với Logistic Balanced
 - Xác nhận rằng:
$$\text{Class-weighted loss} \approx \text{class_weight trong sklearn}$$
 - Một số sai khác nhỏ do:
 - * Thuật toán tối ưu (Gradient Descent vs LBFGS)
 - * Learning rate và batch size

Phân tích trọng số lớp Trọng số lớp được tính theo:

$$c_k = \frac{N}{N_k}$$

Một số giá trị tiêu biểu:

Class	Weight
4	211.53
5	61.20
6	33.45
7	28.33

- Các lớp hiếm được gán trọng số rất lớn
- Điều này buộc mô hình phải:
 - Giảm lỗi trên lớp nhỏ
 - Chấp nhận tăng lỗi trên lớp lớn

Diễn giải dưới góc nhìn hàm mất mát Hàm mất mát có trọng số:

$$\mathcal{L} = - \sum_{i=1}^N c_{y_i} \log p(y_i | x_i)$$

- Nếu c_k lớn $\Rightarrow$ lỗi trên lớp k bị phạt nặng hơn
- Mô hình học cách cân bằng giữa các lớp

Nhận xét quan trọng

- Accuracy **không phải thước đo tốt** trong bài toán mất cân bằng
- Macro F1 phản ánh tốt hơn hiệu năng tổng thể
- Class weighting giúp:
 - Tăng recall cho lớp hiếm
 - Giảm bias của mô hình

Kết luận

- Mô hình không sử dụng class weight đạt accuracy cao nhưng thiếu công bằng giữa các lớp
- Class-weighted loss giúp cải thiện đáng kể khả năng nhận diện lớp thiểu số
- Việc lựa chọn mô hình phụ thuộc vào mục tiêu:
 - Nếu ưu tiên accuracy: dùng Logistic thường
 - Nếu ưu tiên fairness / recall lớp nhỏ: dùng weighted model

Kết quả này nhấn mạnh rằng trong các bài toán thực tế có mất cân bằng dữ liệu, việc thiết kế hàm mất mát phù hợp quan trọng không kém việc lựa chọn mô hình.

2.3 Đánh giá và phân tích các mô hình

2.3.1 Đánh giá trên tập kiểm tra

Bảng 28 thể hiện các kết quả đánh giá các mô hình trên tập kiểm tra. Các chỉ số dùng để đánh giá gồm Accuracy, Precision (macro), Recall (macro) và F1 (macro).

Nhìn vào bảng kết quả, có thể thấy các mô hình thể hiện những đặc điểm rất khác nhau tùy theo từng chỉ số đánh giá, cho thấy sự đánh đổi (trade-off) rõ ràng giữa độ chính xác tổng thể và khả năng nhận diện đầy đủ các lớp.

Trước hết, Softmax đạt Accuracy cao nhất (0.7148), tuy nhiên các chỉ số Recall (0.4306) và F1 (0.4319) lại khá thấp. Điều này cho thấy mô hình có xu hướng dự đoán đúng tổng thể nhưng bỏ sót nhiều mẫu ở các lớp thiểu số, dẫn đến hiệu năng không đồng đều giữa các lớp.

Ngược lại, Weighted Softmax cải thiện đáng kể Recall (0.697) – cao nhất trong tất cả các mô hình – cho thấy khả năng nhận diện tốt hơn các lớp, đặc biệt là lớp ít dữ liệu.

Bảng 28: Kết quả các chỉ số đánh giá các mô hình trên tập kiểm tra

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Softmax	0.7148	0.5901	0.4306	0.4319
LDA	0.6815	0.4856	0.5766	0.5096
QDA	0.2421	0.3589	0.5548	0.256
Weighted Softmax	0.592	0.4603	0.697	0.492

Tuy nhiên, Accuracy (0.592) và Precision (0.4603) lại giảm, phản ánh việc mô hình đánh đổi bằng cách chấp nhận nhiều dự đoán sai hơn để tăng độ bao phủ.

LDA thể hiện sự cân bằng tương đối giữa các chỉ số với Recall (0.5766) và F1 (0.5096) ở mức khá, dù Accuracy (0.6815) không cao bằng Softmax. Điều này cho thấy LDA có hiệu năng ổn định hơn giữa các lớp, phù hợp khi cần một mô hình tổng quát, ít thiên lệch.

Trong khi đó, QDA có hiệu năng kém nhất với Accuracy rất thấp (0.2421) và F1 (0.256), mặc dù Recall (0.5548) không quá thấp. Điều này cho thấy mô hình dự đoán khá “thoảng”, bắt được nhiều mẫu nhưng thiếu chính xác nghiêm trọng, dẫn đến hiệu quả tổng thể kém.

Tổng thể, nếu ưu tiên độ chính xác chung, Softmax là lựa chọn tốt nhất. Nếu mục tiêu là không bỏ sót dữ liệu (Recall cao), Weighted Softmax phù hợp hơn. Trong khi đó, LDA là phương án cân bằng giữa các chỉ số, còn QDA không phù hợp trong bối cảnh này do hiệu năng không ổn định và độ chính xác thấp.

2.3.2 Đánh giá qua cross-validation

Bảng 29 và 30 thể hiện các kết quả trung bình và độ lệch chuẩn các chỉ số Accuracy, Precision (macro), Recall (macro) và F1 (macro) thông qua 5-fold cross-validation.

Bảng 29: Kết quả trung bình và độ lệch chuẩn các chỉ số của các mô hình thông qua cross-validation (phần 1)

Model	Accuracy	Precision (macro)
Softmax	0.7122 ± 0.0009	0.5703 ± 0.0293
LDA	0.6795 ± 0.0013	0.4877 ± 0.0046
QDA	0.2427 ± 0.0009	0.3589 ± 0.0043
Weighted Softmax	0.5903 ± 0.0011	0.4556 ± 0.0015

Kết quả cross-validation trong Bảng 29 và 30 nhìn chung phù hợp và nhất quán với kết quả trên tập kiểm tra, đồng thời cung cấp thêm thông tin về độ ổn định của các mô hình thông qua độ lệch chuẩn.

Trước hết, Softmax tiếp tục duy trì Accuracy cao nhất (0.7122 ± 0.0009), rất gần với kết quả trên tập test (0.7148). Độ lệch chuẩn nhỏ cho thấy mô hình rất ổn định qua các lần chia dữ liệu. Tuy nhiên, Recall (0.4212) và F1 (0.4155) vẫn thấp, củng cố nhận định trước đó rằng mô hình này thiên về tối ưu độ chính xác tổng thể nhưng chưa xử lý tốt sự mất cân bằng giữa các lớp.

Bảng 30: Kết quả trung bình và độ lệch chuẩn các chỉ số của các mô hình thông qua cross-validation (phần 2)

Model	Recall (macro)	F1 (macro)
Softmax	0.4212 ± 0.0015	0.4155 ± 0.0025
LDA	0.5791 ± 0.0060	0.5108 ± 0.0054
QDA	0.5548 ± 0.0030	0.2572 ± 0.0018
Weighted Softmax	0.6899 ± 0.0040	0.4885 ± 0.0015

Weighted Softmax một lần nữa nổi bật với Recall cao nhất (0.6899 ± 0.0040), gần như trùng khớp với kết quả test (0.697). Điều này cho thấy khả năng tổng quát hóa tốt trong việc nhận diện các lớp, đặc biệt là lớp thiểu số. Tuy nhiên, Accuracy (0.5903) và Precision (0.4556) vẫn thấp hơn Softmax, phản ánh sự đánh đổi tương tự như đã quan sát trên tập kiểm tra. Độ lệch chuẩn nhỏ cho thấy mô hình này cũng ổn định.

LDA tiếp tục thể hiện vai trò là mô hình cân bằng nhất, với các chỉ số Recall (0.5791) và F1 (0.5108) khá tốt và ổn định. Kết quả này gần tương ứng với tập test (Recall 0.5766, F1 0.5096), cho thấy LDA có khả năng tổng quát hóa đáng tin cậy và ít bị dao động theo dữ liệu.

Trong khi đó, QDA vẫn có Accuracy rất thấp (0.2427) nhưng Recall tương đối cao (0.5548), giống với kết quả trên tập test. Tuy nhiên, F1 thấp (0.2572) cho thấy mô hình dự đoán thiếu chính xác và không cân bằng. Dù độ lệch chuẩn nhỏ (tức là ổn định), nhưng đây là sự ổn định ở mức hiệu năng kém, nên không có giá trị thực tiễn cao.

Tổng thể, kết quả cross-validation xác nhận lại các kết luận từ tập kiểm tra: Softmax tốt nhất về Accuracy, Weighted Softmax tốt nhất về Recall, LDA cân bằng nhất, và QDA kém hiệu quả. Đồng thời, độ lệch chuẩn nhỏ ở tất cả các mô hình cho thấy các kết luận này đáng tin cậy và không phụ thuộc mạnh vào cách chia dữ liệu.

2.3.3 Đánh giá kiểm định các cặp mô hình

Bảng 31 thể hiện được kết quả kiểm định McNemar các cặp mô hình về sự giống nhau của phân phối lỗi. Các kết quả được làm tròn đến 4 chữ số thập phân.

Bảng 31: Kết quả kiểm định McNemar các cặp mô hình trên tập kiểm tra

Model A	Model B	McNemar statistic	p-value
Softmax	LDA	4700.0	0.0
Softmax	QDA	5574.0	0.0
Softmax	Weighted Softmax	8905.0	0.0
LDA	QDA	5508.0	0.0
LDA	Weighted Softmax	6236.0	0.0
QDA	Weighted Softmax	3335.0	0.0

Kết quả kiểm định McNemar trong Bảng 31 cho thấy tất cả các cặp mô hình đều có p-value xấp xỉ 0.0000 (sau khi làm tròn 4 chữ số thập phân). Điều này cho phép bác bỏ giả thuyết H_0 (H_0 là hai mô hình có cùng phân phối lỗi) ở mức ý nghĩa thông thường (ví

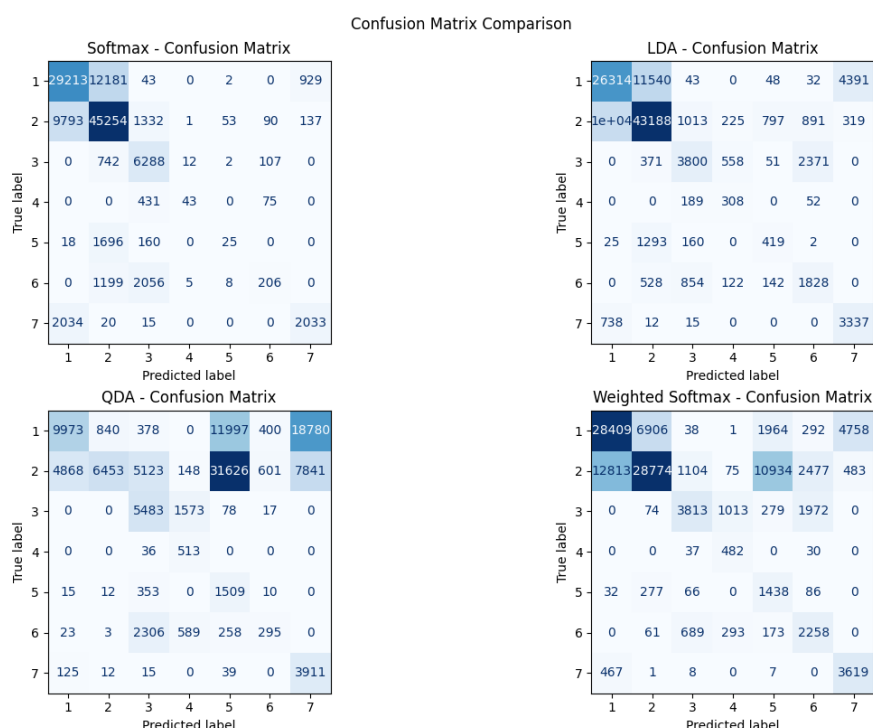
dự 0.05). Nói cách khác, sự khác biệt về hiệu năng giữa mọi cặp mô hình đều có ý nghĩa thống kê, không phải do ngẫu nhiên.

Đáng chú ý, các cặp liên quan đến Softmax có giá trị thống kê rất lớn (đặc biệt với Weighted Softmax: 8905.0), cho thấy sự khác biệt rõ rệt trong cách hai mô hình này dự đoán. Điều này phù hợp với kết quả trước đó: dù cùng họ mô hình, Softmax tối ưu Accuracy trong khi Weighted Softmax thiên về Recall, dẫn đến sự khác biệt đáng kể trong các dự đoán sai/đúng.

Tương tự, các cặp như LDA – QDA (5508.0) hay LDA – Weighted Softmax (6236.0) cũng có thống kê lớn, phản ánh sự khác biệt rõ ràng về bản chất mô hình và hành vi phân loại. Đặc biệt, dù QDA có Recall tương đối cao, kết quả kiểm định cho thấy mô hình này vẫn tạo ra mẫu lỗi rất khác biệt so với các mô hình còn lại, phù hợp với việc Accuracy và F1 của QDA thấp.

Tổng thể, kiểm định McNemar củng cố kết luận từ các bảng trước: các mô hình không chỉ khác nhau về giá trị chỉ số mà còn khác nhau một cách có ý nghĩa thống kê trong dự đoán thực tế. Điều này khẳng định rằng việc lựa chọn mô hình (Softmax, LDA, hay Weighted Softmax) sẽ dẫn đến những hành vi phân loại khác biệt rõ rệt, tùy thuộc vào mục tiêu tối ưu (Accuracy, Recall hay cân bằng tổng thể).

2.3.4 Các biểu đồ so sánh



Hình 33: Ma trận nhầm lẫn của các mô hình trên tập kiểm tra

Ma trận nhầm lẫn Dựa trên các ma trận nhầm lẫn, có thể thấy rõ sự khác biệt trong hành vi phân loại của các mô hình, đặc biệt trong cách xử lý các lớp thiểu số và mức độ thiên lệch về các lớp chiếm đa số.

Đối với Softmax, các giá trị trên đường chéo tập trung chủ yếu ở các lớp lớn, chẳng hạn lớp 1 và lớp 2 với lần lượt 29213 và 45254 mẫu được dự đoán đúng. Tuy nhiên, nhiều lớp nhỏ bị bỏ sót nghiêm trọng. Ví dụ, lớp 5 chỉ có 25 mẫu được dự đoán đúng nhưng bị nhầm sang lớp 2 tới 1696 mẫu; lớp 6 chỉ có 206 mẫu đúng nhưng bị nhầm sang lớp 3 tới

2056 mẫu. Điều này cho thấy mô hình có xu hướng dồn dự đoán về một vài lớp chính, dẫn đến mất cân bằng trong phân loại.

Ngược lại, LDA cho thấy phân phối dự đoán đồng đều hơn giữa các lớp. Các lớp thiếu số được nhận diện tốt hơn, chẳng hạn lớp 5 có 419 mẫu được dự đoán đúng (so với 25 của Softmax) và lớp 6 có 1828 mẫu đúng (so với 206). Tuy nhiên, vẫn tồn tại nhầm lẫn đáng kể giữa các lớp, ví dụ lớp 3 bị nhầm sang lớp 6 tới 2371 mẫu. Dù vậy, mô hình không bị hiện tượng tập trung vào một vài lớp như Softmax, thể hiện sự cân bằng tốt hơn.

Đối với QDA, ma trận nhầm lẫn cho thấy sự phân loại thiếu ổn định với nhiều giá trị lớn nằm ngoài đường chéo. Ví dụ, lớp 1 bị nhầm sang lớp 5 tới 11997 mẫu và sang lớp 7 tới 18780 mẫu. Tương tự, lớp 2 bị nhầm sang lớp 5 tới 31626 mẫu. Những giá trị này cho thấy mô hình đưa ra nhiều dự đoán sai lệch nghiêm trọng. Mặc dù một số lớp có Recall cao, Precision lại rất thấp, dẫn đến hiệu năng tổng thể kém.

Trong khi đó, Weighted Softmax cải thiện đáng kể khả năng nhận diện các lớp thiếu số. Ví dụ, lớp 5 có 1438 mẫu được dự đoán đúng (so với 25 của Softmax). Tuy nhiên, điều này đi kèm với việc gia tăng nhầm lẫn giữa các lớp. Chẳng hạn, lớp 2 bị nhầm sang lớp 5 tới 10934 mẫu và sang lớp 6 tới 2477 mẫu. Ma trận nhầm lẫn cho thấy các dự đoán bị phân tán rộng hơn, dẫn đến việc tăng Recall nhưng giảm Precision và Accuracy.

Một điểm chung rõ ràng giữa các mô hình là đều xảy ra nhầm lẫn mạnh giữa một số cặp lớp cụ thể, đặc biệt là giữa lớp 1 và lớp 2. Cụ thể, ở Softmax, có 12181 mẫu từ lớp 1 bị nhầm sang lớp 2 và 9793 mẫu theo chiều ngược lại. Xu hướng này cũng xuất hiện ở LDA (11540 và 10227 mẫu) và Weighted Softmax (6906 và 12813 mẫu), cho thấy đây là cặp lớp khó phân biệt đối với mọi mô hình.

Ngoài ra, lớp 2 có xu hướng bị nhầm sang nhiều lớp khác ở tất cả các mô hình, đặc biệt là sang lớp 3, lớp 5 và lớp 6. Ví dụ, ở Weighted Softmax, có 10934 mẫu từ lớp 2 bị nhầm sang lớp 5 và 2477 mẫu sang lớp 6; ở Softmax và LDA có tương ứng 1332 và 1013 mẫu bị nhầm từ lớp 2 sang lớp 3. Điều này cho thấy lớp 2 có phân bố đặc trưng chồng lấn với nhiều lớp khác.

Bên cạnh đó, các lớp thiếu số như lớp 5 đều có số lượng dự đoán đúng thấp trong hầu hết các mô hình. Ví dụ, Softmax chỉ dự đoán đúng 25 mẫu cho lớp 5, trong khi LDA và Weighted Softmax cải thiện hơn (419 và 1438 mẫu) nhưng vẫn thấp so với các lớp lớn.

Những điểm chung này cho thấy các lỗi phân loại mang tính hệ thống, đặc biệt liên quan đến sự chồng lấn đặc trưng giữa một số lớp và sự mất cân bằng dữ liệu, thay vì chỉ xuất phát từ từng mô hình riêng lẻ.

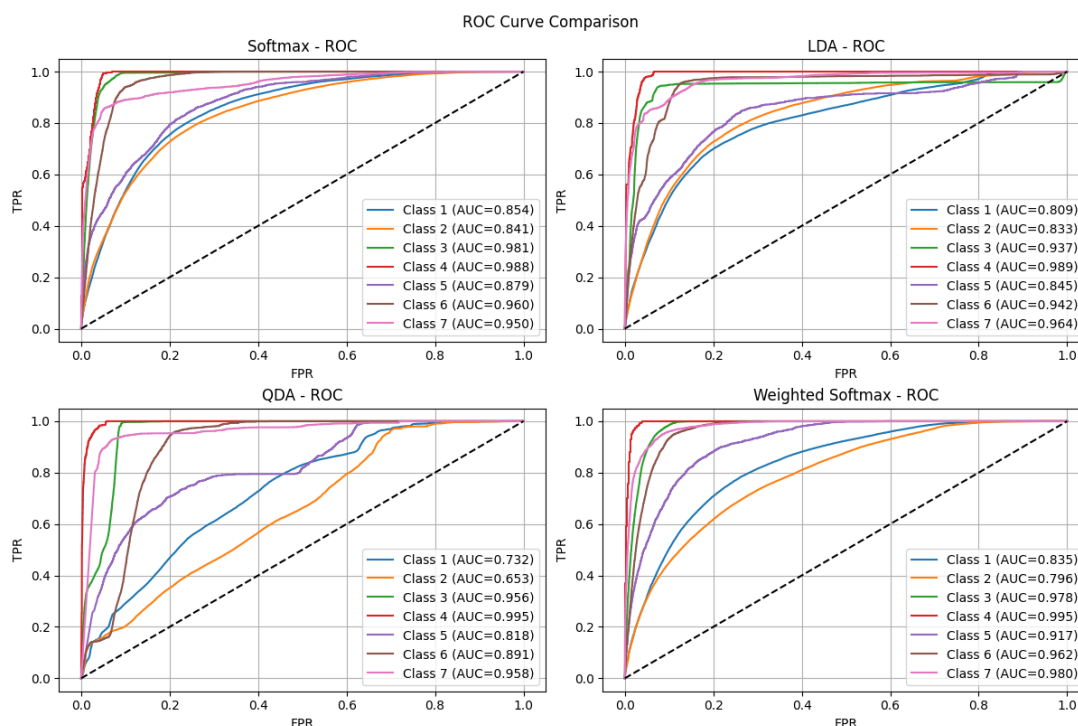
Tổng thể, phân tích ma trận nhầm lẫn cho thấy rõ sự đánh đổi giữa các mô hình: Softmax có xu hướng thiên lệch về lớp lớn, Weighted Softmax mở rộng khả năng nhận diện nhưng kém chính xác hơn, LDA đạt được sự cân bằng tương đối, trong khi QDA thể hiện hiệu năng không ổn định và kém hiệu quả.

Biểu đồ đường cong ROC Hình 34 cho ta thấy kết quả ROC-AUC theo phương pháp One-vs-Rest.

Dựa trên kết quả, có thể nhận thấy sự khác biệt rõ rệt giữa các mô hình Softmax, LDA, QDA và Weighted Softmax trên từng lớp.

Đối với lớp 1, mô hình Softmax đạt hiệu năng cao nhất với $AUC = 0.854$, tiếp theo là Weighted Softmax và LDA. QDA cho kết quả thấp nhất, cho thấy khả năng phân biệt lớp này còn hạn chế.

Ở lớp 2, Softmax và LDA cho kết quả tương đối tương đồng và cao (lần lượt là 0.841 và 0.833), trong khi Weighted Softmax giảm nhẹ. QDA tiếp tục thể hiện kém với AUC chỉ đạt 0.653.



Hình 34: Biểu đồ đường cong ROC trên từng lớp của các mô hình

Đối với lớp 3, tất cả các mô hình đều đạt hiệu năng rất cao ($AUC > 0.93$), trong đó Softmax và Weighted Softmax gần như tương đương và tốt nhất. Điều này cho thấy lớp này tương đối dễ phân loại.

Tại lớp 4, tất cả các mô hình đều đạt kết quả gần như hoàn hảo (AUC xấp xỉ 1.0), đặc biệt QDA và Weighted Softmax đạt giá trị cao nhất là 0.995. Đây có thể xem là lớp dễ phân biệt nhất.

Ở lớp 5, Weighted Softmax vượt trội với $AUC = 0.917$, cao hơn đáng kể so với các mô hình còn lại. QDA tiếp tục có kết quả thấp nhất.

Đối với lớp 6, Softmax và Weighted Softmax đều đạt hiệu năng rất cao (trên 0.96), trong khi LDA và QDA thấp hơn nhưng vẫn ở mức tốt.

Cuối cùng, ở lớp 7, Weighted Softmax cho kết quả tốt nhất với $AUC = 0.980$, tiếp theo là LDA và QDA. Softmax có AUC thấp hơn một chút nhưng vẫn đạt hiệu năng cao.

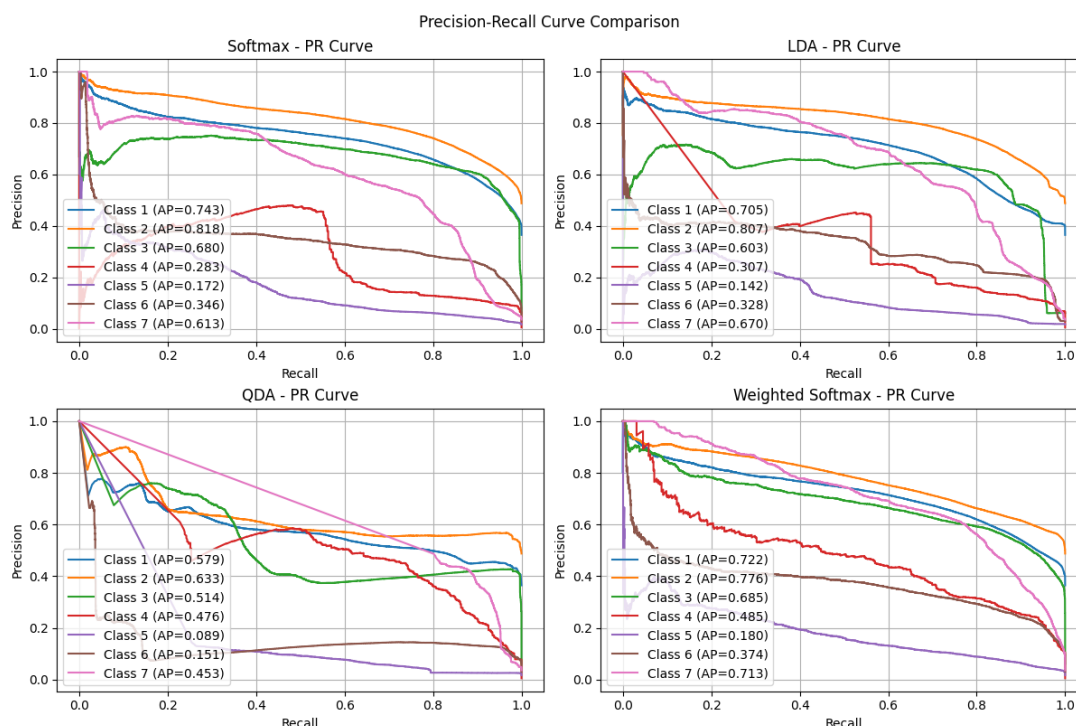
Nhìn chung, Weighted Softmax là mô hình cho hiệu năng tổng thể tốt nhất và ổn định trên hầu hết các lớp, đặc biệt cải thiện rõ rệt ở các lớp khó. Softmax cũng cho kết quả mạnh và ổn định. LDA có hiệu năng trung bình và khá đồng đều. Trong khi đó, QDA có sự biến động lớn, hoạt động tốt ở một số lớp nhưng kém ở các lớp khác.

Biểu đồ đường cong Precision–Recall Hình 35 cho ta thấy kết quả Precision–Recall theo phương pháp One-vs-Rest.

Dựa trên kết quả, có thể nhận thấy sự khác biệt đáng kể giữa các mô hình Softmax, LDA, QDA và Weighted Softmax trên từng lớp.

Đối với lớp 1, mô hình Softmax đạt giá trị AP cao nhất (0.743), trong khi Weighted Softmax và LDA có kết quả gần tương đương. QDA thấp hơn đáng kể, cho thấy khả năng nhận diện lớp này còn hạn chế.

Ở lớp 2, Softmax (0.818) và LDA (0.807) cho kết quả cao và tương đối gần nhau. Weighted Softmax thấp hơn một chút, trong khi QDA tiếp tục cho kết quả kém hơn rõ



Hình 35: Biểu đồ đường cong theo từng lớp của các mô hình

rệt.

Đối với lớp 3, Softmax (0.680) và Weighted Softmax (0.685) đạt hiệu năng tương đương và cao nhất. LDA thấp hơn, còn QDA tiếp tục cho kết quả thấp nhất trong nhóm.

Tại lớp 4, Weighted Softmax (0.485) và QDA (0.476) cải thiện đáng kể so với Softmax và LDA (khoảng 0.28–0.30). Điều này cho thấy lớp này hưởng lợi từ việc xử lý phi tuyến hoặc điều chỉnh trọng số.

Ở lớp 5, tất cả các mô hình đều có giá trị AP rất thấp (dưới 0.2), trong đó Weighted Softmax cao nhất nhưng không đáng kể. Đây là lớp khó phân loại, có thể do mất cân bằng dữ liệu hoặc đặc trưng chồng lấn.

Đối với lớp 6, Weighted Softmax đạt kết quả tốt nhất (0.374), theo sau là Softmax và LDA, trong khi QDA thấp hơn đáng kể.

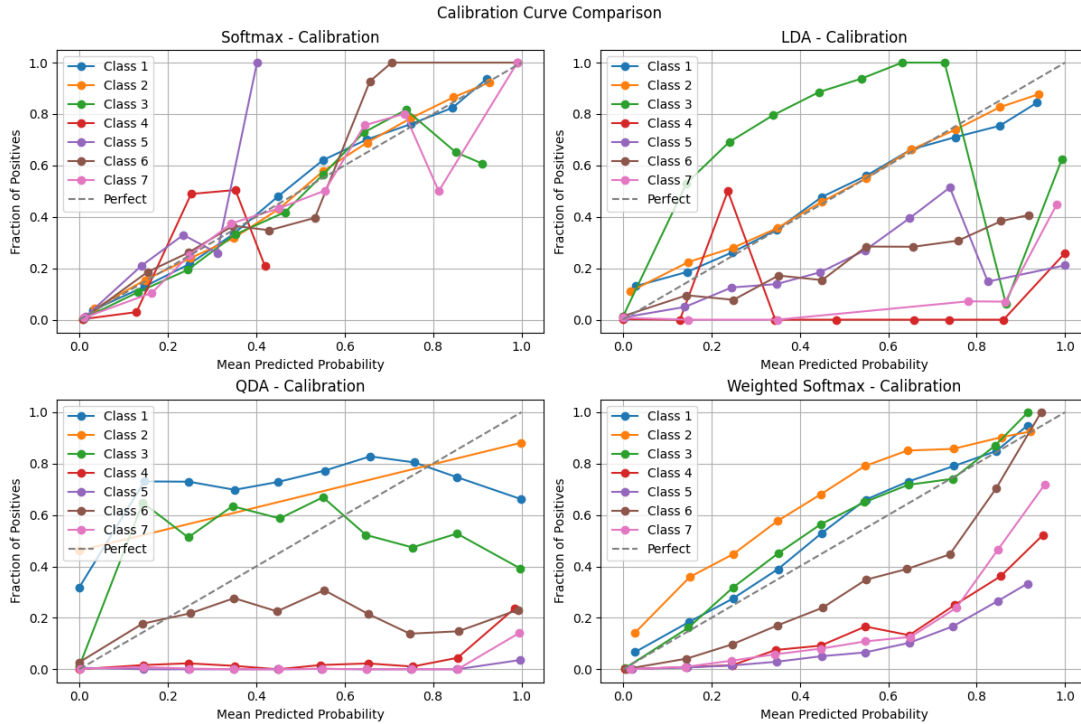
Cuối cùng, ở lớp 7, Weighted Softmax đạt hiệu năng cao nhất (0.713), tiếp theo là LDA (0.670). Softmax và QDA thấp hơn nhưng vẫn duy trì mức chấp nhận được.

Nhìn chung, Weighted Softmax là mô hình có hiệu năng tốt nhất theo thước đo Precision–Recall, đặc biệt ở các lớp khó. Softmax hoạt động tốt trên các lớp phổ biến nhưng kém hiệu quả hơn ở các lớp mất cân bằng. LDA cho kết quả ổn định nhưng không nổi bật, trong khi QDA có hiệu năng không ổn định và phụ thuộc mạnh vào từng lớp cụ thể.

Biểu đồ Calibration (Reliability Curve) Hình 36 cho ta thấy kết quả đường cong tin cậy theo phương pháp One-vs-Rest.

Dựa trên biểu đồ calibration, có thể đánh giá mức độ tin cậy của xác suất dự đoán từ các mô hình Softmax, LDA, QDA và Weighted Softmax thông qua khoảng cách so với đường tham chiếu.

Đối với mô hình Softmax, hầu hết các đường của các lớp đều bám sát đường tham chiếu, cho thấy xác suất dự đoán có độ tin cậy cao và được hiệu chỉnh tốt trên toàn bộ



Hình 36: Biểu đồ calibration các mô hình trên từng lớp

các lớp.

Ở mô hình LDA, các lớp 1 và 2 vẫn duy trì sự bám sát đường tham chiếu, tuy nhiên lớp 3 có xu hướng cao hơn ở vùng xác suất thấp nhưng giảm xuống dưới đường tham chiếu khi xác suất tăng (khoảng từ 0.8 trở đi), cho thấy hiện tượng quá tự tin ở vùng xác suất cao. Các lớp 4, 5, 6 và 7 nằm dưới đường tham chiếu, phản ánh việc mô hình dự đoán xác suất cao hơn thực tế (thiếu độ tin cậy).

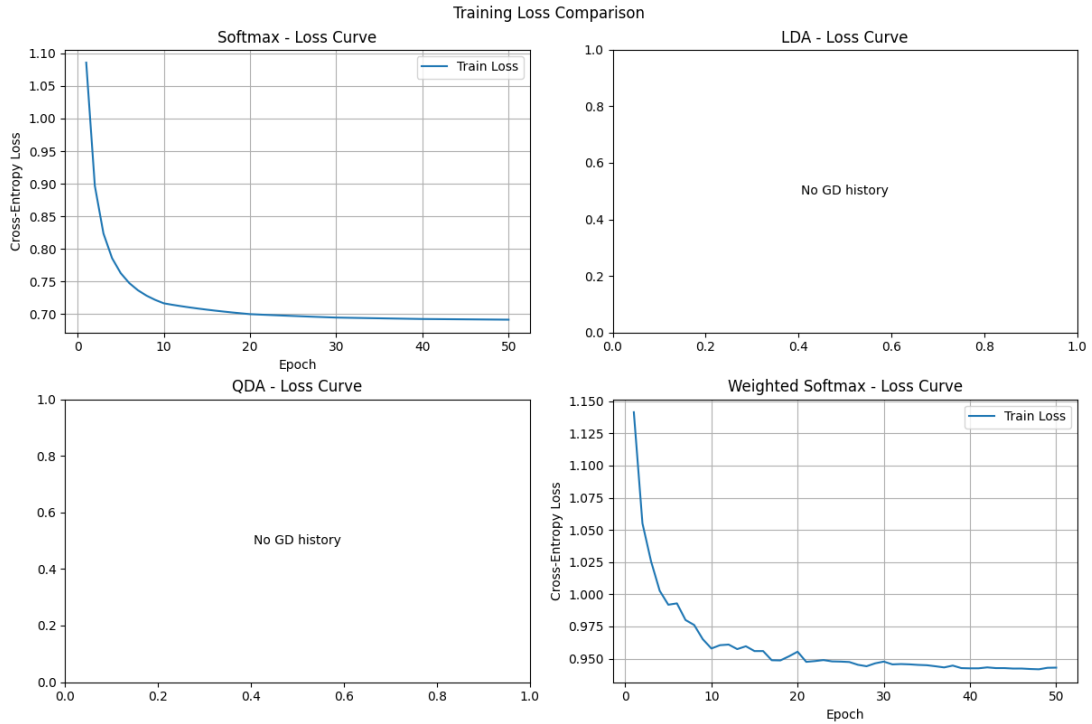
Đối với QDA, các lớp 1, 2 và 3 đều có xu hướng tương tự khi ban đầu nằm trên đường tham chiếu nhưng sau đó giảm xuống dưới ở các mức xác suất cao (lớp 1 và 2 từ khoảng 0.8, lớp 3 từ khoảng 0.6). Điều này cho thấy mô hình có xu hướng không ổn định trong hiệu chỉnh xác suất. Các lớp 4, 5, 6 và 7 phần lớn nằm dưới đường tham chiếu, thể hiện việc dự đoán xác suất không đáng tin cậy.

Đối với Weighted Softmax, các lớp 1 và 3 có xu hướng nằm hơi trên đường tham chiếu, trong khi lớp 2 cũng cao hơn. Điều này cho thấy mô hình có xu hướng hơi thiếu tự tin ở các lớp này. Ngược lại, các lớp 4, 5, 6 và 7 nằm dưới đường tham chiếu, tuy nhiên có xu hướng tiệm cận dần khi xác suất tăng. Đặc biệt, lớp 6 bắt kịp đường tham chiếu ở vùng xác suất cao (khoảng từ 0.9), trong khi các lớp còn lại cải thiện nhưng vẫn chưa đạt mức cân bằng hoàn toàn.

Nhìn chung, Softmax là mô hình có khả năng hiệu chỉnh xác suất tốt nhất. Weighted Softmax có cải thiện ở một số lớp nhưng vẫn chưa đồng đều. LDA và QDA thể hiện sự thiếu ổn định, đặc biệt ở các lớp có xác suất cao, cho thấy hạn chế trong việc cung cấp xác suất đáng tin cậy.

Biểu đồ đường cong mất mát huấn luyện (training loss curve) Hình 37 minh họa đường cong mất mát huấn luyện của các mô hình theo từng epoch.

Dựa trên biểu đồ hàm mất mát theo số epochs (cross-entropy loss), có thể thấy cả hai mô hình Softmax và Weighted Softmax đều có xu hướng giảm mạnh trong giai đoạn



Hình 37: Đường cong mất mát huấn luyện trên từng epoch

đầu và bắt đầu hội tụ sau khoảng 20 epochs, khi giá trị loss gần như ổn định và không còn giảm đáng kể.

So sánh giữa hai mô hình, đường cong loss của Softmax có dạng mượt hơn và ổn định hơn trong suốt quá trình huấn luyện. Điều này cho thấy quá trình tối ưu diễn ra tương đối ổn định khi sử dụng mini-batch.

Ngược lại, Weighted Softmax có đường cong loss dao động nhẹ (rung lắc lên xuống), đặc biệt trong giai đoạn đầu và giữa quá trình huấn luyện. Hiện tượng này là do việc sử dụng trọng số cho các lớp kết hợp với cơ chế mini-batch, dẫn đến gradient cập nhật có độ biến thiên lớn hơn giữa các batch.

Tuy nhiên, mặc dù có dao động, cả hai mô hình đều đạt trạng thái hội tụ tương đương sau khoảng 20 epochs, cho thấy Weighted Softmax vẫn đảm bảo khả năng tối ưu hóa ổn định và không ảnh hưởng đáng kể đến hiệu năng cuối cùng.

2.4 Phần nâng cao

2.4.1 Mô hình Probit

Thay vì sử dụng hàm sigmoid, mô hình Probit định nghĩa xác suất thuộc lớp dương thông qua hàm phân phối tích lũy (CDF) của phân phối Gaussian chuẩn:

$$p(C_1 | \mathbf{x}) = \Phi(\mathbf{w}^\top \mathbf{x} + w_0), \quad (123)$$

trong đó hàm $\Phi(\cdot)$ được định nghĩa bởi:

$$\Phi(a) = \int_{-\infty}^a \mathcal{N}(t | 0, 1) dt. \quad (124)$$

Mô hình Probit có thể được diễn giải từ mô hình biến ẩn (latent variable model). Cụ

thể, ta giả sử tồn tại một biến ẩn z được sinh bởi:

$$z = \mathbf{w}^\top \mathbf{x} + w_0 + \epsilon, \quad (125)$$

trong đó nhiễu $\epsilon \sim \mathcal{N}(0, 1)$. Khi đó, nhãn quan sát được xác định theo quy tắc:

$$t = \begin{cases} 1 & \text{nếu } z > 0, \\ 0 & \text{ngược lại.} \end{cases} \quad (126)$$

Từ đó suy ra xác suất có dạng:

$$p(t = 1 \mid \mathbf{x}) = \mathbb{P}(z > 0) = \Phi(\mathbf{w}^\top \mathbf{x} + w_0). \quad (127)$$

Nhờ cấu trúc dựa trên Gaussian noise, mô hình Probit thường phù hợp hơn hồi quy Logistic trong các trường hợp dữ liệu có nhãn bị nhiễu hoặc có giả định sai số tuân theo phân phối chuẩn.

So sánh với hồi quy Logistic Phần này sẽ so sánh thực nghiệm của mô hình Probit với hồi quy Logistic (sigmoid). Dữ liệu được sử dụng là dữ liệu được sinh bằng hàm `make_blobs` với 1000 điểm dữ liệu, tâm là $(-2, 0)$ và $(0, 1.5)$. Dữ liệu sau đó được biến đổi tuyến tính qua ma trận:

$$A = \begin{bmatrix} 0.4 & 0.2 \\ -0.2 & 0.5 \end{bmatrix}. \quad (128)$$

Dữ liệu được biểu diễn ở Hình 38.

Đường biên quyết định Hình 39, 40, và 41 tương ứng là đường biên quyết định tương ứng của ba mô hình hồi quy Logistic (scikit-learn), hồi quy Logistic (tự cài đặt) và Probit.

Ta thấy rằng đường biên của 3 mô hình sau khi hội tụ là khá giống nhau. Trong đó, mô hình Probit có đường biên có vẻ dốc hơn.

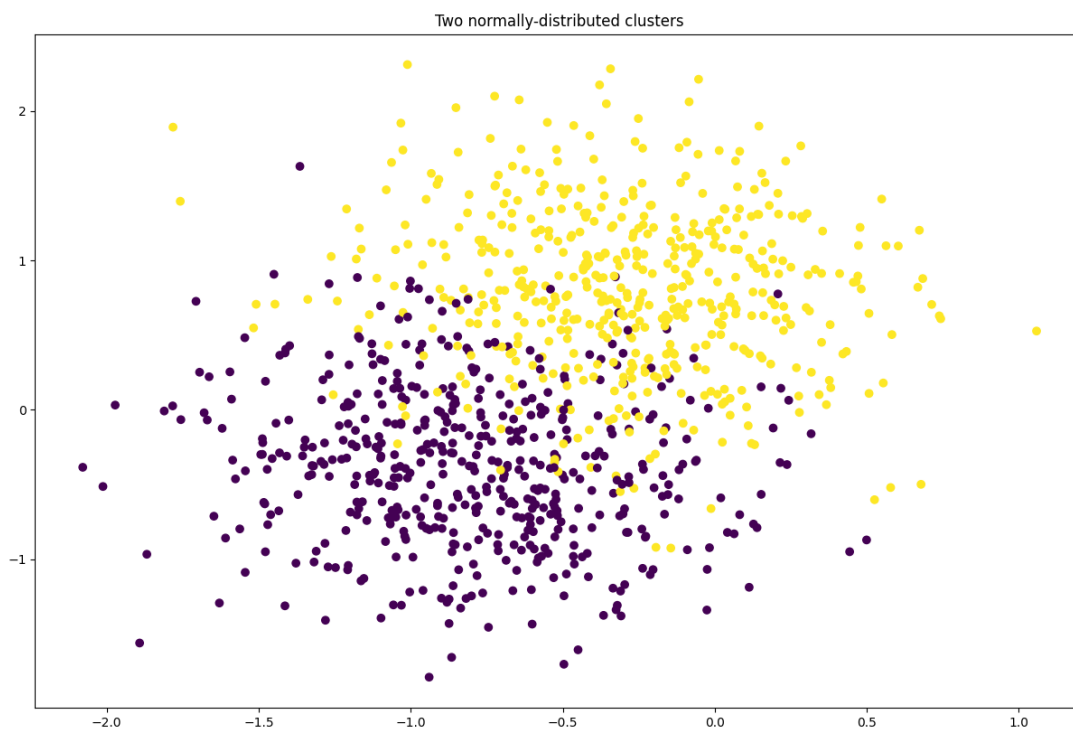
Xác suất đầu ra Hình 42 và 43 tương ứng là xác suất đầu ra của hồi quy Logistic và Probit.

Quan sát các biểu đồ xác suất đầu ra của mô hình hồi quy Logistic và Probit cho thấy sự khác biệt chủ yếu nằm ở vùng chuyển tiếp xác suất quanh đường biên quyết định, tức khu vực mà xác suất thay đổi từ gần 0 sang gần 1.

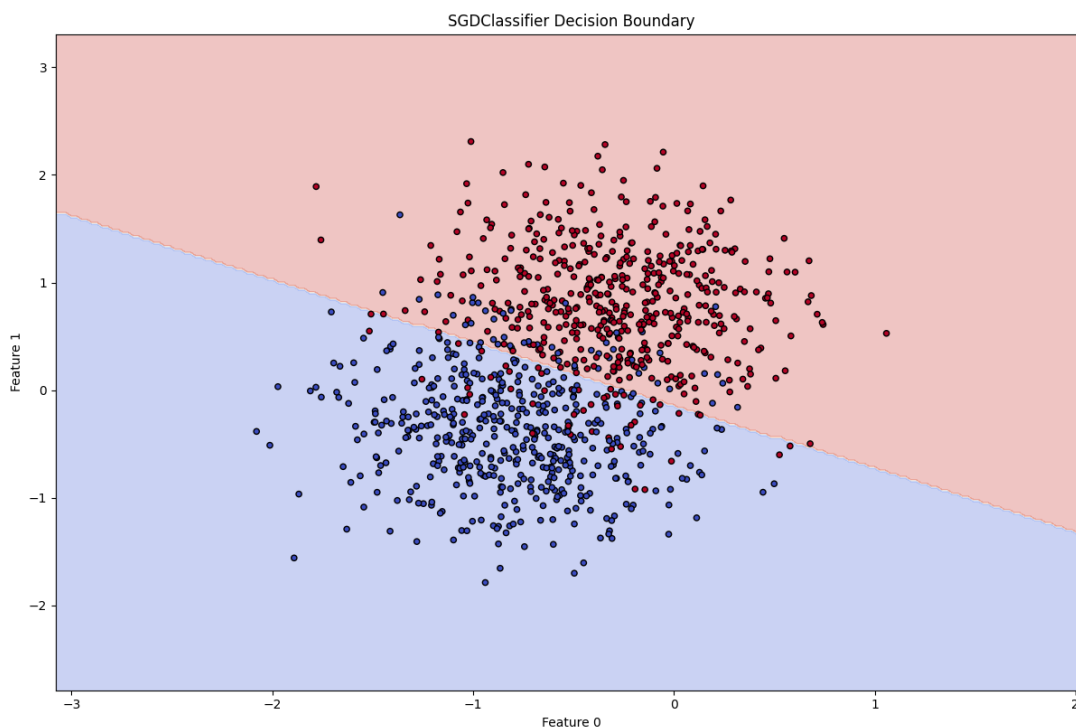
Trong cả hai mô hình, vùng này tương ứng với miền giá trị của $z = \mathbf{w}^\top \mathbf{x}$ lân cận 0, nơi hàm liên kết có độ dốc lớn nhất. Tuy nhiên, hình dạng của vùng chuyển tiếp có sự khác biệt nhẹ: mô hình Sigmoid có xu hướng phân bố sự thay đổi xác suất trên một khoảng giá trị z rộng hơn, trong khi mô hình Probit thể hiện sự tập trung biến thiên xác suất mạnh hơn quanh vùng trung tâm.

Điều này có thể được giải thích từ dạng đạo hàm của hai hàm liên kết. Cụ thể, Sigmoid có gradient phụ thuộc vào chính giá trị đầu ra, trong khi Probit có gradient tương ứng với mật độ của phân phối Gaussian chuẩn. Do đó, Probit có xu hướng tạo ra sự thay đổi xác suất rõ rệt hơn gần $z = 0$, nhưng giảm nhanh hơn khi đi xa khỏi vùng trung tâm.

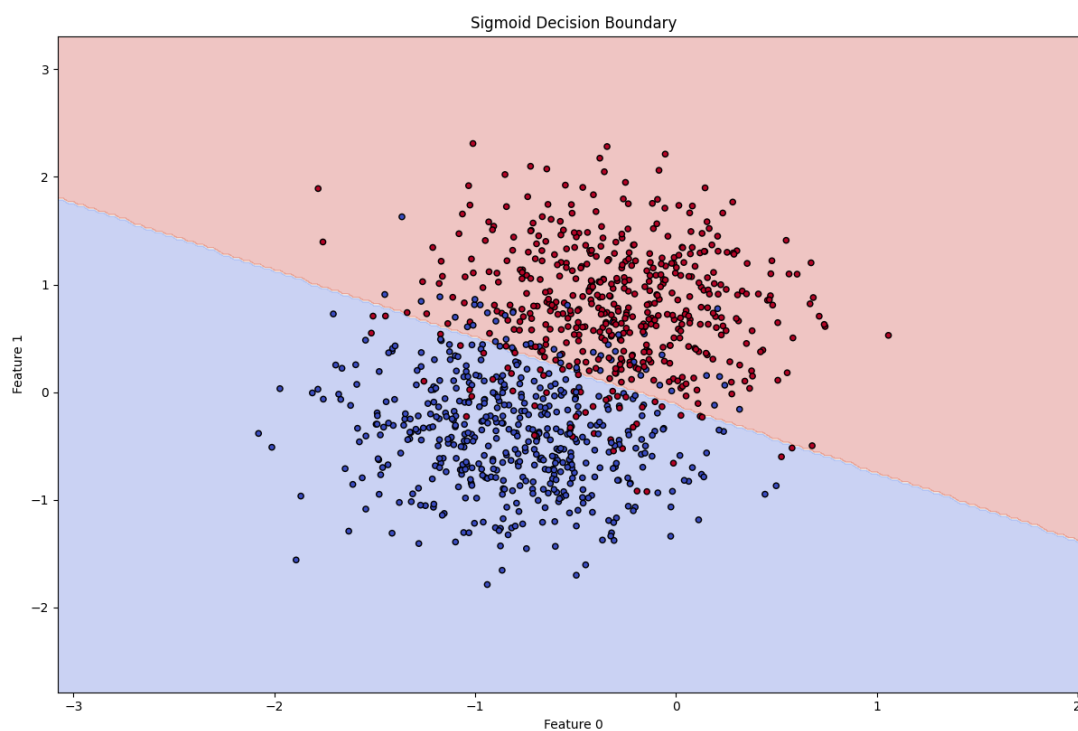
Tuy nhiên, cần lưu ý rằng sự khác biệt về vùng chuyển tiếp giữa hai mô hình không hoàn toàn cố định, mà phụ thuộc vào tham số học được trong quá trình tối ưu. Trên thực nghiệm, cả hai mô hình thường cho đường biên quyết định tương đối tương đồng, và khác biệt chủ yếu thể hiện ở cách phân bố xác suất cục bộ quanh decision boundary.



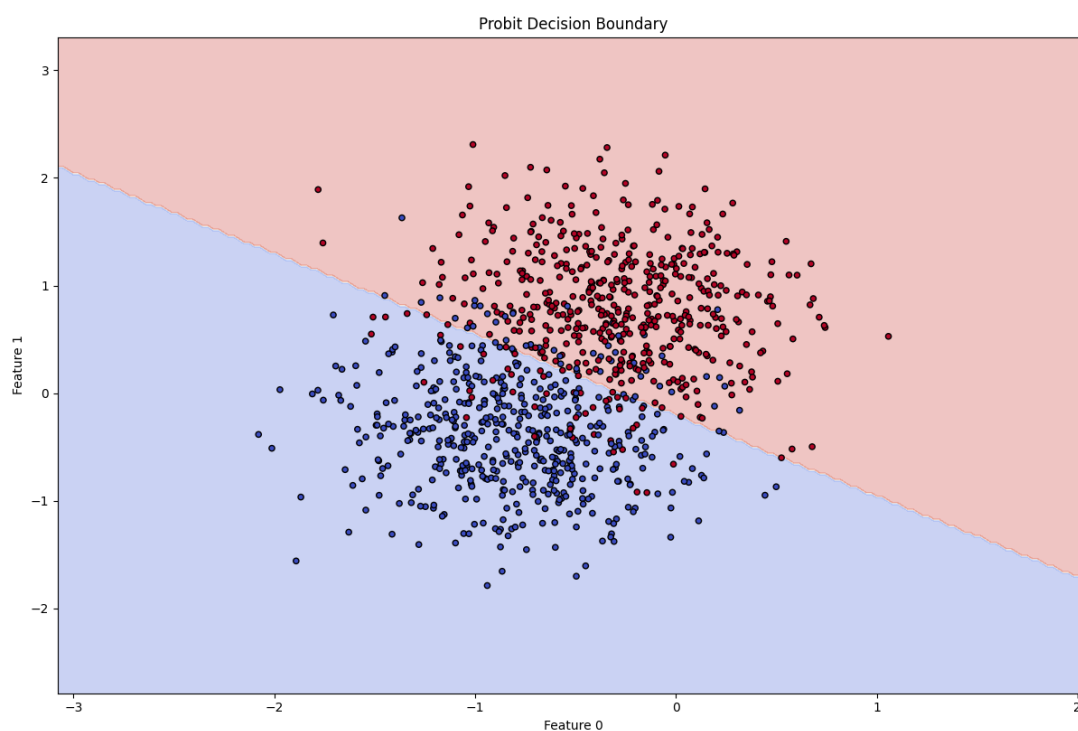
Hình 38: Dữ liệu tạo sinh cho phần mô hình Probit và xấp xỉ Laplace



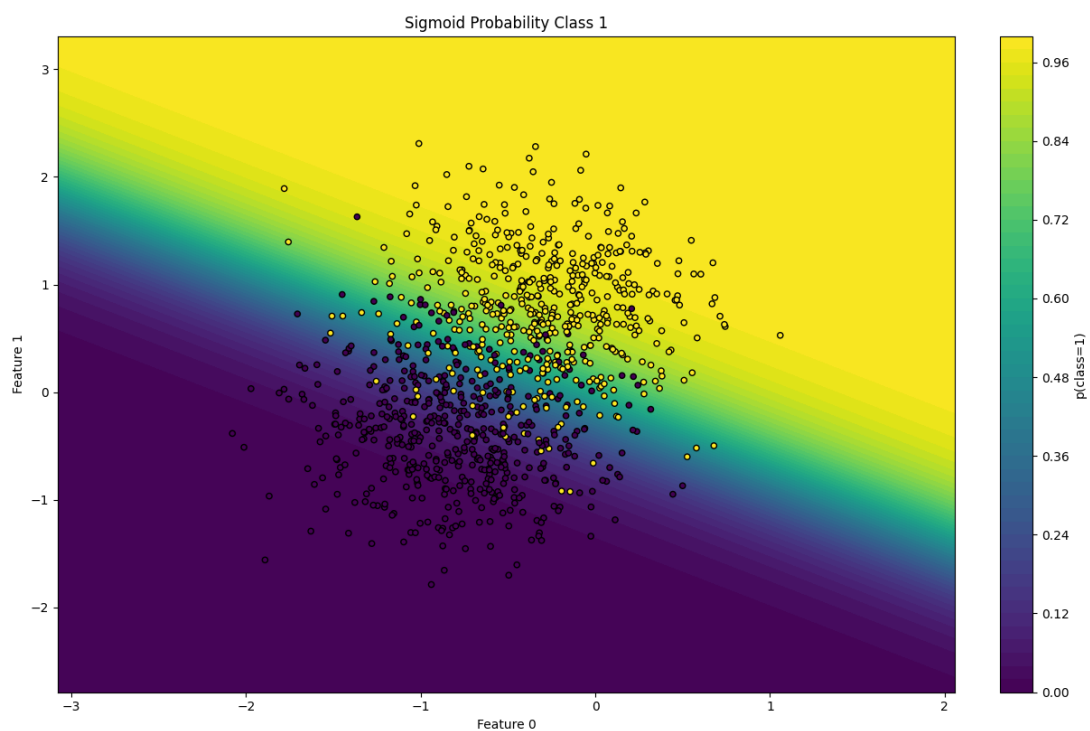
Hình 39: Đường biên quyết định của mô hình SGDClassifier (scikit-learn)



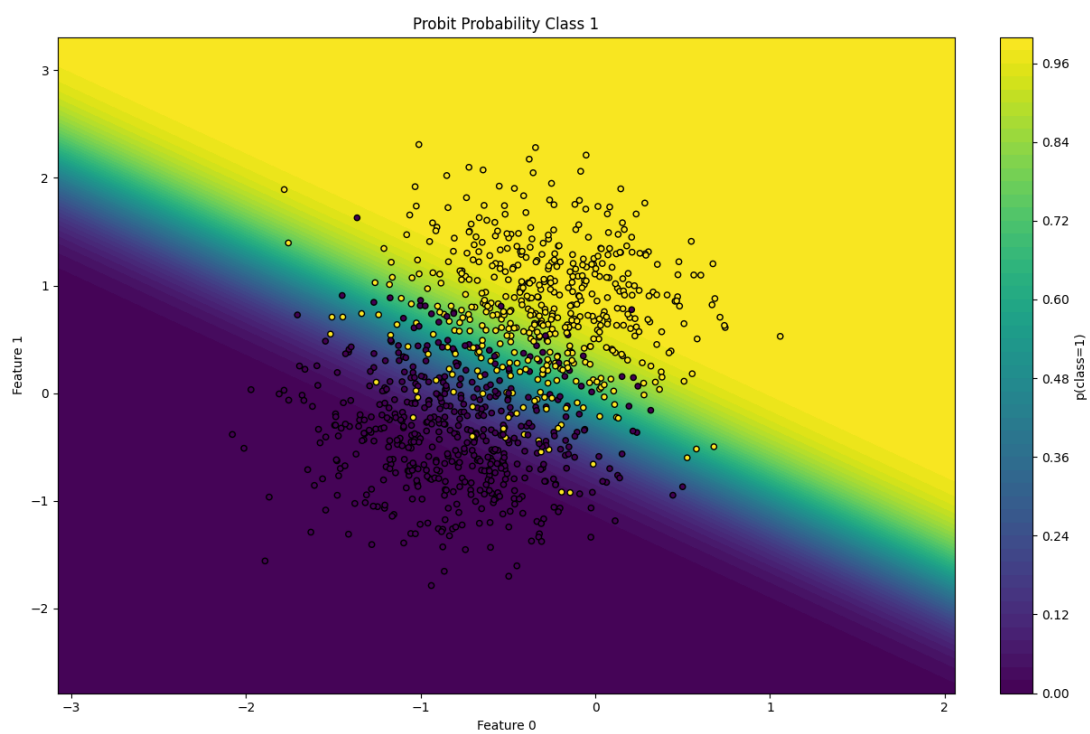
Hình 40: Đường biên quyết định của mô hình hồi quy Logistic



Hình 41: Đường biên quyết định của mô hình Probit



Hình 42: Xác suất đầu ra của mô hình hồi quy Logistic



Hình 43: Xác suất đầu ra của mô hình Probit

Độ nhạy với nhãn nhiễu Bảng 32 và 33 cho ta thấy độ nhạy với nhãn nhiễu của hai mô hình Probit và hồi quy Logistic. Tỷ lệ dữ liệu bị thay đổi nhãn là 0.2. Trong đó xác suất bị thay đổi tỷ lệ với khoảng cách tới đường biên quyết định của model baseline (SGDClassifier của scikit-learn).

Bảng 32: Độ nhạy với nhãn nhiễu trên 2 mô hình Probit và hồi quy tuyến tính (phần 1)

Model	Acc_sensitivity	F1_sensitivity	Precision_sensitivity
Sigmoid	0.0037	0.0037	0.0039
Probit	0.0	0.0	0.0

Bảng 33: Độ nhạy với nhãn nhiễu trên 2 mô hình Probit và hồi quy tuyến tính (phần 2)

Model	Recall_sensitivity	AUC_sensitivity
Sigmoid	0.0037	-0.0001
Probit	0.0	-0.0001

Ta thấy rằng Probit hầu như không thay đổi hiệu suất trên các chỉ số. Điều này cho thấy mô hình này ổn định hơn với nhãn nhiễu. Tuy nhiên sự sai khác so với hồi quy tuyến tính khá nhỏ nên cũng chưa có ý nghĩa rõ ràng.

2.4.2 Xấp xỉ Laplace

Xấp xỉ Laplace là một phương pháp xấp xỉ phân phối hậu nghiệm của mô hình Logistic Regression bằng một phân phối Gaussian, với tâm tại nghiệm MAP (Maximum A Posteriori). Ý tưởng cốt lõi là khai triển bậc hai của log-posterior quanh điểm cực đại $\mathbf{w}_{MAP}$.

Cụ thể, ta có posterior:

$$p(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})p(\mathbf{w}). \quad (129)$$

Ta xấp xỉ log-posterior bằng khai triển Taylor bậc hai tại $\mathbf{w}_{MAP}$:

$$\ln p(\mathbf{w} \mid \mathcal{D}) \approx \ln p(\mathbf{w}_{MAP} \mid \mathcal{D}) - \frac{1}{2}(\mathbf{w} - \mathbf{w}_{MAP})^\top \mathbf{A}(\mathbf{w} - \mathbf{w}_{MAP}), \quad (130)$$

trong đó ma trận Hessian âm được định nghĩa là:

$$\mathbf{A} = -\nabla \nabla \ln p(\mathbf{w} \mid \mathcal{D}) \Big|_{\mathbf{w}=\mathbf{w}_{MAP}}. \quad (131)$$

Từ đó, posterior được xấp xỉ bởi phân phối Gaussian:

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{w}_{MAP}, \mathbf{A}^{-1}), \quad (132)$$

trong đó, $\mathbf{A}^{-1}$ đóng vai trò là ma trận hiệp phương sai, mô tả độ bất định của các tham số mô hình.

Bất định của đường biên quyết định Trong Logistic Regression, đường biên quyết định được xác định bởi:

$$\mathbf{w}^\top \mathbf{x} = 0. \quad (133)$$

Với xấp xỉ Laplace, do $\mathbf{w}$ là một biến ngẫu nhiên Gaussian, biên quyết định trở thành một phân phối thay vì một đường cố định.

Gọi phương sai của logit tại một điểm $\mathbf{x}$ là:

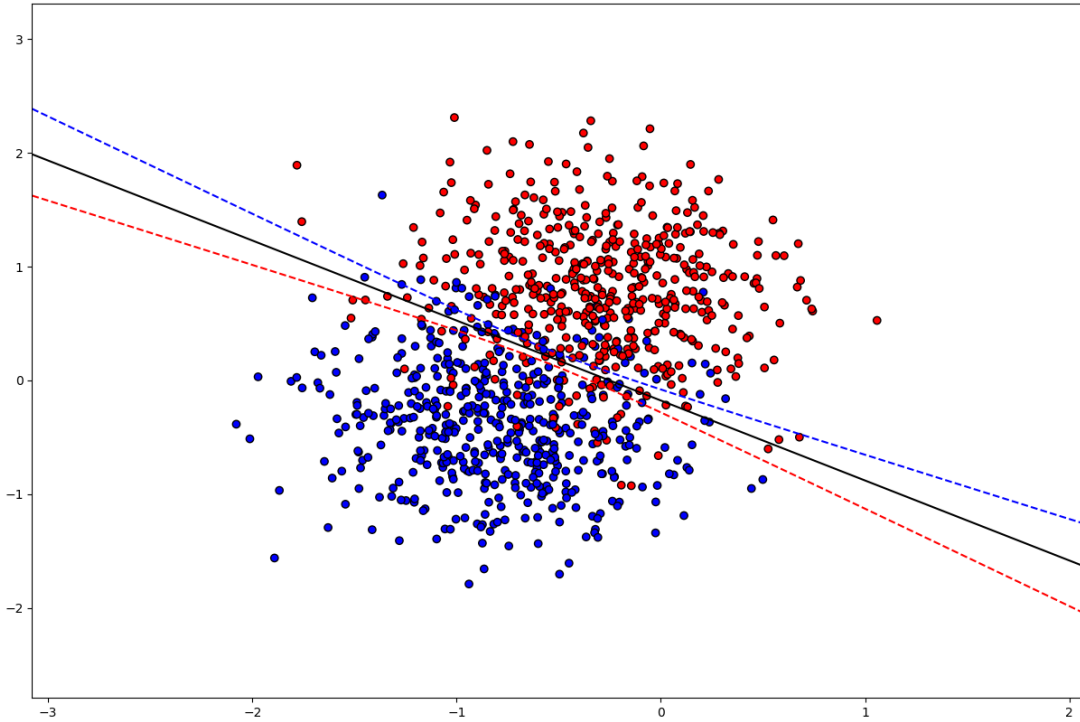
$$\sigma^2(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}^{-1} \mathbf{x}. \quad (134)$$

Khi đó, ta có thể biểu diễn vùng bất định của biên quyết định xấp xỉ bởi:

$$\mathbf{w}_{MAP}^\top \mathbf{x} \pm 2\sigma(\mathbf{x}). \quad (135)$$

Vùng này tương ứng với khoảng tin cậy xấp xỉ 95%, thể hiện mức độ dao động của ranh giới phân lớp do bất định tham số.

Hình 44 cho thấy vùng bất định của đường biên quyết định dựa trên mô hình hồi quy logistic với tập dữ liệu được sinh trong phần Probit (xem biểu diễn dữ liệu này tại Hình 38).



Hình 44: Vùng bất định của đường biên quyết định dựa trên xấp xỉ Laplace

Ta thấy vùng bất định trong hình khá hẹp. Điều này cho thấy mô hình tự tin cao và biên quyết định ổn định. Có thể do dữ liệu tách biệt tốt.

2.4.3 Kernel Logistic Regression

Ý tưởng Kernel Logistic Regression là mở rộng của Logistic Regression nhằm xử lý các bài toán phân loại phi tuyến thông qua *kernel trick*. Thay vì học một vector trọng số trong không gian đặc trưng ban đầu, mô hình được biểu diễn trong dạng đối ngẫu (dual form), trong đó hàm quyết định phụ thuộc vào các điểm huấn luyện thông qua một hàm kernel.

Hàm quyết định của mô hình được viết dưới dạng:

$$f(x) = \sum_{i=1}^n \alpha_i t_i K(\mathbf{x}_i, \mathbf{x}), \quad (136)$$

trong đó α_i là các tham số cần học, $t_i \in \{-1, 1\}$ là nhãn huấn luyện, và $K(\mathbf{x}_i, \mathbf{x})$ là hàm kernel.

Trong trường hợp sử dụng kernel Gaussian RBF, ta có:

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2). \quad (137)$$

Xác suất dự đoán được mô hình hóa thông qua hàm sigmoid:

$$P(t = 1 | x) = \sigma(f(\mathbf{x})) = \frac{1}{1 + \exp(-f(\mathbf{x}))}. \quad (138)$$

Hàm mất mát của Kernel Logistic Regression là log-loss trong không gian kernel, kết hợp với regularization:

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^n \log(1 + \exp(-t_i f(\mathbf{x}_i))) + \lambda \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha}, \quad (139)$$

trong đó $\mathbf{K}$ là ma trận kernel với $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$.

Việc tối ưu mô hình được thực hiện trong không gian đối ngẫu, thay vì tối ưu trực tiếp trên vector trọng số như Logistic Regression tuyến tính. Điều này cho phép mô hình học được các ranh giới quyết định phi tuyến thông qua ánh xạ ngầm sang không gian đặc trưng có chiều cao (thậm chí vô hạn chiều).

So sánh Logistic Regression và Kernel Logistic Regression Logistic Regression là mô hình tuyến tính, trong đó hàm quyết định được biểu diễn dưới dạng:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b. \quad (140)$$

Do đó, ranh giới quyết định của mô hình luôn là một siêu phẳng trong không gian đầu vào. Điều này khiến Logistic Regression phù hợp với các bài toán có cấu trúc tuyến tính hoặc gần tuyến tính, nhưng gặp hạn chế lớn khi dữ liệu có phân bố phi tuyến.

Ngược lại, Kernel Logistic Regression thay thế tích vô hướng trực tiếp bằng hàm kernel:

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i t_i K(\mathbf{x}_i, \mathbf{x}). \quad (141)$$

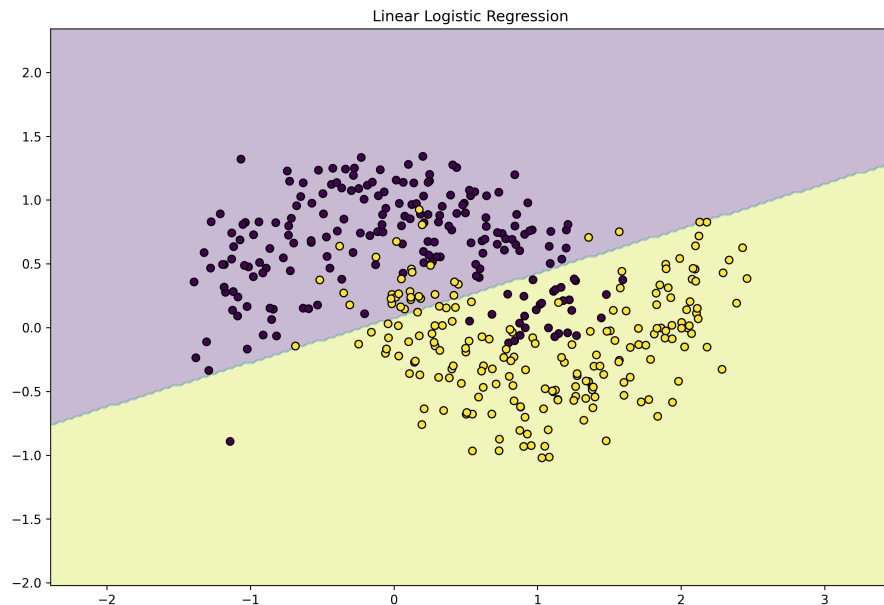
Nhờ kernel trick, mô hình có thể ánh xạ ngầm dữ liệu sang không gian đặc trưng có chiều cao hơn, giúp biểu diễn được các quan hệ phi tuyến phức tạp mà không cần tính toán tường minh ánh xạ.

Về mặt hình học, Logistic Regression tạo ra ranh giới quyết định tuyến tính, trong khi Kernel Logistic Regression có thể tạo ra ranh giới cong linh hoạt tùy thuộc vào lựa chọn kernel (ví dụ RBF kernel).

Về độ phức tạp, Logistic Regression có chi phí tính toán thấp hơn và mở rộng tốt với dữ liệu lớn. Ngược lại, Kernel Logistic Regression phụ thuộc vào ma trận kernel kích thước $n \times n$, dẫn đến chi phí tính toán và bộ nhớ cao hơn.

Tóm lại, Logistic Regression phù hợp với dữ liệu tuyến tính và bài toán lớn, trong khi Kernel Logistic Regression phù hợp với dữ liệu phi tuyến nhưng đánh đổi về chi phí tính toán.

So sánh trên dữ liệu không phân chia tuyến tính Hình 45 và Hình 46 thể hiện đường biên quyết định của Logistic Regression và Kernel Logistic Regression trên dữ liệu không phân chia tuyến tính. Dễ thấy với Logistic Regression thì đường biên này là một đường thẳng do đó vẫn còn nhiều điểm không thể phân loại đúng. Trong khi đó Kernel Logistic Regression có đường biên quyết định là đường cong phi tuyến do đó có thể phân loại được những điểm này và gần như phân loại đúng tất cả trừ vài điểm dữ liệu nhiễu.



Hình 45: Đường biên quyết định của Logistic Regression cho dữ liệu không phân chia tuyến tính

Kết luận Từ kết quả thực nghiệm, có thể thấy rằng Logistic Regression chỉ phù hợp với các bài toán có ranh giới quyết định tuyến tính, do giới hạn trong việc học một siêu phẳng trong không gian đầu vào.

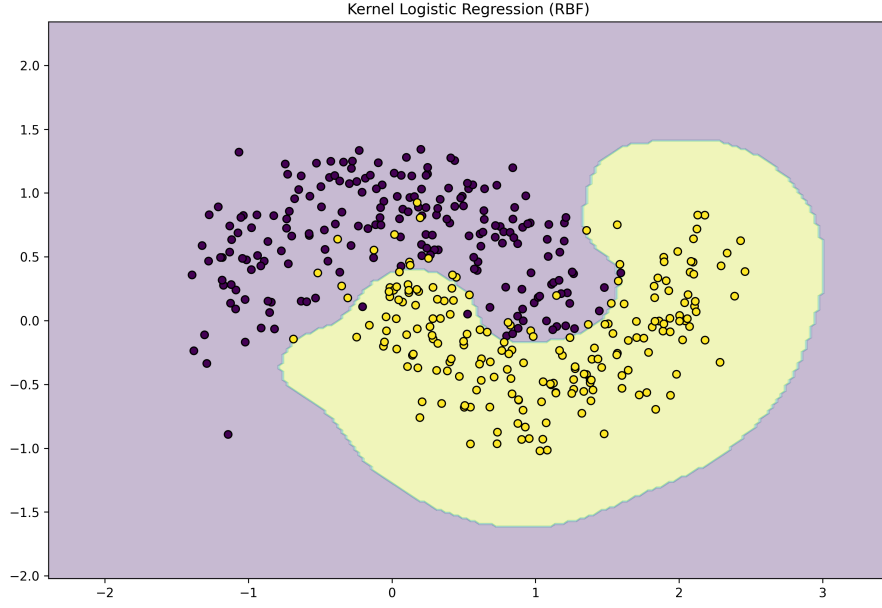
Ngược lại, Kernel Logistic Regression có khả năng mô hình hóa các quan hệ phi tuyến nhờ kernel trick, cho phép xây dựng ranh giới quyết định linh hoạt hơn và phù hợp với các cấu trúc dữ liệu phức tạp như XOR.

Tuy nhiên, khả năng biểu diễn mạnh hơn cũng đi kèm với chi phí tính toán cao hơn và độ nhạy với tham số kernel, đặc biệt là trong các trường hợp dữ liệu có nhiễu hoặc số lượng mẫu lớn.

Do đó, việc lựa chọn giữa hai mô hình cần cân bằng giữa độ phức tạp của dữ liệu và yêu cầu về hiệu năng tính toán.

2.4.4 Gaussian Naive Bayes vs. LDA

Lý thuyết Gaussian Naive Bayes (GNB) và Linear Discriminant Analysis (LDA) đều là các mô hình phân loại dựa trên xác suất, nhưng khác nhau ở mức độ giả thiết về phân phối dữ liệu.



Hình 46: Đường biên quyết định của Kernel Logistic Regression cho dữ liệu không phân chia tuyến tính

Gaussian Naive Bayes giả định rằng các đặc trưng độc lập có điều kiện theo lớp:

$$p(\mathbf{x} \mid t = k) = \prod_{j=1}^d p(x_j \mid t = k), \quad (142)$$

với mỗi đặc trưng tuân theo phân phối Gaussian:

$$p(x_j \mid t = k) = \mathcal{N}(\mu_{kj}, \sigma_{kj}^2). \quad (143)$$

Do giả thiết độc lập, ma trận hiệp phương sai của mỗi lớp là ma trận đường chéo, không mô hình hóa tương quan giữa các đặc trưng.

Ngược lại, LDA giả định rằng dữ liệu mỗi lớp tuân theo phân phối Gaussian đa biến với cùng ma trận hiệp phương sai:

$$p(\mathbf{x} \mid t = k) = \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}). \quad (144)$$

Điều này cho phép LDA mô hình hóa mối tương quan giữa các đặc trưng, vì $\boldsymbol{\Sigma}$ không bị ràng buộc là đường chéo.

Hàm phân biệt của LDA có dạng:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k. \quad (145)$$

Từ đó, biên quyết định của LDA là tuyến tính, trong khi GNB tạo biên phi tuyến do giả thiết độc lập từng chiều.

Thực nghiệm Trong thực tế, hiệu năng của Gaussian Naive Bayes và LDA phụ thuộc mạnh vào đặc điểm dữ liệu.

Gaussian Naive Bayes thường hoạt động tốt hơn LDA trong các trường hợp:

- Số lượng mẫu nhỏ so với số chiều dữ liệu
- Các đặc trưng gần như độc lập hoặc đã được tiền xử lý
- Dữ liệu dạng sparse như văn bản (bag-of-words, TF-IDF)

Trong các trường hợp này, việc ước lượng ma trận hiệp phương sai trong LDA trở nên không ổn định, trong khi GNB chỉ cần ước lượng trung bình và phương sai từng chiều.

Ngược lại, LDA thường cho kết quả tốt hơn khi:

- Các đặc trưng có tương quan đáng kể
- Số lượng mẫu đủ lớn để ước lượng tốt
- Cấu trúc dữ liệu gần với giả định Gaussian đa biến

Tóm lại, GNB có giả thiết mạnh hơn nhưng ít tham số hơn, giúp ổn định trong các bài toán dữ liệu nhỏ hoặc nhiều chiều. Trong khi đó, LDA tổng quát hơn và khai thác tốt mối quan hệ giữa các đặc trưng khi dữ liệu đủ lớn và phù hợp giả định.

2.4.5 Phân tích VC Dimension và Structural Risk Minimization

VC Dimension của bộ phân lớp tuyến tính Xét lớp giả thuyết $\mathcal{H}$ gồm các bộ phân lớp tuyến tính trong không gian $\mathbb{R}^D$:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

VC dimension (Vapnik–Chervonenkis dimension) của $\mathcal{H}$ là số lượng điểm lớn nhất có thể được *shatter* bởi lớp giả thuyết này. Một tập điểm được gọi là *shatter* nếu với mọi cách gán nhãn nhị phân, luôn tồn tại một siêu phẳng phân tách đúng.

Một kết quả kinh điển trong lý thuyết học máy cho thấy:

$$VC(\mathcal{H}) = D + 1$$

Điều này có nghĩa là:

- Với $D + 1$ điểm tổng quát, luôn tồn tại một siêu phẳng phân tách mọi cách gán nhãn
- Nhưng với $D + 2$ điểm, không phải mọi cấu hình nhãn đều có thể phân tách tuyến tính

Do đó, VC dimension phản ánh trực tiếp độ phức tạp (capacity) của mô hình tuyến tính.

Liên hệ với khả năng tổng quát hóa Theo lý thuyết học thống kê, VC dimension cho phép xây dựng các chặn trên cho sai số tổng quát hóa. Cụ thể, với xác suất cao, ta có:

$$R(f) \leq \hat{R}(f) + O\left(\sqrt{\frac{VC(\mathcal{H})}{N}}\right)$$

trong đó:

- $R(f)$ là rủi ro thực (true risk)
- $\hat{R}(f)$ là rủi ro thực nghiệm (empirical risk)
- N là số mẫu

Hệ quả quan trọng:

- VC dimension càng lớn $\Rightarrow$ mô hình càng linh hoạt nhưng dễ overfitting
- VC dimension nhỏ $\Rightarrow$ mô hình đơn giản nhưng có thể underfitting

Do đó, kiểm soát độ phức tạp mô hình là yếu tố then chốt để đảm bảo khả năng tổng quát hóa.

Structural Risk Minimization (SRM) Structural Risk Minimization (SRM) là một nguyên lý lựa chọn mô hình dựa trên việc cân bằng giữa:

- Sai số huấn luyện (empirical risk)
- Độ phức tạp mô hình (model capacity)

Ý tưởng cốt lõi của SRM là xây dựng một dãy các lớp giả thuyết lồng nhau:

$$\mathcal{H}_1 \subset \mathcal{H}_2 \subset \dots \subset \mathcal{H}_m$$

trong đó độ phức tạp (ví dụ VC dimension) tăng dần. Sau đó chọn lớp tối ưu bằng cách tối thiểu hóa chặn trên của rủi ro:

$$\hat{R}(f) + \Omega(\mathcal{H})$$

với $\Omega(\mathcal{H})$ là hàm phạt phụ thuộc vào độ phức tạp của lớp giả thuyết.

Liên hệ với Logistic Regression và tham số C Trong Logistic Regression với regularization L_2 , tham số C (nghịch đảo của λ) đóng vai trò điều khiển độ phức tạp hiệu dụng của mô hình:

- C nhỏ $\Rightarrow$ regularization mạnh $\Rightarrow$ mô hình đơn giản
- C lớn $\Rightarrow$ regularization yếu $\Rightarrow$ mô hình phức tạp hơn

Mặc dù VC dimension lý thuyết của bộ phân lớp tuyến tính là $D+1$, nhưng regularization làm giảm *effective capacity* của mô hình, từ đó cải thiện khả năng tổng quát hóa.

Do đó, việc thay đổi C có thể được xem như xây dựng một chuỗi các mô hình với độ phức tạp tăng dần, phù hợp với nguyên lý SRM.

Kết quả thực nghiệm và phân tích SRM Thực nghiệm được tiến hành trên tập dữ liệu Covtype với:

$$D = 54, \quad \text{VC dimension} = D + 1 = 55$$

Mô hình sử dụng là Logistic Regression với regularization L_2 , trong đó tham số $C = \frac{1}{\lambda}$ điều khiển độ phức tạp hiệu dụng của mô hình. Các giá trị C được lựa chọn theo thang log:

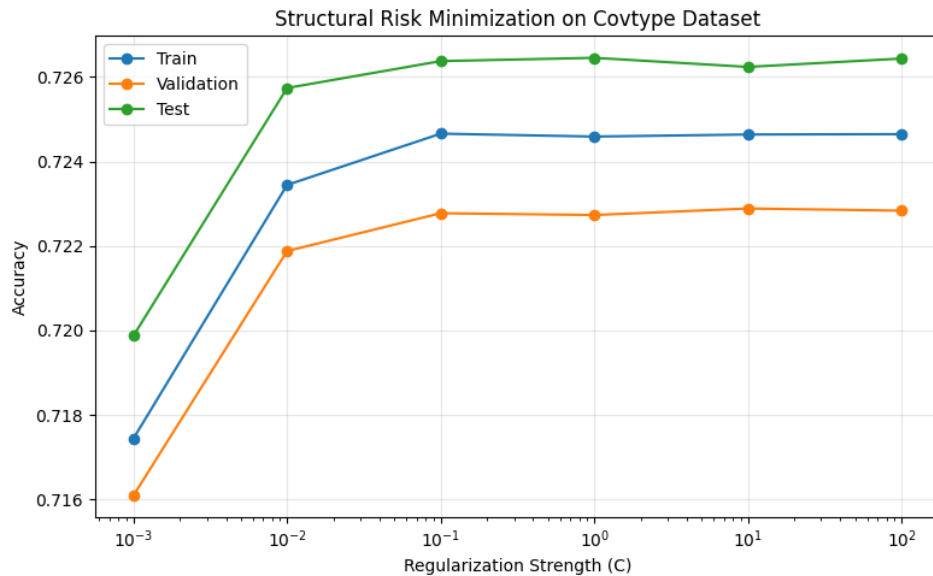
$$C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2\}$$

Kết quả Hiệu năng của mô hình được đo bằng accuracy trên tập train, validation và test:

C	Train	Validation	Test
0.001	0.7174	0.7161	0.7199
0.01	0.7234	0.7219	0.7257
0.1	0.7247	0.7228	0.7264
1	0.7246	0.7227	0.7265
10	0.7246	0.7229	0.7262
100	0.7246	0.7228	0.7264

Giá trị tối ưu theo validation:

$$C^* = 10, \quad \text{Validation Accuracy} = 0.7229$$



Hình 47: Hiệu năng của Logistic Regression theo tham số C trên tập Covtype.

Biểu diễn trực quan

Phân tích xu hướng Từ Bảng kết quả và Hình 47, có thể rút ra các nhận xét quan trọng:

- Khi C nhỏ (10^{-3}), regularization mạnh làm mô hình bị *underfitting*, thể hiện qua accuracy thấp trên cả train và validation.
- Khi C tăng từ 10^{-3} đến 10^{-1} , accuracy tăng nhanh. Điều này cho thấy mô hình dần thoát khỏi trạng thái underfitting và học được cấu trúc dữ liệu hiệu quả hơn.
- Từ $C \geq 10^{-1}$, các đường cong trong Hình 47 bắt đầu **bão hòa**. Cụ thể:

$$\text{Validation Accuracy} \approx 0.723$$

cho thấy mô hình đã đạt đến giới hạn biểu diễn trong không gian đặc trưng tuyến tính.

- Khoảng cách giữa đường train và validation rất nhỏ (dưới 0.002), và đường test gần như trùng với validation, cho thấy:
 - Không có hiện tượng overfitting đáng kể
 - Mô hình tổng quát hóa tốt
- Khi C lớn (≥ 10), việc tăng độ phức tạp không mang lại cải thiện đáng kể, thể hiện qua việc các đường cong gần như phẳng.

Diễn giải dưới góc nhìn SRM Theo nguyên lý Structural Risk Minimization:

$$R(f) \leq \hat{R}(f) + \Omega(\mathcal{H})$$

Trong đó:

- $\hat{R}(f)$: sai số huấn luyện
- $\Omega(\mathcal{H})$: độ phức tạp mô hình

Kết quả thực nghiệm cho thấy:

- Khi C nhỏ: Ω nhỏ nhưng $\hat{R}$ lớn $\Rightarrow$ underfitting
- Khi C tăng: $\hat{R}$ giảm nhanh trong khi Ω tăng vừa phải
- Khi C lớn: cả $\hat{R}$ và validation error gần như không đổi $\Rightarrow$ đạt cân bằng tối ưu giữa bias và variance

Điều này phản ánh chính xác cơ chế của SRM trong việc lựa chọn mô hình tối ưu.

Vai trò của VC Dimension Mặc dù VC dimension lý thuyết của mô hình là:

$$VC = 55$$

trong thực tế, hiệu năng tổng quát hóa còn phụ thuộc mạnh vào kích thước dữ liệu. Với N rất lớn, ta có:

$$O\left(\sqrt{\frac{VC}{N}}\right) \ll 1$$

Do đó:

- Sai số tổng quát hóa được kiểm soát chặt chẽ
- Mô hình ít bị overfitting ngay cả khi giảm regularization

Ngoài ra, regularization đóng vai trò giảm *effective capacity* của mô hình, mặc dù không làm thay đổi VC dimension lý thuyết.

Nhận xét quan trọng

- Ảnh hưởng của C chỉ rõ rệt trong giai đoạn đầu, sau đó giảm dần.
- Điều này cho thấy giới hạn chính của mô hình không nằm ở regularization mà ở:
 - Giả định tuyến tính
 - Khả năng biểu diễn của đặc trưng
- Việc tăng độ phức tạp mô hình (qua C) không thể vượt qua giới hạn này.

Kết luận

- Logistic Regression với regularization L_2 hoạt động ổn định trên dữ liệu lớn
- SRM giúp lựa chọn giá trị C tối ưu một cách hiệu quả
- Tuy nhiên, khi dữ liệu đủ lớn, vai trò của regularization giảm đáng kể
- Để cải thiện hiệu năng hơn nữa, cần sử dụng:
 - Mô hình phi tuyến
 - Hoặc feature engineering

3 Kết nối hồi quy và phân lớp

3.1 Logistic Regression như một bài toán hồi quy

Mặc dù thường được sử dụng cho bài toán phân loại nhị phân, Logistic Regression thực chất là một dạng hồi quy phi tuyến. Cụ thể, mô hình giả định rằng xác suất của biến mục tiêu $t \in \{0, 1\}$ được xác định thông qua một hàm sigmoid áp dụng lên tổ hợp tuyến tính của đầu vào:

$$p(t = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}. \quad (146)$$

Do đó, thay vì dự đoán trực tiếp nhãn lớp, mô hình thực hiện hồi quy trên xác suất (hoặc kỳ vọng) của t . Điều này cho thấy Logistic Regression là một mô hình hồi quy với hàm liên kết phi tuyến (sigmoid), ánh xạ từ không gian tuyến tính sang miền xác suất $[0, 1]$.

3.2 Khung Generalized Linear Models (GLM)

Generalized Linear Models (GLM) cung cấp một khung thống nhất cho nhiều mô hình hồi quy khác nhau (Murphy, 2022, Chap. 12). Trong GLM, ta giả định rằng:

- Biến đầu ra t tuân theo một phân phối thuộc exponential family.
- Kỳ vọng của t , ký hiệu $\mu = \mathbb{E}[t|\mathbf{x}]$, liên hệ với tổ hợp tuyến tính $\eta = \mathbf{w}^\top \mathbf{x}$ thông qua một hàm liên kết g :

$$g(\mu) = \eta = \mathbf{w}^\top \mathbf{x}. \quad (147)$$

Các mô hình quen thuộc là các trường hợp đặc biệt của GLM:

- Linear Regression: phân phối Gaussian, link function là hàm đồng nhất $g(\mu) = \mu$.
- Logistic Regression: phân phối Bernoulli, link function là logit $g(\mu) = \log \frac{\mu}{1-\mu}$.
- Poisson Regression: phân phối Poisson, link function là log $g(\mu) = \log \mu$.

Như vậy, GLM cho phép ta thay đổi phân phối của dữ liệu đầu ra và hàm liên kết một cách linh hoạt nhưng vẫn giữ cấu trúc tuyến tính trong không gian tham số.

3.3 Vai trò của exponential family

Một điểm then chốt của GLM là giả định rằng phân phối của t thuộc exponential family, có dạng:

$$p(t|\theta) = h(t) \exp(\theta^\top \phi(t) - A(\theta)). \quad (148)$$

Các phân phối như Gaussian, Bernoulli, và Poisson đều thuộc họ này. Điều này mang lại một cấu trúc toán học thống nhất với các lợi ích sau:

- Kỳ vọng và phương sai có thể được suy ra trực tiếp từ hàm $A(\theta)$:

$$\mathbb{E}[t] = A'(\theta), \quad \text{Var}(t) = A''(\theta). \quad (149)$$

- Hàm log-likelihood có dạng lồi đối với nhiều trường hợp, giúp tối ưu hóa hiệu quả.
- Tồn tại các sufficient statistics hữu hạn chiều, giúp nén dữ liệu mà không mất thông tin.

Do đó, cả Linear Regression (Gaussian) và Logistic Regression (Bernoulli) đều nằm trong một khuôn khổ thống nhất, nơi sự khác biệt chỉ nằm ở lựa chọn phân phối và hàm liên kết.

3.4 Tổng kết

Logistic Regression có thể được hiểu là một dạng hồi quy với hàm sigmoid. GLM mở rộng ý tưởng này bằng cách cho phép nhiều phân phối đầu ra khác nhau thông qua exponential family và các hàm liên kết tương ứng. Chính exponential family cung cấp nền tảng lý thuyết giúp hợp nhất các mô hình này thành một hệ thống chung, vừa linh hoạt vừa có cấu trúc toán học chặt chẽ.

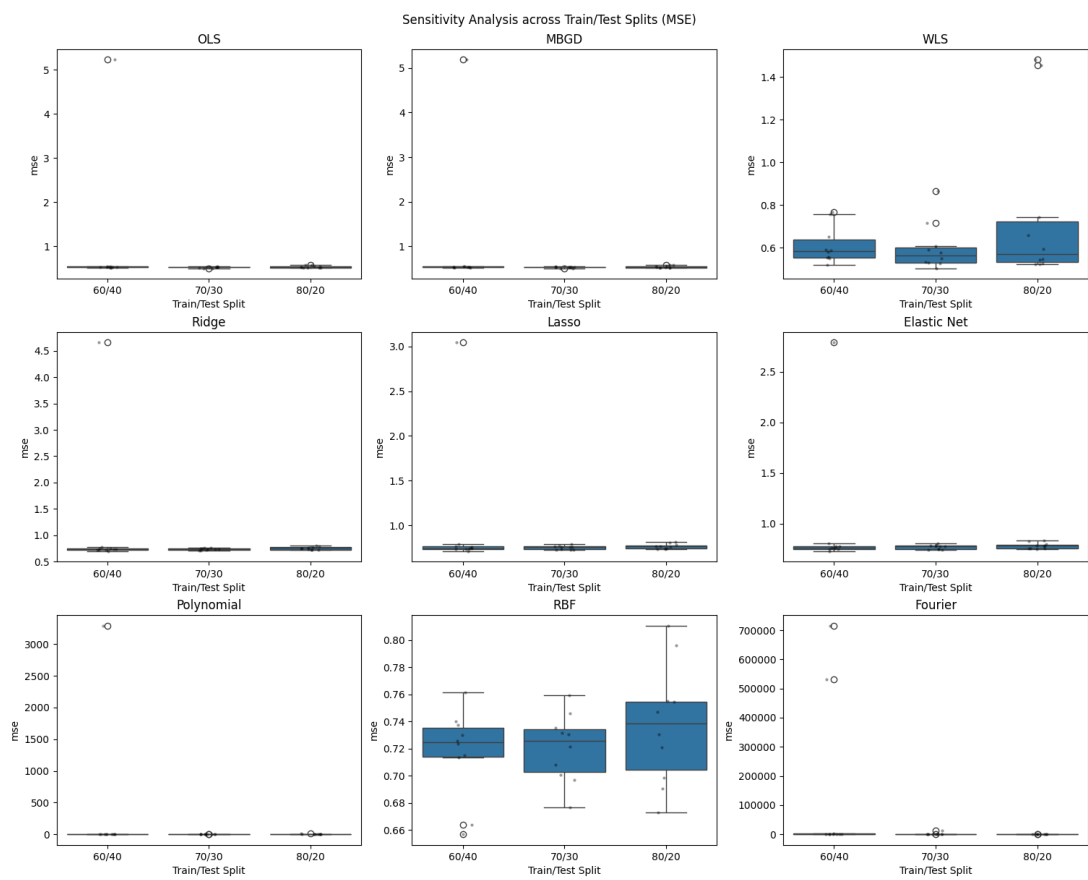
4 Phân tích mức độ nhạy cảm và tính bền vững

4.1 Độ nhạy cảm với tỉ lệ tập huấn luyện và tập kiểm tra

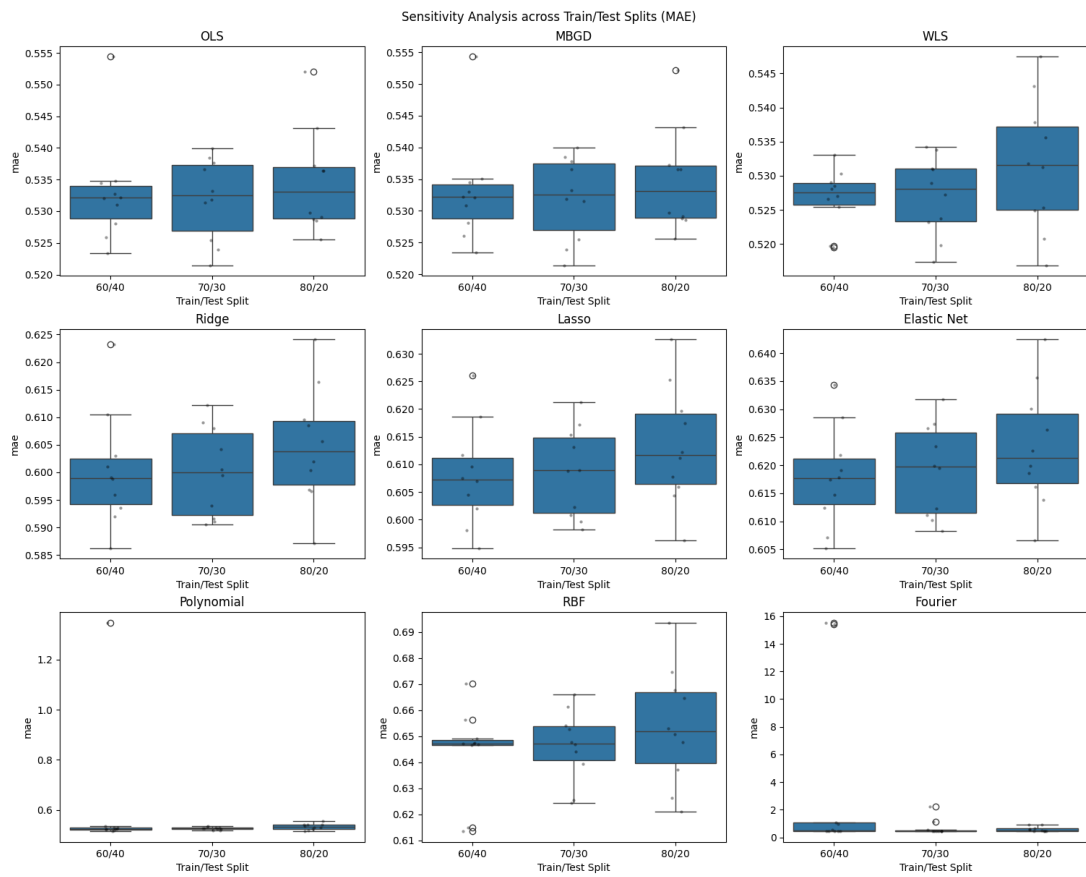
4.1.1 Hồi quy

Hình 48 và 49 trình bày đồ hộp các kết quả của 10 lần chạy ngẫu nhiên với từng tỉ lệ tập huấn luyện/kiểm tra khác nhau trên các chỉ số tương ứng MSE, MAE.

Nhìn vào biểu đồ, ta thấy các mô hình hồi quy tuyến tính cơ bản và chính quy hóa có ổn định khá cao. Tuy nhiên, các mô hình vẫn còn bị ảnh hưởng bởi cách chia dữ liệu, như đối với chỉ số MSE, có một điểm xuất hiện với giá trị trên 5.



Hình 48: Biểu đồ hộp các chỉ số MSE của các mô hình hồi quy đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau



Hình 49: Biểu đồ hộp các chỉ số MAE của các mô hình hồi quy đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau

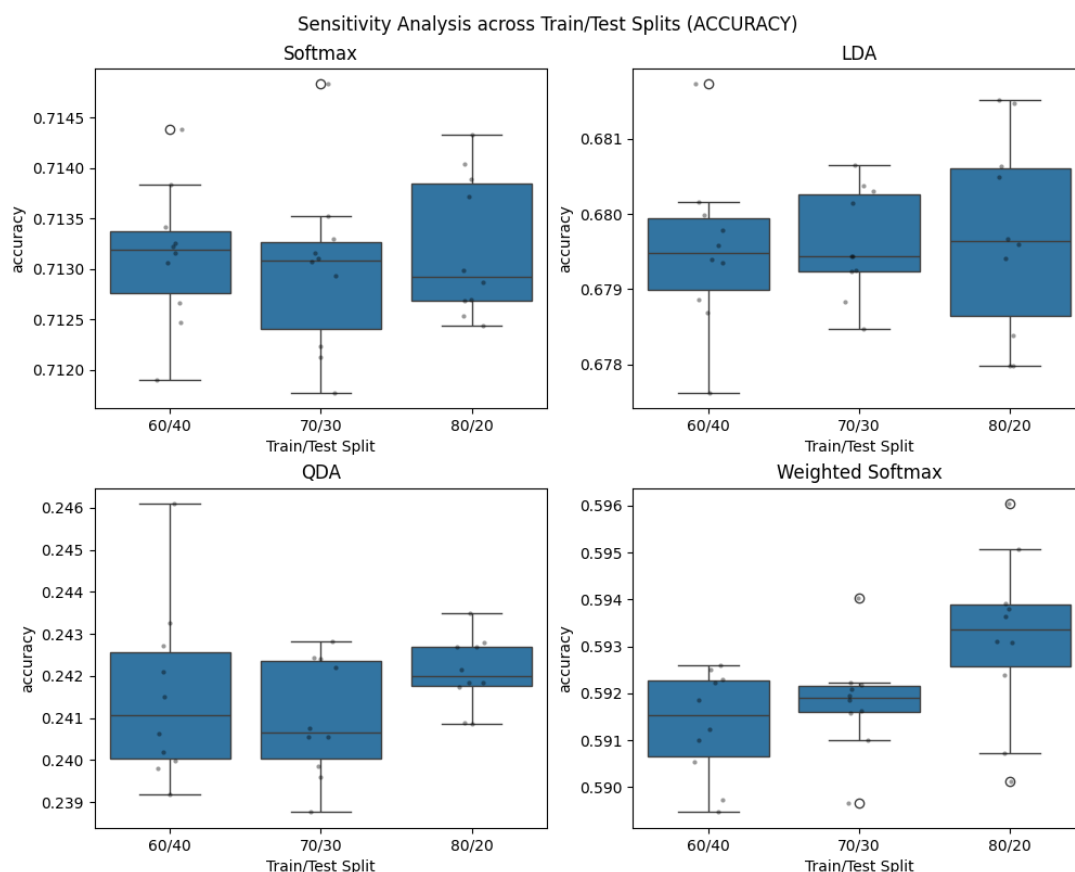
Trong khi đó, mô hình rất ổn định với cả cách chia này, khi mà ở chỉ số MSE, các điểm chỉ phân tán trong khoảng 0.66–0.8. Với các cách phân chia dữ liệu cực đoạn, kết quả cũng không cho thấy sự xuất hiện các ngoại lai quá lớn. Nhưng nếu xét trên chỉ số MAE, thì RBF lại không định bằng các mô hình trên khi dải phân bố rộng hơn đáng kể.

Các mô hình cơ sở phi tuyến Polynomial và Fourier có sự không ổn định khi mà xuất hiện các điểm ngoại lai rất lớn như (700,000+ ở MSE của Fourier, 3,000+ ở MSE của Polynomial, các chỉ số MAE cũng rất lớn nếu so sánh các mô hình khác). Điều này cho thấy sự mất ổn định cũng như dấu hiệu quá khớp ở các mô hình này.

Nhìn chung, RBF là mô hình ổn định nhất khi khó bị ảnh hưởng bởi cách chia dữ liệu và kết quả ở 2 chỉ số đều rất tốt. Các mô hình hồi quy tuyến tính cơ bản và chính quy hóa cũng có độ ổn định cao nhưng có phụ thuộc vào chia dữ liệu nhẹ.

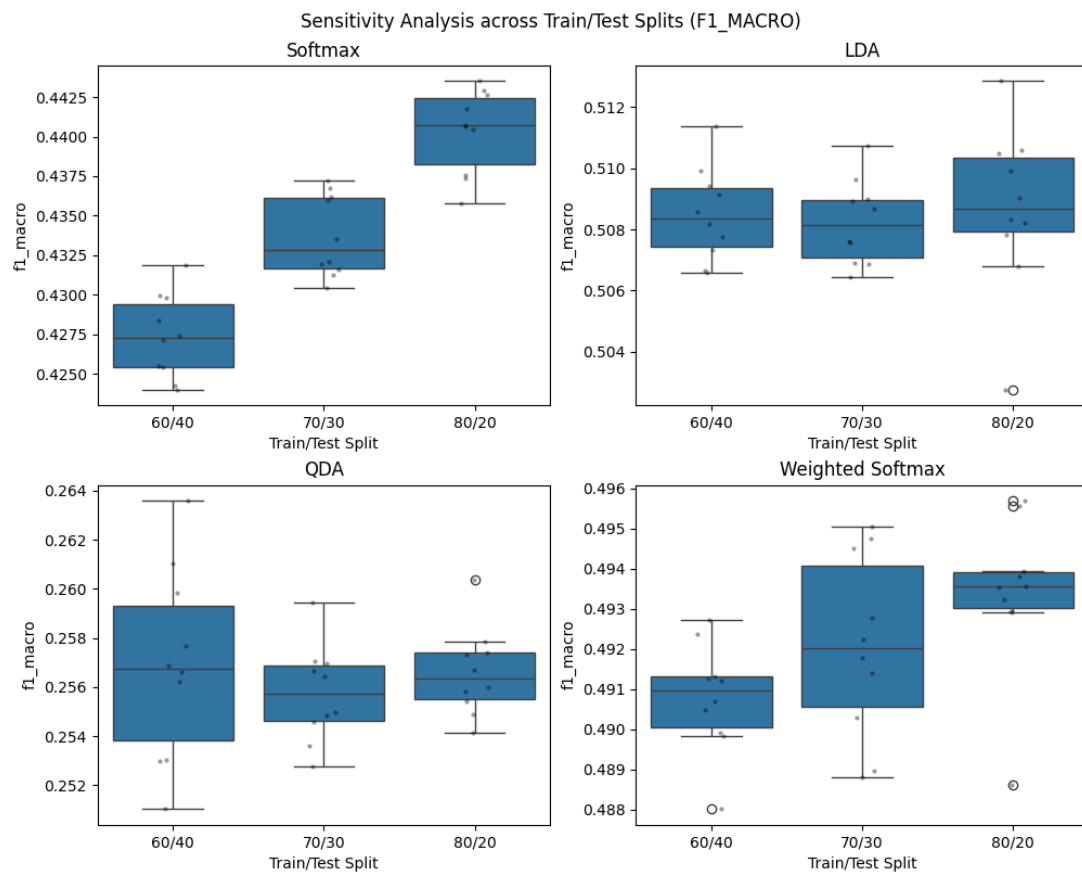
4.1.2 Phân lớp

Hình 50 và 51 trình biểu đồ hộp các kết quả của 10 lần chạy ngẫu nhiên với từng tỉ lệ tập huấn luyện/kiểm tra khác nhau trên các chỉ số tương ứng Accuracy và F1 macro.



Hình 50: Biểu đồ hộp các chỉ số Accuracy của các mô hình phân lớp đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau

Đối với chỉ số Accuracy, mô hình Softmax đạt giá trị cao nhất và ổn định nhất, dao động quanh mức khoảng 0.711–0.714 ở cả các tỉ lệ chia 60/40, 70/30 và 80/20. Mô hình LDA đứng thứ hai với Accuracy thấp hơn một chút (xấp xỉ 0.678–0.681) nhưng cũng rất ổn định giữa các seed. Trong khi đó, Weighted Softmax có Accuracy thấp hơn đáng kể (khoảng 0.589–0.596), và QDA cho kết quả thấp nhất (khoảng 0.24) với phương sai nhỏ nhưng hiệu năng kém rõ rệt so với các mô hình còn lại.



Hình 51: Biểu đồ hộp các chỉ số F1 của các mô hình phân lớp đối với các tỉ lệ tập huấn luyện và kiểm tra khác nhau

Đối với chỉ số F1-macro, xu hướng có sự khác biệt rõ hơn. Mô hình LDA thể hiện hiệu năng tốt nhất và ổn định nhất với F1 khoảng 0.506–0.512, phản ánh khả năng cân bằng tốt giữa precision và recall trên các lớp. Weighted Softmax đứng thứ hai với F1 khoảng 0.488–0.495; mặc dù Accuracy thấp hơn, mô hình này có xu hướng cải thiện recall nên F1 tương đối cạnh tranh. Softmax dù có Accuracy cao nhất nhưng F1 chỉ quanh mức 0.43–0.44, cho thấy mô hình thiên lệch hơn về lớp chiếm ưu thế. QDA tiếp tục cho kết quả thấp nhất với F1 khoảng 0.25 và không cải thiện đáng kể theo thay đổi seed hay tỉ lệ chia dữ liệu.

Nhìn chung, có thể rút ra rằng Softmax tối ưu về Accuracy nhưng không tối ưu về F1-macro, trong khi LDA là mô hình cân bằng tốt nhất giữa hai tiêu chí và ổn định nhất trên nhiều lần chạy. Weighted Softmax có cải thiện về khả năng thu hồi (recall) nhưng chưa vượt trội về tổng thể, còn QDA không phù hợp với bài toán do hiệu năng thấp và thiếu tính phân biệt giữa các lớp.

4.2 Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng

4.2.1 Hồi quy

Bảng 34 trình bày các kết quả về độ nhạy (tỉ lệ thay đổi tương đối khi chạy trên dữ liệu gốc) các chỉ số MSE, RMSE, MAE và R^2 đối với các độ lệch chuẩn nhiễu khác nhau. Các chỉ số độ nhạy cảm càng cao cho thấy kết quả càng tệ so với khi chạy trên dữ liệu gốc.

Bảng 34: Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng đối với các mô hình hồi quy

Model	σ	MSE sens	RMSE sens	MAE sens	R^2 sens
OLS	0.1	0.0229	0.0114	0.0129	0.0147
MBGD	0.1	0.0228	0.0113	0.0128	0.0146
WLS	0.1	0.015	0.0074	0.018	0.0098
Ridge	0.1	0.0043	0.0022	0.003	0.0051
Lasso	0.1	0.0041	0.0021	0.003	0.0052
Elastic Net	0.1	0.004	0.002	0.0029	0.0053
Polynomial	0.1	0.0348	0.0173	0.0207	0.0209
RBF	0.1	0.0993	0.0485	0.0738	0.0948
Fourier	0.1	0.0415	0.0205	0.0188	0.0168
OLS	0.5	0.1994	0.0952	0.1161	0.1274
MBGD	0.5	0.199	0.095	0.1158	0.1273
WLS	0.5	0.1873	0.0896	0.1328	0.1224
Ridge	0.5	0.0611	0.0301	0.0468	0.0726
Lasso	0.5	0.0626	0.0308	0.0459	0.0784
Elastic Net	0.5	0.0594	0.0292	0.0423	0.0787
Polynomial	0.5	0.2349	0.1113	0.1365	0.1407

Còn tiếp trang sau

Bảng 34: Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng đối với các mô hình hồi quy (Tiếp theo)

Model	σ	MSE sens	RMSE sens	MAE sens	R^2 sens
RBF	0.5	0.2127	0.1012	0.1408	0.2029
Fourier	0.5	0.4421	0.2009	0.2281	0.1789
OLS	1.0	0.4848	0.2185	0.277	0.3099
MBGD	1.0	0.4844	0.2184	0.2766	0.3098
WLS	1.0	0.4806	0.2168	0.3006	0.314
Ridge	1.0	0.2097	0.0998	0.1472	0.2491
Lasso	1.0	0.2077	0.0989	0.1429	0.2604
Elastic Net	1.0	0.1973	0.0942	0.1342	0.2615
Polynomial	1.0	0.5262	0.2354	0.2858	0.3152
RBF	1.0	0.2901	0.1358	0.1847	0.2768
Fourier	1.0	0.9761	0.4057	0.4817	0.395

Khi tăng nhiễu lên $\sigma = 0.1$, độ nhạy tương đối của tất cả các mô hình vẫn còn khá thấp (đa số MSE/RMSE/MAE sens chỉ khoảng 0.2–3%), cho thấy mô hình chưa bị ảnh hưởng đáng kể bởi nhiễu nhỏ. Tuy nhiên, một số mô hình phi tuyến như Polynomial và RBF bắt đầu xuất hiện độ nhạy cao hơn nhẹ so với các mô hình tuyến tính.

Khi tăng lên $\sigma = 0.5$, độ nhạy tăng rõ rệt. Các mô hình tuyến tính như OLS, MBDG và WLS có mức tăng sai số tương đối khoảng 10–20%, cho thấy chúng bắt đầu bị ảnh hưởng rõ rệt dưới nhiễu trung bình. Các mô hình regularization như Ridge, Lasso và Elastic Net có xu hướng ổn định hơn tương đối, với mức tăng sai số thấp hơn nhóm tuyến tính thuần túy. Trong nhóm phi tuyến, Polynomial và Fourier thể hiện độ nhạy cao, trong khi RBF chỉ tăng ở mức trung bình và vẫn giữ được độ ổn định tương đối tốt hơn so với hai mô hình còn lại.

Ở mức nhiễu cao nhất $\sigma = 1.0$, sự khác biệt trở nên rõ rệt hơn. Các mô hình tuyến tính truyền thống tăng sai số tương đối lên khoảng 20–40%. Các mô hình phi tuyến có xu hướng nhạy hơn, tuy nhiên mức độ không đồng đều: Polynomial tăng mạnh, trong khi RBF chỉ ở mức trung bình (ví dụ MSE sens khoảng 0.29), cho thấy khả năng chống chịu nhiễu tốt hơn so với nhiều mô hình phi tuyến khác. Đặc biệt, Fourier cho thấy mức độ biến động rất lớn (ví dụ MSE sens vượt gần 1), cho thấy mô hình này rất nhạy với nhiễu đầu vào trong cấu hình hiện tại. Ngược lại, các mô hình có regularization (Ridge, Lasso, Elastic Net) vẫn giữ được mức tăng tương đối thấp hơn so với phần lớn các mô hình phi tuyến, dù vẫn suy giảm đáng kể khi nhiễu lớn.

Tổng thể, kết quả cho thấy độ nhạy tăng theo σ một cách nhất quán trên tất cả mô hình. Các mô hình tuyến tính và regularization có xu hướng ổn định hơn trong nhiễu thấp đến trung bình. Các mô hình phi tuyến đạt hiệu năng tốt trên dữ liệu sạch nhưng nhạy hơn với nhiễu, tuy nhiên RBF là một trường hợp đáng chú ý khi vẫn duy trì được độ ổn định tương đối ngay cả khi nhiễu lớn.

4.2.2 Phân lớp

Bảng 35 cho thấy độ nhạy cảm tương đối của các mô hình phân lớp khi thêm nhiều Gaussian vào dữ liệu đầu vào. Nhìn chung, các giá trị sens có xu hướng tăng (dương hơn) khi σ tăng, cho thấy hiệu năng suy giảm theo mức nhiễu.

Bảng 35: Độ nhạy cảm với nhiễu Gaussian ở các biến đặc trưng đối với các mô hình phân lớp

Model	σ	Accuracy sens	Precision macro sens	Recall macro sens	F1 macro sens
Softmax	0.1	0.0101	0.0308	0.0708	0.0615
LDA	0.1	-0.0033	-0.0172	0.0264	-0.0086
QDA	0.1	-1.5435	-0.331	-0.1753	-1.0717
Weighted Softmax	0.1	0.0154	0.0228	0.0211	0.0292
Softmax	0.5	0.0327	0.2476	0.2472	0.2399
LDA	0.5	-0.0145	-0.049	0.2487	0.1269
QDA	0.5	-1.7787	-0.489	-0.0387	-1.1447
Weighted Softmax	0.5	0.0782	0.0786	0.0599	0.1054
Softmax	1.0	0.0376	0.2731	0.2829	0.2899
LDA	1.0	-0.0107	-0.0021	0.3298	0.21
QDA	1.0	-1.7544	-0.4878	0.0302	-1.0686
Weighted Softmax	1.0	0.0997	0.0999	0.0713	0.1326

Ở mức nhiễu thấp $\sigma = 0.1$, đa số các mô hình vẫn khá ổn định. Softmax và Weighted Softmax có sens dương nhỏ (khoảng 1–7%), cho thấy hiệu năng chỉ suy giảm nhẹ. LDA có một số chỉ số mang giá trị âm, tức là hiệu năng thậm chí cải thiện nhẹ trong điều kiện nhiễu nhỏ, phản ánh tính ổn định cao. Ngược lại, QDA có giá trị sens âm rất lớn về độ lớn (ví dụ Accuracy sens xấp xỉ -1.54), cho thấy sự thay đổi cực mạnh và không ổn định (dù một phần là cải thiện, nhưng biến động lớn là dấu hiệu kém bền vững).

Khi tăng lên $\sigma = 0.5$, xu hướng suy giảm trở nên rõ rệt hơn. Softmax có sens dương lớn ở các chỉ số macro (khoảng 24–25%), cho thấy hiệu năng giảm đáng kể. Weighted Softmax vẫn duy trì sens thấp hơn, thể hiện khả năng chống nhiễu tốt hơn. LDA tiếp tục ổn định ở Accuracy và Precision (sens gần 0 hoặc âm nhẹ), nhưng Recall và F1 bắt đầu có sens dương đáng kể, cho thấy mô hình bị ảnh hưởng khi xét đến cân bằng giữa các lớp. QDA tiếp tục có biến động rất lớn (cả âm và dương), phản ánh sự thiếu ổn định nghiêm trọng.

Ở mức nhiễu cao $\sigma = 1.0$, sự khác biệt càng rõ. Softmax có sens dương lớn (xấp xỉ 27–29%), cho thấy suy giảm mạnh. Weighted Softmax vẫn kiểm soát tốt hơn với sens thấp hơn đáng kể (xấp xỉ 7–13%). LDA duy trì Accuracy và Precision ổn định (sens gần 0), nhưng Recall và F1 có sens dương lớn, cho thấy hiệu năng tổng thể vẫn bị ảnh hưởng. QDA tiếp tục là mô hình kém ổn định nhất với các giá trị sens có độ lớn rất cao, cho thấy phản ứng mạnh và khó kiểm soát trước nhiễu.

Tổng thể, Weighted Softmax là mô hình ổn định nhất (sens nhỏ và nhất quán), LDA ổn định ở một số chỉ số nhưng không đồng đều, Softmax suy giảm rõ khi nhiễu tăng, còn

QDA có độ nhạy rất lớn và thiếu ổn định trong mọi mức nhiễu.

4.3 So sánh các chiến lược bổ sung dữ liệu thiếu (Imputation)

4.3.1 Hồi quy

Bảng 36 trình bày độ nhạy của các mô hình hồi quy trước các tỉ lệ dữ liệu thiếu khác nhau khi áp dụng các chiến lược bổ sung dữ liệu (mean, median và kNN).

Bảng 36: Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình hồi quy

Model	Imputer	Missing rate	MSE sens	RMSE sens	MAE sens	R^2 sens
OLS	mean	0.1	0.2733	0.1284	0.1372	0.1747
MBGD	mean	0.1	0.273	0.1283	0.1369	0.1746
WLS	mean	0.1	0.2568	0.1211	0.1469	0.1678
Ridge	mean	0.1	0.09	0.044	0.0504	0.1069
Lasso	mean	0.1	0.0883	0.0432	0.0482	0.1107
Elastic Net	mean	0.1	0.0838	0.041	0.0458	0.1111
Polynomial	mean	0.1	0.2971	0.1389	0.1475	0.178
RBF	mean	0.1	0.1756	0.0842	0.1045	0.1675
Fourier	mean	0.1	0.3162	0.1473	0.1634	0.128
OLS	median	0.1	0.2967	0.1387	0.1445	0.1896
MBGD	median	0.1	0.2964	0.1386	0.1443	0.1896
WLS	median	0.1	0.279	0.1309	0.155	0.1823
Ridge	median	0.1	0.0964	0.0471	0.0535	0.1146
Lasso	median	0.1	0.0925	0.0452	0.0505	0.116
Elastic Net	median	0.1	0.0878	0.043	0.0482	0.1163
Polynomial	median	0.1	0.3158	0.1471	0.1526	0.1892
RBF	median	0.1	0.2431	0.115	0.1502	0.232
Fourier	median	0.1	0.3126	0.1457	0.1611	0.1265
OLS	knn	0.1	0.1796	0.0861	0.0907	0.1148
MBGD	knn	0.1	0.1794	0.086	0.0905	0.1147
WLS	knn	0.1	0.1622	0.0781	0.094	0.106
Ridge	knn	0.1	0.0578	0.0285	0.0329	0.0687
Lasso	knn	0.1	0.0559	0.0276	0.0308	0.0701
Elastic Net	knn	0.1	0.0529	0.0261	0.0291	0.0701
Polynomial	knn	0.1	0.1956	0.0934	0.0969	0.1172

Còn tiếp trang sau

Bảng 36: Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình hồi quy (Tiếp theo)

Model	Imputer	Missing rate	MSE sens	RMSE sens	MAE sens	R^2 sens
RBF	knn	0.1	0.1686	0.081	0.109	0.1608
Fourier	knn	0.1	0.2303	0.1092	0.1181	0.0932
OLS	mean	0.2	0.4468	0.2028	0.2184	0.2856
MBGD	mean	0.2	0.4464	0.2027	0.2181	0.2855
WLS	mean	0.2	0.4211	0.1921	0.2302	0.2752
Ridge	mean	0.2	0.1885	0.0902	0.1012	0.224
Lasso	mean	0.2	0.1832	0.0877	0.0971	0.2297
Elastic Net	mean	0.2	0.1737	0.0834	0.092	0.2303
Polynomial	mean	0.2	0.4514	0.2048	0.2195	0.2704
RBF	mean	0.2	0.2399	0.1135	0.1372	0.2289
Fourier	mean	0.2	0.5102	0.2289	0.2678	0.2065
OLS	median	0.2	0.4665	0.211	0.2266	0.2982
MBGD	median	0.2	0.4661	0.2108	0.2263	0.2981
WLS	median	0.2	0.441	0.2004	0.2377	0.2882
Ridge	median	0.2	0.1957	0.0935	0.1011	0.2326
Lasso	median	0.2	0.1883	0.0901	0.0962	0.2362
Elastic Net	median	0.2	0.1786	0.0856	0.0915	0.2367
Polynomial	median	0.2	0.4659	0.2107	0.2238	0.2791
RBF	median	0.2	0.2711	0.1275	0.1462	0.2587
Fourier	median	0.2	0.4837	0.2181	0.2526	0.1958
OLS	knn	0.2	0.4331	0.1971	0.2068	0.2769
MBGD	knn	0.2	0.4327	0.197	0.2065	0.2768
WLS	knn	0.2	0.4112	0.1879	0.2205	0.2687
Ridge	knn	0.2	0.1894	0.0906	0.102	0.225
Lasso	knn	0.2	0.1855	0.0888	0.0988	0.2326
Elastic Net	knn	0.2	0.1755	0.0842	0.0934	0.2326
Polynomial	knn	0.2	0.4429	0.2012	0.2099	0.2653
RBF	knn	0.2	0.309	0.1441	0.1746	0.2948
Fourier	knn	0.2	0.5246	0.2348	0.27	0.2123
OLS	mean	0.3	0.6267	0.2754	0.309	0.4006
MBGD	mean	0.3	0.6262	0.2752	0.3086	0.4005
WLS	mean	0.3	0.6048	0.2668	0.3246	0.3952

Còn tiếp trang sau

Bảng 36: Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình hồi quy (Tiếp theo)

Model	Imputer	Missing rate	MSE sens	RMSE sens	MAE sens	R^2 sens
Ridge	mean	0.3	0.2891	0.1354	0.1546	0.3436
Lasso	mean	0.3	0.2761	0.1297	0.1462	0.3463
Elastic Net	mean	0.3	0.2612	0.123	0.1377	0.3462
Polynomial	mean	0.3	0.6723	0.2932	0.3263	0.4027
RBF	mean	0.3	0.4002	0.1833	0.2137	0.3819
Fourier	mean	0.3	0.7711	0.3308	0.3817	0.3121
OLS	median	0.3	0.6399	0.2806	0.3152	0.4091
MBGD	median	0.3	0.6394	0.2804	0.3148	0.409
WLS	median	0.3	0.6174	0.2718	0.3307	0.4034
Ridge	median	0.3	0.2971	0.1389	0.1573	0.3531
Lasso	median	0.3	0.2843	0.1333	0.149	0.3565
Elastic Net	median	0.3	0.269	0.1265	0.1405	0.3565
Polynomial	median	0.3	0.6805	0.2963	0.3285	0.4076
RBF	median	0.3	0.4357	0.1982	0.2289	0.4158
Fourier	median	0.3	0.7465	0.3215	0.3679	0.3021
OLS	knn	0.3	0.6332	0.278	0.304	0.4047
MBGD	knn	0.3	0.6327	0.2778	0.3037	0.4046
WLS	knn	0.3	0.6131	0.2701	0.3213	0.4006
Ridge	knn	0.3	0.3003	0.1403	0.1608	0.3569
Lasso	knn	0.3	0.2884	0.1351	0.1528	0.3616
Elastic Net	knn	0.3	0.2732	0.1284	0.1441	0.3621
Polynomial	knn	0.3	0.6883	0.2994	0.3227	0.4123
RBF	knn	0.3	0.4262	0.1942	0.2238	0.4066
Fourier	knn	0.3	0.7906	0.3381	0.3833	0.32

Trước hết, có thể nhận thấy một xu hướng nhất quán rằng khi tỉ lệ dữ liệu thiếu tăng từ 0.1 lên 0.3, độ nhạy của tất cả các mô hình đều tăng đáng kể trên mọi chỉ số (MSE, RMSE, MAE và R^2). Điều này cho thấy việc bổ sung dữ liệu không thể hoàn toàn bù đắp thông tin bị mất, đặc biệt trong trường hợp dữ liệu thiếu ở mức cao.

Xét theo từng phương pháp, kNN imputation cho kết quả tốt nhất một cách nhất quán trên hầu hết các mô hình và mức thiếu dữ liệu. Cụ thể, các giá trị độ nhạy của kNN luôn thấp hơn đáng kể so với mean và median, ví dụ với OLS tại mức thiếu 0.1, MSE sensitivity giảm từ 0.2733 (mean) và 0.2967 (median) xuống còn 0.1796. Xu hướng này tiếp tục được duy trì ở các mức thiếu cao hơn, cho thấy kNN có khả năng bảo toàn cấu trúc cục bộ của dữ liệu tốt hơn so với các phương pháp dựa trên thống kê toàn cục.

Mean imputation đóng vai trò như một phương pháp trung gian, thường cho kết quả

tốt hơn median nhưng vẫn kém hơn kNN. Phương pháp này có ưu điểm về tính đơn giản và chi phí tính toán thấp, tuy nhiên việc thay thế bằng giá trị trung bình làm giảm độ biến thiên của dữ liệu, từ đó ảnh hưởng đến khả năng học của mô hình.

Ngược lại, median imputation thường cho kết quả kém nhất trong các trường hợp thử nghiệm. Độ nhạy cao hơn cho thấy phương pháp này không phù hợp khi dữ liệu không bị lệch hoặc khi mối quan hệ giữa các biến mang tính phi tuyến.

Xét theo từng nhóm mô hình, các mô hình tuyến tính (OLS, MBGD, WLS) thể hiện độ nhạy cao hơn rõ rệt so với các mô hình có chính quy hóa (Ridge, Lasso, Elastic Net). Các mô hình chính quy hóa cho thấy khả năng chống chịu tốt hơn trước nhiễu do dữ liệu thiếu, thể hiện qua các giá trị độ nhạy thấp hơn đáng kể trên mọi chiến lược imputation.

Đối với các mô hình phi tuyến (Polynomial, RBF, Fourier), độ nhạy thường cao hơn so với các mô hình tuyến tính, đặc biệt khi tỉ lệ dữ liệu thiếu tăng. Điều này cho thấy các mô hình này phụ thuộc mạnh vào cấu trúc dữ liệu ban đầu, do đó dễ bị ảnh hưởng khi dữ liệu bị thiếu và được bổ sung lại.

Tóm lại, có thể xếp hạng hiệu quả của các phương pháp bổ sung dữ liệu theo thứ tự giảm dần: kNN, Mean, Median. Ngoài ra, khi tỉ lệ dữ liệu thiếu tăng, việc lựa chọn mô hình (đặc biệt là các mô hình có chính quy hóa) đóng vai trò quan trọng không kém so với việc lựa chọn phương pháp imputation.

4.3.2 Phân lớp

Bảng 37 trình bày độ nhạy của các mô hình phân lớp trước các tỉ lệ dữ liệu thiếu khác nhau khi áp dụng các chiến lược bổ sung dữ liệu (mean, median).

Bảng 37: Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình phân lớp

Model	Imputer	Missing rate	Accuracy sens	Precision macro sens	Recall macro sens	F1 macro sens
Softmax	mean	0.1	0.024	0.0569	0.0562	0.0388
LDA	mean	0.1	0.0254	0.0305	0.0697	0.044
QDA	mean	0.1	0.0575	0.0294	0.0795	0.0572
Weighted Softmax	mean	0.1	0.0503	0.0627	0.0604	0.074
Softmax	median	0.1	0.0248	0.0532	0.0589	0.043
LDA	median	0.1	0.0266	0.0335	0.0724	0.0464
QDA	median	0.1	0.1155	0.0793	0.062	0.1469
Weighted Softmax	median	0.1	0.052	0.0677	0.0619	0.079
Softmax	mean	0.2	0.0478	0.067	0.0998	0.0681
LDA	mean	0.2	0.0502	0.0521	0.1301	0.0831
QDA	mean	0.2	0.129	0.0581	0.1353	0.1444

Còn tiếp trang sau

Bảng 37: Độ nhạy cảm với tỉ lệ dữ liệu thiếu và chiến lược bổ sung dữ liệu thiếu đối với các mô hình phân lớp (Tiếp theo)

Model	Imputer	Missing rate	Accuracy sens	Precision macro sens	Recall macro sens	F1 macro sens
Weighted Softmax	mean	0.2	0.0981	0.1125	0.109	0.1326
Softmax	median	0.2	0.0476	0.0777	0.1081	0.0809
LDA	median	0.2	0.0519	0.055	0.1339	0.0871
QDA	median	0.2	0.2019	0.1319	0.1262	0.2735
Weighted Softmax	median	0.2	0.0995	0.1165	0.1055	0.1369
Softmax	mean	0.3	0.0738	0.0906	0.1583	0.1196
LDA	mean	0.3	0.0711	0.0658	0.1835	0.116
QDA	mean	0.3	0.2064	0.1054	0.2064	0.2495
Weighted Softmax	mean	0.3	0.1472	0.159	0.148	0.187
Softmax	median	0.3	0.0738	0.1247	0.1692	0.1339
LDA	median	0.3	0.0741	0.0693	0.1901	0.1217
QDA	median	0.3	0.2873	0.1678	0.1972	0.3769
Weighted Softmax	median	0.3	0.1502	0.1685	0.1474	0.1965

Trước hết, có thể nhận thấy một xu hướng nhất quán rằng khi tỉ lệ dữ liệu thiếu tăng từ 0.1 lên 0.3, độ nhạy của tất cả các mô hình đều tăng đáng kể trên mọi chỉ số (Accuracy, Precision macro, Recall macro và F1 macro). Điều này cho thấy việc bổ sung dữ liệu không thể bù đắp hoàn toàn thông tin bị mất, đặc biệt khi mức thiếu dữ liệu lớn.

So sánh giữa hai phương pháp, mean imputation thường cho kết quả tốt hơn hoặc tương đương so với median trên hầu hết các mô hình và chỉ số. Cụ thể, các giá trị độ nhạy khi sử dụng mean nhìn chung thấp hơn hoặc chênh lệch không đáng kể so với median, đặc biệt ở các mô hình Softmax và LDA.

Ngược lại, median imputation có xu hướng kém ổn định hơn, thể hiện rõ ở các mô hình nhạy như QDA. Ví dụ, tại mức thiếu 0.2 và 0.3, độ nhạy của QDA với median tăng đột biến so với mean trên tất cả các chỉ số, cho thấy phương pháp này dễ làm sai lệch phân phối dữ liệu trong bối cảnh hiện tại.

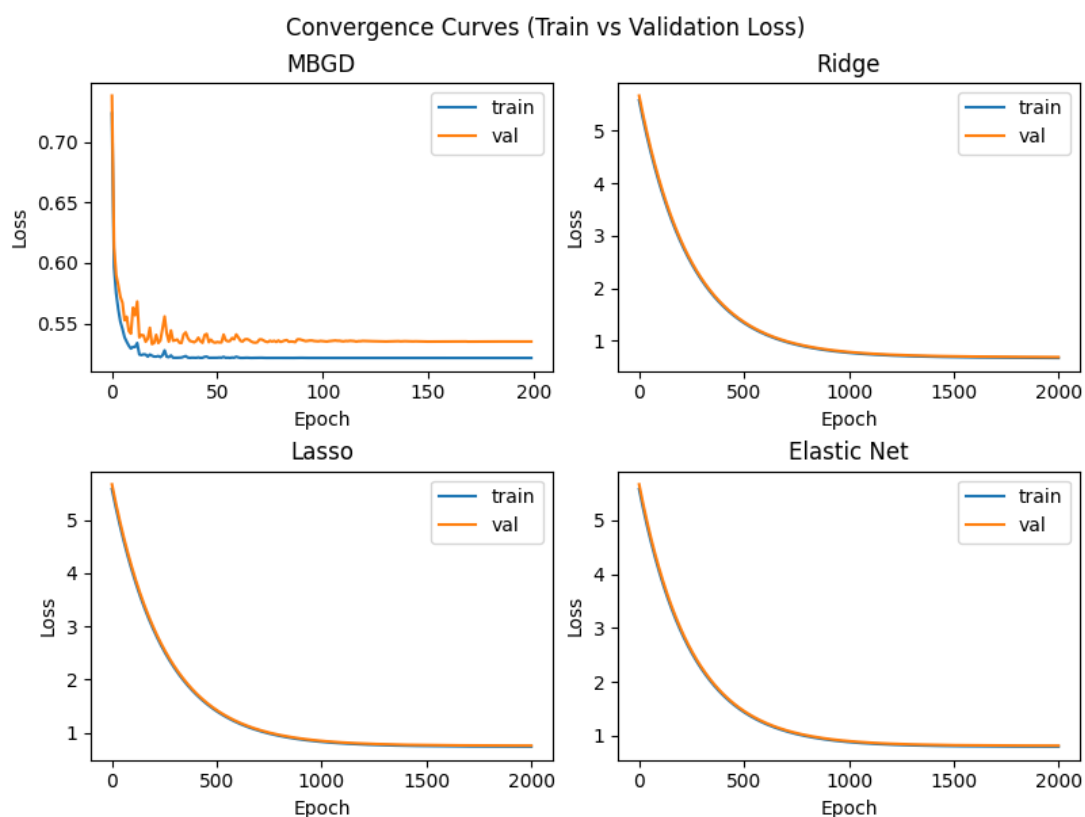
Xét theo từng mô hình, QDA là mô hình nhạy nhất với dữ liệu thiếu, với độ nhạy tăng rất mạnh khi missing rate tăng, đặc biệt khi sử dụng median. Các mô hình Softmax và LDA cho thấy độ ổn định tốt hơn, trong khi Weighted Softmax có độ nhạy cao hơn Softmax chuẩn nhưng vẫn thấp hơn đáng kể so với QDA.

Tổng hợp lại, theo thứ tự hiệu quả giảm dần: mean, median. Ngoài ra, do hạn chế về chi phí tính toán trên tập dữ liệu lớn, phương pháp kNN imputation không được xem xét trong thí nghiệm này.

4.4 Khả năng hội tụ

4.4.1 Hồi quy

Hình 52 mô tả đường cong lỗi của các mô hình hồi quy sử dụng thuật toán lặp.



Hình 52: Biểu đồ đường cong lỗi của các mô hình hồi quy sử dụng thuật toán lặp

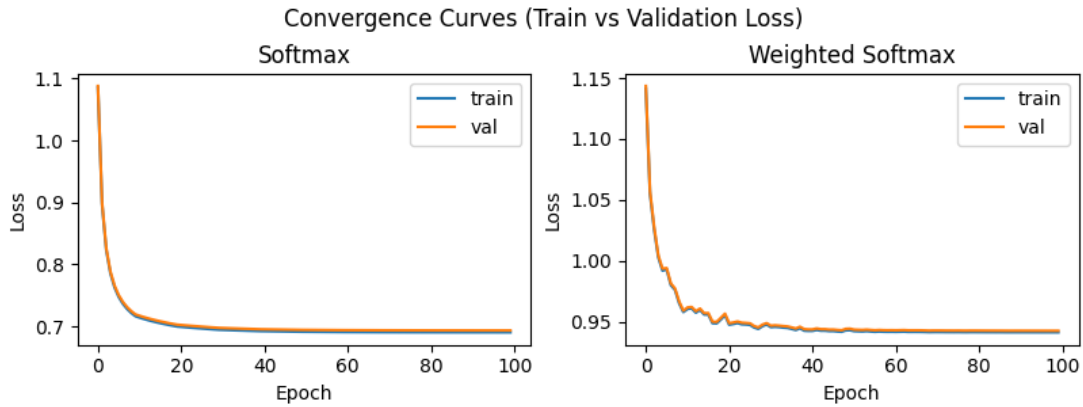
Qua biểu đồ, có thể quan sát rằng mô hình MBGD—sử dụng phương pháp lặp lịch tốc độ học dạng step decay—giảm lỗi rất nhanh trong giai đoạn đầu và hội tụ chỉ sau khoảng 150 epochs. Tuy nhiên, đường cong lỗi của MBGD xuất hiện các dao động nhẹ quanh xu hướng giảm. Đây là đặc trưng của phương pháp mini-batch, khi gradient chỉ được ước lượng trên các tập con dữ liệu nên mang tính nhiễu, dẫn đến quá trình cập nhật không hoàn toàn mượt.

Ngược lại, các mô hình có áp dụng chính quy hóa hội tụ chậm hơn đáng kể, cần khoảng 1500 epochs để đạt trạng thái ổn định. Một nguyên nhân quan trọng là các mô hình này không sử dụng các chiến lược lặp lịch tốc độ học như step decay. Do đó, để đảm bảo quá trình học ổn định và tránh dao động mạnh, tốc độ học phải được chọn nhỏ ngay từ đầu. Điều này làm cho mỗi bước cập nhật tham số trở nên thận trọng hơn, dẫn đến tốc độ giảm lỗi chậm và kéo dài thời gian hội tụ.

Bù lại, các mô hình chính quy hóa có đường cong lỗi mượt hơn và ít dao động hơn so với MBGD, cho thấy quá trình tối ưu ổn định hơn. Nhìn chung, tồn tại sự đánh đổi rõ rệt giữa tốc độ hội tụ và độ ổn định: MBGD hội tụ nhanh nhưng dao động nhẹ, trong khi các mô hình chính quy hóa hội tụ chậm hơn nhưng ổn định hơn và có xu hướng tổng quát hóa tốt hơn.

4.4.2 Phân lớp

Hình 53 mô tả đường cong lỗi của các mô hình hồi quy sử dụng thuật toán lặp.



Hình 53: Biểu đồ đường cong lỗi của các mô hình phân lớp sử dụng thuật toán lặp

Từ biểu đồ, có thể thấy rằng cả hai mô hình Softmax và Weighted Softmax đều hội tụ khá nhanh, chỉ trong khoảng 50 epochs. Điều này phản ánh đặc trưng của hàm mất mát cross-entropy, với độ dốc lớn ở giai đoạn đầu giúp mô hình nhanh chóng tiến gần tới vùng cực tiểu.

Tuy nhiên, mô hình Weighted Softmax có đường cong lỗi dao động nhẹ và kém mượt hơn so với Softmax. Nguyên nhân là do việc áp dụng trọng số lớp làm thay đổi phân phối gradient giữa các lớp, khiến quá trình cập nhật tham số trở nên kém ổn định hơn, đặc biệt trong bối cảnh dữ liệu mất cân bằng.

Dù tồn tại dao động, cả hai mô hình đều hội tụ nhanh và không có dấu hiệu phân kỳ, cho thấy cấu hình huấn luyện là hợp lý. Sự khác biệt về độ mượt của đường cong lỗi cũng cho thấy một đánh đổi giữa việc xử lý mất cân bằng dữ liệu và độ ổn định trong quá trình tối ưu.

5 Kiểm chuẩn (Benchmark)

Trong phần này, các mô hình được đánh giá và so sánh dựa trên nhiều tiêu chí khác nhau như thời gian huấn luyện, mức sử dụng bộ nhớ trong quá trình huấn luyện, cũng như thời gian và bộ nhớ khi suy luận. Mục tiêu của việc kiểm chuẩn là nhằm đánh giá hiệu năng tính toán của từng mô hình trong các điều kiện thực nghiệm giống nhau.

5.1 Hồi quy

Kết quả kiểm chuẩn của các mô hình hồi quy được trình bày trong Bảng 38 và Bảng 39. Kết quả được tính trên 5 lần chạy. Các tiêu chí đánh giá gồm trung bình và độ lệch chuẩn thời gian (theo đơn vị giây) và bộ nhớ (tính theo đơn vị byte) quá trình huấn luyện và suy luận.

Từ các kết quả kiểm chuẩn, có thể thấy sự khác biệt rõ rệt giữa các mô hình về thời gian huấn luyện và mức sử dụng bộ nhớ.

Các mô hình tuyến tính đơn giản như OLS và WLS có thời gian huấn luyện rất thấp, gần như tức thời, đồng thời mức tiêu thụ bộ nhớ cũng nhỏ. Điều này cho thấy đây là các mô hình có độ phức tạp tính toán thấp, phù hợp với các bài toán cần tốc độ xử lý cao.

Ngược lại, MBGD có thời gian huấn luyện lớn hơn đáng kể so với OLS và WLS do phải cập nhật tham số theo từng lô dữ liệu, kéo theo chi phí tính toán tăng lên. Tuy nhiên, mức bộ nhớ sử dụng vẫn tương đối ổn định.

Bảng 38: Kết quả kiểm chuẩn các mô hình hồi quy (phần 1)

model	train time mean	train memory mean	inference time mean	inference memory mean
OLS	0.001	1321608.0	0.0	66344.0
MBGD	2.1302	2518412.4	0.0	66368.0
WLS	0.0063	4758081.0	0.0	66344.0
Ridge	0.6169	2511440.0	0.0	66344.0
Lasso	0.634	2511440.0	0.0	66344.0
Elastic Net	0.6453	2511627.2	0.0	66344.0

Bảng 39: Kết quả kiểm chuẩn các mô hình hồi quy (phần 2)

model	train time std	train memory std	inference time std	inference memory std
OLS	0.0002	0.0	0.0	0.0
MBGD	0.0401	1037.7972	0.0	0.0
WLS	0.001	0.0	0.0	0.0
Ridge	0.0226	0.0	0.0	0.0
Lasso	0.023	0.0	0.0	0.0
Elastic Net	0.0268	310.4	0.0	0.0

Các mô hình Ridge, Lasso và Elastic Net có thời gian huấn luyện cao hơn OLS nhưng vẫn ở mức chấp nhận được. Sự khác biệt giữa ba mô hình này không quá lớn, tuy nhiên Elastic Net có xu hướng tốn thời gian hơn nhẹ do kết hợp đồng thời hai dạng chính quy.

Về suy luận, tất cả các mô hình hồi quy đều có thời gian suy luận rất nhỏ, gần như không đáng kể, cho thấy khả năng dự đoán nhanh sau khi huấn luyện.

5.2 Phân lớp

Kết quả kiểm chuẩn của các mô hình phân lớp được trình bày trong Bảng 40 và Bảng 41.

Bảng 40: Kết quả kiểm chuẩn các mô hình phân lớp (phần 1)

model	train time mean	train memory mean	inference time mean	inference memory mean
Softmax	32.7608	496501522.4	0.0376	71648448.0
LDA	2.4893	810431965.0	0.0254	13081910.0
QDA	1.9705	293849960.4	0.3616	157108606.0
Weighted Softmax	34.5564	496498120.2	0.0395	71648448.0

Trong nhóm mô hình phân lớp, có sự khác biệt rõ rệt hơn về cả thời gian huấn luyện lẫn bộ nhớ.

Bảng 41: Kết quả kiểm chuẩn các mô hình phân lớp (phần 2)

model	train time std	train mem- ory std	inference time std	inference memory std
Softmax	0.4819	6838.8137	0.0007	0.0
LDA	0.0108	0.0	0.0033	0.0
QDA	0.0157	308.8	0.0117	0.0
Weighted Softmax	1.7707	1015.3767	0.0047	0.0

LDA và QDA có thời gian huấn luyện tương đối nhỏ, trong đó QDA nhanh hơn LDA nhưng lại có thời gian suy luận lớn hơn đáng kể. Điều này phản ánh sự đánh đổi giữa độ phức tạp mô hình và chi phí dự đoán.

Softmax và Weighted Softmax có thời gian huấn luyện cao hơn so với LDA và QDA, trong đó Weighted Softmax là mô hình tốn thời gian huấn luyện nhiều nhất. Tuy nhiên, chi phí suy luận của các mô hình này khá thấp và ổn định.

Về bộ nhớ, LDA có mức tiêu thụ bộ nhớ cao hơn so với các mô hình còn lại, trong khi Softmax và Weighted Softmax sử dụng bộ nhớ ở mức trung bình nhưng ổn định giữa các lần chạy.

6 Bảng so sánh tất cả mô hình

Bảng 42 trình bày bảng so sánh tổng hợp các mô hình đã được cài đặt trong hai phần, bao gồm hồi quy tuyến tính, Ridge/Lasso, Logistic Regression và LDA/QDA, nhằm đối chiếu sự khác nhau về giả thiết mô hình, cách tìm nghiệm, độ phức tạp huấn luyện và các đặc tính thực nghiệm.

Trong bảng này, các ký hiệu được sử dụng có ý nghĩa như sau. N là số lượng mẫu trong tập dữ liệu. M là số lượng đặc trưng của mỗi mẫu. K là số lớp trong bài toán phân loại đa lớp. E là số vòng lặp khi sử dụng các phương pháp tối ưu lặp như gradient descent. Các ký hiệu $\mathbf{x}$ và $\mathbf{w}$ tương ứng biểu diễn véc-tơ đặc trưng đầu vào và véc-tơ tham số của mô hình, trong khi t là biến mục tiêu cần dự đoán.

Bảng 42: So sánh toàn diện các mô hình hồi quy và phân lớp

Tiêu chí	Linear Regression	Ridge/Lasso	Logistic Regression	LDA/QDA
Giả thiết phân phối	$t = \mathbf{w}^\top \mathbf{x} + \varepsilon$, với $\mathbb{E}[\varepsilon \mathbf{X}] = 0$	$t = \mathbf{w}^\top \mathbf{x} + \varepsilon$ với $\ \mathbf{w}\ _2^2$ (Ridge) hoặc $\ \mathbf{w}\ _1$ (Lasso)	$P(t = 1 \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$ cho bài toán nhị lớp, $P(t = k \mathbf{x}) = \exp(\mathbf{w}_k^\top \mathbf{x}) / (\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x}))$ cho bài toán đa lớp	$\mathbf{x} t = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \Sigma)$ (LDA), $\mathbf{x} t = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$ (QDA)
Nghiệm dạng đóng	$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$	Ridge có $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{t}$; Lasso thì không có	Không có	LDA có nghiệm đóng từ $\boldsymbol{\mu}_k, \Sigma$, QDA từ $\boldsymbol{\mu}_k, \Sigma_k$
Độ phức tạp huấn luyện	$\Theta(NM^2 + M^3)$ khi dùng Normal Equation, $\Theta(ENM)$ khi dùng gradient descent	Ridge thì $\Theta(NM^2 + M^3)$ khi dùng nghiệm đóng, $\Theta(ENM)$ khi dùng gradient descent; còn Lasso thì $\Theta(ENM)$ khi dùng gradient descent	$\Theta(ENM)$ cho nhị lớp khi dùng gradient descent; $\Theta(EKNM)$ cho bài toán đa lớp với softmax khi dùng gradient descent	LDA $\Theta(NM^2)$ khi ước lượng thống kê lớp QDA $\Theta(KNM + KM^2)$ khi ước lượng riêng theo từng lớp
Khả năng giải thích	Cao	Ridge cao hơn Lasso ổn định, Lasso thưa	Trung bình	LDA cao, QDA trung bình thấp
Nhạy với outlier	Cao	Ridge giảm nhạy, Lasso vẫn nhạy	Trung bình	Cao
Hiệu năng thực nghiệm	Tốt	Trung bình	Tốt	LDA tốt, QDA kém

A Thông tin nhóm

Báo cáo được thực hiện bởi nhóm 13 của lớp 23_24 môn Nhập môn học máy.

A.1 Thông tin thành viên

Thông tin các thành viên nhóm được trình bày ở Bảng 43.

Bảng 43: Thông tin thành viên nhóm

Tên thành viên	Mã số sinh viên
Hoàng Ngọc Phú [†]	23120010
Hoàng Ngọc Quý	23120077
Nguyễn Duy Bảo	23120113

[†] Nhóm trưởng.

A.2 Phân chia công việc và mức độ hoàn thành

Bảng 44 trình bày các nhiệm vụ và thành viên thực hiện. Mức độ hoàn thành được tính riêng cho từng thành viên thay vì so với tỉ lệ hoàn thành trong từng công việc—tức là ai hoàn thành công việc của mình đều được đánh giá 100%.

Bảng 44: Nhiệm vụ và mức độ hoàn thành yêu cầu được giao của mỗi thành viên

Nhiệm vụ	Thành viên		
	H. N. Phú	H. N. Quý	N. D. Bảo
Bài toán hồi quy			
Chọn bộ dữ liệu	100%	100%	100%
Phân tích khám phá và tiền xử lí	100%		
Hồi quy tuyến tính	100%		
Hồi quy có Regularization và Lựa chọn Đặc trưng		100%	
Mô hình với Hàm Cơ sở Phi Tuyến và Ablation Study			100%
Hồi quy Bayesian đầy đủ + vùng bất định (nâng cao)			100%
Evidence Maximization (nâng cao)			100%
Kernel Ridge Regression (nâng cao)		100%	
Gaussian Process Regression (nâng cao)	100%		
Robust Regression (nâng cao)	100%		
Phân tích Bias–Variance thực nghiệm (nâng cao)		100%	

Còn tiếp trang sau

Bảng 44: Nhiệm vụ và mức độ hoàn thành yêu cầu được giao của mỗi thành viên (Tiếp theo)

Nhiệm vụ	Thành viên		
	H. N. Phú	H. N. Quý	N. D. Bảo
Đánh giá mô hình	100%	100%	100%
Phân tích và thảo luận	100%	100%	100%
Bài toán phân lớp			
Chọn bộ dữ liệu	100%	100%	100%
Phân tích khám phá và tiền xử lí	100%		
Logistic Regression	100%		
Linear Discriminant Analysis-LDA và QDA		100%	
Perceptron và Logistic Regression có Regularization			100%
Mô hình Probit (nâng cao)	100%		
Xấp xỉ Laplace (nâng cao)	100%		
Kernel Logistic Regression (nâng cao)		100%	
Gaussian Naive Bayes vs. LDA (nâng cao)		100%	
Phân tích VC dimension (nâng cao)			100%
Đánh giá mô hình	100%	100%	100%
Phân tích và thảo luận	100%	100%	100%
Kết nối hồi quy và phân lớp		100%	100%
Bảng so sánh toàn diện	100%	100%	100%
Phân tích mức độ nhạy cảm (sensitivity) và tính bền vững (robustness)	100%		
Khác			
Cài đặt	100%	100%	100%
Chạy thí nghiệm	100%	100%	100%
Viết báo cáo	100%	100%	100%

A.3 Mức độ hoàn thành đề án

Bảng 45 trình bày từng yêu cầu của đề án và mức độ hoàn thành.

Bảng 45: Yêu cầu và mức độ hoàn thành từng yêu cầu

Yêu cầu	Mức độ hoàn thành
Bài toán hồi quy	

Còn tiếp trang sau

Bảng 45: Yêu cầu và mức độ hoàn thành từng yêu cầu (Tiếp theo)

Yêu cầu	Mức độ hoàn thành
Cơ sở lý thuyết: Normal Eq., Gauss–Markov, Bias–Variance, Bayesian, Evidence	100%
EDA, kiểm định Breusch–Pagan, WLS nếu cần, tiền xử lý đầy đủ	100%
Linear Regression: Normal Equations + Mini-batch GD + learning rate schedule	100%
Elastic Net + Lasso path + Forward/Backward feature selection	100%
Mô hình phi tuyến (3 loại) + validation curve + ablation study	100%
Đánh giá: k -fold CV $k = 10$, kiểm định thống kê, calibration	100%
Hồi quy Bayesian (nâng cao)	100%
Evidence Maximization (nâng cao)	100%
Kernel Ridge (nâng cao)	100%
GP Regression (nâng cao)	100%
Robust Regression (nâng cao)	100%
Bias–Var bootstrap	100%
Bài toán phân lớp	
Lý thuyết: gradient/Hessian LR, IRLS, Perceptron, Kernel LR, Laplace, Probit	100%
EDA, phân tích mất cân bằng, tiền xử lý có chiều sâu	100%
Logistic Regression: GD + Newton–Raphson + so sánh 3 chiến lược đa lớp	100%
LDA + QDA: Fisher ratio, so sánh lý thuyết, quy chiếu 2D	100%
Perceptron + L1/L2 LR + class-weighted loss + stratified CV	100%
Đánh giá: CV, McNemar, PR curve, calibration, phân tích lỗi sâu	100%
Probit (nâng cao)	100%
Laplace (nâng cao)	100%
Kernel LR (nâng cao)	100%

Còn tiếp trang sau

Bảng 45: Yêu cầu và mức độ hoàn thành từng yêu cầu (Tiếp theo)

Yêu cầu	Mức độ hoàn thành
GNB vs LDA (nâng cao)	100%
VC dimension (nâng cao)	100%
Kết nối Hồi quy và Phân lớp	
Hồi quy logistic như hồi quy	100%
Generalized Linear Models (GLM)	100%
Exponential family	100%
Bảng so sánh toàn diện	100%
Phân tích mức độ nhạy cảm (sensitivity) và tính bền vững (robustness)	
Sensitivity analysis	100%
Noise injection	100%
Feature corruption	100%
Phân tích khả năng hội tụ (convergence)	100%
Khả năng tái hiện lại thí nghiệm	
Cố định tất cả random seed	100%
Ghi log đầy đủ: hyperparameter, train/val/test split indices, metric từng fold, ...	80% [†]
Báo cáo thời gian thực thi/ huấn luyện/ kiểm tra và bộ nhớ tiêu tốn của từng thuật toán	100%
Cung cấp file <code>environment.yml</code> (conda) hoặc <code>requirements.txt</code> với version cụ thể	100%
Kết quả phải tái hiện được hoàn toàn khi chạy lại notebook từ đầu	100%
Yêu cầu về Code	
Tất cả code phải chạy lại được	100%
Phần cài đặt từ đầu phải rõ ràng, tách biệt với phần sử dụng thư viện kiểm chứng	100%
Phiên bản Python và thư viện phải được ghi rõ trong file <code>requirements.txt</code>	100%
Đặt seed ngẫu nhiên để đảm bảo kết quả tái hiện được	100%
Yêu cầu về Báo cáo	

Còn tiếp trang sau

Bảng 45: Yêu cầu và mức độ hoàn thành từng yêu cầu (Tiếp theo)

Yêu cầu	Mức độ hoàn thành
Báo cáo được nộp dưới dạng PDF, không giới hạn độ dài trang	100%
Danh mục tài liệu tham khảo theo chuẩn IEEE hoặc APA rõ ràng	100%
Khuyến nghị sử dụng $\text{\LaTeX}$. Nếu sử dụng $\text{\LaTeX}$, yêu cầu nhóm nộp cả mã nguồn <code>.tex</code>	100%
Mỗi thành viên trong nhóm phải có mục đóng góp rõ ràng	100%

[†] Chưa thực hiện ở các phần nâng cao.

B Mã nguồn bài làm đồ án

Mã nguồn của đồ án được công bố tại GitHub: <https://github.com/kanumi-2005/supervised-learning-project>

Tài liệu tham khảo

- Blackard, J. (1998). *Covertime*. UCI Machine Learning Repository. (DOI: <https://doi.org/10.24432/C50K5N>)
- Géron, A. (2019). *Hands-on machine learning with scikit-learn, keras, and tensorflow: Concepts, tools, and techniques to build intelligent systems* (2nd ed.). O'Reilly Media.
- Murphy, K. P. (2022). *Probabilistic machine learning: An introduction*. MIT Press. Retrieved from <http://probml.github.io/book1>
- Pace, R. K., & Barry, R. (1997, May). Sparse spatial autoregressions. *Statistics and Probability Letters*, 33(3), 291–297.